

Application lifecycle management (ALM) with Microsoft Power Platform

Learn how to implement application lifecycle management (ALM) using Microsoft Power Platform (Power Apps, Power Automate, Microsoft Dataverse, Microsoft Copilot Studio).

About ALM with Power Platform

OVERVIEW

[What is ALM?](#)

CONCEPT

[ALM basics with Power Platform](#)

[Environment strategy](#)

REFERENCE

[Tools and apps used for ALM](#)

Understand and work with solutions

CONCEPT

[Solution concepts](#)

[Understand solution layers](#)

[Use solutions to customize](#)

[Segment solutions](#)

Implement healthy ALM

CONCEPT

[Scenario 0: ALM for a new project](#)

[Scenario 1: Citizen development ALM](#)

[Scenario 2: Moving from a single environment](#)

[Scenario 3: Moving from unmanaged to managed solutions](#)

[Scenario 4: Use DevOps for automation](#)

[Scenario 5: Support team development](#)

[Scenario 6: Embrace citizen developers](#)

For developers

CONCEPT

[ALM for developers](#)

[Work with solution components](#)

[Solution and packaging tools](#)

[Manage solutions using custom code](#)

ALM automation

OVERVIEW

[Use Power Platform Build Tools for Azure DevOps](#)

[Use GitHub Actions for Power Platform](#)

[Deploy solutions using pipelines](#)

[CoE: Set up ALM Accelerator](#)

[CoE: Use ALM Accelerator](#)

Related docs

OVERVIEW

[Create apps in Power Apps](#)

[What is Microsoft Dataverse?](#)

[Work with solutions](#)

[Power Platform admin guide](#)

[Power Platform Center of Excellence \(CoE\) Kit](#)

[Best practices and guidance when using Microsoft Dataverse](#)

Overview of application lifecycle management with Microsoft Power Platform

Article • 01/29/2025

The articles in this section describe how you can implement application lifecycle management (ALM) using Power Apps, Power Automate, Power Pages, Microsoft Copilot Studio, and Microsoft Dataverse.

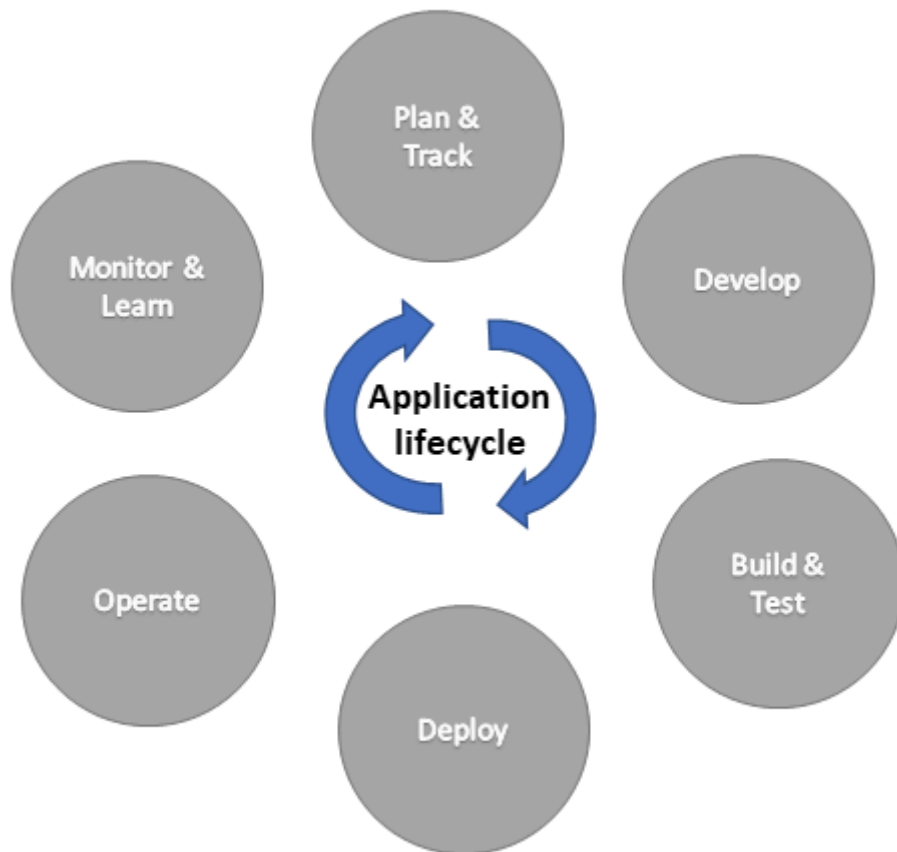
What is ALM?

ALM is the lifecycle management of applications, which includes governance development and maintenance. Moreover, ALM includes these disciplines: requirements management, software architecture, development, testing, maintenance, change management, support, continuous integration, project management, deployment, release management and governance. ALM tools provide a standardized system for communication and collaboration between software development teams and related departments, such as test and operations. These tools can also automate the process of software development and delivery. To that end, ALM combines the disciplines concerned with all aspects of the process to achieve the goal of driving efficiency through predictable and repeatable software delivery.

Key areas of ALM

- **Governance** includes requirements management, resource management, nurturing and system administration such as data security, user access, change tracking, review, audit, deployment control, and rollback.
- **Application development** includes identifying current problems, and planning, design, building, and testing the application and its continuous improvements. This area includes traditional developer and app maker roles.
- **Maintenance** includes deployment of the app, and maintenance of optional and dependent technologies.

The *application lifecycle* is the cyclical software development process that involves these areas: plan and track, develop, build and test, deploy, operate, monitor, and learn from discovery.



ALM for Power Apps, Power Automate, Microsoft Copilot Studio, and Dataverse

[Dataverse](#) in Microsoft Power Platform lets you securely store and manage data and processes that are used by business applications. To use the Power Platform features and tools available to manage ALM, all environments that participate in ALM must include a Dataverse database.

The following concepts are important for understanding ALM using the Microsoft Power Platform.

- *Solutions* are the mechanism for implementing ALM; you use them to distribute components across environments through export and import. A component represents an artifact used in your application and something that you can potentially customize. Anything that can be included in a solution is a component, such as tables, columns, canvas and model-driven apps, Power Automate flows, agents, charts, and plug-ins.
- *Dataverse* stores all the artifacts, including solutions and in-product deployment [pipelines](#)
- *Source control* should be your source of truth for storing and collaborating on your components.

- *Continuous integration and continuous delivery (CI/CD) platform* such as [Azure DevOps](#) that allows you to automate your build, test, and deployment pipeline. Can also be used in with in-product [pipelines](#).

For more information about how ALM and Azure DevOps technologies—combined with people and processes—enable teams to continually provide value to customers, go to [DevOps tools on Azure](#) .

Related articles

- [ALM basics with Microsoft Power Platform](#)
- [Set up pipelines in Power Platform](#)
- [What is Dataverse?](#)
- [Application lifecycle management for Finance and Operations apps](#)
- [ALM for copilots in Copilot Studio](#)
- [Using solutions with Power Pages](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

ALM basics with Microsoft Power Platform

Article • 01/30/2025

This article describes the components, tools, and processes needed to implement application lifecycle management (ALM).

Environments

Environments are a space to store, manage, and share your organization's business data, apps, and business processes. They also serve as containers to separate apps that might have different roles, security requirements, or target audiences. Each environment can have only one Microsoft Dataverse database. More information: [Environments overview](#)

Important

When you create an environment, you can choose to install Dynamics 365 apps, such as Dynamics 365 Sales and Dynamics 365 Marketing. It's important to determine at that time if these apps are required or not because they can't be uninstalled or installed later. If you aren't building on these apps and will not require them in the future, we recommend that you not install them in your environments. This helps avoid dependency complications when you distribute solutions between environments.

Types of environments used in ALM

Using the Power Platform admin center, you can create these types of Power Platform environments:

- **Sandbox.** A sandbox environment is any nonproduction environment of Dataverse. Isolated from production, a sandbox environment is the place to safely develop and test application changes with low risk. Sandbox environments include capabilities that would be harmful in a production environment, such as reset, delete, and copy operations. More information: [Manage sandbox environments](#)
- **Production.** The environment where apps and other software are put into operation for their intended use.
- **Developer** (formally called Community). A developer environment is a single-user environment, and can't be used to run or share production apps. The Power Apps

Developer Plan gives you access to Power Apps premium functionality, Dataverse, and Power Automate for individual use. This plan is primarily meant to build and test with Power Apps, Power Automate, and Dataverse or for learning purposes.

- **Default.** A single default environment is automatically created for each tenant and shared by all users in that tenant. The tenant identifies the customer, which can have one or more Microsoft subscriptions and services associated with it.

Whenever a new user signs up for Power Apps, they're automatically added to the Environment Maker role of the default environment. The default environment is created in the closest region to the default region of the Microsoft Entra tenant and is named: "{Microsoft Entra tenant name} (default)"

Create and use the correct environment for a specific purpose, such as development, test, or production.

For more information on environments, go to [Environments overview](#).

Who should have access?

Define and manage the security of your resources and data in Dataverse. Power Platform provides environment-level admin roles to perform tasks. Dataverse includes security roles that define the level of access to apps, app components, and resources app makers and users have within Dataverse.

 Expand table

Environment purpose	Roles that have access	Comments
Development	App makers and developers.	App users shouldn't have access. Developers require at least the Environment Maker security role to create resources.
Test	Admins and people who are testing.	App makers, developers, and production app users shouldn't have access. Test users should have just the privileges to perform testing.
Production	Admins and app users. Users should have just the access to perform their tasks for the apps they use.	App makers and developers shouldn't have access, or should only have user-level privileges.
Default	By default, every user in your tenant can create and edit apps	We strongly recommend that you create environments for a specific purpose, and

Environment purpose	Roles that have access	Comments
	in a Dataverse default environment that has a database.	grant the appropriate roles and privileges only to those people who need them.

More information:

- [Environments overview](#)
- [Control user access to environments: security groups and licenses](#)
- [Create users and assign security roles](#)
- [Create environments](#)

Solutions

Solutions are used to transport apps and components from one environment to another, or to apply a set of customizations to existing apps.

Solutions have these features:

- They include metadata and certain tables with configuration data. Solutions don't contain any business data.
- They can contain many different Power Platform components, such as model-driven apps, canvas apps, site maps, flows, tables, forms, custom connectors, web resources, choices, charts, and columns. Notice that not all tables can be included in a solution. For example, the Application User, Custom API, and Organization Setting system tables can't be added to a solution.
- They're packaged as a unit to be exported and imported to other environments, or deconstructed and checked into source control as source code for assets. Solutions are also used to apply changes to existing solutions.
- Managed solutions are used to deploy to any environment that isn't a development environment for that solution. This includes test, user acceptance testing (UAT), system integration testing (SIT), and production environments. Managed solutions can be serviced (upgrade, patch, and delete) independently from other managed solutions in an environment. As an ALM best practice, managed solutions should be generated by a build server and considered a build artifact.
- Updates to a managed solution are deployed to the previous version of the managed solution. This doesn't create an additional solution layer. You can't delete components by using an update.
- A patch contains only the changes for a parent managed solution. You should only use patches when making small updates (similar to a hotfix) and you require it to

possibly be uninstalled. When patches are imported, they're layered on top of the parent solution. You can't delete components by using a patch.

- Upgrading a solution installs a new solution layer immediately above the base layer and any existing patches.
 - Applying solution upgrades involves deleting all existing patches and the base layer.
 - Solution upgrades delete components that existed but are no longer included in the upgraded version.

More information: [Solution concepts](#)

Source control

Source control, also known as version control, is a system that maintains and securely stores software development assets and tracks changes to those assets. Change tracking is especially important when multiple app makers and developers are working on the same set of files. A source control system also gives you the ability to roll back changes or restore deleted files.

A source control system helps organizations achieve healthy ALM because the assets maintained in the source control system are the "single source of truth"—or, in other words, the single point of access and modification for your solutions.

Branching and merging strategy

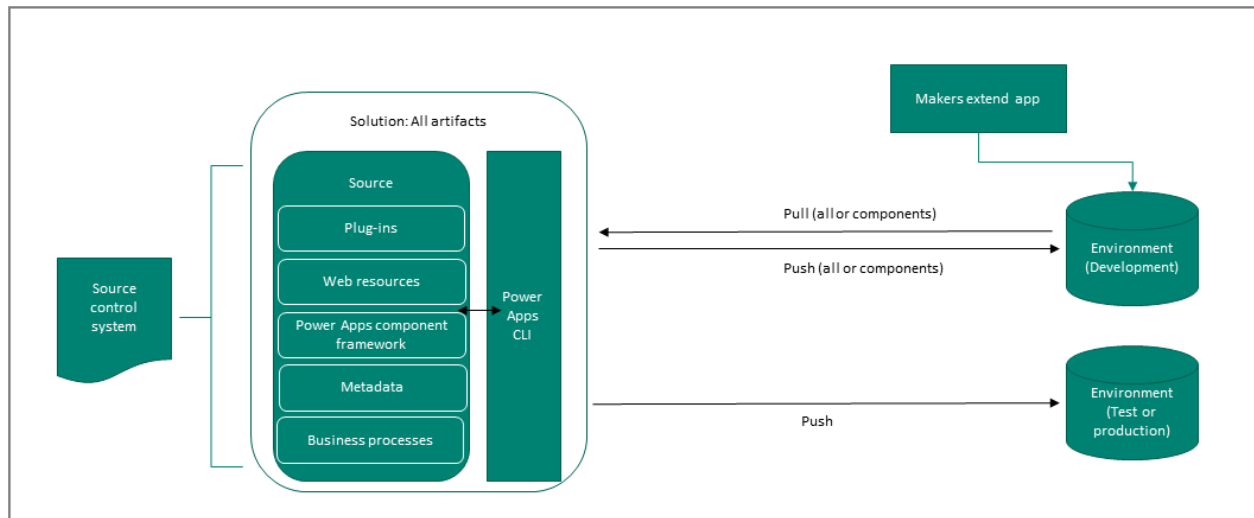
Nearly every source control system has some form of branching and merging support. Branching means you diverge from the main line of development and continue to do work without changing the main line. The process of merging consists of combining one branch into another, such as from a development branch into a main line branch. Some common branching strategies are trunk-based branching, release branching, and feature branching. More information: [Adopt a Git branching strategy](#)

Source control process using a solution

There are two main paths you can use when working with solutions in a source control system:

- Export the unmanaged solution and place it as unpacked in the source control system. The build process imports the packed solution as unmanaged into a temporary build environment (sandbox environment). Then, export the solution as managed and store it as a build artifact in your source control system.

- Export the solution as unmanaged and also export the solution as managed, and place both in the source control system. Although this method doesn't require a build environment, it does require maintaining two copies of all components (one copy of all unmanaged components from the unmanaged solution and one copy of all managed components from the managed solution).



More information: [Build tool tasks](#)

Automation

Automation is a key part of the application lifecycle that improves the productivity, reliability, quality, and efficiency of ALM. Automation tools and tasks are used to validate, export, pack, unpack, and export solutions in addition to creating and resetting sandbox environments.

More information: [What are Microsoft Power Platform Build Tools?](#)

Team development using shared source control

It's important to consider how you and your development team will work together to build the project. Breaking down silos and fostering views and conversations can enable your team to deliver better software. Some tools and workflows—such as those provided in Git, GitHub, and Azure DevOps—were designed for the express purpose of improving communication and software quality. Note that working with configurations in a solution system can create challenges for team development. Organizations must orchestrate changes from multiple developers to avoid merge conflicts as much as possible, because source control systems have limitations on how merges occur. We recommend that you avoid situations where multiple people make changes to complex components—such as forms, flows, and canvas apps—at the same time.

More information: [Scenario 5: Supporting team development](#)

Continuous integration and deployment

You can use any source control system and build a pipeline to start with for continuous integration and continuous deployment (CI/CD). However, this guide focuses on GitHub and Azure DevOps. GitHub is a development platform used by millions of developers. Azure DevOps provides developer services to support teams to plan work, collaborate on code development, and build and deploy applications.

To get started, you need the following:

- A GitHub account, where you can create a repository. If you don't have one, you can [create one for free](#) .
- An Azure DevOps organization. If you don't have one, you can [create one for free](#) .

More information: [Create your first pipeline](#)

Licensing

To create or edit apps and flows by using Power Apps and Power Automate, respectively, users are required to have a per-user license for Power Apps or Power Automate or an appropriate Dynamics 365 application license. For more information, go to [Licensing overview for Microsoft Power Platform](#). We also recommend contacting your Microsoft account representative to discuss your licensing needs.

ALM considerations

When you consider ALM as an integral part of building apps on Microsoft Power Platform, it can drastically improve speed, reliability, and user experience of the app. It also ensures that multiple developers, both traditional developers writing code and citizen developers, can jointly contribute to the application being built.

See the following articles that discuss several items to consider at the outset of any application development:

- [ALM environment strategy](#)
 - [Solution concepts](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Environment strategy for ALM

Article • 05/23/2024

To follow application lifecycle management (ALM) principles, you'll need separate environments for app development and production. Although you can perform basic ALM with only separate development and production environments, we recommend that you also maintain at least one test environment that's separate from your development and production environments. When you have a separate test environment, you can perform end-to-end validation that includes solution deployment and application testing. Some organizations might also need more environments for user acceptance testing (UAT), systems integration testing (SIT), and training.

Separate development environments can be helpful to help isolate changes from one work effort being checked in before it's completed. Separate development environments can also be helpful to reduce situations when one person negatively affects another while making changes.

Every organization is unique, so carefully consider what your organization's environment needs are.

Development environments

You should answer questions such as:

- How many development environments do I need?
 - More information: [Environments overview](#)
- How can I automatically provision environments from source code?
 - More information: [Microsoft Power Platform Build Tools for Azure DevOps](#)
- What are the dependencies on my environments?
 - More information: [Multiple solution layering and dependencies](#)

Other environments

You should also answer the question, "Which types of non-development environments do I need?"

For example, in addition to your production environment, you might need separate test, UAT, SIT, and pre-production environments. Notice that, at a minimum, any healthy ALM practice should include using a test environment prior to deploying anything to the

production environment. This ensures that you have a place to test your app, but also ensures that the deployment itself can be tested.

More information: [Establishing an environment strategy for Microsoft Power Platform](#)

Multi-geographical considerations

Power Platform environments follow a specific service update schedule as environments are updated across the world. There are six stations in total that are primarily defined by geographical location. Service updates are applied in sequence for each station. So, station 2 service updates are applied before station 3. Therefore, it's common for environments that are in different stations to have different versions at a certain point in time. For more information about the environment service update schedule, go to [Released versions of Microsoft Dataverse](#)

Solution import and environment version

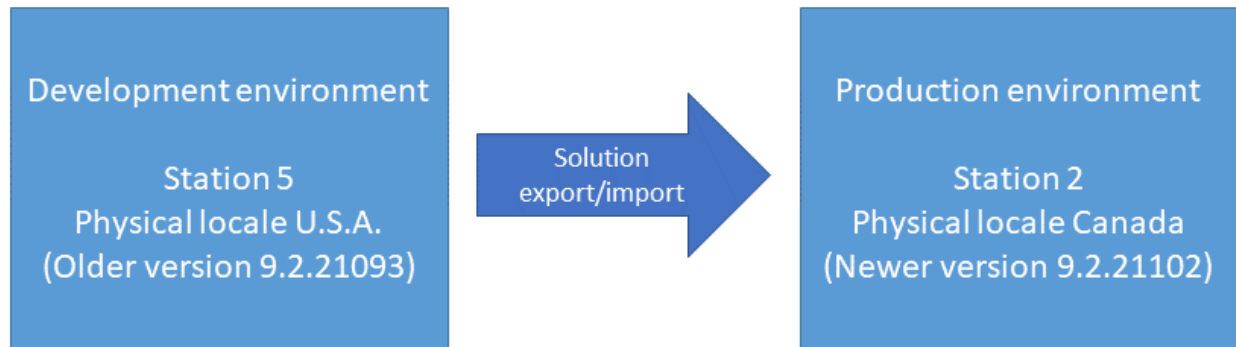
When you have multiple environments in different regions, it's important to understand the following when you import a solution:

- You *can* import a solution into an environment that is a newer version than the environment where the solution was exported.
- You *can't* reliably import a solution into an environment that's an older version than the environment where the solution was exported. This is because there might be missing components or required functionality in the older environment.

Example of successfully aligning environments with service update stations

Imagine that you have production environments in Canada and the United States. In that case, your development environments should be in North America (station 5) and not in Canada (station 2). Then, your development environments will always be the same or an earlier version than your production environments, which will curtail solution

import version conflicts.



See also

[Solution concepts](#)

Feedback

Was this page helpful?

[Provide product feedback](#) [↗](#)

Solution concepts

Article • 01/29/2025

Solutions are the mechanism for implementing application lifecycle management (ALM) in Power Apps and Power Automate. This article describes the following key solution concepts:

- Two types of solutions (managed or unmanaged)
- Solution components
- Lifecycle of a solution
- Solution publisher
- Solution and solution component dependencies

Managed and unmanaged solutions

A solution is either *managed* or *unmanaged*.

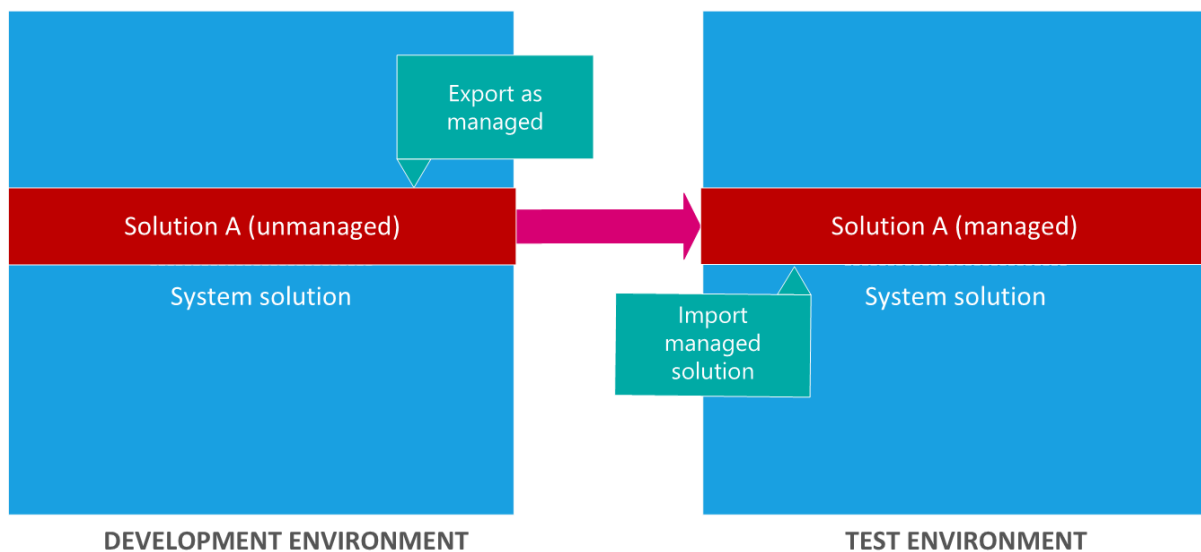
- **Unmanaged solutions are developed.** Unmanaged solutions are used in development environments while you make changes to your application. Unmanaged solutions can be exported either as unmanaged or managed. Exported unmanaged versions of your solutions should be checked into your source control system. Unmanaged solutions should be considered your source for Microsoft Power Platform assets. When an unmanaged solution is deleted, only the solution container of any customizations included in it's deleted. All the unmanaged customizations remain in effect and belong to the default solution.
- **Managed solutions are deployed.** Managed solutions are deployed to any environment that isn't a development environment for that solution. These environments include test, user acceptance testing (UAT), system **integration testing (SIT)**, and production environments. Managed solutions can be serviced independently from other managed solutions in an environment. As an ALM best practice, managed solutions should be generated by exporting an unmanaged solution as managed and considered a build artifact. Additionally:
 - **You can't edit components directly within a managed solution.** To edit managed components, first add them to an unmanaged solution.
 - When you edit a managed component, you create a dependency between your unmanaged customizations and the managed solution. When a dependency exists, the managed solution can't be uninstalled until you remove the dependency.

- Some managed components can't be edited. To verify whether a component can be edited, view the [Managed properties](#).
- You can't export a managed solution. But you can export an unmanaged solution as managed.
- When a managed solution is deleted (uninstalled), all the customizations and extensions included with it are removed.

i Important

- You can't import a managed solution into the same environment that contains the originating unmanaged solution. To test a managed solution, you need a separate environment to import it into.
- When you delete a managed solution, the following data is lost: data stored in custom tables that are part of the managed solution and data stored in custom columns that are part of the managed solution on other tables that aren't part of the managed solution.

Makers and developers work in development environments using unmanaged solutions, then import them to other downstream environments—such as test—as managed solutions.



i Note

When you customize in the development environment, you're working in the unmanaged layer. Then, when you export the unmanaged solution as a managed

solution to distribute to another environment, the managed solution is imported into the environment in the managed layer. More information: [Solution layers](#)

Solution components

A **component**, also known as **objects**, represents something that you can potentially customize. Anything that can be included in a solution is a component. To view the components included in a solution, open the solution you want. The components are listed in the **Components** list.

+ New ▾ + Add existing ▾ 🗑 Delete ➡ Export 🗨 Publish all customizations ...							≡ All ▾ 🔍 Search
Solutions > Contoso							
Display name ▾	Name	Type ▾	Managed...	Modified	Owner	Status	
Account	... account	Entity	🔒	-	-	-	
Contoso 🗨	... constoso_Contoso	Model-driven a	🔒	3 wk ago	-	-	
Contoso 🗨	... constoso_Contoso	Site map	🔒	3 wk ago	-	-	
Contoso dashboard 🗨	... Contoso dashboard	Dashboard	🔒	-	-	On	
Custom entity	... cr167_customentity	Entity	🔒	-	-	-	

ⓘ Note

- A solution can be up to 95 MB in size.
- You can't edit components directly within a managed solution.

To view a list of component types that can be added to any solution, go to [ComponentType Options](#).

Some components are nested within other components. For example, a **table** contains **forms, views, charts, columns, tables relationships, messages, and business rules**. Each of those components requires a table to exist. Except for choice columns, all other columns can't exist outside of a table. We say that the column is dependent on the table. There are twice as many types of components as shown in the preceding list, but most of them are nested within other components and not visible in the application.

The purpose of having components is to keep track of any limitations on what can be customized using managed properties and all the dependencies so that it can be

exported, imported, and (in managed solutions) deleted without leaving anything behind.

Solution lifecycle

Solutions support the following actions that help support application lifecycle processes:

- **Create.** Author and export unmanaged solutions.
- **Update.** Create updates to a managed solution that are deployed to the **parent managed** solution. You can't delete components with an update.
- **Upgrade.** Import the solution as an upgrade to an existing managed solution, which removes unused components and implements upgrade logic. Upgrades involve rolling up (merging) all patches to the solution into a new version of the solution. Solution upgrades delete components that existed but are no longer included in the upgraded version. You can choose to upgrade immediately or to stage the upgrade so that you can do some additional actions prior to completing the upgrade.
- **Patch.** A patch contains only the changes for a parent managed solution, such as adding or editing components and assets. Use patches when making small updates (similar to a hotfix). When patches are imported, they're **layered** on top of the parent solution. You can't delete components with a patch.

Solution publisher

Every app and other solution components such as tables you create or any customization you make is part of a solution. **Because every solution has a publisher, you should create your own publisher rather than use the default.** You specify the publisher when you create a solution.

ⓘ Note

- By default, if you don't use a custom solution you'll be working in the default system solutions, which are known as the **Common Data Service Default Solution** and the **Default** solutions. More information: [Default Solution and Common Data Service Default Solution](#)
- The preferred solution is a solution you specify that becomes your default solution. More information: [Set the preferred solution](#)

The **publisher** of a solution where a component is created is considered the **owner** of that component. The owner of a component controls what changes other publishers of solutions including that component are allowed to make or restricted from making. It's possible to move the ownership of a component from one solution to another within the same publisher, but not across publishers. Once you introduce a publisher for a component in a managed solution, you can't change the publisher for the component. Because of this restriction, it's best to define a single publisher so you can change the layering model across solutions later.

The solution publisher specifies who developed the app. For this reason, you should create a solution publisher name that's meaningful.

Solution publisher prefix

A **solution publisher** includes a prefix. The **publisher prefix** is a mechanism to help avoid naming collisions. This allows for solutions from different publishers to be installed in the same environment with few conflicts. For example, the Contoso solution displayed here includes a solution publisher prefix of *contoso*.

Solutions > Contoso solution					
Display name ▾		Name	Type ▾	Managed...	Modified
Custom entity	...	contoso_customentity	Entity	🔒	-
Make a choice	...	contoso_makeachoice	Option set	🔒	-
My app 📄	...	contoso_Myapp	Model-driven a	🔒	14 min ago
My app 📄	...	contoso_Myapp	Site map	🔒	14 min ago
html page 📄	...	contoso_webresource	Web Resource	🔒	10 min ago

⚠ Note

When you change a solution publisher prefix, you should do it before you create any new apps or metadata items because you can't change the names of metadata items after they're created.

More information:

- [Create a solution publisher prefix](#)
- [Change a solution publisher prefix](#)

Solution dependencies

Because of the way that managed solutions are layered, some managed solutions can be dependent on solution components in other managed solutions. Some solution publishers take advantage of this to build solutions that are modular. You might need to install a "base" managed solution first and then install a second managed solution that further customizes the components in the base managed solution. The second managed solution depends on solution components that are part of the first solution.

The system tracks these dependencies between solutions. If you try to install a solution that requires a base solution that isn't installed, you won't be able to install the solution. You get a message saying that the solution requires another solution to be installed first. Similarly, because of the dependencies, you can't uninstall the base solution while a solution that depends on it's still installed. You have to uninstall the dependent solution before you can uninstall the base solution. More information: [Removing dependencies](#)

Solution component dependencies

A solution component represents something that you can potentially customize. Anything that can be included in a solution is a solution component and some components are dependent on other components. For example, the website column and account summary report are both dependent on the account table. More information: [Dependency tracking for solution components](#)

See also

[Solution layers](#)

[Create and manage environments in the Power Platform admin center](#)

Feedback

Was this page helpful?

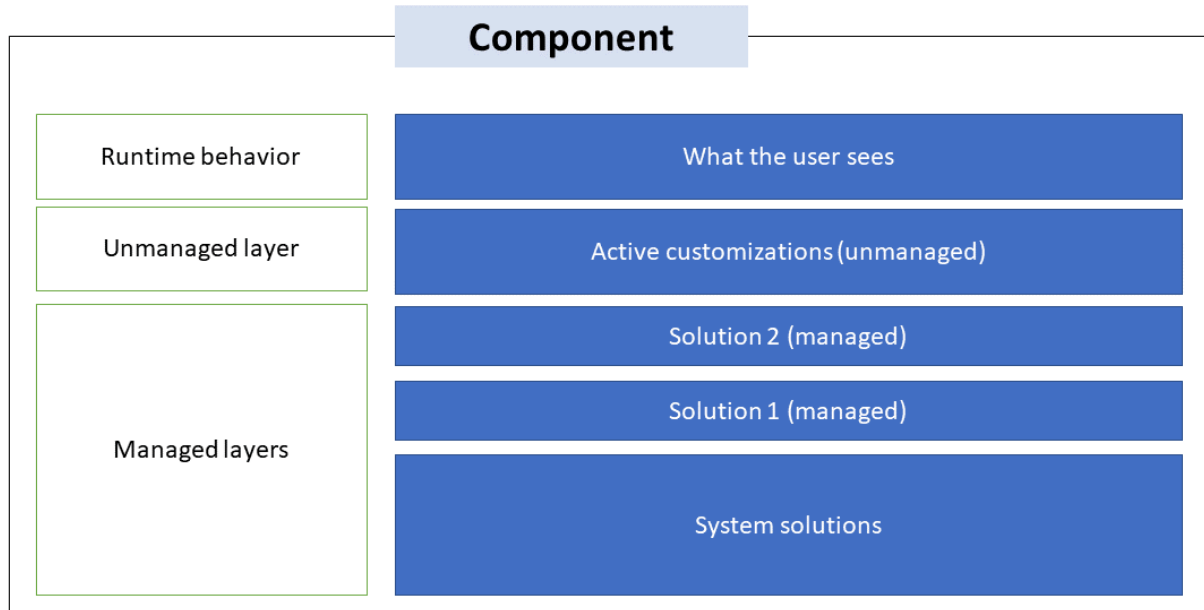
[Provide product feedback](#) 

Solution layers

Article • 01/30/2025

Solution layering is implemented at a component level. Managed and unmanaged solutions exist at different layers within a Microsoft Dataverse environment. In Dataverse, there are two distinct layers:

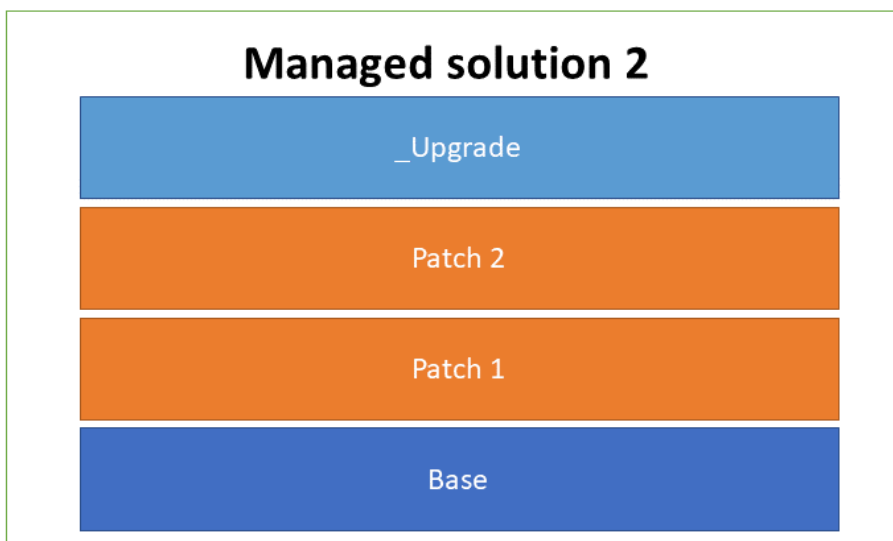
- **Unmanaged layer** All imported unmanaged solutions and ad-hoc customizations exist at this layer. All unmanaged solutions share a single unmanaged layer.
- **Managed layers** All imported, managed solutions, and the system solution exist at this level. When multiple managed solutions are installed, the last one installed is above the managed solution installed previously. This means that the second solution installed can customize the one installed before it. When two managed solutions have conflicting definitions, the runtime behavior is either "Last one wins" or a merge logic is implemented. If you uninstall a managed solution, the managed solution below it takes effect. If you uninstall all managed solutions, the default behavior defined within the system solution is applied. At the base of the managed layers level is the system layer. The system layer contains the tables and components that are required for the platform to function.



Layering within a managed solution

For each managed component, there are layers within a solution, which—depending on whether one or more patches or a pending upgrade to the solution has been imported—can include the following layers:

- **Base** Located at the bottom of the solution layer "stack" is the base layer. This layer includes the solution publisher, which identifies the owner of the component and the managed properties associated with it.
- **Top** The top layer is considered the current layer and defines the runtime behavior of the component. The top layer can be an upgrade or a patch, or if no patches or upgrades have been applied to the solution, the base solution determines the component runtime behavior.
- Layers added from updates:
 - **Patches** If the component has one or more solution patches imported, they're stacked on top of the base layer, with the most recent patch residing above the previous patch.
 - **Pending upgrade** If a staged upgrade (named `_Upgrade`) is imported, it resides on top of the base and patch (if any) layers.



ⓘ Important

Using patches isn't recommended. More information: [Create solution patches](#)

The following image shows an example of solution layers for a custom column that displays the base solution, a patch, and a pending upgrade.

Solution Layers		
Name: contoso_custom		
Type: Attribute		
SOLUTIONS		
Solution	Publisher	Order
Contoso_Upgrade	Contoso Inc	3
Contoso_Patch_801c1acf	Contoso Inc	2
Contoso	Contoso Inc	1

For information about how to view layers for a component within a solution, go to [Solution layers](#).

Merge behavior

Makers who work with solutions should understand the merge behavior when a solution is updated or when multiple solutions are installed that affect the same component. Notice that only model-driven app, form, and site map component types are merged. All other components use "top level wins" behavior.

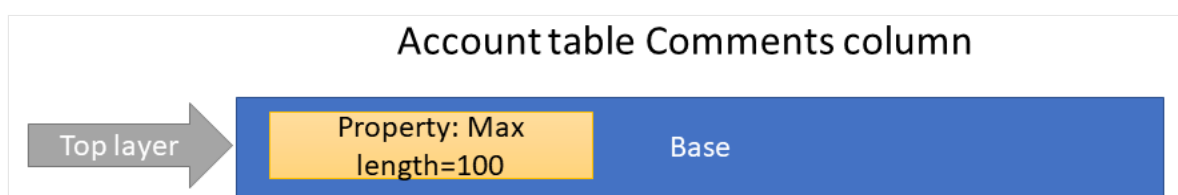
"Top wins" behavior

Except for the model-driven app, form, and site map components, other solution components use a "top wins" behavior where the layer that resides at the top determines how the component works at app runtime. A top layer can be introduced by a staged (pending) upgrade.

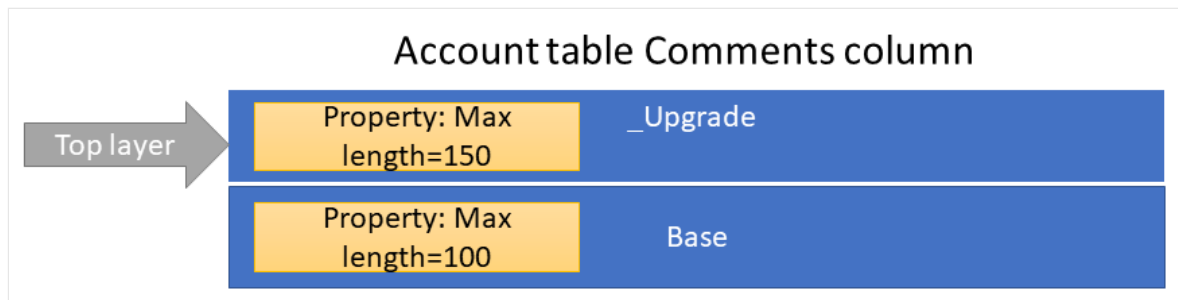
Top layer introduced by a pending upgrade

Here's an example of a top wins component behavior introduced by a stage for upgrade update to a solution. More information: [Apply the upgrade or update in the target environment](#)

1. The current top (base) layer has the **Max length** property of the **Comments** text column for the account table using the default setting of 100.



2. A solution upgrade is imported using the stage for upgrade option, which creates a new top layer. The pending upgrade includes the **Comments** text column for the account table with the **Max length** property value changed to 150.



In this situation, the **Comments** column for account records allows up to a maximum of 150 characters during app run time.

Solution update and upgrade merge behavior

As described in the previous section, patches and a staged upgrade are stacked on top of the base solution. These can be merged by selecting **Apply upgrade** from the **Solutions** area in Power Apps, which flattens the layers and creates a new base solution.

Multiple solutions merge behavior

When you prepare your managed solution for distribution, remember that an environment might have multiple solutions installed or that other solutions might be installed in the future. Construct a solution that follows best practices so that your solution won't interfere with other solutions. More information: [Use segmented solutions](#)

The processes that Dataverse uses to merge customizations emphasize maintaining the functionality of the solution. Although every effort is made to preserve the presentation, some incompatibilities between customizations might require that the computed resolution change some presentation details in favor of maintaining the customization functionality.

See also

[Understand how managed solutions are merged](#)

Feedback

Was this page helpful?



[Provide product feedback](#)

Understand how managed solutions are merged

Article • 02/05/2025

When you prepare your managed solution to be installed, remember that an environment might already have multiple solutions installed or that other solutions might be installed in the future. Construct a solution that follows best practices so that your solution doesn't interfere with other solutions.

The processes that Microsoft Dataverse uses to merge customizations emphasize maintaining the functionality of the solution. Although every effort is made to preserve the presentation, some incompatibilities between customizations might require that the computed resolution change some presentation details in favor of maintaining the customization functionality.

Merge form customizations

The only form customizations that have to be merged are those that are performed on any table forms that are already in the environment. Typically, this means that form customizations only have to be merged when your solution customizes the forms that were included for tables created when Dataverse was installed. One way to avoid form merging is to provide new forms for any Dataverse tables. Forms for custom tables won't require merging unless you're creating a solution that updates or modifies an existing managed solution that created the custom tables and their forms.

When a solution is packaged as a managed solution, the form definitions stored in `FormXML` are compared to the original `FormXML` and only the differences are included in the managed solution. When the managed solution is installed in a new environment, the form customization differences are then merged with the `FormXML` for the existing form to create a new form definition. This new form definition is what the user sees and what a system customizer can modify. When the managed solution is uninstalled, only those form elements found in the managed solution are removed.

Form merge occurs on a section-by-section basis. When you add new elements to an existing tab or section, your changes can affect or conceal the elements from the managed layers, including when the managed element is updated. This behavior occurs because the managed layers are underneath the unmanaged layer you're introducing with your customization. If you don't want to affect or conceal managed elements on the form, we recommend that you include your new elements within new container

elements, such as a section or tab. This isolates your elements and reduces the possibility of affecting or concealing the elements from the managed layers. More information: [Solution layers](#)

Managed solutions that contain forms that use new security roles depend on those roles. You should include these security roles with your managed solution.

When you import a solution that includes table forms, the **Overwrite Customizations** option, even if selected, doesn't apply. The form being imported merges with any existing solution layers for the form.

! Note

When a managed solution table contains multiple forms and the environment table form also contains multiple forms, the new forms aren't appended to the bottom of the list of available forms—they're interleaved with the original table forms.

Identifying and resolving form merge conflicts

After you import a solution that includes a form, you might notice that the imported form displays a tab named **Conflicts Tab**. This is an autogenerated tab, which is created when certain form components are unable to merge. To avoid any data loss, the form components that aren't able to merge are placed under the **Conflicts** tab. Merge conflicts usually happen when the source and target customizations are out of sync, which leads to conflicting form customizations.

The screenshot shows a web form titled "New Project Team Member" with a subtitle "Project Team Member". At the top, there are four tabs: "General", "Administration", "Conflicts Tab", and "Related". The "Conflicts Tab" is highlighted with a red rectangle. The form is divided into two main sections. The left section contains fields for "Position Name", "Project", "Bookable Resource", "Role" (with a dropdown menu showing "Experience Designer"), "Role Description", and "Resourcing Unit" (with a dropdown menu showing "New Zealand"). The right section contains fields for "From" (1/01/2019), "To" (20/05/2019), "Billing Type" (Chargeable), and "Project Approver" (No).

Avoid these situations that can cause form merge conflicts:

- You import two different solutions that add a component, such as a form tab, that uses the same ordinal value.
- You customize a component of the form, such as a section, in the source environment but also make the same or similar customization to the component in

the target environment. Then, you export the customization from the source environment and import it into the target environment.

When the **Conflicts** tab appears on an imported form, you can move the component displayed somewhere on the form. Once all the components are moved from the **Conflicts** tab, you can delete or hide the **Conflicts** tab.

Merge navigation (SiteMap) customizations

When a solution is packaged as managed, the SiteMap XML is compared to the original SiteMap XML and any other customizations made to the site map. Only the differences are included in the managed solution. These differences include items that are changed, moved, added, or removed. When the managed solution is installed in a new environment, the site map changes are merged with the SiteMap XML found for the environment where the managed solution is being installed. A new site map definition is what people observe.

At this point, a customizer can export the site map to an unmanaged solution and that site map definition includes all the elements of the active site map. A customizer can then modify the site map and reimport it as an unmanaged customization. Later, if the managed solution is uninstalled, the SiteMap XML that was imported with the managed solution will be referenced to remove the changes introduced with that managed solution. A new active site map is then calculated.

Whenever a new visible element is added to the site map, it appears at the bottom of whatever container it belongs. For example, a new area appears at the bottom of the navigation area. To position the elements that have been added, you must export the site map, edit it to set the precise position of the elements, and then import it again as an unmanaged solution.

ⓘ Note

Only one site map customization can be applied between publishing. Any unpublished site map customizations are lost when a new site map definition is imported.

Merge choice columns options

Each new choice column option is initialized with an integer value assigned that includes an option value prefix. The option value prefix is a set of five digits prepended to the

option value. An option value prefix is generated based on the solution publisher's customization prefix, but can be set to any value. The option value prefix helps differentiate new option set options created in the context of a specific solution publisher and reduces the opportunity for collisions of option values. Using the option value prefix is recommended but not required.

A managed solution usually updates or adds options for choice columns that are already in the environment, for example, the Category or Industry choice columns for an account. When a managed solution modifies the options available in a choice column, all the options defined in the managed solution are available in the environment. When the managed solution is uninstalled, the options in the choice column are returned to their original state.

Merge security role privileges

When a security role is imported from a managed solution into an environment, all the manually added privileges of that security role are removed. However, the modified privileges of the security role where the privilege level was changed, for example from basic to global scope or vice versa, remain.

Tip

Use the same custom solution to manage updating security roles. If you use a new custom solution to update a security role that was previously updated in another solution, some of the privilege updates won't be applied.

Don't use a custom solution to modify [predefined security roles](#). These updates are removed when the predefined roles are updated by the system. Create a copy of the predefined role and manage the copied role in your custom solution.

See also

[Use a solution to customize](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Use a solution to customize

Article • 02/07/2025

We recommend that you create a solution to manage your customizations. By using a custom solution, you can easily find just the solution components you customize, consistently applying your solution publisher prefix, and exporting your solution with the right prefix for distribution to other environments.

If you don't use a custom solution, you're working in one of these default solutions in the unmanaged layer:

- **Common Data Service Default Solution.** This solution is available for all makers to use by default for their customizations in an environment. The Common Data Service Default Solution is useful when you want to evaluate or learn Power Platform. However, if you're planning on deploying your apps or flows to other environments, we recommend that makers work in their own unmanaged solutions. Additionally, you can set your preferred solution so that, by default, all your solution components are included in the preferred solution that you specify that's used when you create or add solution components outside a solution context.
- **Default Solution.** This is a special solution that contains all components in the system. The default solution is useful for discovering all the components and configurations in your system.

More information:

- [Common Data Service Default Solution](#)
- [Set the preferred solution](#)

Why you shouldn't use the default solutions to manage customizations

There are a few reasons why you shouldn't create apps/flows and make customizations in either of the default solutions:

- When you use either default solution to create components, you'll also use the default publisher assigned to the solution. This often results in the wrong publisher prefix being applied to some components. More information: [Solution publisher](#)
- The default solution can't be exported; therefore, you can't distribute the default solution to another environment.

- If you aren't consistently using the same solution while you're configuring the environment, you might accidentally leave components behind that are necessary to deploy your application to another environment.

Common Data Service Default Solution

The default solution in the Power Platform is the Common Data Service Default Solution, which is associated with the Microsoft Dataverse Default Publisher. The default publisher prefix is randomly assigned for this publisher, for example it might be `cr8a3`. This means that the name of every new item of metadata created in the default solution has this prepended to the names used to uniquely identify the items. If you create a new table named *Animal*, the unique name used by Dataverse is `cr8a3_animal`. The same is true for any new columns (attributes), relationships, or choice columns. If you're customizing using this special solution, consider changing the publisher prefix.

See also

[Managed properties](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Managed properties

Article • 02/05/2025

You can use managed properties to control which of your managed solution components can be customized. If you're creating solutions for other organizations, you should allow them to customize solution components where it makes sense for their unique requirements. However, you have to be able to predictably support and maintain your solution, so you should never allow any customization of critical components that provide the core functionality of your solution.

Consider using managed properties to lock down your components unless you want them customizable in the destination environment. For example, imagine a scenario where your users might want to change many of the labels to fit their own business requirements.

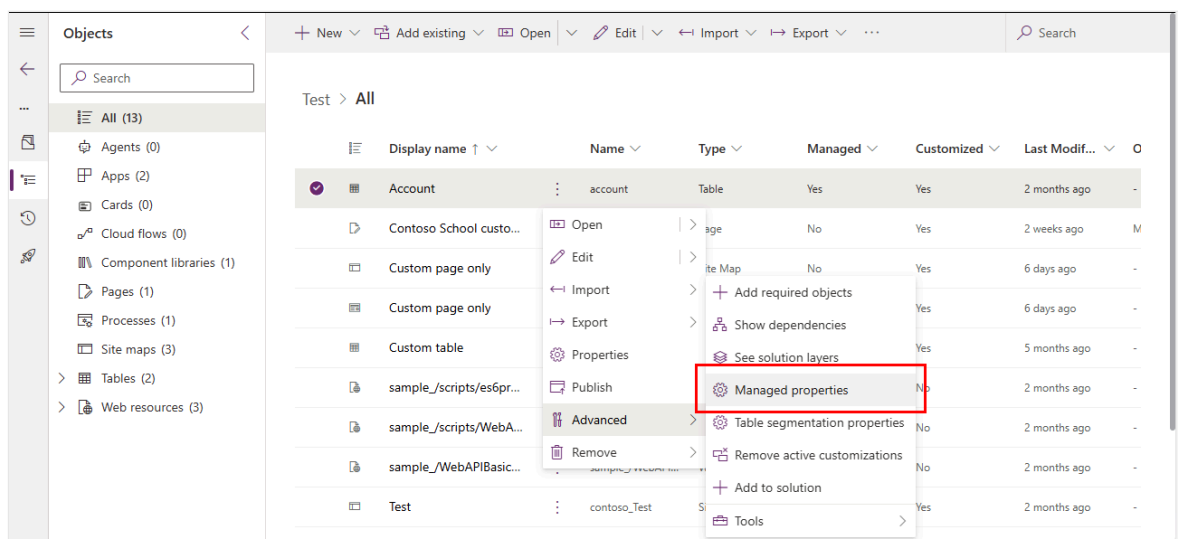
Managed properties are intended to protect your solution from modifications that might cause it to break. Managed properties don't provide digital rights management (DRM), or capabilities to license your solution or control who might import it.

You apply managed properties when the solution is unmanaged in the unmanaged layer of your development environment. The managed properties take effect after you package the managed solution and install it in a different environment. After the managed solution is imported, the managed properties can't be updated except by an update of the solution by the original publisher.

Most solution components have a **Managed properties** menu item available in the list of solution components. When you import the managed solution that contains the components, you can view—but not change—the managed properties.

View and edit table managed properties

1. Sign in to [Power Apps](#) or [Power Automate](#) and select **Solutions** from the left pane.
2. Open the solution that you want.
3. From the list of components in the solution, select the three vertical dots next to the table where you want to view the managed properties, and then select **Advanced > Managed properties**.



The managed properties page is displayed.

Managed Properties

The following properties will take effect only after the component is exported and imported as part of a managed solution.

- ☐ Allow customizations
- ☒ Display name can be modified
- ☒ Can change additional properties
- ☒ New forms can be created
- ☒ New charts can be created
- ☒ New views can be created
- ☒ Can change hierarchical relationship
- ☒ Can change tracking be enabled
- ☒ Can enable sync to external search index

Done
Cancel

Tables have more managed properties than any other type of solution component. If the table is customizable, you can set the following options:

[Expand table](#)

Option	Description
Allow customizations	Controls all the other options. If this option is <code>False</code> , none of the other settings apply. When it's <code>True</code> , you can specify the other customization options. When <code>False</code> , it's equivalent to setting all other options to false.
Display name can be modified	Whether the table display name can be modified.
Can Change Additional Properties	Applies to anything not covered by other options.
New forms can be created	Whether new forms can be created for the table.
New charts can be created	Whether new charts can be created for the table.
New views can be created	Whether new views can be created for the table.
Can Change Hierarchical Relationship	Whether hierarchical relationships settings can be changed. More information: Define and query hierarchically related data
Can Change Tracking Be Enabled	Whether the table Change Tracking property can be changed.
Can Enable sync to external search index	Whether the table can be configured to enable Dataverse search. More information: Configure Dataverse search to improve search results and performance

View and edit column managed properties

Next to a custom column in a solution select the three vertical dots, and then select **Advanced > Managed properties**.

The **Managed Properties** pane opens.

Managed Properties

The following properties will take effect only after the component is exported and imported as part of a managed solution.

☒ Allow customizations

☒ Display name can be modified

☒ Can change additional properties

☒ New forms can be created

☒ New charts can be created

☒ New views can be created

☒ Can change hierarchical relationship

☒ Can change tracking be enabled

☒ Can enable sync to external search index

DoneCancel

The **Allow customizations** option controls all the other options. If this option is disabled, none of the other settings apply. When it's enabled, you can specify the other customization options.

If the column is customizable, you can enable the following options.

- **Display name can be modified**
- **Can change additional properties:** This property controls any other customizations that don't have a specific managed property.
- **New forms can be created**
- **New charts can be created**
- **New views can be created**
- **Can change hierarchical relationship**
- **Can change tracking be enabled**
- **Can enable sync to external search index**

Disabling all the individual options is equivalent to disabling **Allow customizations**.

Apply your choices and select **Done** to close the pane.

ⓘ Note

If this column is a **Date and Time** column, an additional **Can change date and time behavior** property is available. More information: [Behavior and format of the Date and Time field](#)

Go to [Update or delete a column](#) for information about how to edit columns.

View and edit other component managed properties

You can view and edit managed properties for many other solution components, such as a web resource, process, chart, or dashboard. Next to the component in a solution select the three vertical dots, and then select **Advanced > Managed properties**.

View and edit relationship managed properties

While viewing table relationships in a solution, select a relationship from an unmanaged solution, select the three vertical dots, and then select **Advanced > Managed Properties**.

With relationships, the only managed property is **Allow customizations**. This single setting controls all changes that can be made to the table relationship.

See also

[Use segmented solutions](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Create and update solutions

Article • 02/05/2025

To locate and work with just the components you've customized, create a solution and do all your customization there. Then, always remember to work in the context of the custom solution as you add, edit, and create components. This makes it easy to export your solution for import to another environment or as a backup. More information

[Create a solution](#)

Update a solution

Make changes to your unmanaged solution, such as adding or removing components. Then, when you import a managed solution that was previously imported, the import logic detects the solution as an update and displays the following screen of options.

Import a solution



Environment

Environment

 This solution package contains an update for a solution that is already installed.

Details

Name

Mysolution

Type

Managed

Publisher

Contoso Inc

Current version installed

1.0.0.3

Version contained in the update

1.0.0.4

Patch

No

Advanced settings 

Solution action [Learn more](#)

☒ Upgrade

Upgrades your solution to the latest version. Any objects not present in the newest solution will be deleted.

☐ Stage for upgrade

Upgrades your solution to the higher version, but defers the deletion of the previous version and any related patches until you apply an upgrade later.

☐ Update

Replaces your older solution with this one.

☒ Enable Plugin steps and flows included in the solution

Import

Cancel

More information: [Apply an update or upgrade for a solution](#)

Create solution patches

You can create a patch for a parent solution and export it as a minor update to the base solution. When you clone a solution, the system rolls up all related patches into the base solution and creates a new version.

Warning

Using clone a patch and clone solution to update a solution isn't recommended because it limits team development and increases complexity when storing your solution in a source control system. For information about how to update a solution, go to [Update a solution](#).

Creating updates using clone solution and clone to patch

When you're working with patches and cloned solutions, keep the following information in mind:

- A patch represents an incremental minor update to the parent solution. A patch can add or update components and assets in the parent solution when installed on the target system, but it can't delete any components or assets from the parent solution.
- A patch can have only one parent solution, but a parent solution can have one or more patches.
- A patch is created from an unmanaged solution. You can't create a patch from a managed solution.
- When you import a patch into a target system, you should export it as a managed patch. Don't use unmanaged patches in production environments.
- The parent solution must be present in the target system to install a patch.
- You can delete or update a patch.
- If you delete a parent solution, all child patches are also deleted. The system gives you a warning message that you can't undo the delete operation. The deletion is performed in a single transaction. If one of the patches or the parent solution fails to delete, the entire transaction is rolled back.
- After you have created the first patch for a parent solution, the solution becomes locked, and you can't make any changes in this solution or export it. However, if you delete all of its child patches, the parent solution becomes unlocked.
- When you clone a base solution, all child patches are rolled up into the base solution and it becomes a new version. You can add, edit, or delete components and assets in the cloned solution.

- A cloned solution represents a replacement of the base solution when it's installed on the target system as a managed solution. Typically, you use a cloned solution to ship a major update to the preceding solution.

When you clone a solution, the version number you specify includes the major and minor positions.

Clone to solution ✕

Create a new version for the selected unmanaged solution. Any patches that have been created will be rolled up into the newly created solution.

Base solution name:

Myapp

Display name:

Contoso solution

Version number:

1

1

0.0

Save

Close

When you clone a patch, the version number you specify includes the build and revision positions.

Clone to patch

Create a patch for the selected unmanaged solution. A patch contains changes to the existing solution.

Base solution name:

Myapp

Display name:

Contoso solution

Version number:

1.0.1.0

Save Close

For more information about version numbers, go to [Clone solution and clone patch version numbers](#) in this article.

Create a solution patch

A patch contains changes to the parent solution, such as adding or editing components and assets. You don't have to include the parent's components unless you plan to edit them.

Create a patch for an unmanaged solution

1. Go to the Power Apps (make.powerapps.com), and then select **Solutions**.
2. In the solutions list, select an unmanaged solution where you want to create a patch. On the command bar, select ... > **Clone** > **Clone a Patch**. The right pane that opens contains the base solution's name and the patch version number. Select **Save**.

Clone to patch ✕

Create a patch for the selected unmanaged solution. A patch contains changes to the existing solution.

Base solution name:

Testsolution3

Display name:

Test solution 3

Version number:

1.1.

1

0

Save

Close

3. In the solutions list, find and open the newly created patch. Notice that the unique name of the solution has been appended with *Patchhexnumber*. Just like with the base solution, add the components and assets you want.

Create a patch using solution explorer

The following illustrations provide an example of creating a patch for an existing solution by using the legacy solution explorer. Start by selecting **Clone a Patch** (in the compressed view, the **Clone a Patch** icon is depicted as two small squares, as shown here).

All Solutions ▾				
Name	Display Name	Version	Installed On ↑	Package Typ...
✓ SegmentedSolutionExample	Segmented Solution Exam...	1.0.1	10/19/2015	Unmanaged

In the **Clone To Patch** dialog box you see that the version number for the patch is based on the parent solution version number, but the build number is incremented by one. Each subsequent patch has a higher build or revision number than the preceding patch.

Clone To Patch

Create a patch for the selected unmanaged solution. A patch contains changes to the existing solution.

Base Solution Name

SegmentedSolutionExample

Display Name

Segmented Solution Example

Version Number

1.0.2.0

Save

Cancel

The following screenshot shows the base solution **SegmentedSolutionExample**, version 1.0.1.0, and the patch **SegmentedSolutionExample_Patch**, version 1.0.2.0.

All Solutions ▾		
Name	Display Name	Version
✓ SegmentedSolutionExample_Patch_c630693b	Segmented Solution Exam...	1.0.2.0
SegmentedSolutionExample	Segmented Solution Exam...	1.0.1

In the patch, we added a new custom table called **Book**, and included all assets of the **Book** table in the patch.

Book - 1 of 1

Select Entity Assets to Include in the solution

☒ Include entity metadata
 ☒ Add All Assets

Forms Views Charts Fields Keys 1:N N:1 N:N Business Rules Hierarchy Settings

System Forms

Active Forms

✓	Name	Form State	Form Type ↑	State	Customizable	Version	
	Information	Active	Main	Unmanaged	True	1.0	A f
	Information	Active	Mobile - Express	Unmanaged	True	1.0	This
	Information	Active	Quick View Form	Unmanaged	True	1.0	

<

1 - 3 of 3 (0 selected)

>

Finish

Cancel

Clone a solution

When you clone an unmanaged solution, the original solution and all patches related to the solution are rolled up into a newly created version of the original solution. After cloning, the new solution version contains the original tables plus any components or tables that are added in a patch.



Important

Cloning a solution merges the original solution and associated patches into a new base solution and removes the original solution and patches.

1. Go to the Power Apps (make.powerapps.com), and then select **Solutions**.
2. In the solutions list, select an unmanaged solution to create a clone. On the command bar, select ... > **Clone** > **Clone solution**. The right pane displays the base solution's name and the new version number. Select **Save**.

Clone solution and clone patch version numbers

A patch must have a higher build or revision number than the parent solution. It can't have a higher major or minor version. For example, for a base solution with version 3.1.5.7, a patch could be a version 3.1.5.8 or version 3.1.7.0, but not version 3.2.0.0. A cloned solution must have the version number greater than or equal to the version number of the base solution. For example, for a base solution version 3.1.5.7, a cloned solution could be a version 3.2.0.0, or version 3.1.5.7. When you clone a solution or patch, you set the major and minor version values for a cloned solution, and the build or revision values for a patch.

See also

[Overview of tools and apps used with ALM](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Removing dependencies overview

Article • 05/09/2025

Solution components often depend on other solution components. You can't delete any solution component that has dependencies from another solution component. Dependencies are records created automatically by the solutions framework to prevent required components from being deleted while one or more dependent components still include references to them.

Examples of a dependency is, given a field is required for a form to function, when you try to execute an action that results in the deletion of that field, the form will stop working.

Another example is when you try to delete the site map for a model-driven app. The system indicates that there's a dependency on the app. The app is the dependent component, and the site map is the required component. The system prevents you from deleting the site map until you remove the dependency.

Power Apps

Search

Environment
Matts dev environment

🔔 ⚙️ ?

Objects

Search

All (47)

Agents (0)

Apps (6)

Cards (0)

Choices (1)

Cloud flows (0)

Component libraries (1)

Data Workspace (0)

Pages (1)

Processes (1)

Site maps (7)

Tables (27)

Web resources (3)

+ New Add existing Edit Object checker Publish Advanced

Failed to delete Site Map (Custom page only). Object dependencies exist, please review before deleting.

View Dependencies

Test > All

	Display name ↑		Name	Type	Managed	Customized	Last Modif..
	Account	:	account	Table	Yes	Yes	5 months ago
	Action Card	:	actioncard	Table	Yes	No	2 years ago
	Address	:	customeraddress	Table	Yes	No	1 year ago
	Appointment	:	appointment	Table	Yes	No	2 years ago
	Connection	:	connection	Table	Yes	No	2 years ago
	Contact	:	contact	Table	Yes	No	1 year ago
	Contoso School custo...	:	contoso_contoso...	Page	No	Yes	3 months ago
👇	Custom page only	:	contoso_Custom...	Site Map	No	Yes	3 months ago
	Custom page only	:	contoso_Custom...	Model-Driven App	No	Yes	3 months ago

ⓘ Note

Delete means that the component is completely removed from Microsoft Dataverse.

In this article, we discuss how to handle these dependencies and the strategies you can use to remove dependencies that you no longer need.

Dependencies of unmanaged vs. managed components

First, it's important to understand that dependencies only prevent operations that delete a required component. The actions that can delete a component are different, depending on whether it's unmanaged or managed.

It's good to be familiar with the concept of solution and component layers when describing dependencies. If you aren't familiar, we recommend that you read this article before continuing: [Solution layers](#)

Unmanaged components

These components are represented by a single layer in the active solution. Any **Delete** operation on such a component results in the complete removal of the component.

Managed components

The deletion of managed components depends on multiple factors: the number of solution layers, the relative position of the layer that's being uninstalled, and the component publishers. For example, when a component is deleted, consider the following scenarios and what will be the expected behavior when you uninstall the various layers.

Example scenarios

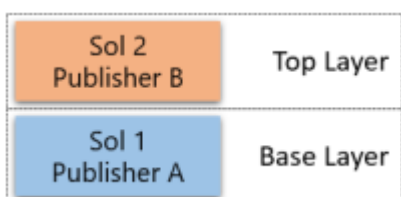
The following example scenarios illustrate what happens to solution layers when solutions are uninstalled.

Scenario 1: Uninstall a single solution layer



Uninstalling Solution 1 causes a component deletion because it's the only layer for the component.

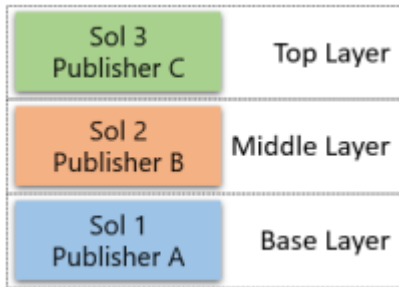
Scenario 2: Uninstall solution layers from different publishers



- Uninstalling Solution 2 doesn't cause a component deletion. Only that layer is removed.

- Uninstalling Solution 1 causes a component deletion, because the action happens in the base layer. In fact, Solution 1 can't be uninstalled in this scenario, because a solution from a different publisher extends the component.

Scenario 3: Uninstall multiple solution layers from different publishers



- Uninstalling Solution 3 doesn't cause a component deletion. Only that layer is removed.
- Uninstalling Solution 2 doesn't cause a component deletion. Only that layer is removed.
- Uninstalling Solution 1 doesn't cause a component deletion, because in this case there's another solution from the same publisher (Publisher A = Publisher C). The platform removes the layer from Solution 1 and replaces it with the layer from Solution 3.

Scenario 4: Uninstall solution layers in an unmanaged customization



- Uninstalling the active (unmanaged) layer doesn't cause a component deletion. Only that layer is removed. Note that you can't uninstall the active solution, but you can remove components by using the **Remove Active Customization** feature.
- Uninstalling Solution 1 causes a component deletion. The action happens in the base layer. Unlike scenario 2, you can uninstall Solution 1. The Active solution isn't considered an extension, and both layers are removed.

Show dependencies page

The **Show dependencies** command lists the dependencies for the selected solution or solution component. It can be invoked by:

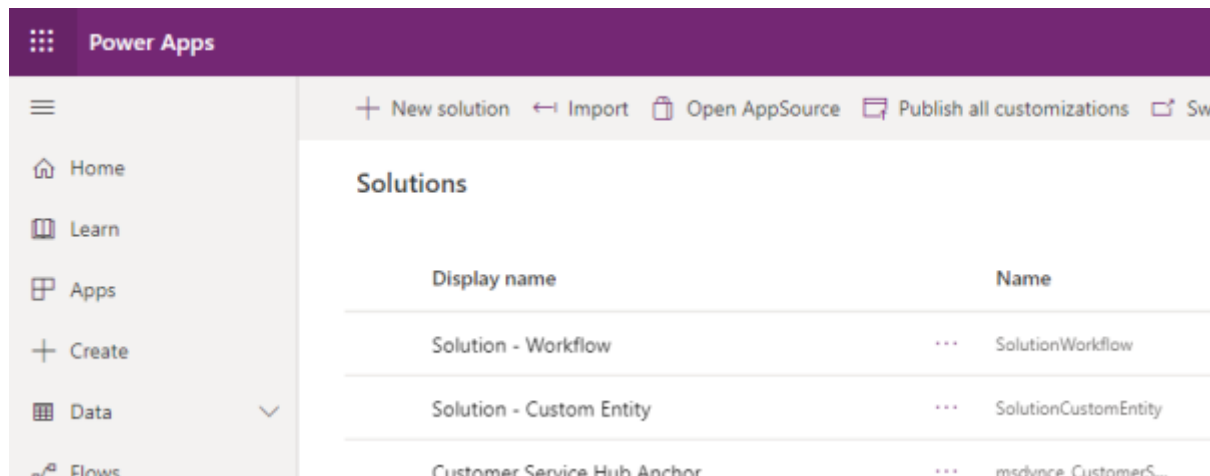
- Selecting **Show dependencies** on the solution page.
- Selecting **Advanced > Show dependencies** within a solution when a solution component is selected.

- Trying to uninstall a solution, which causes the platform to detect that dependencies exist.

From the **Dependencies** page you can open, remove, or delete, the component. More information: [View dependencies for a component](#)

Diagnosing dependencies

Consider this scenario. The environment here has two solutions: **Solution - Workflow** and **Solution - Custom Entity**.



A maker of the environment decides that they no longer require **Solution - Custom Entity**. The maker tried to delete it, and was presented with the following page:

Dependency Details ×

Display Name ↑	Name/Id	Type	Required by ↑	Dependent Type..	Solution Layers
Custom Entity	new_customentity	Entity	MyApp	Site Map	Solution Layers
Custom Entity	new_customentity	Entity	MyApp	Model-driven App	Solution Layers
Custom Entity	new_customentity	Entity	Test Workflow	Process	Solution Layers
Custom Entity (Custom Entity)	new_customentityid	Field	Test Workflow	Process	Solution Layers
Custom Entity (Name)	new_name	Field	Test Workflow	Process	Solution Layers
Custom Entity (Number Field)	new_numberfield	Field	Test Workflow	Process	Solution Layers

The uninstall of the solution is trying to delete a table named **Custom Entity** and three fields—**Custom Entity**, **Name**, and **Number Field**—and all four components have dependencies.

ⓘ Note

Uninstalling the solution might potentially delete more components, but because they don't have dependencies, they won't appear on the list.

The next step is to check the **Solution Layers** link (rightmost column) for each dependency. That helps you decide what to do to remove the dependency.

The following figure shows dependency details between the table (Custom Entity) and process (Test Workflow).

Dependency Details

REQUIRED COMPONENT SOLUTION LAYERS

Name: new_customentity

Type: Entity

SOLUTIONS

Solution	Publisher	Order
SolutionCustomEntity	Default Publisher for i	1

DEPENDENT COMPONENT SOLUTION LAYERS

Name: Test Workflow

Type: Workflow

SOLUTIONS

Solution	Publisher	Order
SolutionWorkflow	Default Publisher for i	1

Based on the data displayed, you can observe that the dependent component belongs to a solution named SolutionWorkflow. To remove this dependency, you can either:

- Update the definition of the workflow in SolutionWorkflow by removing any references to the table or its subcomponents. Then **Update** or **Upgrade** the solution.
- Uninstall the SolutionWorkflow solution.
- Remove the workflow from a new version of the SolutionWorkflow solution, and then perform an **Upgrade**.

Because any one dependent component can prevent the removal of the solution, we recommend that you check all the dependencies and make all the required changes in a single operation.

The following figure shows dependency details between the table (Custom Entity) and a model-driven app (MyApp).

Dependency Details

REQUIRED COMPONENT SOLUTION LAYERS

Name: new_customentity

Type: Entity

SOLUTIONS

Solution	Publisher	Order
SolutionCustomEntity	Default Publisher for i	1

DEPENDENT COMPONENT SOLUTION LAYERS

Name: MyApp

Type: AppModule

SOLUTIONS

Solution	Publisher	Order
Active	Default Publisher for i	1

Based on the data displayed, you can observe that the dependent component belongs to a solution named Active. This indicates that the dependency was created by importing an unmanaged solution, or through an unmanaged customization that was executed through the modern UI or API.

To remove this dependency, you can either:

- Edit the definition of the model-driven app to remove any reference to the entity or its subcomponents. Because model-driven apps support publishing, you must publish your changes.
- Delete the model-driven app.

ⓘ Note

Uninstalling an unmanaged solution isn't an option for removing this dependency, because unmanaged solutions are just a means to group components.

Actions to remove a managed dependency

Managed dependencies are the ones where the dependent component is associated to a managed solution. To resolve this kind of the dependency, you must act on the solution where the component was added. That action can be different depending on what you're trying to do.

If you're trying to uninstall a solution

Follow these steps:

1. In the target environment, inspect the **Solution Layers** link to find what is the topmost solution on the list of the dependent component.
2. In the source environment, prepare a new version of that solution where the solution either doesn't contain the dependent component, or has an updated version of the dependent component that doesn't contain references to the required component. Your goal is to remove any reference to the required components in the new version of the solution.
3. Export the new version of the solution.
4. In the target environment, **Upgrade** that solution.
5. Retry the uninstall.

If you're trying to upgrade a solution

In this case, you must confirm that you wanted to delete the required component (remember that dependencies are enforced only on components that are being deleted).

If you didn't want to delete the component, you can fix the new version of the solution by adding the component back by doing the following:

1. In the target environment, uninstall the staged solution (the solution that ends in **_Upgrade**).
2. In the source environment, add the required components back to the solution.
3. Export the new version.
4. Retry the upgrade.

If the deletion is intentional, you must remove the dependency. Try the steps outlined in the preceding section, [If you're trying to uninstall a solution](#).

Layers and dependencies

The dependent components can be layered, so you might need to change more than one solution to completely remove a dependency. The dependency framework only calculates dependencies between the topmost layers for the required and dependent components. That means you need to work your way from the top to the bottom of the solutions of the dependent component.

Consider the following scenario:

Solutions

Display name		Name	Created ↓
Solution - Workflow - 03	...	SolutionWorkflow03	4/10/2020
Solution - Workflow - 02	...	SolutionWorkflow02	4/10/2020
Solution - Workflow - 01	...	SolutionWorkflow01	4/10/2020
 Solution - Custom Entity	...	SolutionCustomEntity	4/10/2020

You try to uninstall **Solution - Custom Entity**, and the operation is blocked by dependencies.

Dependency Details

×

Display Name ↑	Name/Id	Type	Required by ↑	Dependent Type...	Solution Layer
Custom Entity	new_customentity	Entity	Test Workflow	Process	Solution Layers
Custom Entity (Custom Entity)	new_customentityid	Field	Test Workflow	Process	Solution Layers
Custom Entity (Name)	new_name	Field	Test Workflow	Process	Solution Layers
Custom Entity (Number Field)	new_numberfield	Field	Test Workflow	Process	Solution Layers

You start diagnosing the dependency by selecting **Solution Layers** on the **new_numberfield** attribute. You observe the following screen:

Dependency Details

REQUIRED COMPONENT SOLUTION LAYERS

Name: new_numberfield

Type: Attribute

SOLUTIONS

Solution	Publisher	Order
SolutionCustomEntity	Default Publisher for i	1

DEPENDENT COMPONENT SOLUTION LAYERS

Name: Test Workflow

Type: Workflow

SOLUTIONS

Solution	Publisher	Order
SolutionWorkflow03	Default Publisher for i	3
SolutionWorkflow02	Default Publisher for i	2
SolutionWorkflow01	Default Publisher for i	1

Because dependencies are created only between the topmost layers of each component, the first step is to deal with the dependency between the **new_numberfield** attribute in SolutionCustomEntity and the **Test Workflow** workflow in SolutionWorkflow3.

To remove the dependency, you decide to uninstall SolutionWorkflow3. You do so, but when you try to uninstall the solution once more, you're presented by the same page of dependencies:

Dependency Details

×

Display Name ↑	Name/Id	Type	Required by ↑	Dependent Type..	Solution Layers
Custom Entity	new_customentity	Entity	Test Workflow	Process	Solution Layers
Custom Entity (Custom Entity)	new_customentityid	Field	Test Workflow	Process	Solution Layers
Custom Entity (Name)	new_name	Field	Test Workflow	Process	Solution Layers

However, the **new_numberfield** column is no longer listed, even if it existed in more layers.

Actions to remove an unmanaged dependency

To remove unmanaged dependencies, you need to act directly on the components, not in the solutions they belong to. For example, if you want to remove the dependencies between a column and a form, you must edit it in the form designer and remove the column from the form. The dependency is removed after you select **Save** and **Publish**.

ⓘ Note

- The **Show dependencies** command lets you take action to view, remove the dependency, edit the component, or delete the component. More information: [View dependencies for a component](#)
- The **View Dependencies** button appears when you try to delete a component that has dependencies. Although this screen lets you view the dependency, it doesn't let

you take action.

- When you delete the dependent component, the action deletes *all dependencies*, along with the component.

To view the dependencies of a component, from the **Solutions** area, open the solution you want, select the vertical ellipses next to the component, and then select **Advanced** > **Show dependencies**.

The page of dependencies has two distinct parts:

- Dependent components: A list of components that depend on the selected column. In other words, these components have this column as their required component.
- Required components: A list of components that this column requires in order to work. In other words, these components have this column as their dependent component.

Number Field: Dependencies

x

Dependent components

You cannot delete this component while the following components depend on it.

☐	Display Name ↑	Name/Id	Component T...	Dependent Entity	Managed Solution	Dependency ...	↻
	Form	Information	System Form	Custom Entity		Published	
	Test Workflow	Test Workflow	Process			Published	
1 - 2 of 2 (0 selected)							
⏪ ⏩ Page 1							

Required components

When you import a solution, the required components must already be present on the target system or included within the solution being imported. You cannot delete the following components because this component requires them.

☐	Display Name ↑	Name/Id	Component T...	Parent Entity	Managed Solution	Dependency ...	↻
No records are available in this view.							
0 - 0 of 0 (0 selected)							
⏪ ⏩ Page 1							

Next steps

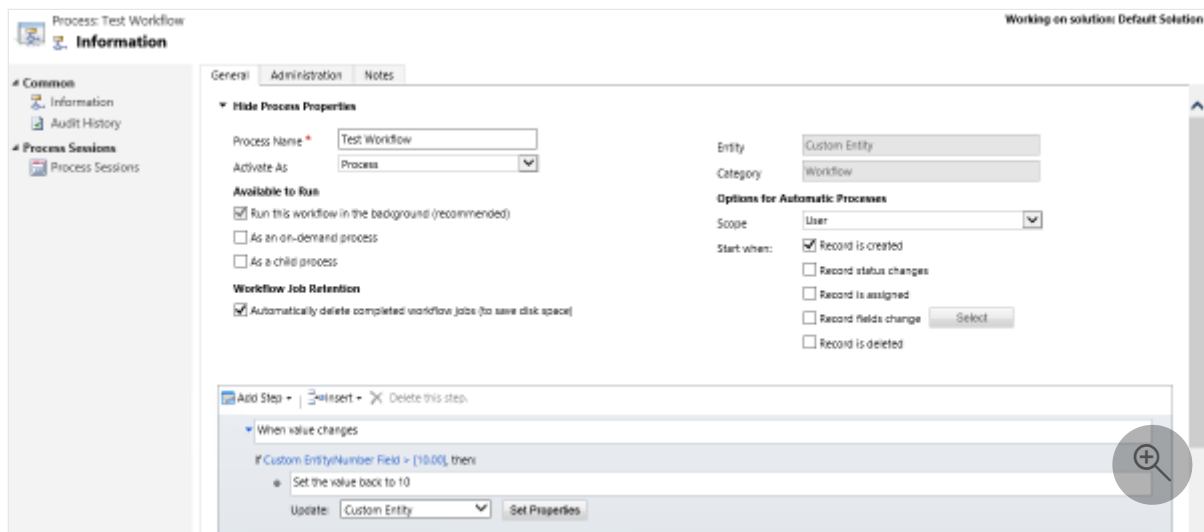
- Remove dependencies between fields and workflows
- Remove dependencies between fields and views
- Remove dependencies between tables and model-driven apps
- Solution layers

Remove dependencies between fields and workflows

Article • 03/15/2025

To remove dependencies between fields (attributes) and classic workflows (processes), locate the workflow in the customizations page.

When open the workflow to view the workflow details, and then find the reference to the component you no longer want the workflow to depend on. In this example, you observe the field **Number Field** that's referenced in a process step.



Delete (or change) the step, and then save the workflow.

Related articles

- [Removing dependencies overview](#)
- [Remove dependencies between fields and views](#)
- [Remove dependencies between tables and model-driven apps](#)

Feedback

Was this page helpful?

☐ Yes

☐ No

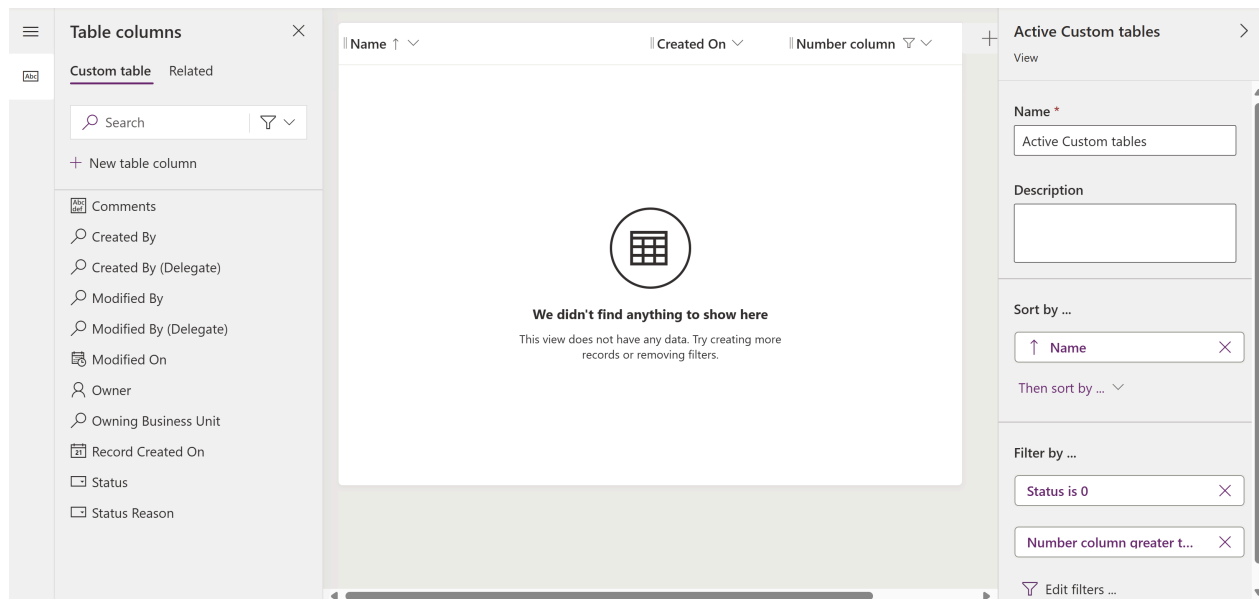
[Provide product feedback](#)

Remove dependencies between columns and views

Article • 03/15/2025

To remove dependencies between columns (formerly fields and also known as attributes) and views (saved queries), locate the view in the table and open it.

In the view editor, find the reference to the component you no longer want the view to depend on. In this example, you observe the column named **Number column** being used as a view column and a filter.



Remove both the filter and column, save, and then publish the view.

Related articles

- [Removing dependencies overview](#)
- [Remove dependencies between fields and workflows](#)
- [Remove dependencies between tables and model-driven apps](#)

Feedback

Was this page helpful?

Yes

No

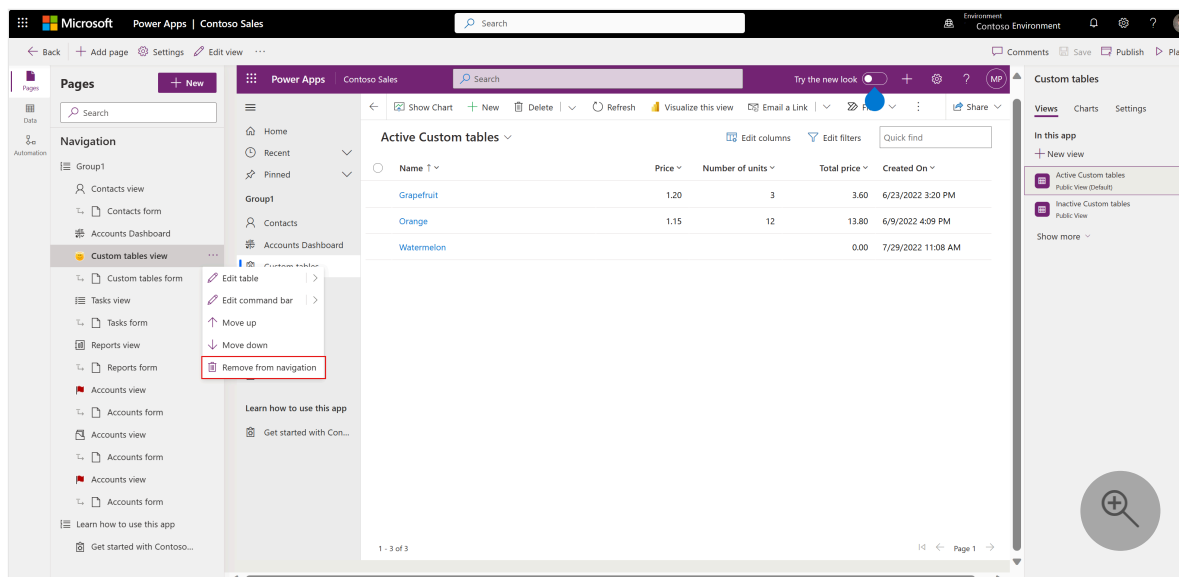
[Provide product feedback](#)

Remove dependencies between tables and model-driven apps

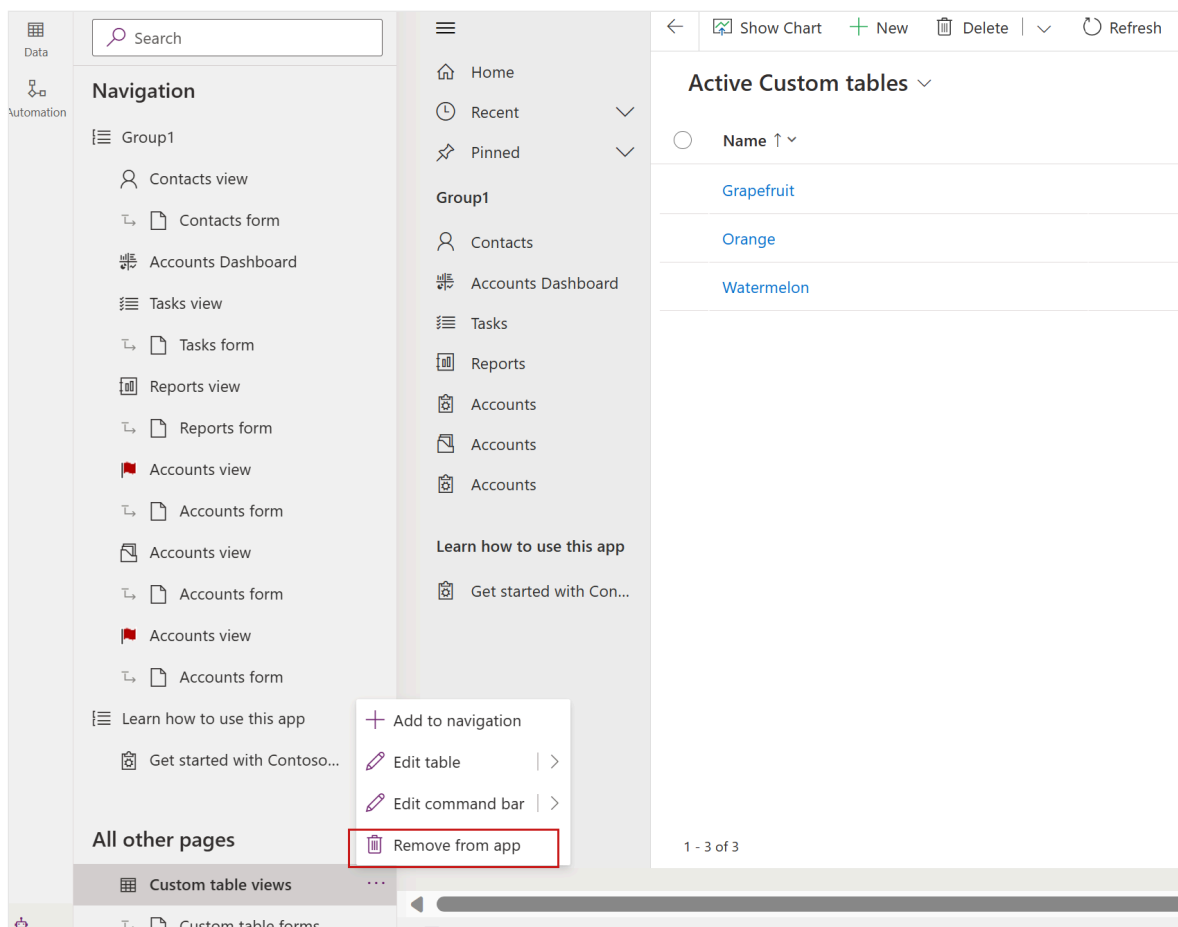
Article • 03/15/2025

To remove dependencies between tables and model-driven apps, open the app in the app designer, remove the component, such as a table, from the app navigation, and then remove the component from the app.

1. Sign into Power Apps (make.powerapps.com), go to in the **Solutions** on the left navigation pane, and then open the solution that has the model-driven app.
2. Open the model-driven app in the app designer, then find the component you no longer want the app to depend on. In this example, you observe the table **Custom table** in the **Pages** left pane. Select ... next to the item, and then select **Remove from navigation**.



3. The component appears under the **All other pages** area near the bottom of the **Pages** left pane. Select ... next to the item, and then select **Remove from app**.



4. Save and publish the app.

! Note

After you remove the component from app designer in your developer environment, the component can be added to an unmanaged solution and then exported as a managed solution to other environments, such as production, to remove the managed dependency.

See also

- [Removing dependencies overview](#)
- [Remove dependencies between fields and workflows](#)
- [Remove dependencies between fields and views](#)

Feedback

Was this page helpful?

Yes

No

Organize your solutions

Article • 02/05/2025

Before you create solutions, take some time to plan ahead. For example, think about how many solutions you want to release and whether the solutions share components.

Also, determine how many Microsoft Dataverse environments you need to develop your line of solutions. You can use a single development environment for most strategies described in this article. However, if you decide to have only one development environment and later realize that you need more, it can be challenging to change the solutions if people have already installed them. When you have multiple environments, although introducing more complexity, can provide better flexibility.

The following sections describe different strategies for managing solutions listed in order from simple to more complex.

Single solution

By creating a solution, you establish a working set of customizations. A single solution makes it easier to find items that you have customized.

This approach is recommended when you only want to create a single managed solution. If you think you might have to split up the solution in the future, consider using multiple solutions.

Multiple solutions

If you have two unrelated solutions that don't share components, the most direct approach is to create two unmanaged solutions.

ⓘ Note

It's very common in solutions to modify the application ribbons or the site map. If both of your solutions modify these solution components, they're shared components. See the following section to see how to work with shared components.

Multiple solution layering and dependencies

When you import different solutions into your target environment, you're often creating layers where the existing solution lies underneath the one being imported. When it comes to solution layering, it's important that you don't have cross-solution dependencies. Having multiple solutions in the same environment using the same unmanaged component should be avoided. This cross-solution dependency issue is especially true with tables.

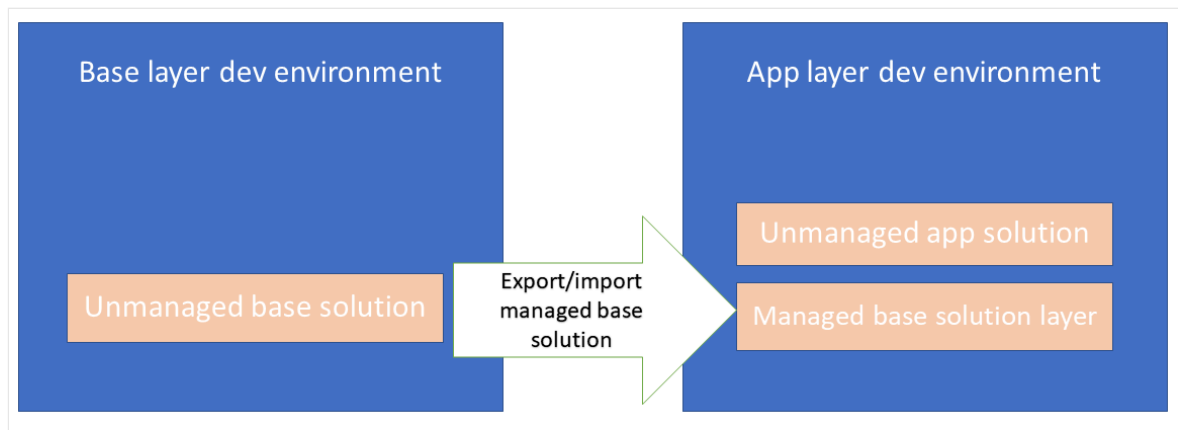
Segment your solutions by component type when there are no cross-dependency risks. For example, have one solution that includes all of your tables, another solution that has all of your plug-ins, and a third solution that has all of your flows. These different components don't have risks of cross-solution dependencies. Therefore, it's safe to have multiple solutions formed this way in the same environment.

Don't have two different solutions in an environment where both contain tables. This is because there are frequently risks of a single relationship between tables, which creates a cross-solution dependency and causes solution upgrade or delete issues in the target environment at a later point in time.

When you're designing your solution layers and you want to have a structured approach for apps, you should start with a base layer. Later, you import additional solutions that will reside on top of the base layer. Subsequently, you have a base layer and extension layers on top that extend that base layer.

When you manage your projects this way, we recommend that you use a separate environment for each layer. Build your solution layering using these steps.

1. Before you create the solutions in the following steps, use a single publisher for all your solutions across your environments. More information: [Solution publisher](#)
2. In the "base" environment, you have your base solution with the unmanaged tables from that environment and no other tables. You then export this solution as managed.
3. You set up a second environment for the extension or "app" layer that will later reside on top of the base layer.
4. You import the managed base layer into the app layer environment and create an unmanaged solution for the app layer.



You can now extend the data model by adding additional tables, columns, table relationships, and so on, into the app solution. Then, export the app solution as managed. Notice that the app solution will have dependencies on the base layer solution.

In your production environment, you import the managed base layer and then import the managed app layer. This creates two managed layers in the environment with clear dependencies between the two managed solutions. Managing multiple solutions this way doesn't create cross-solution dependencies, which can cause solution maintenance issues, such as removing the top layer if needed.

Repeat this segmentation pattern to have as many different solutions as you need to maintain. Although we recommend that you keep the number of solutions as small as possible to keep your solution layering manageable.

See also

[Use segmented solutions](#)

[Scenario 5: Supporting team development](#)

Feedback

Was this page helpful?

[Provide product feedback](#) 

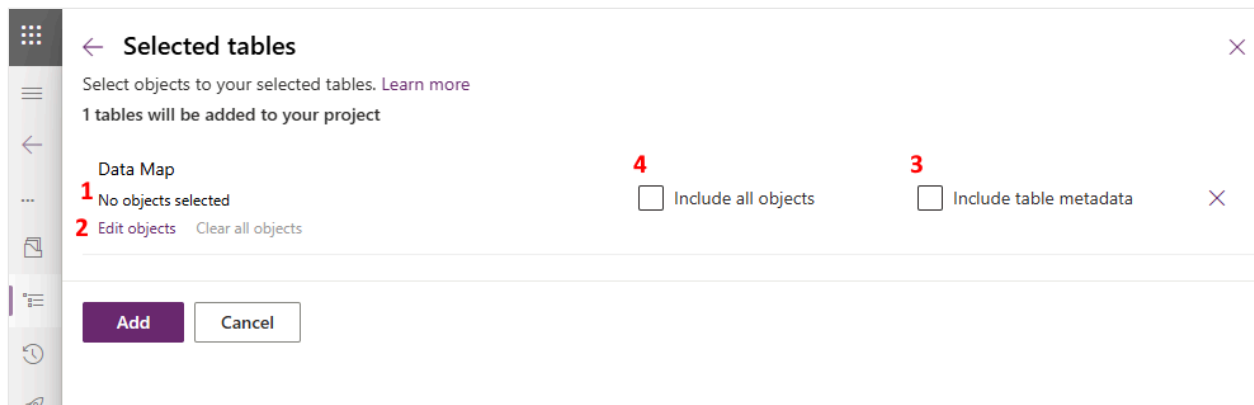
Use segmented solutions

Article • 02/05/2025

Use solution segmentation so that you only include table components that are updated when you distribute solution updates. With solution segmentation, you export solution updates with selected table assets, such as table columns, forms, and views, rather than entire tables with all their assets. To create a segmented solution, you use the **Solutions** area in Power Apps.

You can segment a solution when you select from the following options when you add an existing table to the solution:

1. **No objects selected.** When you don't select any objects or metadata, the minimal table information is added to the solution. Therefore, apart from the friendly name, table attributes (metadata), or components won't be included. This is the default option so no objects are added when you select **Add** without any other selections when adding an existing component.
2. **Edit objects.** You can segment your solution by individually selecting **Edit objects**, and then select each component that's associated with the table, such as columns, relationships, business rules, views, forms, and charts. Use the select components option to select only the components that you added or changed with the table, such as a new custom column or adding a form.
3. **Include table metadata.** This option includes no components—such as columns, forms, views, or related tables—but does include all the metadata associated with the table. Metadata includes the table properties, such as auditing, duplicate detection, and change tracking.
4. **Include all objects.** This option includes all components and metadata associated with the table. It can include other tables or table components such as business process flows, reports, connections, and queues. You should only use this option when you're distributing an unmanaged table that doesn't exist in the target environment. Notice that after you select this option, you can't undo it. To segment the solution, you must remove the table from the solution and readd it by selecting only the changed or new components.



Warning

Don't add components to the solution that aren't new or changed components. When your update is imported to the target environment, a solution with unintended components can cause unexpected behavior to the existing components that now lay below the layer you introduced with your solution update. For example, if you add a view for a table that isn't updated and the view in the existing layer has customizations, the existing customizations might become inactive. More information: [Solution layers](#)

More information: [Create segmented solutions](#)

See also

[Update a solution](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Overview of tools and apps used for ALM

Article • 02/07/2025

This article gives a brief overview of the tools and apps used for application lifecycle management (ALM) in Microsoft Power Platform. These categories are categorized for the specific audience or persona.

Tools and apps used by admins and makers

Administrators and makers use the following tools for implementing ALM in Power Platform.

Power Platform admin center

The [Power Platform admin center](#) provides a unified portal for administrators to manage environments and settings for Power Apps, Power Automate, and model-driven apps in Dynamics 365 (such as Dynamics 365 Sales and Dynamics 365 Customer Service). From the admin center, administrators can manage environments, data integration, gateways, data policies, and get key Power Platform metrics through Microsoft Dataverse analytics, Power Automate analytics, and Power Apps analytics.

More information:

- [Power Platform admin center features](#)
- [Administer Power Apps](#)
- White paper: [Administering a low-code development platform](#)

Power Apps

[Power Apps](#) is part of a suite of apps, services, connectors, and data platform that provides a rapid application development environment to build custom apps for your business needs. Using Power Apps, you quickly build custom business apps that connect to your business data stored either in the underlying data platform (Dataverse) or in various online and on-premises data sources, such as Microsoft 365, Dynamics 365, SQL Server, and so on.

More information: [What is Power Apps?](#)

Configuration Migration Tool

The Configuration Migration Tool enables you to move configuration and/or reference data across environments. Configuration and reference data is different from user and transactional data and is used to define custom functionality in apps based on Dataverse. More information: [Move configuration data across environments and organizations with the Configuration Migration Tool](#)

ⓘ Note

The Configuration Migration Tool is best suited for migrating relational configuration data. [Environment variables](#) are recommended for storing and migrating nonrelational configuration parameters.

PowerShell modules

With PowerShell cmdlets for administrators, app makers, and developers, you can automate many of the monitoring, management, and quality assurance tasks that are only possible manually today in Power Apps or the Power Platform admin center.

More information: [Manage solutions using PowerShell](#)

Pipelines

With Power Platform pipelines, admins create and configure pipelines so makers can use them to deploy solutions to environments with a single choice.

More information: [Overview of Power Platform pipelines \(preview\)](#)

Tools and apps used by developers

Developers use the following tools and apps for implementing ALM in Power Platform.

DevOps

DevOps is the combining of two historically disparate disciplines: software development and IT operations. The primary goal of DevOps is to shorten the software development lifecycle and provide continuous integration and continuous delivery (CI/CD) with high software quality. You use Power Apps build tools to automate common build and deployment tasks related to Power Apps if your DevOps platform is Azure DevOps. This

includes synchronization of solution metadata between development environments and your version control system, generating build artifacts, deploying to downstream environments, provisioning or deprovisioning of environments, and the ability to perform static analysis checks against your solution by using the Power Apps checker service.

More information: [Microsoft Power Platform Build Tools for Azure DevOps overview](#)

Version control system

A version control system is a category of software tools that help record changes to files by keeping track of changes committed to software code. A version control system is a database of changes, which contains all the edits and historical versions of a software project. Version control systems allow you to maintain a single "source of truth" and recall specific versions when needed. Git is a popular example of a version control system.

Important

Notice that *source control* applies both to [Dataverse solutions](#) and "traditional" source code. Dataverse solutions should always be part of the source code and never stored solely in Microsoft Power Platform environments.

More information: [Getting started: What is Git?](#) 

Package Deployer

Package Deployer lets administrators or developers deploy comprehensive packages of relevant assets to Dataverse instances. Packages can consist of not only solution files, but also flat files, custom code, and HTML files. Common Data Service provides you with a Visual Studio template for creating these packages that can be used with the Package Deployer tool or with PowerShell to deploy them to a Common Data Service instance.

More information: [Create packages for the Package Deployer](#)

Solution Packager

Solution Packager is a tool that can unpack a compressed solution file into multiple XML files and other files, so they can be easily managed by a source control system.

More information: [Use the Solution Packager tool to compress and extract a solution file](#)

Power Platform CLI

Microsoft Power Platform command line interface (CLI) is a simple, single-stop developer command-line interface that empowers developers and app makers to create code components.

More information: [What is Microsoft Power Platform CLI?](#)

See also

[Implementing healthy ALM](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

About healthy ALM

Article • 02/04/2025

This section provides information about the various scenarios that help you reach the end goal of implementing and practicing healthy application lifecycle management (ALM) by using Power Apps, Power Automate, and Microsoft Dataverse in your organization.

The scenarios range from people who are new to Microsoft Power Platform to existing users practicing unhealthy ALM, and show how you can move to a recommended, successful ALM implementation.

[Scenario 0: ALM for a new project](#)

[Scenario 1: Citizen development ALM](#)

[Scenario 2: Moving from a single environment](#)

[Scenario 3: Moving from unmanaged to managed solutions](#)

[Scenario 4: Use DevOps for automation](#)

[Scenario 5: Support team development](#)

[Scenario 6: Embrace citizen developers](#)

[Scenario 7: Add canvas apps in solutions by default](#)

See also

[ALM basics in Power Apps](#)

Feedback

Was this page helpful?

[Provide product feedback](#) 

Scenario 0: ALM for a new project

Article • 02/06/2025

If you're new to Power Apps and creating your first app, follow the tasks described in this article to successfully deploy a functioning application to your production environment using a healthy application lifecycle management (ALM) strategy.

 Expand table

Task	Description	More information
1. Plan and implement your environment strategy.	Determining the environments you'll need and establishing an appropriate governance model is a critical first step. At a minimum, you should have two environments: dev and production. However, we recommend that you have three environments: dev, test, and production.	Environment strategy
2. Create a solution and publisher.	Start with a blank solution, and create a custom publisher for that solution.	Solution publisher
3. Set up your DevOps project.	Set up a DevOps project in which you'll later add several pipelines to perform required processing like export and deployment of your solution.	Setting up continuous integration and deployment Create a project
4. Create your export from development pipeline in DevOps.	Create a DevOps pipeline to export your completed unmanaged solution to a managed solution.	Create your first pipeline Build and release pipelines Build pipeline: Export a solution from development (DEV)
5. Configure and build your app.	Create your app within the solution you created.	Model-driven apps: Build your first model-driven app from scratch Canvas apps: Create an app from scratch

Task	Description	More information
		using Microsoft Dataverse
6. Add any additional customizations to that solution.	Add additional components as needed. Choose from a vast selection of components, such as flows, AI models, export to data lake configuration, web resources, plug-ins, and even other apps.	Create components in a solution Add an existing component to a solution
7. Create a deployment pipeline in DevOps.	Create a DevOps pipeline to deploy your managed solution to one or more target production environments.	Build and release pipelines Release pipeline: Deploy to production (PROD)
8. Grant access to the app.	Assign licensing and assign security roles to share applications with users.	Licensing Share a model-driven app Share a canvas app

See also

[Scenario 1: Citizen development \(app and flow makers\)](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Scenario 1: Citizen development

Article • 03/15/2025

This scenario aims at the legacy canvas app makers and flow makers in Power Apps and Power Automate, respectively, who work in a Microsoft Dataverse environment *without* a Dataverse database.

You currently build canvas apps and flows that aren't part of a solution, and share your apps and flows by specifying each user by name or specifying a security group in Microsoft Entra who can access your app or flow.

The end goal is to move your legacy canvas app and flows to a managed application lifecycle management (ALM) model by creating apps and flows in a Dataverse solution.

ⓘ Note

You can use the default Dataverse environment as a playground to get acquainted with the app-building experience. However, if you're shipping shared components (such as tables and Copilot agents) along with your apps and flows as part of a solution, we recommend you move to a model where you have multiple environments, each dedicated for the development, testing, and release of your apps. For more information about environments, go to [Types of environments used in ALM](#).

For legacy canvas app and flow makers to participate and move to a healthy ALM model, follow these steps:

1. Work with your IT admin/organization to discuss your business processes and environment strategy, such as the number of environments, access permissions, groups, and security. More information: [ALM environment strategy](#)
2. Understand the concept of solutions in Dataverse as a mechanism to transport and share components with others. You can export a solution from one environment to another to easily and securely share your apps and flows. More information: [Solution concepts](#)
3. Follow the steps in [Scenario 0: ALM for a new project](#) to move to a healthy ALM.

See also

[Scenario 2: Moving from a single production environment](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

Scenario 2: Moving from a single production environment

Article • 03/15/2025

This scenario aims at users or organizations that have a single Microsoft Dataverse environment with a database that they use to create, test, and deploy their apps. Having a single environment poses challenges in terms of change management and control.

To move to healthy application lifecycle management (ALM), you must:

1. At a minimum, separate your development and production environments. The ideal solution is to have three environments, one each for development, testing, and production/deployment. More information: [ALM environment strategy](#)
2. Use solutions to ship changes across environments: More information: [Use a solution to customize](#)
3. Automate the application lifecycle process using DevOps. More information: [Microsoft Power Platform Build Tools for Azure DevOps](#)

See also

[Scenario 3: Moving from unmanaged to managed solutions in your organization](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Scenario 3: Moving from unmanaged to managed solutions in your organization

Article • 02/05/2025

This scenario addresses a situation where your production environment contains several unmanaged solutions or your customizations were made in the default solution. The tasks described here show you how to convert all of your unmanaged model-driven app components to managed using a single solution that is used to create a single managed layer in your test and production environments. Later, you might want to create additional solutions to develop different layering strategies and dependencies between solutions.

Except for your development environment, the end result is to only have managed solutions in your environments. More information: [Managed and unmanaged solutions](#).

Prerequisites

- Separate development and production environments. Additionally, we recommend that you also maintain at least one test environment that's separate from your development and production environments.
- Use a single publisher for all your solutions across all your environments. More information: [Solution publisher](#)

Convert an unmanaged solution to managed

1. Identify and remove unnecessary tables and components.
 - a. Create a backup of your production environment. You can bring back components that might be inadvertently deleted through solution export and import in the next step.
 - b. Remove tables and components that aren't needed from your production environment. For example, consider deleting tables with no records or very old records, or tables and components that have no dependencies. More information: [View dependencies for a component](#)
2. Create a solution to use to convert components from unmanaged to managed.
 - In your development environment, create a *single* unmanaged solution that is used to contain *all* Microsoft Dataverse model-driven apps, tables, and dependent components, such as forms, views, columns, charts, and

dashboards. Incorporating all of these components together can help reduce the chances of cross-solution layering issues that might occur later when you update or introduce new model-driven apps and customizations. More information: [Create a solution](#)

- For *unmanaged* components, such as custom unmanaged tables, you won't use segmentation. Instead, select **Include all components** when adding those components to the solution.
- If there are *managed* components that you've customized, use segmentation when adding those components to the solution. For example, if you've added a custom column or changed the display name for an existing column to a Power Apps standard table, such as **Account** and **Contact**, use segmentation so you only export the customized components your project needs and not additional components that you don't intend to service. To do this, choose **Select components**, and then add only your customized components to the solution.

 Tip

To see if a managed component has been customized, look for an unmanaged layer that appears above the base managed layer of the component. More information: [View solution layers for a component](#)

- If you have canvas apps, flows, Power Pages apps, or plug-ins to convert, you can add them to a separate unmanaged solution now, or at a later time.
- Remember to use a single publisher for all your solutions. More information: [Solution publisher](#)

3. Deploy the managed solution.

- a. If you have an existing test environment, you can go to the next step. However, we recommend that you make a copy of your production environment to use as the test environment. More information: [Copy an environment](#)
- b. Export the unmanaged solutions from your development environment as *managed*. More information: [Export solutions](#)
- c. If there's an unmanaged solution in your test environment that has the same name as the managed solution you want to import, delete the unmanaged solution record in the test environment. To delete the unmanaged solution record, go to [Power Apps](#), select the test or production environment, select **Solutions**, select the unmanaged solution, and then on the command bar, select **Delete**. Notice that deleting the unmanaged solution doesn't delete the solution's unmanaged components.

- d. Import the solution into your test environment using [Power Platform CLI](#) solution import commanding with the *convert-to-managed* parameter or the [DevOps tooling](#) solution import task. Currently, you can't successfully import the solution and convert all components to managed using Power Apps (make.powerapps.com).
- e. There are various types of tests you can use to check whether specific functions or features of your app are working correctly. Some of these tests include unit tests, end-to-end, and user acceptance tests (UAT).
- f. After testing is completed and validated, repeat steps c-e, but instead of performing the step in your test environment perform the step in your *production* environment.
- g. If you have canvas apps, flows, or Power Pages apps, import the managed solutions first into your test and then into your production environments in a similar fashion as described earlier.

Next steps

Now that you've moved to managed solutions, we recommend that you understand solution and component layering. Moreover, except for your development environments, there shouldn't be any unmanaged customizations or unmanaged solutions in your environments. More information: [Solution layers](#)

See also

[Scenario 5: Supporting team development](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Scenario 4: Using DevOps to move from manual to automated ALM

Article • 02/06/2025

This scenario aims at a situation where you're shipping your solutions manually and don't have a source code control system in place.

For this scenario, you should:

1. Consider adopting a source code control system like [Azure DevOps](#). Azure DevOps provides developer services for support teams to plan work, collaborate on code development, and build and deploy applications.
2. Export a solution from your development environment containing your apps and customizations, unpack your solution, and store the components in your source control system. Use Azure Pipelines to manage your components, and then deploy them to the target environment for testing. Finally, deploy to the production environment for user consumption.

See also

[Microsoft Power Platform Build Tools for Azure DevOps](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Scenario 5: Supporting team development

Article • 03/13/2025

Supporting team development consists of multiple apps and development team members in a structured release management process. Team development includes the following strategies:

- **Conflict and merge resolution** It's important to consider which pieces will merge well and which should only be worked on by a single developer at a time, such as forms or site maps. More information: [Understand how managed solutions are merged](#)
- **Branching strategy** Your branching strategy should make sure that you can service an existing release while working on the next version.
- **Environment strategy** Developer isolation, and an environment strategy that supports it, is critical when multiple developers work on the same application. Environments should be short-lived and spun up on demand from the latest build. Each developer should have their own development environment in which only the required unmanaged solution is installed, and all dependent solutions are installed as managed.
- **Power Apps component library** Components are reusable building blocks for canvas apps so that app makers can create custom controls to use inside an app, or across apps using the component library. More information: [Component library](#)

See also

[Scenario 6: Embrace your citizen developers in the ALM process](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Scenario 6: Embrace your citizen developers in the ALM process

Article • 02/15/2022

Use the [Center of Excellence \(CoE\) kit](#) that provides customizable apps, flows, and dashboards to showcase the admin and governance capabilities in Microsoft Power Platform. Use the kit to adapt application lifecycle management (ALM) at scale in your organization.

See also

[ALM for developers](#)

Scenario 7: Add canvas apps and cloud flows to solutions by default for a healthy ALM (preview)

Article • 03/14/2025

[This article is prerelease documentation and is subject to change.]

Important

- This is a preview feature.
- Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.

Add canvas apps and cloud flows into solutions by default by enabling the **Canvas apps** and **Cloud flows** switches under the environment setting for **Create new canvas apps and cloud flows in Dataverse solutions (preview)**.

To learn about enabling the setting, go to [Environment settings](#). More information: [Add canvas apps and cloud flows to solution by default \(preview\)](#)

See also

[ALM for developers](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Maintaining healthy model-driven app form ALM

Article • 03/13/2025

This article provides you with information about the various scenarios on how to implement and practice healthy application lifecycle management (ALM) for customizing forms in your model-driven app solutions.

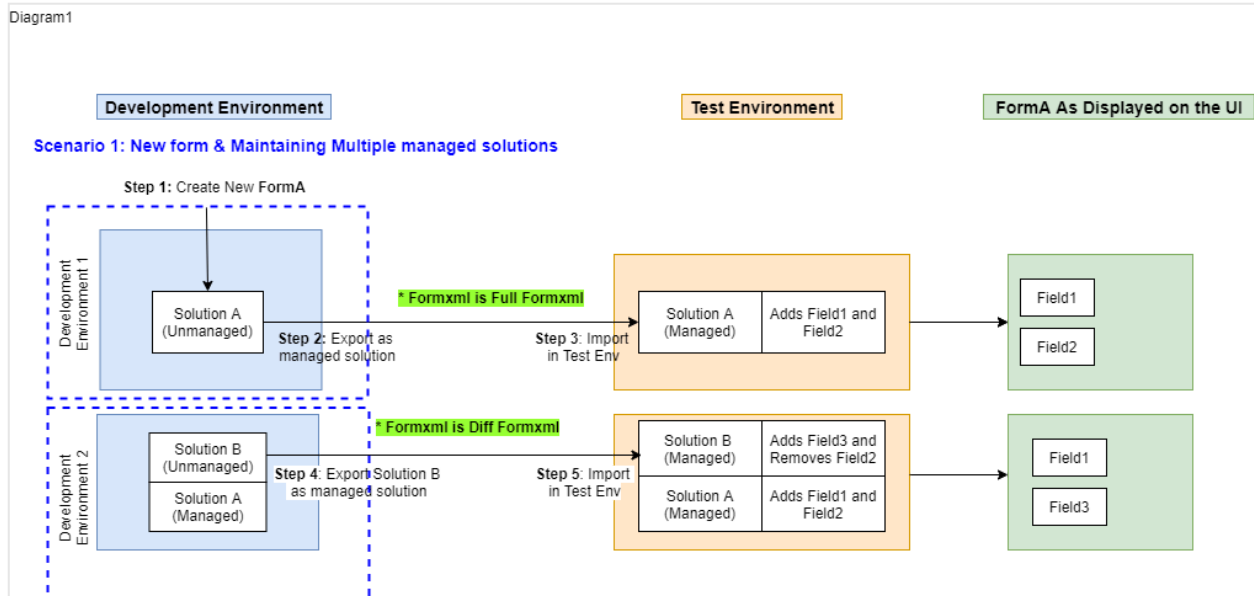
The following sections describe how form merge works and how to maintain customizations. The basic development scenarios with recommendations for maintaining successful ALM for a model-driven app form are covered in detail within each section that follows. Every scenario includes steps to follow that can help you implement a proper ALM process when updating your solution or model-driven app.

Creating a new form and maintaining it using multiple managed solutions

Follow these steps to implement healthy form ALM for this scenario.

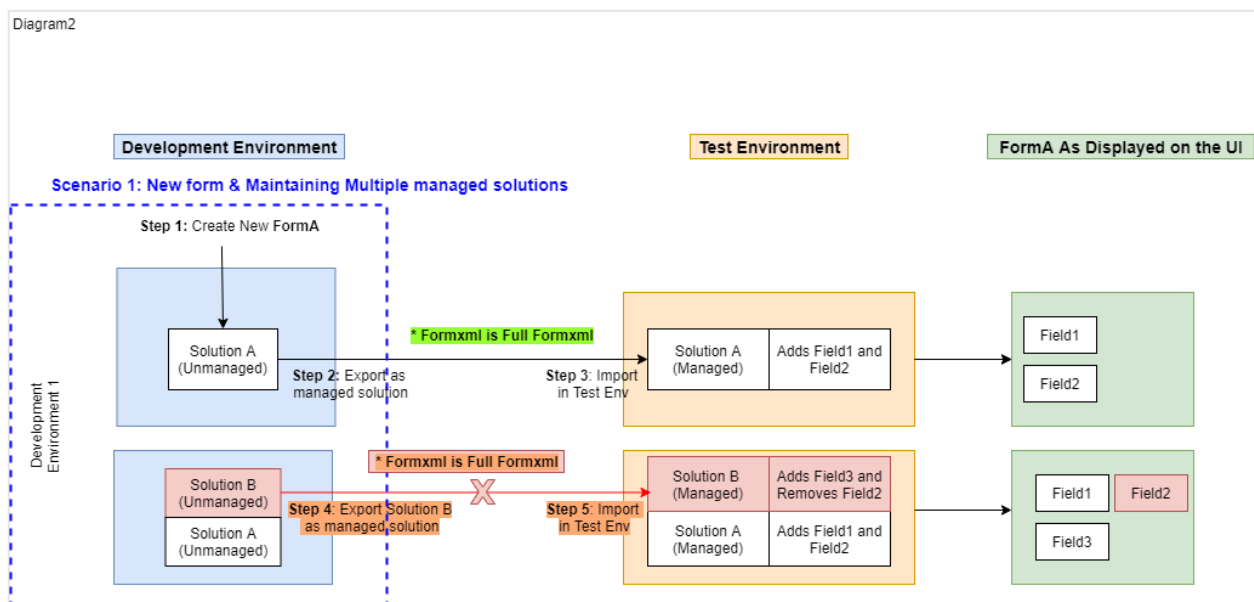
1. Create a new form named *FormA* in your development environment and perform customizations on the form.
2. Create a new solution (named *Solution A* in the diagram) in the development environment, which will be an unmanaged solution and add your new form. Export the solution as managed. This step exports a [full FormXml](#) for the form.
3. In your test environment, import the managed solution from step 2, which creates *FormA* in the test environment. In the diagram, *FormA* gets created in the test environment and the UI for the form shows *Field1* and *Field2* that *Solution A* added to the form.
4. When you further customize the form you created in step 1 using a new development (source) environment, import the managed **Solution A** created in step 2, make sure the development instance you're using has **FormA** in a managed state. As shown in the diagram, managed *Solution A* is imported in the development environment and the form is customized creating active customizations. Then, *FormA* can then be added to a new unmanaged solution (*Solution B* in the diagram) and exported as a managed solution from the development environment. This step exports a [differential \(diff\) FormXml](#) for the form.
5. In your test environment, import the managed solution (*Solution B*) from step 4. As shown in the diagram *Solution B* is adding a new *Field3* to *FormA* and removing

Field2, which was added by *Solution A*. The UI for the form in the test environment now shows *Field3* and *Field1* on the form but not *Field2* after the merge.



Unhealthy example for this scenario

As seen in the this diagram, it isn't a healthy ALM practice to create multiple managed solutions from the development environment where the base solution (*Solution A*) is in an unmanaged state. This is because, when you create another unmanaged solution (*Solution B*) for the unmanaged form, the FormXml is exported as a full FormXml, instead of a diff FormXml as shown in the valid scenario above. Subsequently, changes like removing a column won't take effect.



Creating a new form and making customizations using patches and upgrades

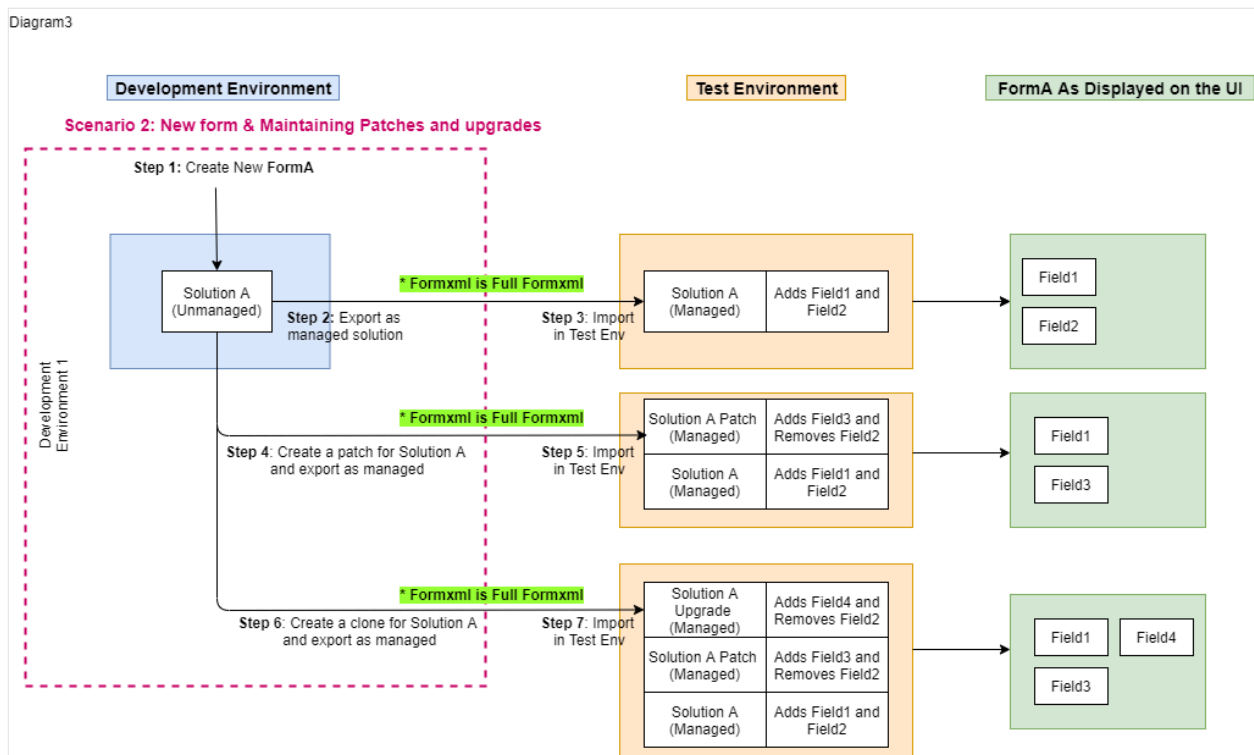
Follow these steps to implement healthy form ALM for this scenario.

1. Create a new form named *FormA* in your development environment and perform customizations on the form.
2. Create a solution (*Solution A* in the diagram), which will be an unmanaged solution and add your new form. Export the solution as managed. This step exports a [full FormXml](#) for the form.
3. In your test environment, import the managed solution from step 2, thus creating the form in the test environment. In the diagram *FormA* gets created in the test environment and the UI for the form shows *Field1* and *Field2* that *Solution A* added to the form.
4. When you further customize the form you created in Step 1 using patches, use the same environment where *Solution A* is in an unmanaged state and create a patch for the solution and customize the form. Next, export the patch as a managed solution. This step exports a [full formXml](#) for the form.
5. In your test environment, import the managed patch solution from step 4. As shown in the diagram, the *Solution A* patch is adding a new *Field3* to *FormA* and removing *Field2*, which was added by *Solution A*.

ⓘ **Note**

Patches containing [full formXml](#) are always compared to the base layer that the patch was created from and ignores any intermediate patches between the base and the current patch. As a result, *Field2* is removed since it exists in the base layer *Solution A*,* and the removal is detected. On the other hand, *Field3* is added by this patch solution and can't be removed by subsequent patches. Thus, fields added through patch solutions are additive in nature.

6. When you further customize the form you created in Step 1 using upgrades, use the same environment where *Solution A* is in an unmanaged state and clone *Solution A* to create the upgrade solution and customize the form. Then, export the *Solution A* upgrade as a managed solution. This step exports a full FormXml for the form.
7. In your test environment, import the managed *Solution A* upgrade from step 6. As shown in the diagram, the *Solution A* upgrade is adding a new *Field4* to *FormA* and removing *Field2*, which was added by *Solution A*. The UI for the form in the test environment now shows *Field1*, *Field3*, and *Field4* on the form, but *Field2* will be removed after the form is merged from the import.

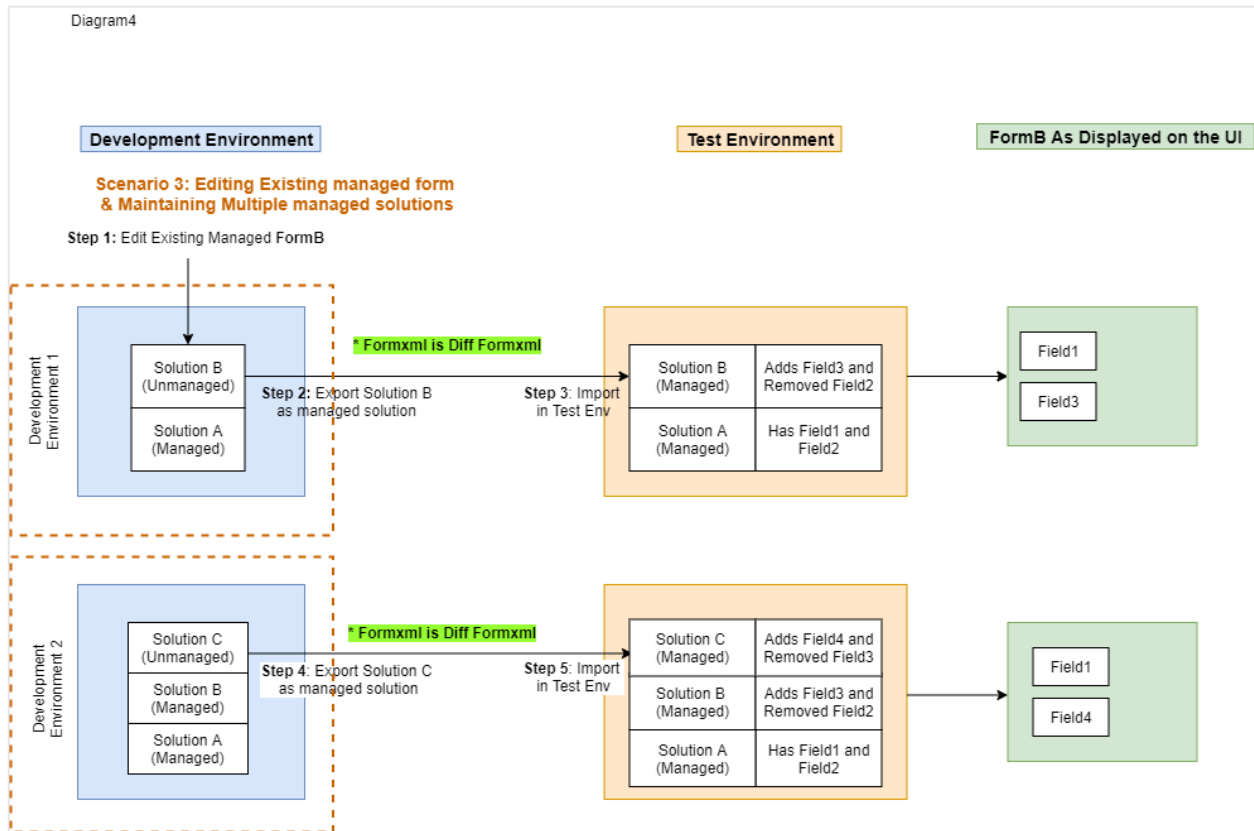


Customizing an existing managed form and maintaining it using multiple managed solutions

Follow these steps to implement healthy form ALM for this scenario.

1. Edit an existing managed form, named *FormB* in this example, in your development environment and perform customizations on the form. Note that solution A is the managed solution already installed for the form in the development environment.
2. Create a new solution (*Solution B* in the below diagram), which is an unmanaged solution, and add *FormB*. Export the solution as managed. This step exports a [differential \(diff\) FormXml](#) for the form.
3. In your test environment, import the managed solution from step 2, thus creating a second solution layer for the form. In the diagram, *FormB* gets the merged changes from *Solution A* and *Solution B* in the test environment and the UI for the form shows *Field1* and *Field3* on the form but not *Field2*, which was removed by *Solution B*.
4. When you further customize the form you customized in step 1 using new managed solutions, make sure to use a new development environment which has *FormB* in a managed state. As shown in the diagram, *Solution A* and *Solution B* managed solutions are imported in the new development environment. *FormB* is customized creating active customizations, which can then be added to a new solution (*Solution C* in the diagram) and exported as a managed solution.

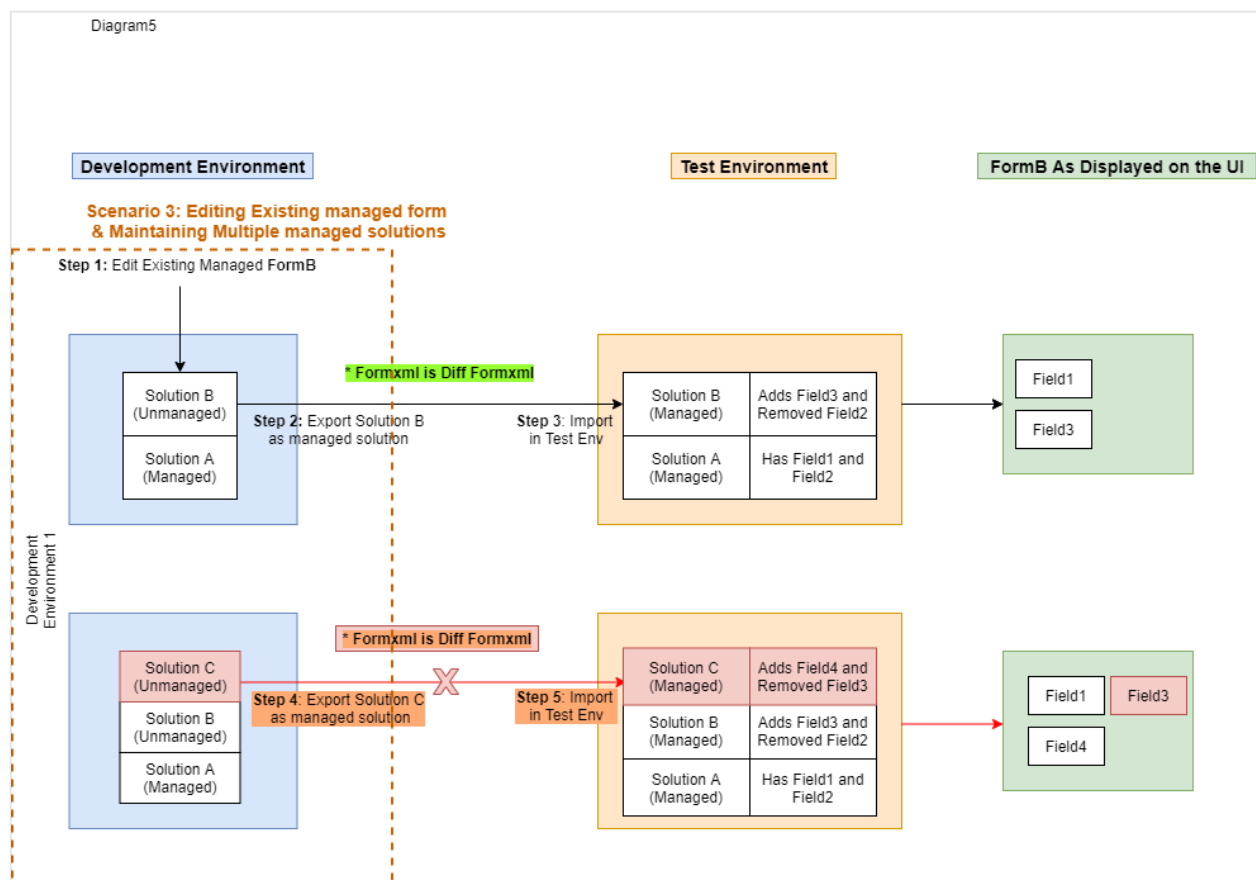
5. In your test environment, import the managed *Solution C* from step 4. As shown in the diagram, *Solution C* is adding a new *Field4* to *FormB* and removing *Field3*, which was added by *Solution B*. The UI for the form in the test environment now shows *Field1* and *Field4* on the form, but not *Field2* and *Field3*.



Unhealthy example for this scenario

As shown in the diagram, it isn't a healthy ALM practice to create multiple managed solutions from the development environment that contains another unmanaged solution you created for the same form. Notice that *Solution B* is in unmanaged state. When you create another unmanaged solution (*Solution C*) for *FormB*, the FormXml is exported as a diff FormXml as shown in step 4 in the above scenario. But, *FormB* also contains the changes from *Solution B*, which will get overwritten by your new changes.

For example, as seen in the diagram, *Field3* is added to *FormB* in *Solution B*. But now when you create a new *Solution C* in this environment, with *Solution B* in unmanaged state, and remove *Field3*, *Field3* will also be removed in the development environment. *Field3* won't be tracked in the diff FormXml when the solution is exported, since the change of adding and removing this column was made in the same active layer. That means when managed *Solution C* is imported in the test environment, the form will still render the *Field3* because the diff FormXml never records it as removed (like it was removed in step 5 in the healthy form ALM scenario above). Performing your form customizations this way will lead to the development environment being inconsistent with the test environment.



Customizing an existing managed form and maintaining it using patches and upgrades

Follow these steps to implement healthy form ALM for this scenario.

1. Customize an existing managed form, named *FormB* in this example, in your development environment and perform customizations on the form. Note that *Solution A* is the managed solution already installed for the form in the development environment.
2. Create a solution (*Solution B*), which will be an unmanaged solution and add *FormB*. Export the solution as managed. This step exports a [diff FormXml](#) for the form.
3. In your test environment, import managed *Solution B* from step 2, thus creating a second solution layer for the form. In the diagram, *FormB* gets the merged changes from *Solution A* and *Solution B* in the test environment. Additionally, the UI for *FormB* shows *Field1* and *Field3* on the form but not *Field2*, which was removed by *Solution B*.
4. When you further customize the form you customized in Step 1 using a patch solution, you can use the same development environment as step 1 where *Solution B* exists in an unmanaged state. As shown in the diagram, *Solution A* is in a

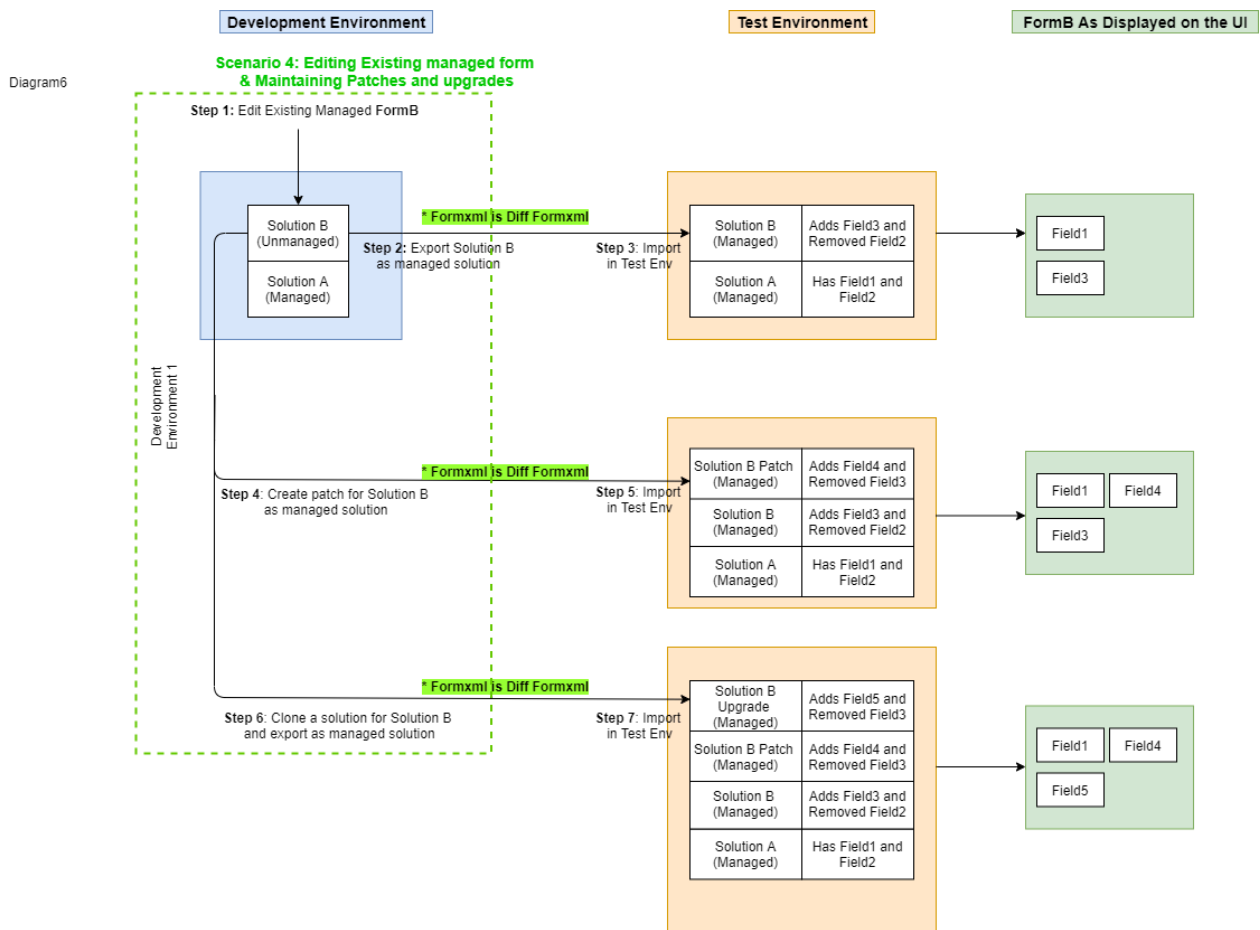
managed state and *Solution B* is in an unmanaged state. The form is further customized and you create a patch for *Solution B* adding your form to this solution and exporting it as a managed patch solution. This step exports a diff FormXml.

5. In your test environment, import managed patch *Solution B* from step 4. As shown in the diagram, *Solution B Patch* is adding a new *Field4* to *FormB* and removing *Field3*, which was added by *Solution B*.

ⓘ **Note**

Patches are additive in nature and can't remove components, such as columns, from the form. So, *Field3* won't be removed from the form. The UI for the form in the test environment now shows *Field1*, *Field3*, and *Field4* on the form, but not *Field2*.

6. When you further customize the form you created in Step 1 using upgrades, use the same environment where *Solution B* is in an unmanaged state and clone *Solution B* to create the upgrade solution and customize *FormB*. Export the upgrade as a managed solution. This step exports a diff FormXml for the form.
7. In your test environment, import the managed *Solution B* upgrade solution from step 6. As shown in the diagram, *Solution B Upgrade* is adding a new *Field5* to *FormB* and removing *Field3*, which was added by *Solution B*. The UI for the form in the test environment now shows *Field1*, *Field4*, and *Field5* on the form, but *Field2* and *Field3* are removed.



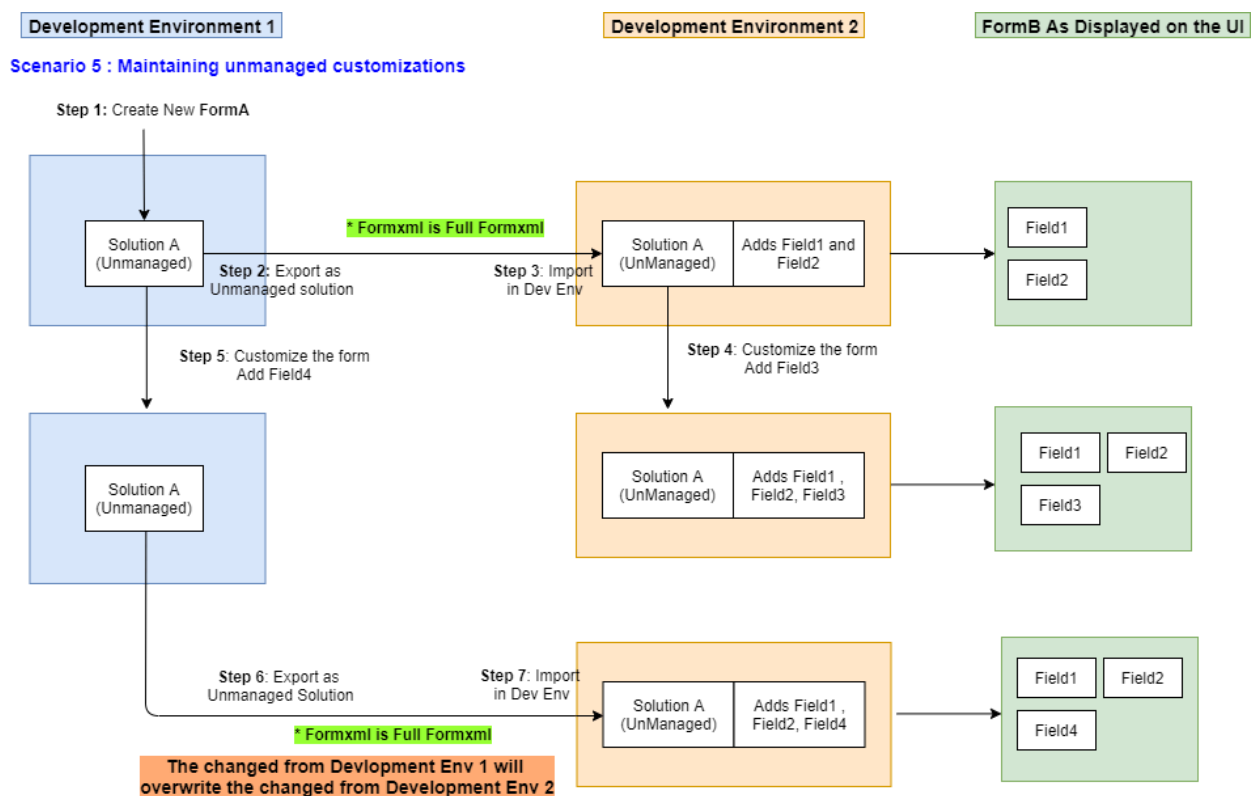
Maintaining unmanaged solutions and customizations for a new form across multiple development environments

Follow these steps to implement healthy form ALM for this scenario.

1. In *Development Environment 1*, create a new *FormA* and perform customizations on the form.
2. Create a solution (*Solution A* in the diagram), which will be an unmanaged solution, and add your new form. Export the solution as unmanaged. This step exports a **full FormXml** for the form.
3. In *Development Environment 2*, import the unmanaged solution from step 2, which creates the form in *Development Environment 2*. In the diagram, *FormA* gets created and the UI for the form shows *Field1* and *Field2* that *Solution A* added to the form.
4. You further customize the form in *Development Environment 2* making active customizations in the environment, such as adding a new column named *Field3*. *FormA* now shows *Field1*, *Field2*, and *Field3*.
5. In your *Development Environment 1*, you further customize the form also by adding *Field4*. The UI for the form in the *Development Environment 1* now shows *Field1*,

Field2, and *Field4*.

6. Export unmanaged *Solution A* with the changes made in step 5. This step exports a full FormXml for the form.
7. In *Development Environment 2*, import unmanaged *Solution A Upgrade* from step 6. Since the solution you're importing contains the full FormXml for *FormA*, it overwrites the active customization made in *Development Environment 1*. So, the form now shows only *Field1*, *Field2*, and *Field4*, but not *Field3*, which was the additional active customization done in *Development Environment 1*. This behavior occurs with any unmanaged solution import that has the full FormXml for the form.

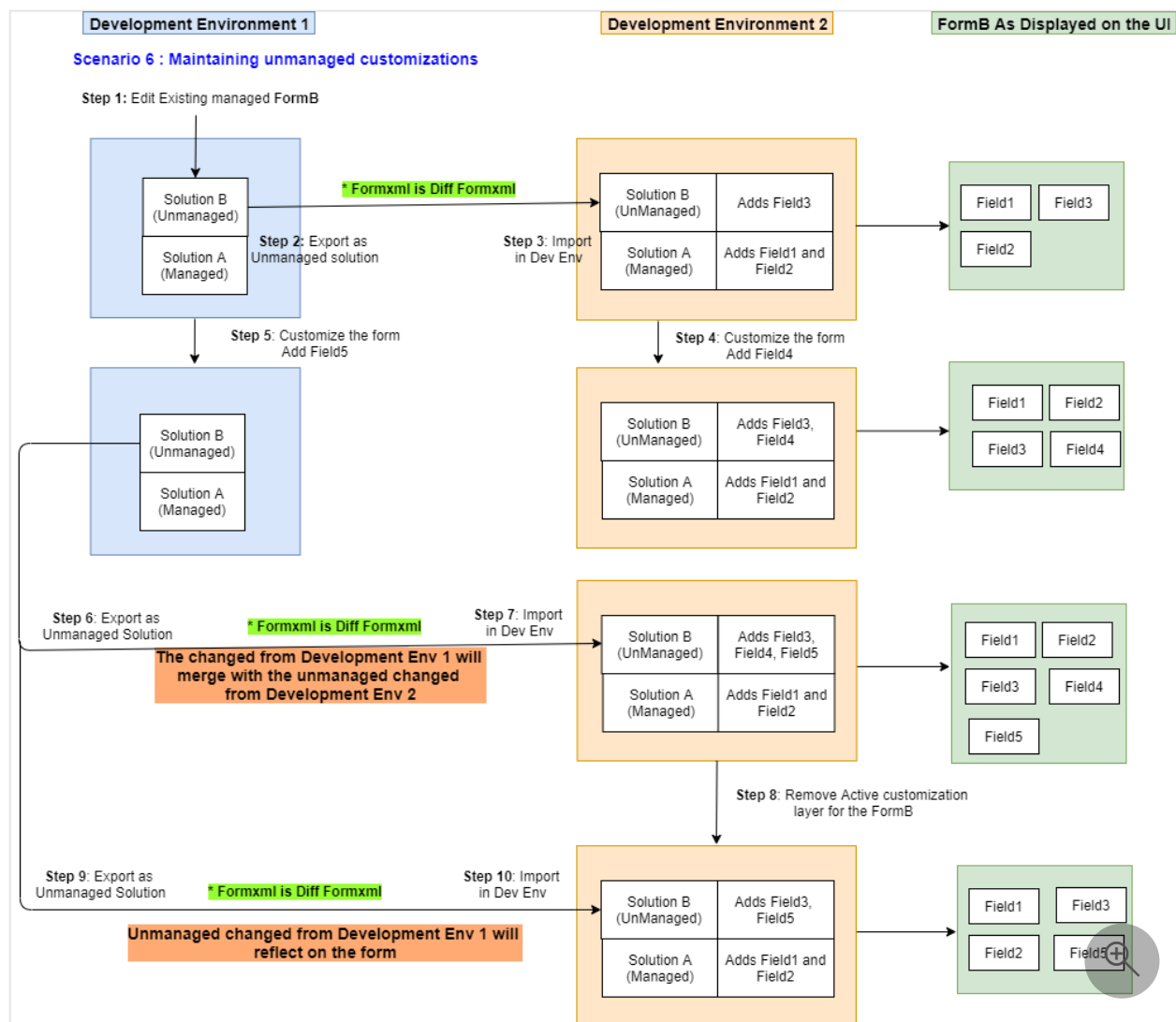


Maintaining unmanaged solutions and customizations for an existing form across multiple development environments

Follow these steps to implement healthy form ALM for this scenario.

1. In *Development Environment 1*, customize an existing form, named *FormB* in this example. Then perform customizations on the form.
2. Create a solution (*Solution B* in the diagram), which will be an unmanaged solution, and add *FormB*. Export the solution as unmanaged. This step exports a [diff FormXml](#) for the form.

3. In *Development Environment 2*, import the unmanaged solution from step 2, thus creating a second solution layer for the form. The *FormB* UI shows *Field1*, *Field2*, and *Field3* after the form merge.
4. You further customize the form in *Development Environment 2*, making active customizations in the environment, such as adding a new column named *Field4*. *FormB* now shows *Field1*, *Field2*, *Field3*, and *Field4*.
5. In *Development Environment 1*, you further customize the form adding a new column named *Field5*. The UI for the form in *Development Environment 1* now shows *Field3* and *Field5*.
6. Export unmanaged *Solution B* with the changes made in step 5. This step exports a [diff FormXml](#) for the form.
7. In *Development Environment 2*, import unmanaged *Solution B Upgrade* from step 6. Since the solution you're importing contains the diff FormXml for *FormB*, it will merge with the active customization made in *Development Environment 1*. So, the form now shows *Field1*, *Field2*, *Field3*, *Field4*, and *Field5*. This behavior occurs for any unmanaged solution import that has the diff FormXml for the form.
8. If the form merge in step 7 isn't what you want even though you're importing a diff FormXml with the unmanaged solution and you want to be able to overwrite the active customizations made in *Development Environment 2*, then remove the active layer for *FormB*. More information: [Remove an unmanaged layer](#).
9. Export unmanaged *Solution B* with the changes made in step 5. This step exports a diff FormXml for the form.
10. In *Development Environment 2*, import unmanaged *Solution B Upgrade* from step 9. Since there's no active layer for the form in *Development Environment 2*, (step 8), all the changes from unmanaged *Solution B* are imported even though you're importing diff FormXml for *FormB*. So, the form now shows only *Field1*, *Field2*, *Field3*, and *Field5*. This behavior occurs for any unmanaged solution import that has the diff FormXml for the form. This is the same result as step 7 in the [Maintaining unmanaged solutions and customizations for an existing form across multiple development environments](#) scenario.



Full and differential form XML

Every exported solution package includes a customizations.xml file. Whenever a form is included in a solution, the related form definition exists within the FormXml sections of the customizations.xml file. FormXml can either be *full* or *differential (diff)*.

Full FormXml

The FormXml you get on exporting a solution for a form in an unmanaged state is called a *full* FormXml. Full means it contains the entire form definition. When you create a new form and export it, the form will always be a full FormXml because the form in the environment you're exporting from is in an unmanaged state and also is in a create state. If you export any further solutions from this same environment, those will also include a full FormXml. Because the `solutionaction` attribute indicates a diff FormXml, the full FormXml in the customization.xml file in the solution you export won't contain any `solutionaction` attributes.

Differential (diff) FormXml

The FormXml you get when exporting a solution for a form in a managed state is called a differential or *diff* FormXml. Diff means the FormXml contains only the changes done in the active customizations in that environment and not the entire form definition.

When you customize an existing managed form and export it, the form will always be a diff FormXml because it will only contain the active changes done to it. The diff FormXml in the customization.xml file in the solution you export will contain `solutionaction` attributes defining what the changes are, like **Added**, **Removed**, **Modified**.

Diff FormXml ensures that your solution will only express the changes your app needs and will be impacted less by changes from other layers. Diff FormXml also makes the solution less bulky and helps it import faster.

See also

[Recommendations for healthy form ALM](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Recommendations for healthy model-driven app form ALM

Article • 03/15/2025

This article provides best practices for managing the application lifecycle of model-driven app forms in Power Apps. It covers recommendations for creating, updating, and importing forms, as well as managing solutions to ensure a healthy and efficient ALM process.

Follow these recommendations when you customize forms:

- To create a new form, don't make manual edits to the FormXml in the customizations.xml file. Instead, use the form designer to create a new form or copy an existing form by doing a **Save as**. The form designer ensures that new forms have unique IDs, which avoid conflicts with existing forms. More information: [Create, edit, or configure forms using the form designer](#).
- Don't import the same localized labels when there are existing translated labels that have translation text you want to keep. This reduces dependencies and improves solution import performance.
- If your solution publisher is the owner of a form, only the base solution in your solution package should be a full FormXml. Similarly, a solution upgrade or patch for this base solution can be full FormXml. Any other managed solution you package for this form other than the base shouldn't be a full FormXml, but should be diff FormXml. More information: [Full and differential form XML](#)
- Forms use a merge approach when you import an update to a form. Therefore, using the **Overwrite customization** option during import doesn't have any effect on forms. We recommend that you keep this in mind when you update forms. If you want to overwrite customizations in the target environment, remove the active customizations in the unmanaged layer and then import the solution update. More information: [Merge form customizations](#) and [Remove an unmanaged layer](#)
- Don't use unmanaged solutions in your production instances. More information: [Scenario 3: Moving from unmanaged to managed solutions in your organization](#)

See also

[Maintaining healthy model-driven app form ALM](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

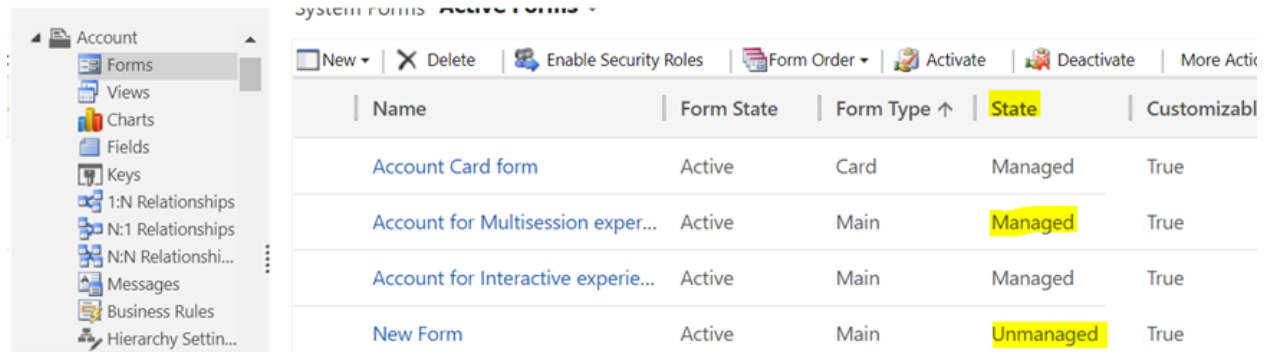
Model-driven app form ALM FAQ

FAQ

This article provides information on frequently asked questions about application lifecycle management with model-driven app forms.

How to recognize when a form is in a managed or unmanaged state?

In the customization settings, check the **State** column next to the form to know if it is in a managed state. A managed form has managed solutions installed for the form and might or might not have active unmanaged customizations. Or the form might be in an unmanaged state and has only an unmanaged layer for the form and no managed solutions.



Name	Form State	Form Type ↑	State	Customizable
Account Card form	Active	Card	Managed	True
Account for Multisession exper...	Active	Main	Managed	True
Account for Interactive experie...	Active	Main	Managed	True
New Form	Active	Main	Unmanaged	True

How to create a form?

Use the modern form designer. More information: [Create and design model-driven app forms](#)

Why do form label translations not appear for base language?

On translation export, if no changes were made to the base language label, no translations are exported in CrmTranslations.xml. On translation import, if translations for any label in the base language column were changed in CrmTranslations.xml, the changes won't take effect. More information: [Form label translations don't appear specifically for the base language](#)

Why is there an active unmanaged layer created after importing a managed solution?

If during import there isn't a fallback form specified for a table, then the import creates an unmanaged active layer for one of the main forms and the unmanaged customization indicates the form as the fallback form. More information: [There's an active unmanaged layer created after importing a managed solution](#)

Why does the form not appear in the target environment after importing a solution containing the form?

During export of unmanaged solutions, some forms that aren't modified get exported with the attribute `unmodified=1` in the form XML of the customizations.xml file located in the solution package. More information: [The form doesn't appear in target environment after importing the unmanaged solution](#)

Why do I receive the message "Full formXml is expected to create a form" during solution import?

This error can occur when the form you are importing doesn't exist in the target environment and the form is imported for the first time. More information: [Microsoft.Crm.CrmInvalidOperationException: full formXml is expected to create a form message during solution import](#)

Why do I receive the message "You cannot delete this form because it is the only fallback form of type main for the 'table' table. Each table must have at least one fallback form for each form

type during solution upgrade or uninstall?""

This error occurs when a solution upgrade or uninstall attempts to delete the last remaining form for a table. More information: [Microsoft.Crm.CrmException: You cannot delete this form because it is the only fallback form of type main for the 'table' table.](#) Each table must have at least one fallback form for each form type message during solution upgrade or uninstall

Why do I receive the message "Solution cannot be deleted due to dependencies from other components in the system message when uninstalling a solution?""

This issue can occur when the solution contains components that are referenced by other solutions on top of it in the layer stack. More information: [Solution cannot be deleted due to dependencies from other components in the system message when uninstalling a solution](#)

See also

[Maintaining healthy model-driven app form ALM Recommendations for healthy form ALM](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

ALM for admins and makers

Article • 02/07/2025

This section describes how you as a Power Platform administrator or app maker can use available features, tools, and other resources to implement application lifecycle management (ALM) using Microsoft Power Platform.

Git integration

Source control integration allows development teams to sync solutions and solution objects across one or more Microsoft Dataverse environments using an Azure DevOps Git repository. The source control integration functionality is available natively within the solutions experiences, ensuring citizen developers, code-first developers, and administrators can benefit from version control, change tracking, and seamless team collaboration across different tools and environments.

More information: [Overview of Dataverse Git integration \(preview\)](#)

Pipelines

With pipelines in Power Platform, admins create and configure pipelines so makers can use them to deploy solutions to environments with a single click.

More information: [Overview of pipelines in Power Platform](#)

Manage solutions using PowerShell

PowerShell cmdlets let admins automate many of the monitoring, management, and quality assurance tasks in Power Platform.

More information: [Manage solutions using PowerShell](#)

Feedback

Was this page helpful?

☐ Yes

☐ No

[Provide product feedback](#) 

Manage solutions using PowerShell

Article • 12/24/2022

The PowerShell cmdlets let you automate many of the monitoring, management, and quality assurance tasks in Power Platform.

Microsoft.PowerApps.Administration.PowerShell

The Power Apps administration module is used to manage users, databases, connectors, apps, flows, and permissions in Dataverse environments. More information: [Use Microsoft.PowerApps.Administration.PowerShell](#)

Microsoft.Xrm.Tooling.CrmConnector

The connector module is used to connect to and retrieve environment or organization details from Dataverse and Dynamics 365 Customer Engagement (on-premises) deployments. More information: [Use Microsoft.Xrm.Tooling.CrmConnector](#)

Microsoft.Xrm.Tooling.PackageDeployment

The package deployment module is used to deploy packages. More information: [Use Microsoft.Xrm.Tooling.PackageDeployment](#)

Microsoft.PowerApps.Checker.PowerShell

The Power Apps checker module interacts with the Power Apps checker service providing the ability to run static analysis jobs and download the results. The module is a community supported tool that's available in the [PowerShell Gallery](#). More information: [Use Microsoft.PowerApps.Checker.PowerShell](#)

The current restriction is that it does require Windows PowerShell in your project pipeline. If you're unable to meet this requirement, interacting directly with the [Power Checker Web API](#) might be a better approach.

There is a checker task in the Azure DevOps build tools include a checker task. More information about using that task in your build pipeline: [Quality check](#).

📌 Note

Microsoft.Xrm.Data.PowerShell [↗](#) module is another community supported tool that lets you easily manage solutions using PowerShell as part of your custom automation. See sample code [here](#) [↗](#). For example:

```
Export-CrmSolution "MySolution"
```

```
Export-CrmSolution -conn $conn -SolutionName "MySolution" -Managed -  
SolutionFilePath "C:\temp" -SolutionZipFileName "MySolution_Managed.zip"
```

```
Import-CrmSolutionAsync -SolutionFilePath c:\temp\mysolution.zip -  
ActivateWorkflows -OverwriteUnManagedCustomizations -MaxWaitTimeInSeconds 600
```

See also

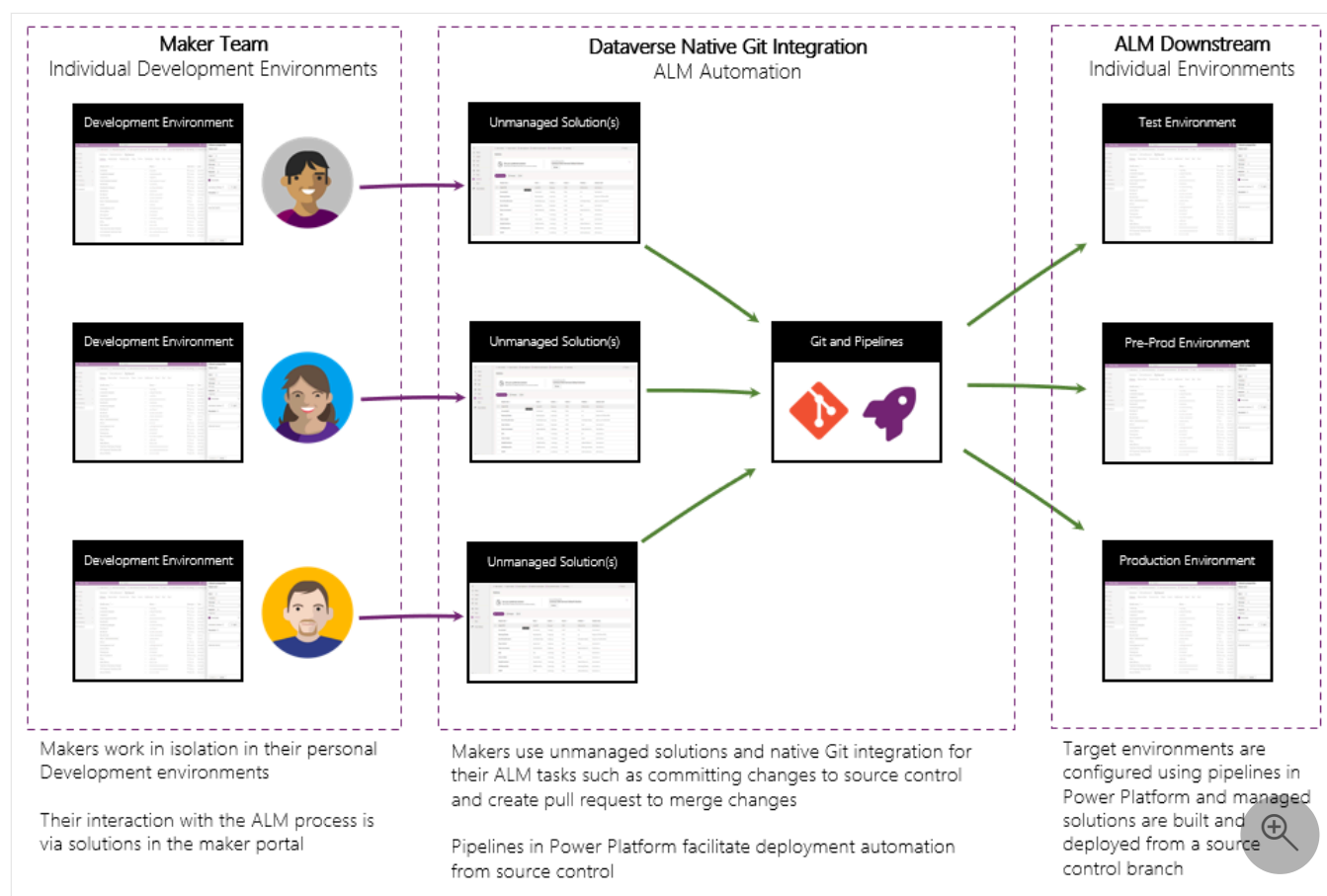
[PowerShell modules](#)

Overview of Git integration in Power Platform

Article • 04/22/2025

Source control integration allows development teams to sync solutions and solution objects across one or more Microsoft Dataverse environments using an Azure DevOps Git repository. The source control integration functionality is available natively within the solutions experiences, ensuring citizen developers, code-first developers, and administrators can benefit from version control, change tracking, and seamless team collaboration across different tools and environments. Git integration is intended to be used with developer environments and not in your test or production environments where deployments can be made using builds to create solution artifacts and pipelines in Power Platform to deploy.

In this article, you find some of the key concepts and benefits to using Git-enabled source control with your Dataverse environments and solutions. For information about Git in Azure DevOps, go to [Azure DevOps Git repository](#).



ALM in the Power Platform and Dataverse

Power Platform provides many out-of-the-box capabilities that enable organizations to manage application lifecycle management (ALM) for their solutions. Included are the ability to

package solutions as containers for the many different types of objects in the platform, manage environments involved in the application lifecycle, and deploy solutions using [pipelines in Power Platform](#). There are also several ways to integrate Git repositories with Power Platform using developer tooling. With native integration of Git in Dataverse, the process is simplified and streamlined for makers to work with their solutions in a familiar way and interact with source control through simplified interfaces in Power Apps (make.powerapps.com).

Benefits

- **Source control as source of truth:** Within some organizations, the source of truth for deployments in Dataverse is the maker environments in which solutions are built. The primary driver for this behavior is that the non-native Git integration uses advanced techniques and tools, which require professional IT expertise to get started. With the native integration of Git in Dataverse, source control can be enabled in only a few steps and provides a familiar interface for makers to work with their solutions.
- **Safety, auditing, and compliance using SDLC best practices:** Software development lifecycle (SDLC) best practices are a set of guidelines and processes that help you manage your software development projects effectively. By using Git integration in Dataverse, you follow SDLC practices such as version control, code reviews, and static source code analysis to ensure the quality, reliability, and security of your solutions. Git integration in Dataverse also provides features such as auditing, compliance, and traceability that help you track changes to your solutions and collaborate with other team members effectively.
- **Short-lived dev environments:** By storing a copy of your environments' customizations and configurations in source control, you can rehydrate development environments from source control quickly and easily in Dataverse. This allows you to create short-lived environments for development and testing purposes. Short-lived environments allow you to free up storage, experiment with new features, test, and iterate on your solutions without relying on permanent environments.
- **Fusion development teams:** Fusion development teams are teams that consist of both developers and makers who work together to build solutions. By using Git integration in Dataverse, these users can build independently in separate environments and collaborate with others by syncing with a common source control repository. Source control integration allows you to use the skills and expertise of both developers and makers to build high-quality solutions that meet the needs of your organization.
- **Protection:** Using source control as the source of truth for your solutions allows you to recover from unintended changes in your solutions quickly and easily. By storing your solutions in source control, you can restore to a previous state or version.

Key concepts

Unmanaged versus managed solutions

When you're using Git integration with Dataverse, solutions stored in source control come from unmanaged solutions in a maker's environment. Unmanaged solutions allow makers to add, remove, and update objects that are synced with source control when you commit and push changes. Managed solutions are built from source control and deployed into downstream environments, such as test or production, and aren't editable in those environments. Managed solutions are used to ensure that the source of truth for your solutions is always source control and that changes are only made in a maker's environment before they're added to source control and deployed elsewhere.

File formatting for solution objects

With the introduction of Git integration in Dataverse, there are changes to the way that solutions and solution objects are represented in source control. When you commit and push changes to source control, the solution objects are stored in a specific format that is compatible with Git. This format is used to represent the solution objects in a way that is easy to read and understand and can be used to track changes to the solution objects over time. The file format for solution objects is designed to be human-readable and can be used to view changes to the solution objects in source control. Additionally, to allow for multiple solutions to be stored in the same repository and folder, the solution objects in source control are no longer duplicated for each solution. Instead, the solution objects are stored in a single location and can be shared across multiple solutions in the same repository and folder.

Code-first development with Git

Code-first development in Power Platform is enabled using development tooling like the Power Platform CLI, Visual Studio, and Visual Studio Code extensions. Involving code-first developers in the solution development process is difficult without source control integration as objects like Power Apps component framework controls and Dataverse plug-ins are deployed to solutions as packaged assets built from source code and not directly editable in Power Apps (make.powerapps.com). Without source control as a part of the development process for both low-code and code-first objects, it's difficult to manage changes to the solution and ensure that changes are tracked and deployed in a controlled manner.

By enabling Git integration in Dataverse, you can meet code-first developers where they work and provide a seamless experience for both low-code and code-first developers. However, there are some considerations to keep in mind when managing code-first objects in a low-code environment.

Fusion development with Dataverse Git integration

Power Platform provides capabilities for both low-code and code-first development. This article discusses code-first development processes associated with Dataverse and Git integration and provides guidance about how to manage code-first and low-code objects in a single environment. Objects like Power Apps component framework controls, Dataverse plug-ins, and custom workflow activities are examples of code-first objects that can be managed in source control.

Code-first and low-code objects in a single environment

Code-first objects can be included in solutions through a build process that generates a managed or unmanaged solution that can be imported into a Dataverse environment. However, code-first objects can also be deployed directly into an unmanaged solution in a maker environment once they're built without employing the solution build process to deploy them. There's the build process to consider, given this flexibility.

If you're deploying code-first objects directly to an unmanaged solution in a maker environment, when those objects are committed to source control, only their compiled (built) version is stored in source control. For example, the binary DLL if a plug-in, or the transpiled and optimized bundle JavaScript for a Power Apps component framework control. As a result, you end up with two copies of the object in source control—one represented by the built version and the other represented by the source code. Storing binaries in your repository can lead to confusion and potential conflicts if the source code and the built version aren't kept in sync. This practice isn't recommended because the source code should be the single source of truth for the object and only a single copy should be stored.

The recommended approach is to build code-first objects as part of a solution build process and import the generated unmanaged solution into the maker environment. This approach ensures that the source code and the built version are kept in sync and that the source code is the single source of truth for the object. However, this approach requires that you have a build process in place to generate the managed or unmanaged solution for use in the import process and deployment process. You can create Azure Pipelines or GitHub workflows, for instance, that create artifacts for pipelines in Power Platform and for the Git sync processes to consume.

Next steps

[Dataverse Git integration setup](#)

Dataverse Git integration setup

Article • 04/22/2025

Git integration is initiated from Power Platform in the **Solutions** area within Power Apps, Copilot Studio, Power Automate, and Power Pages. This article discusses connecting environments and solutions to an Azure DevOps Git repository.

As the team develops, customizations within an environment can be committed to Git and changes made within Git can be pulled into connected environments.

Prerequisites for Git integration with Dataverse

- Dataverse Git integration is a feature of [Managed Environments](#). Development and target environments must be enabled as Managed Environments.
- An Azure DevOps subscription and licenses for users who interact with source control are required when connecting your Dataverse environment to Git.

ⓘ Note

A limited number of licenses are available in Azure DevOps in the Basic Plan of the service. Go to [Azure DevOps Services](#) [↗] for more information on pricing and licensing for Azure DevOps.

- Azure DevOps Services permission for making commits and reading source files is required. Members of the Contributors security group have permission.
- A Power Platform environment with Dataverse and the system administrator security role is required to connect to Git, which completes the initial binding of your environment or solution to a project in Azure DevOps.

Azure DevOps setup

To get started, you need an [Azure DevOps project and repository](#) to connect your Dataverse environment. If you don't have an Azure DevOps project and repository setup, follow these steps to create a new project and repository in Azure DevOps.

1. Sign in to [Azure DevOps](#) [↗] and select your organization.
2. Select **New project**, enter a name, and ensure that your version control is set to Git for your project, and select **Create**.

3. In the new project, select **Repos** and then select **Initialize** at the bottom of the page to initialize the default repository.

Make sure that all users who are making changes in your environment have access to your repository and to commit changes. Review your Azure DevOps licensing configuration and security groups to ensure that they're able to use the solution source control panel.

Connect to Git

Connecting to Git binds solutions in your environment to a specific location and branch within the Azure DevOps repository. You can bind your environment to a project using either environment or solution binding. To learn more about the differences between environment and solution binding, go to [How to choose between environment and solution binding](#).

1. Enable all development and target environments as Managed Environments within the [Power Platform admin center](#).
2. Sign in to [Power Apps](#) (or [Copilot Studio](#), [Power Automate](#), [Power Pages](#)) and then go to **Solutions**.
3. On the **Solutions** page, or the **Source control** page within a custom unmanaged solution, select **Connect to Git**.
4. Select either **Environment** or **Solution** from the connection type options, and select your Azure DevOps organization and project. For more information about the differences between environment and solution binding, go to [How to choose between environment and solution binding](#).
5. Depending on your binding type, you can select a project, repository, or branch and enter a folder to bind the environment or solution to.
6. Optionally, create a new branch.
7. Select **Connect**.

ⓘ Note

By design, the Default Solution and Common Data Service Default Solution can't be connected to Git. Use a custom solution for app and solution development.

How to choose between environment and solution binding

If you're not sure which binding strategy to use, environment binding is the recommended approach to get started with Git integration in Dataverse. Here are some key differences between environment and solution binding to help you decide which approach is right for you.

Environment binding

Environment binding is a single process for binding your entire Dataverse environment and all of the unmanaged solutions and objects in the environment to a single repository and folder. When you choose to bind the entire environment, all unmanaged customizations done in any custom unmanaged solution are stored in a single Git folder and branch and you don't need to set up any other solutions once selected. The system allows for multiple solutions to use a single root folder location and keeps track of which objects belong to each solution in a separate file. It's no longer a requirement to use a unique root folder for each solution. If you're using environment binding, you can select the repository, branch, and folder to bind the environment to during the initial setup.

Solution binding

Solution binding can be used to source control multiple solutions within the same environment to separate repositories or folders. Solution binding provides additional flexibility in your source files organization but requires more management on the part of the makers to onboard new solutions to source control. The system requires that there only be one location in source control for each and every solution object, which means that if you're going to choose to bind solutions to different source locations, the system doesn't allow you to have the same object in multiple solutions. Solution binding doesn't require a repository or solution to be selected during the initial setup. Instead, you select the repository and folder to bind each solution to when you're ready to bind the solution to source control.

Other considerations

If you select environment binding, all new unmanaged solutions are automatically synced with source code with no further actions, and if you select per solution binding, creation of a new solution later requires that the solution also be configured for source control before being synced to source control. Consider the safety of having all solutions synced versus the flexibility of having different folders and source code linkages for each solution.

Binding a solution to a repository and folder (when using the solution binding strategy)

Once you've connected your environment to Git using the solution binding strategy, you can bind a solution to a repository and folder in the same Azure DevOps project.

1. In the **Solutions** area, select the three vertical dots next to the solution you want to bind to source control, and then select **Connect to Git**.
2. Select an existing branch, or create a new branch, enter a Git folder to bind the solution to, and then select **Connect**.

Validate your connection

To validate your connection to Git, create a new solution or make changes to an existing solution and view the changes using these steps:

1. On the **Solutions** page, open the solution where you want to view changes.
2. On the left pane, select **Source control** to view the changes made to the solution.

Now you can view the changes made to the solution and can commit and push the changes to the repository as well as view the current branch your solution is bound to.

Disconnect from Git

Environments and solutions can be disconnected and then reconnected to different Git locations. With environment or solution binding, all solutions can be disconnected. With solution binding, you can also disconnect a specific solution.

Disconnect all solutions

1. On the Solutions page, select **Git connection** (environment binding) or **Connect to Git** (solution binding).
2. Select **Disconnect all solutions from Git**.
3. Select **Continue** on the confirmation dialog.

Disconnect a specific solution

1. On the **Solutions** or **Source control** page, select **Git connection**.
2. Select **Disconnect solution from Git**.
3. Select **Continue** on the confirmation dialog.

Connect multiple development environments to Git

Multiple development environments can be connected to the same Git location. This feature provides developer isolation with the ability to quickly push your changes to Git and pull others' changes into your environment.

 Important

Every environment must be connected with the same binding type, repository, branch, and Git folder.

1. Export the desired solutions as unmanaged from the original development environment, then import into the second environment.
2. Alternatively, create a solution in the second environment with the same exact name and publisher.
3. In other environments, repeat the [Connect to Git](#) steps described earlier. Use the same connection type (binding), organization, project, repository, branch, and Git Folder.

Now you can synchronize changes across multiple environments.

Next steps

After your environments and solutions are connected to Git, you can start making changes to your solutions and committing those changes to the repository. For more information about source control operations, go to [Source control repository operations \(preview\)](#).

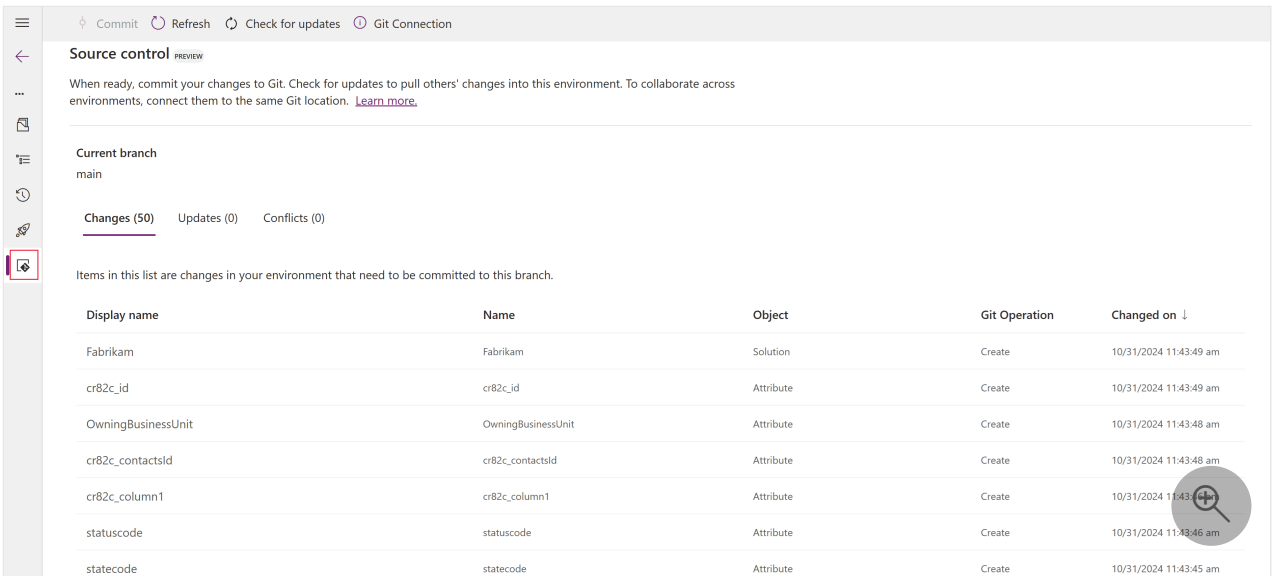
Source control repository operations

Article • 04/22/2025

After you connect to Git, your changes are tracked in Microsoft Dataverse. You can view a list of unmanaged changes on the **Source control** page within your unmanaged **Solution**. Objects that are part of the managed layer aren't included in the commit.

View changes in the solutions area

1. After making a change or adding an object in an unmanaged solution, go to the **Solutions** area in Power Apps.
2. Open the solution that you made changes to and select **Source Control** on the left navigation pane.



The source control screen provides a list of **Changes** available to commit from your environment to source control. Select **Refresh** to refresh the list if you made changes in another browser tab. Selecting **Check for updates** queries your source code repository and identifies whether there are **Updates** in Git available to pull into your environment. When detected, you need to resolve **Conflicts** detected between objects in your environment and Git.

Conflict resolution

The system detects if there are conflicting changes to your source repository. It lists these issues on the **conflicts** tab. You should resolve all conflicts before pulling or committing. Select each conflict listed, then choose to **keep existing changes** if you want to keep the changes originated in your environment or **accept incoming changes** if you want to accept the changes that were made in your source code repository.

If you choose to keep existing changes, those objects are moved to the **Changes** tab and are available to commit to your source code repository. If you choose to accept incoming changes, those objects are moved to the **Updates** tab and are available to pull into your environment. Making the selection in the **Conflicts** tab doesn't perform any immediate action with source control, but it resolves the conflict so that your next pull or commit action performs the changes that you selected without further conflict.

Pull changes available from source control repository

To import updates available from your source control repository into your environment, switch to the **Updates** tab and select **Check for updates**. Next, review the updates available, and then select **Pull** on the command bar. We recommend that you validate the application experiences that might have been impacted by those incoming changes.

If an object was deleted in Git, you're prompted to either remove the object from your solution (object remains in the environment) or delete the object from your environment. This is necessary before updates are pulled from Git.

Commit changes to source control repository

To commit the changes to source control, switch to the **Changes** tab and review the changes. Select **Commit** from the top menu, enter a comment about your commit, and then select **Commit** again.

Once your changes are committed, you can either navigate directly to Azure DevOps to view your commit or select the commit number provided on the information bar item that notifies you of your successful commit.

Note

It's a best practice to resolve your conflicts first, then pull any changes from source code into your environment, validate your application is still functional and operating as designed, and then commit your changes to source control. This practice ensures that source code commits are of the best quality.

Next steps

[Git integration setup](#)

Source control canvas apps (preview)

Article • 05/14/2025

We expect changes to this feature, so you shouldn't use it in production. Use it only in test and development environments.

Makers and developers use Git integration in Power Platform to manage canvas apps' source code effectively using version control. This feature helps you review changes made by makers in Power Apps Studio without encountering noisy diffs. Additionally, it allows for minimal edits to be made directly in the repository, ensuring a streamlined and efficient workflow.

How to commit canvas apps using Git integration

You can commit your canvas apps when you publish them. Changes aren't available to commit until you publish your app. This behavior ensures that the source code reflects the latest published version of your app, making it ready for use with Git integration.

Steps to commit canvas apps

1. Open Power Apps Studio and make changes to your canvas app.
2. Publish the app to make the changes available for Git integration.
3. Navigate to your Git repository and commit the changes.

Accessing source code files

Canvas apps' source code is stored in `.pa.yaml` files. These files are human-readable and designed for use with source control. When you use Git integration, the source code for canvas apps is automatically available in the repository without requiring any conversion. The source code is stored within the `canvas` folder in the repository.

File structure for canvas apps

The extracted source code files are located in the `\Src` folder and include:

- `App.pa.yaml`: Represents the app.
- `[screen Name].pa.yaml`: One file for each screen, representing the screen.
- `\Component`: A folder containing one file for each component, represented as `[component Name].pa.yaml`.

Only use `.pa.yaml` files within the `\Src` folder with source control. Don't use other files, such as JSON files, as source code because they aren't stable between save and load cycles.

Editing `.pa.yaml` files

You can do minor edits directly in the repository. Any changes are restored with the app when you pull changes to your environment. You must consider any code in your branch as a published app.

Best practices for editing

- Avoid making significant changes directly in the `.pa.yaml` files.
- Always test changes in a development environment before merging them into the main branch.

Known limitations

You can't edit the `.pa.yaml` files directly in your repository if your app contains:

- **Canvas components.**
- **Code components** (also known as PCF).

Editing files with these components prevent the app from running. If you change these files by mistake, you can load the previous version of your canvas app in the Maker portal.

Troubleshooting

This section contains common issues and tips to help you be more successful using Git integration with canvas apps.

Common issues

- **Merge Conflicts:** Resolve conflicts carefully to avoid corrupting the `.pa.yaml` files.
- **App Fails to Load:** Ensure no unsupported edits were made to the `.pa.yaml` files.

Tips

- Use a Git client with visual diff tools to review changes before committing.
- Regularly back up your repository to avoid data loss.

Next steps

- [Overview of Dataverse Git Integration](#)
- [View Source Code for Canvas Apps](#)

FAQs about source code integration

Article • 04/22/2025

This article includes answers to commonly asked questions about Git integration with Microsoft Dataverse.

What is source code integration in Power Platform?

Git integration provides an easier way of syncing Power Platform customizations (solutions and their objects) with a Git repository. The solutions and their objects are stored in human readable formats in the repository, which allows you to easily search, track changes, and manage your solutions in a source control system.

What's the difference between Git integration and pre-existing Developer tools?

Git integration is built into Power Platform and is more performant, democratized for makers and admins, has more robust safeguards, and uses an optimized file format. Developer tools currently provide more flexibility for code-first developers.

What are the prerequisites for using Git integration?

You need a Power Platform environment with the necessary permissions to connect to Azure DevOps and bind your environment to a project in Azure DevOps. It requires a system administrator role in the Dataverse environment to bind to Git.

You need an Azure DevOps subscription and licenses for users who interact with source control. These users need **Basic** access level in the Azure DevOps organization and access to contribute to the project and repository.

Development and target environments must be enabled as Managed Environments.

What other Git providers are supported?

Azure DevOps Git repositories are currently the only Git provider supported. This capability uses the Git provider model so that others might be supported later.

What licenses do I need to use Git integration with Dataverse?

All users within the environment must meet license requirements for [Managed Environments](#). Managed Environments is required regardless of environment type.

Developers using source code integration also need an Azure DevOps license to gain access to the repository. For more information, go to [Azure DevOps Services](#) 

Why am I getting the message "Failed to retrieve the default branch for the selected repository. Choose a default branch to allow creating new branches?"

Make sure that your Azure DevOps Git repo is initialized. New projects and repos by default have an uninitialized repository, and you have to manually initialize the repo to create the default branch.

Do I integrate my dev, test, and prod environments with source code and use branch merging to push configuration to another environment?

This feature is designed to only have your unmanaged solutions in development environments connected to Git. Deployment to upstream environments currently requires the managed solution from a development environment or using pipelines in Power Platform.

How do I work with branches?

The feature currently works with a single branch that you specify when you bind the environment or solutions to your repository. You can disconnect and reconnect to different branches as well as use Azure DevOps for pull requests (PRs), merging, and other advanced Git operations.

For example, you can connect your development to a developer branch, PR the changes into a feature branch, then disconnect your solution and reconnect to the feature branch to pull the latest changes into an environment.

Why isn't there unmanaged and managed solution representation for different objects in Git?

Before this feature became available, it was common to store managed and unmanaged solution versions in Git. That's no longer needed. Git should only include your source code and unmanaged customizations. Managed versus unmanaged is determined when building and releasing the solution.

Why are my source code files in YAML yet the solution export is still primarily XML?

The feature uses YAML to represent solution content because it's easier to read, understand, and facilitates easier merges.

How can I build and deploy a solution from source code?

Currently, deployment requires synchronizing the Git release branch with a development environment and exporting the managed artifact from the environment.

How do I develop with code-first objects where I don't want my compiled binaries checked into source code?

Currently, you can still author different objects, such as Power Apps component framework controls and plug-ins, using a different solution and Git folder.

How do I resolve conflicts?

Simple conflicts can be resolved on the solution's source control page by selecting which version to keep (environment versus Git).

Merge conflicts detected when merging branches is currently managed within Git.

Can I commit specific changes instead of all changes for a solution?

Currently, you need to commit all pending object changes within a solution. However, you can move objects to a different solution and add them back later.

Can I use an API to bind to Git and trigger commits and pulls?

Yes. Go to the [Web API reference](#) for developer documentation about connecting to Git, `CommitToGit`, `RefreshChangesFromGit`, and `PullChangesFromGit`.

Can I now make changes directly in source control in a supported manner?

Support for direct modifications to solution customizations isn't changing with this feature. Our recommendation is to continue the practice of making your changes directly in the environment, then committing those changes to source control. Changes to localized labels are permitted directly in source. Code-first objects that are supported using our developer tools are also supported, and include plug-ins, PCF controls, and web resources.

Can I commit large solutions?

Yes. However, there's a 17 MB limit for single file commits withing Azure DevOps. The system chunks large solutions, containing multiple files into multiple commits and squash-merges them.

Are all object types supported?

Currently, a small number of low-usage legacy object types are unsupported. You receive an error when unsupported object types are detected.

How can I upgrade existing solutions?

You can connect existing solutions in an environment to Git and commit them. If the solution is only in Git, first use developer tools to pack and import the unmanaged solution into a new development environment. We recommend a new source code location to avoid disruptive changes between old and new file formats.

Can I use Git integration to audit metadata changes? Even for citizen developers?

Yes, this is common. Dataverse API's can also be used to commit changes automatically.

Is Git integration available in sovereign clouds?

Not currently.

Can I automatically deploy changes pushed to Git?

Support is currently limited, but you can use pipelines in Power Platform and pac pipeline deploy.

Can I connect environments located in a different geo than the ADO repo?

Consent is needed when the environment is in a different geography than the ADO repo. A message and consent are shown when connecting.

Related content

[Overview of Dataverse Git integration](#)

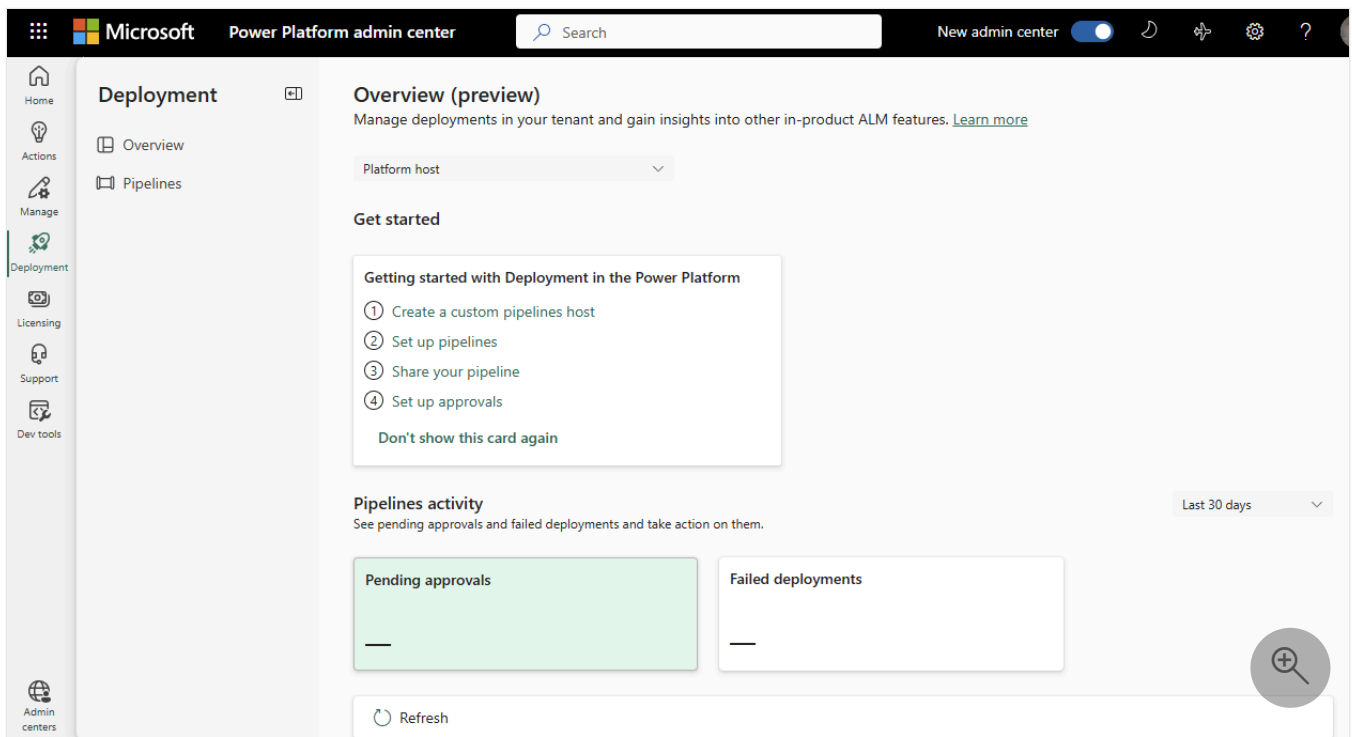
[Dataverse Git integration setup](#)

[Source control repository operations](#)

Admin deployment page

Article • 05/14/2025

The **Deployment** page in the Power Platform admin center provides a streamlined experience to help administrators navigate the complexities of managing Power Platform application lifecycle management (ALM) workloads, including managing pipelines deployments at enterprise scale. Admins have visibility to all the deployments in their tenant and can approve deployment requests and troubleshoot issues.



! Note

Currently the deployment page doesn't have all the capabilities available within the [Deployment Pipelines Configuration app](#).

Use the deployment page

1. Sign in to the [Power Platform admin center](#).
2. Enable **New admin center** in the upper right corner.
3. In the navigation pane, select **Deployment**.

Learn about the ALM process in Power Platform

The **Get started** section provides helpful learning content and guidance to set up deployments using best practices.

Manage deployments from one central location

Pipelines in Power Platform allow you to manage the end-to-end deployment process across your organization. The deployment page makes it easy to view all the pipelines and deployment activity within the tenant.

Admins can view all [pipelines host](#) environments in the tenant, including the platform host, and **select a host** to view all the pipelines and deployment history managed by that host.

A dedicated **Failed deployments** view helps admins quickly identify and troubleshoot failures. Select **Pipelines** on the left navigation, then **Run history** to view all deployments.

Review and approve deployment requests

On the deployment page, admins approve or reject deployment requests assigned to them. You'll first need to [setup delegated deployments](#) with service principals, which is recommended as the secure way to securely deploy to production environments.

It's important admins review changes in the solution and the sharing request. Once approved, the solution is deployed, and solution objects and security roles are shared automatically. Notice other types of approvals within the pipelines host environment can also be managed.

Retry failed deployments

Deployments shown as **Failed** in the run history view can be deployed by selecting **Retry** in the details panel if the operation was **Deploy**. A confirmation message appears when you confirm the retry.

Related articles

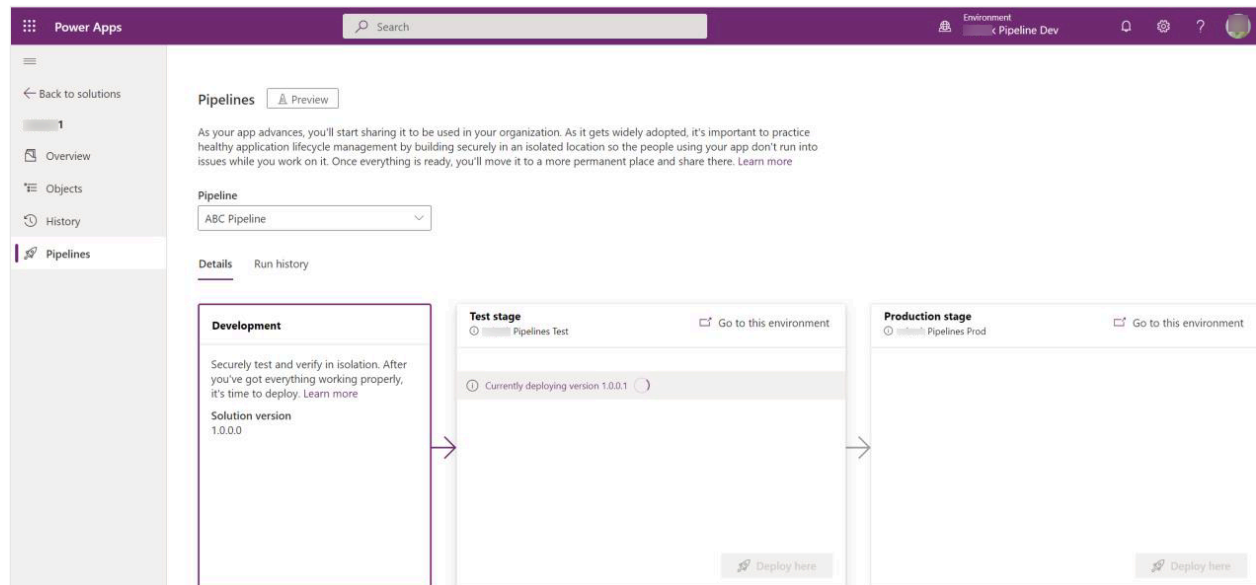
[Overview of pipelines in Power Platform](#)

[View solutions on the deployment page for makers](#)

Overview of pipelines in Power Platform

Article • 01/30/2025

Pipelines in Power Platform aim to democratize application lifecycle management (ALM) for Power Platform and Dynamics 365 customers by bringing ALM automation and continuous integration and continuous delivery (CI/CD) capabilities into the service in a manner that's more approachable for all makers, admins, and developers.



Pipelines significantly reduce the effort and domain knowledge previously required to realize the ROI from adopting healthy, automated ALM processes within your team or organization.

- Admins easily configure automated deployment pipelines in minutes rather than days or weeks.
- Makers have an intuitive user experience for easily deploying their solutions.
- Professional developers can (optionally) [extend pipelines](#) and run them using the Power Platform command line interface (CLI).

Admins centrally manage and govern pipelines

Pipelines enable admins to centrally govern citizen-led and pro-dev-led projects at scale with less effort. Admins set up the appropriate safeguards that govern and facilitate solution development, testing, and delivery across the organization. Other admin benefits include:

- Lower total cost of ownership:
 - Pipelines significantly improve maker, developer, and admin productivity.
 - Pipelines enable your business solutions to come to market faster, with higher

- quality, through a safe and governed process.
- Minimal effort to implement a secure and custom-tailored change management processes across your organization or team.
- Save time and money:
 - The system handles the heavy lifting and ongoing maintenance so you don't have to.
- Scale ALM at your own pace:
 - Regardless of where you're at in your ALM journey, you can extend pipelines to accommodate your evolving business needs. We aim for this upward transition to be as seamless and effortless as possible. More information: [Microsoft Power Platform CLI pac pipeline command group](#)
- Achieve compliance, safety, monitoring, and automation goals with:
 - Secure production environments with approval based [delegated deployments](#).
 - Customizations and audit log saved automatically and are easily accessible.
 - Out-of-the-box analytics provides better visibility within a central location.
 - The ability to view out-of-the-box Power BI reports within the pipelines app or create your own. More information: [Reporting overview for model-driven apps](#)
 - Custom tailor pipelines to the needs of your organization with [pipelines extensibility](#) and Power Automate.

Makers run preconfigured pipelines

Once pipelines are in place, makers can initiate in-product deployments with a few clicks. They do so directly within their development environments. Other benefits to makers include:

- No prior knowledge of ALM processes or systems required. Citizen developers often view pipelines as a guided change management process.
- Solution deployments are prevalidated against the target environment to prevent mistakes and improve success rates. For example, missing dependencies and other issues are detected before deployment and makers are immediately guided to take the appropriate action.
- Connections and environment variables are provided upfront and validated before the deployment begins.
 - This helps ensure applications and automation are deployed without needing manual post-processing steps, and are connected to the appropriate data sources within each environment.
 - Admins can even preconfigure certain connections that will be used.

Developers can use and extend pipelines

Professional developers are more productive with pipelines now handling the complex background operations. Developers can tell the system what they want to accomplish instead of executing the various underlying tasks necessary to accomplish the same goal. Using the Power Platform CLI, developers can:

- List pipelines to view pertinent details such as which stages and environments are ready to deploy their solutions to.
- Deploy a solution with a single command:
 - With pipelines, developers simply provide the required parameters and the system orchestrates all the end-to-end deployment operations in compliance with the organizational policies.
 - No need to connect to multiple environments, export solutions, download solution files, manually create connections and populate deployment settings files, import solutions, or handle various other tasks that were required previously.

Additionally, developers can [extend pipelines](#) and integrate with other CI/CD tools.

Frequently asked questions

What do pipelines deploy?

Pipelines deploy solutions as well as configuration for the target environment such as connections, connection references, and environment variables. Any Power Platform customization contained in your solution can be deployed using pipelines. Pipelines, or solutions in general, don't contain data stored within Dataverse tables.

Important

Power BI Dashboards (preview) and Power BI Datasets (preview) are not currently supported in pipelines.

Why can't I see my pipeline from my environment?

First, ensure that your source and target environments are linked properly. You'll only be able to view your pipeline in the assigned source environments, such as your development environments. When linking each of your environments to your pipeline during configuration, you have an option of **Development Environment** or **Target**

Environment environment type. If your pipeline-associated environments are assigned their proper type, your pipeline appears as an option in your source development environment.

Does pipelines automatically store solution backups?

Yes. Both managed and unmanaged solutions are automatically exported and stored in the pipelines host for every deployment.

Can customization bypass a deployment stage such as QA?

No. Solutions are exported as soon as a deployment request is submitted (when the maker selects **Deploy** from within their development environment), and the same solution artifact will be deployed. Similarly, the system doesn't re-export a solution for deployments to subsequent stages in a pipeline. The same solution artifact must pass through pipeline stages in sequential order. The system also prevents any tampering or modification to the exported solution artifact. This ensures customization can't bypass QA environments or your approval processes.

Are standalone licenses required to use pipelines?

- Developer environments aren't required to be Managed Environments. They can be used for development and testing with the developer plan.
- The pipelines host should be a production environment, but the pipelines host doesn't have to be a Managed Environment.
- All other environments used in pipelines must be enabled as Managed Environments.
- Licenses granting premium use rights are required for all Managed Environments.

A common setup example:

 Expand table

Environment purpose	Environment type	Standalone license required
Host	Production	No
Development	Developer	No
QA	Developer	No
Production	Production	Yes

Can I configure approvals for deployments?

Yes. See [delegated deployments](#).

Can I use different service principals for different pipelines and stages?

Yes. More information: [Deploy with a service principal](#)

What connections can be used?

Similar to authoring experiences, makers running pipelines can either provide their own connections or connections they have access to. Service principal connections can also be used for connectors that support service principal authentication, including custom connectors.

Why can't I update existing connection references?

Currently, connection references without a value in the solution or targeted environment can't be updated during deployment. If a value was deployed previously, it can be updated in the targeted environment.

Who owns deployed solution objects?

The deploying identity. For standard deployments, the owner is the requesting maker. For delegated deployments, the owner is the delegated service principal or user.

Can pipelines deploy to a different tenant?

No. We recommend using Azure DevOps or GitHub for this scenario.

What should I do if my development or target environment is reset or deleted?

You should delete the environment record and update the pipeline configuration when needed. If an environment is reset, you re-create the environment record then associate it with your pipeline.

Can I use pipelines in the default environment?

Yes. However, using the default environment as the pipelines host isn't recommended for all customers.

Can I deploy using my own service principal?

Yes. More information: [Deploy pipelines as a service principal or pipeline owner](#).

Can pipelines be used with Azure DevOps, GitHub, or the ALM Accelerator?

Yes, together these tools are powerful while keeping maker experiences simple. More information: [extend pipelines](#)

Can I roll back to a previous version?

Yes. If the pipeline setting is enabled, you can [redeploy previous solution versions](#) from the run history view on the Pipelines page. If the setting is disabled, only higher solution versions can be deployed or imported. As a work-around, admins can download the artifact from the pipelines host, increment the solution version in the solution.xml file, then manually import it into the target environment.

Can I set retention policies for pipelines data?

Yes. You can configure bulk delete jobs in the Dataverse pipelines host to delete data on a defined schedule.

Can I specify advanced solution import behaviors such as update versus upgrade?

Not currently. Pipelines default import behavior is *Upgrade* without *Overwrite customizations*.

Can an environment be associated with multiple hosts?

No. However, one environment can be linked to multiple pipelines within the same host. In order to associate an environment with a different host, add it to a pipeline in the new host. Then delete the environment record from the original host and verify everything works as expected.

Can I customize or extend the first-party deployment pipeline app and tables?

Not currently. However, intentional extension hooks are available to customize pipelines logic. More information: [extend pipelines](#).

Where can I view and run pipelines?

Navigate to an unmanaged solution in development to an environment associated with your pipeline. Pipelines can't be viewed or run from the default solution, managed solutions, or in target environments. Notice you can also retrieve and run pipelines from the Power Platform CLI.

Can I deploy across regions?

Yes, but only if the [Cross-Geo Solution Deployments](#) setting is enabled in the host. If the setting is disabled, the host and all environments associated with pipelines in a host must be located within the same geographic location (as specified when creating environments). For example, if the setting is disabled, a pipeline can't deploy from Germany to Canada and a host in Germany can't manage environments in Canada. In a case where the tenant administrator would like to prevent cross-geo solution deployments, separate hosts should be used for Germany and Canada.

Can I deploy the same solution using different pipelines?

Yes, this is possible, although we recommend starting with the same pipeline for a given solution. This helps avoid confusion and inadvertent mistakes. Pipeline run information is displayed in the context of one pipeline and one solution (within the solution experience). Therefore other pipelines might not show the latest deployed solution version or other important run information associated with different pipelines. Notice that the Deployment Pipeline Configuration app shows run information across all pipelines and all solutions for the current host.

Can the host environment also be used as a development or target environment?

Using the same environment for development and the host isn't supported; other combinations aren't recommended as a best practice.

How can I view what changed between different versions?

Within the target environment, you can see layers of deployed objects as well as what changed between layers. Additionally, you can see XML diffs between layers for model-driven apps, site maps, and forms. Pipelines can also be extended to integrate with GitHub and other source control systems to compare granular diffs.

Should my host environment be the same as where I installed the COE toolkit?

This is a valid configuration and should be evaluated based on the needs and policies within your organization.

Can I deploy unmanaged solutions?

No. We recommend that you always deploy managed solutions to nondevelopment environments. Notice unmanaged solutions are automatically exported and stored in the pipelines host so you can download and import them to other development environments or put them in source control.

Can I deploy multiple solutions at once?

Not currently. You'll need to submit a different deployment for each solution. However, the same pipeline can be used for multiple solutions.

Do pipelines publish unmanaged customizations before exporting the solution?

Not currently. We recommend you publish individual objects as they're saved. Note that only certain solution objects require publishing.

Can I use pipelines for multi-developer teams working in isolated development environments?

The current implementation uses a single development environment for a given solution.

How are pipelines different from the ALM Accelerator?

Both offer many valuable capabilities and the owning teams work together closely in developing the pipelines and broader ALM vision for Power Platform. Pipelines are more simplistic in nature and can be set up and managed with less effort. Access to other products and technologies isn't required as everything is managed in-house. The ALM Accelerator, on the other hand, is sometimes a better fit for more advanced ALM scenarios.

While there are many additional functional differences, the fundamental difference is that pipelines are an official Microsoft Power Platform product feature—meaning it's designed, architected, engineered, tested, maintained, and supported by Microsoft product engineering. Pipelines are built into the product and can be accessed within native product experiences.

When should I use pipelines versus another tool?

We encourage customers to use pipelines for core deployment functionality, and when needed, extend pipelines to integrate with other CI/CD tools. When used together, the workloads required within CI/CD tools often become less complicated and costly to maintain.

Next steps

[Set up pipelines](#) [Extend pipelines](#)

Related information

[Deploy solutions using Pipeline in Power Apps \(video\)](#) [↗](#)

Feedback

Was this page helpful?

[👍 Yes](#)

[👎 No](#)

[Provide product feedback](#) [↗](#)

Set up pipelines in Power Platform

Article • 05/14/2025

Create and run pipelines to easily deploy solutions to environments. There are two different ways to set up pipelines:

- [Platform host](#). The default tenant-wide platform host, which can be configured by makers.
- [Custom host](#). Admins configure a custom host to centrally govern citizen-led and pro-dev-led projects.

Information within each section of this article pertains to the specified host method for setting up pipelines.

Frequently asked questions

Will personal pipelines conflict with any pipelines that I have already set up?

No. Thanks to the host separation dynamic that we have in place, there's no way for a maker creating a personal pipeline (in the platform host) to associate an environment that is already associated with a custom host. By default, makers don't have permissions to create lightweight personal pipelines in environments already associated with a custom host. This means that your current pipelines UX, if in place, won't change.

Important

Makers also don't receive elevated access to environments as a result of this feature. Selectable target environments are filtered to include only environments that a maker can already import to. This feature ensures that all personal pipelines are stored in the platform host that is accessible to administrators, and provides an easier way for makers to self-service their application lifecycle management (ALM).

Why can't I select or view certain environments when I create a pipeline?

The target environment picker filters out any environments that:

- The current user doesn't have import-access to, or are outside of the geographical region that the pipelines host is located in if host-wide setting is disabled. More information: [Enable cross-geo solution deployments](#)

You also can't create a pipeline with a target environment that's already associated to the host as a development environment. To change an environment's type distinction in a host, you must [play the Deployment Pipeline Configuration app](#), delete the environment record, and re-create the environment record with the desired type.

Why am I seeing an error that states "this environment is already associated with another pipelines host?"

This error indicates that another host already contains an active environment record that you're trying to associate with the current host. To resolve this, go to [Using Force Link to associate an environment with a new host](#) or [Disassociating environments from one host and associating them with another host](#).

Do the pipelines and data within the platform host count towards my Dataverse capacity?

No. The data consumption in the *platform host* doesn't count against your current plan since the pipelines data for the platform host is stored in Power Platform infrastructure. This data is stored within your tenant and accessible by administrators, but due to its implementation details, doesn't consume data capacity within a plan.

However, capacity does apply to a *custom host*, which isn't an implementation in the platform but is instead in a customizable environment.

Can I enable makers to create personal pipelines in a custom host?

Yes. As an administrator, you can assign the **Deployment Pipeline Default** role to anyone you want to grant lightweight pipeline creation permissions to. Administrators can also add users to the **Deployment Pipeline Maker** team via the **Security Teams** page in the Deployment Pipeline Configuration app.

This Deployment Pipelines Default role isn't assigned to anyone by default in the case of custom host, so the lightweight personal pipeline creation experience is only visible by default in environments that aren't already associated with a custom host.

As an admin, how do I prevent makers from creating personal pipelines by default?

Because custom hosts don't grant pipeline create-access by default like the platform host does. You can [set up a custom host](#) and then [use force link](#), if necessary, to associate development environments with a custom host.

1. If there's already a custom host available skip this step. If not, you have to create one following the steps to [create a pipeline using a custom pipelines host](#).
2. Once there's a custom host available, as an admin, navigate to the Deployment Pipeline Configuration app for the custom host. The app is located in the environment that you installed the Power Platform Pipelines package in.
3. Go to **Environments** from the side navigation pane, and **create new environment record(s)** for the development environments that you would like to prevent makers from creating new personal pipelines from. If the environment was already linked to another host, such as the platform host, the validation fails. If this occurs, select **Force Link** on the command bar after validation failure to override the current link to the other pipelines host.

Following these steps effectively disables the **create pipeline** capability for any makers who access in pipelines feature in these development environments because they don't have pipelines permissions. Existing pipelines in the custom host, if any, are also not shared with any users by default. Admins are able to apply with workaround with any existing custom host as well.

Why am I not seeing the latest features for pipelines?

The pipelines package is always being updated to give you the latest and greatest for your ALM processes. Ensure that you have the latest Power Platform pipelines package in your **custom host**:

1. Go to the [Power Platform admin center](#) [↗],
2. Select your pipelines host environment.
3. Select **Dynamics 365 apps**, and locate **Power Platform Pipelines**. Notice if there's an update available.

For **platform hosts**, the pipelines package is updated automatically, and might not be available as soon as the manual package update is made available for custom hosts.

How can I recover unmanaged solutions from Power Platform Pipelines past deployment history?

The [Import Solutions from a Pipelines Host](#) feature in Power Platform allows users to recover unmanaged and managed solutions from past deployments. This capability is particularly useful in scenarios where a solution is accidentally lost or needs to be restored to a

development environment. Alternatively, Pipelines admins can also use the pipeline configuration management app to download the unmanaged solution from the deployment history record easily.

Next steps

- [Extend pipelines in Power Platform](#)
- [Run pipelines in Power Platform](#)

Create a personal pipeline using the platform host

Article • 01/06/2025

Pipelines are available to all who navigate to the pipelines page for the first time. Create personal pipelines from your development environments to other target environments that you have import access to. Apply the benefits of pipelines such as no downloads or uploads required, deployment scheduling, inline target environment solution validation, upfront connection reference and environment variable value validation, and Copilot-generated deployment notes with an easy setup process.

Prerequisites for personal pipelines using the platform host

- Three environments can be associated to a personal pipeline created from the **Pipelines** page: one current development environment and two target environments. More information: [Add a stage to a single-stage pipeline you own](#).

ⓘ Note

Default environments, Teams environments, and environments without a Microsoft Dataverse store can't be used as target environments for personal pipelines.

- All environments used in pipelines must have a Dataverse database.
- All target environments used in a pipeline must be enabled as [Managed Environments](#).

Current limitations with personal pipelines using the platform host

- Personal pipelines can't be extended.
- Only three environments, a source, and two subsequent targets, can be configured.
- Personal pipelines can't be shared with other users.

Setting up the platform host for the first time

If you're the first user in your tenant to navigate to the **Pipelines** page in an environment not already associated with a custom pipelines host, the platform host is provisioned for everyone in the tenant to use.

ⓘ Note

The platform host is always created in the tenant's home geographical region. If you would like to create pipelines using the platform host that include environments not present in this geographical region, you must [enable cross-geo solution deployments](#) as a tenant administrator.

Create a pipeline in Power Apps

First, ensure that you are in the development environment where you want your pipeline to start.

Select **Create pipeline** on the command bar or from the **Pipeline** dropdown list.

A panel opens, where you enter:

- **Pipeline name:** A name for the pipeline.
- **Description:** An optional description to describe what the pipeline is used for.
- **Target environment:** The environment that solutions in the current development environment are deployed to.

ⓘ Note

The **Target environments** list is filtered based on import access and geographical region (if the [cross-geo solution deployments](#) setting is disabled).

Once you select **Save**, the current (source) environment and the selected target environment are associated with the platform host, and environment records for each are created in the platform host. Then, the newly created pipeline is autoselected and you're ready to [run the pipeline](#).

ⓘ Important

- Environments can only be associated with one pipelines host. If you would like to use a governed pipeline with advanced extensibility, shareability, and more than two environments, ensure that environment records in the platform host

are deleted within the **Manage pipelines** experience so they are freed up to be associated with a custom host. Deleting a pipeline doesn't delete the environment records linked to that pipeline.

- Environments can only be classified as either development or target, and once set, this classification can only be changed by an admin in the **Manage pipelines** experience by deleting or re-creating the environment record.

Add a stage to a single-stage pipeline you own

Select **Add stage** on the command bar to append a stage to the current single-stage pipeline. As part of this step, you're prompted for a stage name, description, and a final target environment for your pipeline.

ⓘ Note

- You must be the owner of the current pipeline to add a stage.
- The add stage capability is limited to single-stage pipelines, but administrators can add more stages in the [Deployment Pipeline Configuration app](#).

Delete a pipeline you own from Power Apps

Select **Delete** on the command bar to delete the currently selected pipeline in the dropdown list. A confirmation message appears regarding your intent and knowledge of the consequence.

This action doesn't remove the run history for the pipeline nor does it delete the associated environment records from the host.

Use your personal pipeline with other development environments

If you have the same pipeline process for an additional development environment, such as your user acceptance testing (UAT) and production environments are the same for another source environment, you can reuse the pipeline that you already created.

Choose the created pipeline from the dropdown menu. When you select **Next** in the first step of the deployment configuration process, the current environment is linked to

the pipeline (and the host) as a development environment.

Accessing the Deployment Pipeline Configuration app from Power Apps

From the **Pipelines** page within any solution, the **Manage pipelines** button in the command bar navigates to the configuration app of the pipelines host that is associated with the current environment:

- If the current environment is associated with a custom pipelines host, the button links to the Deployment Pipeline Configuration app within the dedicated Host environment.
- If the current environment is associated with the platform host, the button links to an embedded Deployment Pipeline Configuration app within Power Apps.

The Deployment Pipeline Configuration app can be accessed by anyone with the Deployment Pipeline Administrator role if using a custom host, and any tenant administrator for the app associated with the Platform host.

Assigning pipeline roles via Security Teams in the Deployment Pipeline Configuration app

As a **Deployment Pipeline Administrator**, you can assign out-of-the-box pipelines roles easily in the **Deployment Pipeline Configuration** app.

1. Play the **Deployment Pipeline Configuration** app.
2. Select **Security Teams** under **Settings** in the left navigation pane.

There are three teams:

- **Deployment Pipeline Administrators.** These users have full access to all pipelines and can play the Deployment Pipeline Configuration app.
- **Deployment Pipeline Makers.** These users can create and consume personal pipelines that are managed in the custom host. If a maker needs to use a shared pipeline, we recommend that the user is assigned the **Deployment Pipeline Users** role to gain access to shared artifacts and shared stage runs.
- **Deployment Pipeline Users.** These users can trigger a pipeline that is shared with them. Deployment Pipeline Users also have read access to all deployment stage runs within the same business unit.

Frequently asked questions

When I navigate to "Manage pipelines", why am I receiving a privilege error when I'm a system administrator or why is it loading the app indefinitely?

Sometimes, when another user authentication cookie is present in your browser, the embedded model-driven app experience can display an error message related to privileges to access the app. In this case, clear your web browser's cookies or open the app in an InPrivate window in your web browser to ensure that your authorization token is the only one considered.

Next steps

- [Run pipelines in Power Platform](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Create a pipeline using a custom pipelines host

Article • 08/21/2024

Power Platform administrators can create one or more pipelines, associate any number of environments, then share access with those people that administer or run pipelines.

Prerequisites for custom pipelines host

- Four environments are recommended, but you can use as few as three Power Platform environments to create a pipeline.
- All environments used in pipelines must have a Microsoft Dataverse database.
- You must have a Power Platform administrator or Dataverse system administrator role to install the pipelines application.
- All target environments used in a pipeline must be enabled as [Managed Environments](#).

Create or choose environments for pipelines

Before you begin, you need to identify which environments participate in pipelines. Be sure to enable all target environments as [Managed Environments](#). Managed Environments isn't required for the pipelines host or developer environments.

- **Host environment.** This special-purpose environment acts as the **storage** and management plane for all pipeline configuration, security, and run history.
 - We recommend you keep this setting as a **dedicated production environment** separate from the development and target environments.
 - If desired, you can configure multiple different hosts within a tenant, such as for separately managing pipelines for different business organizations or geographic locations. One host for the entire tenant is also acceptable as long as all environments are in the same geographic location.

Important

Deleting the host environment deletes all pipelines and run data. Use caution and understand the impact of data and configuration loss as well as maker access to pipelines hosted in the environment.

- **Development environment.** This environment is where you develop solutions. A pipeline can be run from within any development environment linked to it.
- **Target environment.** The destination environment a pipeline deploys to. For example, integration testing, user acceptance testing (UAT), production, and so on.

Tip

Use environment names that indicate their purpose. For example, *Contoso Host*, *Contoso Development*, *Contoso QA*, and so forth.

Install the pipelines application in your host environment

This step is only required for the initial host setup. You might skip to the next section if you already have access to a host environment where you'll create pipelines.

1. Sign in to the [Power Platform admin center](#), go to **Environments** > **New**, and create a new environment with a Dataverse database. Be sure to choose the same region that your development, QA, and production environments are created in.
2. Install the **Power Platform Pipelines** application in your host environment by selecting the host environment, then select **Resources** > **Dynamics 365 apps**.
3. Select **Install app** and scroll down within the right-side panel until you find **Power Platform Pipelines**.
4. Select **Next**, if you agree, accept the terms, and then select **Install**.

Important

You only need to install the deployment pipelines application in the host environment. You don't need to install it in other environments, such as development, QA or production environments that will be associated with your pipelines.

Once installed, the deployment pipelines configuration application appears in the list of installed apps.

Configure a deployment pipeline

1. Copy the environment IDs of all development and target [Managed Environments](#) that are linked to pipelines. You need these IDs later. More information: [Find your environment and organization ID](#)
2. Once the **Deployment Pipeline** package installation is completed, go to [Power Apps](#), then select the dedicated host environment (where you installed the application).
3. Play the **Deployment Pipeline Configuration** app.
4. Select **Environments** on the left pane, and then select **New** to create the environment records in Dataverse:
 - **Name:** Enter the name for the environment record. It's a good idea to include the same name as the actual environment, such as **Contoso Dev**.
 - **Environment Type:** Select either **Development Environment** or **Target Environment**. Select **Development Environment** for all source environments where unmanaged solutions will be developed. Select **Target Environment** for QA and production environments where the managed solution artifacts are deployed.
 - **Environment Id:** Be sure to select the correct ID. You can find the environment ID for the current environment within make.powerapps.com. Select the **Settings** icon on the command bar, and then select **Developer resources**. More information: [Find your environment and organization ID](#)
5. Select **Save**.
6. Refresh the form, then verify **Validation Status** equals **Success**.
7. Repeat steps 4-6 until all environments that participate in the pipeline have environment records created.
8. Select **Pipelines** on the left navigation pane, and then select **New** to create a new deployment pipeline:
 - **Name:** Enter a name for the pipeline, such as *Contoso Standard Deployment Pipeline* or *Contoso Hotfix Pipeline*.

- **Description:** Optionally, enter a meaningful description for the pipeline.

Pipelines is only supported in Managed Environments.

Home Recent Pinned Overview Pipelines Dashboard Pipeline Setup Environments Pipelines

← Save Save & Close + New Flow

New Deployment Pipeline - Unsaved

General Deployment stages Run history

Pipeline Details

Name * Contoso Standard Deployment Pipeline

Description To be used for all deployments except when requesting a hotfix deployment.

Owner * (Available)

9. Within the **Linked Development Environments** grid, select **Add Existing Development Environment**, then associate one or more development environments. A pipeline must have at least one development environment and one stage before it can run.

Home Recent Pinned Overview Pipelines Dashboard Pipeline Setup Environments Pipelines

Contoso Standard Deployment Pipeline - Saved

Deployment Pipeline

General Deployment stages Run history Related

Linked Development Environments

Name	Environment Type	Environment ID	Validation Status
No Rows To Show			

+ New Development Environment

Add Existing Deployment Environment Refresh Flow

10. Within the **Deployment Stages** grid, select **New Deployment Stage**, to display the quick create pane.

Power Apps Deployment Pipeline Configuration

Home Recent Pinned Overview Pipelines Dashboard Pipeline Setup Environments Pipelines

Contoso Standard Deployment Pipeline - Saved

Deployment Pipeline

General Deployment stages Run history Related

To enable this content, save the record.

Linked Development Environments

Name	Environment Type	Environment ID
Contoso Standard Deployment Pipeline (Dev)	Development Environment	Contoso Standard Deployment Pipeline (Dev)

1 - 1 of 1

Deployment Stages (Deployment Pipeline)

Name	Deployment Pipeline	Previous Deployment Stage
No Rows To Show		

0 - 0 of 0

Quick Create: Deployment Stage

Deployment Pipeline Contoso Standard Deployment Pipeline

Name * Deploy to QA

Description Deploys solution from development to QA.

Previous Deployment Stage ---

Target Deployment Environment ID Test Environment

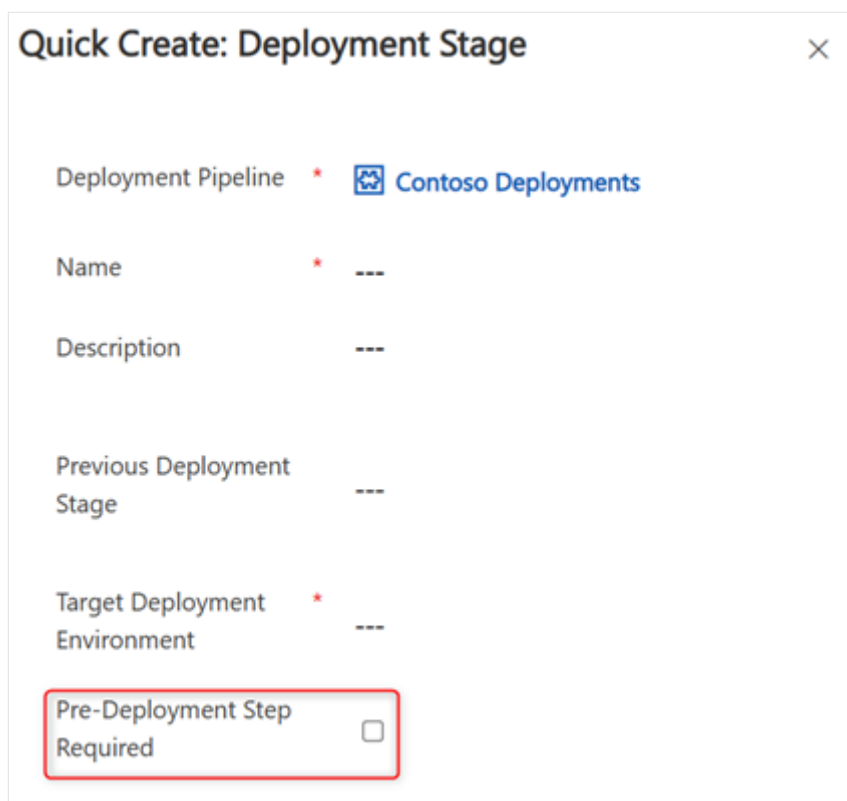
Save and Close Cancel

11. Enter the details for each stage, and then select **Save and Close**:

- **Name:** The name of the stage.
- **Description** (optional): Optional description for the stage.

- **Previous Deployment Stage** (optional): Specifies a deployment stage that must be deployed to before deploying to the current stage. For example, when creating a production stage, you can add the test stage as the **Previous Deployment Stage**. For the first stage, or pipelines containing only one stage, this stage should be left blank.
- **Target Deployment Environment**: This is the target environment where this stage deploys to.
- **PreDeployment Step Required** (optional): Deployment requests are pending until approved via custom business logic. Requires additional configuration. More information: [Extend pipelines in Power Platform](#)

12. Repeat the previous two steps for each stage you'd like to add to the pipeline. You must have at least one stage. You can add up to seven stages.



Quick Create: Deployment Stage [Close]

Deployment Pipeline * Contoso Deployments

Name * ---

Description ---

Previous Deployment Stage ---

Target Deployment Environment * ---

Pre-Deployment Step Required ☐

Grant access to edit or run pipelines

Pipeline security is managed within the host environment by assigning security roles. Additionally, users must currently have access to all environments associated with a pipeline in order to create or run the pipeline.

When you install the Power Platform pipelines application, two security roles are added:

- **Deployment Pipeline User**: Has privileges to run pipelines that have been shared with them.

- **Deployment Pipeline Administrator:** Has full control over all pipeline configuration, without needing system administrator security role membership.

Share pipelines with makers

The **Deployment Pipeline User** security role grants access to run one or more pipelines. It doesn't grant access to create, edit, or delete pipelines. Users with the Deployment Pipeline User security role can't view the host environment within the environment picker in Power Apps or Power Automate, or otherwise need to be aware of it.

To share pipelines:

- Assign makers the **Deployment Pipeline User** security role within the host environment. This security role is installed with the Power Platform pipelines application. More information: [Assign a security role to a user](#)
- In the deployment pipeline configuration app, **Share** the pipeline record with makers (or Microsoft Entra groups). **Read** privilege is sufficient to view and run a pipeline from within the development environment. More information: [Share rows with a user or team](#)
- Makers must also have privileges to export solutions from the source development environments, as well as privileges to import solutions to the target test and production environments for which the pipeline deploys to. By default, system customizer and environment maker roles have these privileges.

Share with pipeline administrators

The Deployment Pipeline Administrator security role grants full privileges to all pipelines (and tables, which store pipeline information) within the host environment. The role also grants access to run all pipelines associated with the current host. The Deployment Pipeline Administrator security role doesn't grant privileges to perform other activities within the host environment.

Important

Pipeline administrators must also have access to all development, test, and production environments that are associated with pipelines they create or run.

Assign the **Deployment Pipeline Administrator** security role to users or Microsoft Entra groups within the host environment. This security role is installed with the **Power Platform Pipelines** application. More information: [Assign a security role to a user](#)

Assigning pipeline roles via Security Teams in the Deployment Pipeline Configuration app

As a **Deployment Pipeline Administrator**, you can assign out-of-the-box Pipelines roles easily in the **Deployment Pipeline Configuration** app:

1. Play the **Deployment Pipeline Configuration** app.
2. Select **Security Teams** under **Settings** in the left navigation pane.

There are three teams:

- **Deployment Pipeline Administrators.** These users have full access to all pipelines and can play the Deployment Pipeline Configuration app.
- **Deployment Pipeline Makers.** These users can create and consume personal pipelines that are managed in the custom host. If a maker needs to use a shared pipeline, we recommend that the user is assigned the Deployment Pipeline Users role to gain access to shared artifacts and shared stage runs.
- **Deployment Pipeline Users.** These users can trigger a pipeline that is shared with them. Deployment Pipeline Users also have read access to all deployment stage runs within the same business unit.

Using a pipeline with other development environments that aren't already associated

If a user has **write-access** on a given pipeline, they're able to associate other development environments with that pipeline in the Pipelines experience.

Choose the created pipeline from the dropdown menu. When you select **Next** in the first step of the deployment configuration process, the current environment is linked to the pipeline (and the host, if not already) as a development environment.

This is useful if you have the same pipeline process for an additional development environment. For example, your UAT and production environments are the same for another source environment. You can reuse the pipeline you already created or have been given write-access to.

Centrally manage and monitor deployments

The pipeline configuration app and host environment provide many other benefits such as:

- Centrally view all deployment activity. Filtered views are also accessible within the maker experience for a given pipeline and solution.
- Audit where a certain solution version is deployed as well as who initiated the request.
- View error logs, validation results, and deployment settings. Environment variables and connections are provided during deployment.
- Retain backups of all solution artifacts by version.
- Schedule bulk delete jobs to remove unwanted data and conserve database capacity. More information: [Remove a large amount of specific, targeted data with bulk deletion](#)
- Use a dashboard to visualize deployment metrics. You might also build your own reports using data stored within the host.

Accessing the Deployment Pipeline Configuration app from Power Apps

From the **Pipelines** page within any solution, the **Manage pipelines** command on the command bar navigates to the configuration app of the pipelines host that is associated with the current environment:

1. If the current environment is associated with a custom pipelines host, the button links to the Deployment Pipeline Configuration app within the dedicated Host environment.
2. If the current environment is associated with the Platform host, the button links to an embedded Deployment Pipeline Configuration app within Power Apps.

The Deployment Pipeline Configuration app can be accessed by anyone with the Deployment Pipeline Administrator role if using a custom host, and any tenant administrator for the app associated with the platform host.

Manually disassociating environments from one host and associating them with another host

1. From Power Apps (make.powerapps.com), begin in an environment that you want to disassociate, and select **Solutions**.
2. Choose any solution that you have access to, and select into it.
3. Navigate to the **Pipelines** page from the left navigation pane, and then select **Manage pipelines** on the command bar. If you're beginning with a custom host, you can also go directly to the dedicated host environment and run the **Deployment Pipeline Configuration** app from there.

4. Now that you are in the **Deployment Pipeline Configuration** app, go to **Environments** from the left navigation pane.
5. In the **Added Deployment Environments** table shown by default, select the environment records for the environments that you want to disassociate from this host.
6. Select **Delete** on the command bar, and confirm deletion.
7. For associating the deleted environments to the new host, repeat steps 1-4 but in the context of the desired host to associate the environments with.
8. In the **Added Deployment Environments** table shown by default, select **New** on the command bar.
9. Create the environment record by assigning a name, type, environment ID, and optional description.

Now you're able to link the environments to pipelines in the new host.

Using Force Link to associate an environment with a new host

After creating an environment record in the Deployment Pipeline Configuration app, you might encounter an error message indicating "this environment is already associated with another pipelines host." To take over the association, resulting in delinking the environment in the previous host and a successful link to the new host, select **Force Link** on the command bar.

Important

- Makers lose access to any pipelines in the previous host that were accessible within that environment when you perform this action on a development (source) environment.
- This action can be undone by performing a **Force Link** in the previous host.
- The environment record in the previous host will have a stale validation status until it's updated, so it will show as **Succeeded** unless it's updated.

Next steps

- [Extend pipelines in Power Platform](#)
- [Run pipelines in Power Platform](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Extend pipelines in Power Platform

Article • 03/18/2024

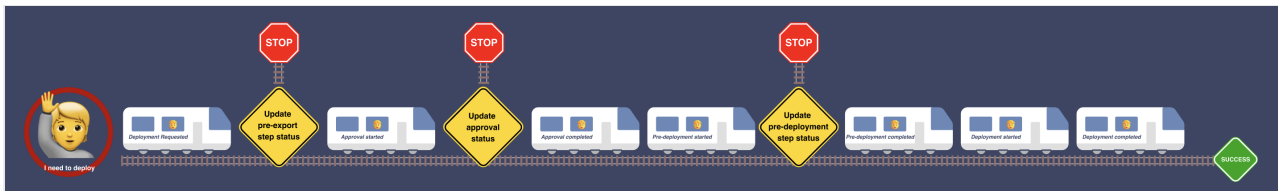
Pipelines can be custom tailored to serve the unique needs of an organization. For example, you can add approvals, deploy via service principals, and integrate with internal systems of record, Azure DevOps, GitHub, and much more. Because [Microsoft Dataverse business events](#) are used, business logic can be executed within Power Automate or various other subscribers. Whether your pipeline is basic or sophisticated on the inside, the deployment experience remains simple for makers on the outside.

Important

- New pipelines extensions are being gradually rolled out across regions and might not be available yet in your region.
- Existing pipelines customers may need to update the Power Platform pipelines application to enable new features. You can manage the updates within the Power Platform admin center.
- Personal pipelines created within Power Apps (make.powerapps.com) can't be extended.

Gated extensions available

Pipelines deployments progress through multiple predefined steps until deployment completion. Gated extensions insert custom steps into the progression where custom business logic can be executed. It's like your own personal train where you're in control of where it stops and whether it continues or not.



When enabled, each extension inserts a custom step at a different point within a deployment's progression. Extensions can be used alone or together.

- **Pre-export Step Required** allows running custom validation logic when a deployment request is submitted. Pipelines won't export the solution from a development environment until this step is marked as completed. Only enable this for the first stage in a pipeline. For example, user assistance testing (UAT).
- **Is Delegated Deployment** carries out deployments using a service principal or pipeline stage owner's identity instead of the requesting maker's. This identity ensures makers can request deployments without elevated (or any) access within target environments. Requires approval from an authorized identity.
- **Pre-deployment Step Required** provides additional flexibility to insert a custom step after a deployment is approved. For example, if a final approval is required.

Power Apps

Deployment Pipeline Configuration

Home

Recent

Pinned

Overview

Pipelines Dashboard

Pipeline Setup

Environments

Pipelines

Deployments

Run history

Solution artifacts

Contoso Production - Saved

Deployment Stage

General Related

Name

*

Contoso delegated pipeline

Owner

*

Description

You may deploy here without having access to the environment. Approval required.

The basic user role in needed to use the deployed application, however.

Name

*

Contoso Production

Owner

*

Description

Deployment Pipeline

*

Contoso delegated pipeline

Previous Deployment Stage

Target Deployment Environment ID

*

Prod

Pre-Export Step Required

Recommendation

Additional configuration required

By checking this box, you must also configure an external process (e.g., cloud flow) in this environment that determines the pre-export step(s) required for this stage. Learn more at <https://aka.ms/setupPipelines>

Apply

Pre-Deployment Step Required

Recommendation

Additional configuration required

By checking this box, you must also configure an external process (e.g., cloud flow) in this environment to execute the pre-deployment step(s) required for this stage. Learn more at <https://aka.ms/setupPipelines>

Apply

Is Delegated Deployment

Recommendation

Additional configuration required

By checking this box, you must also configure an external process (e.g., cloud flow) in this environment to approve or reject the delegated deployment. Learn more at <https://aka.ms/setupPipelines>

Apply

Steps inserted by enabling a gated extension are in a pending state until your business logic executes and finally signals the pipelines host to complete or reject the step. Makers can see when a deployment is pending as well as cancel their deployment request up until the final step of the deployment.

Note

Once the managed and unmanaged solution artifacts are exported, the system stores them in the pipelines host and prohibits any tampering or modification. The same managed artifact, per version,

will be deployed to all subsequent stages in the pipeline in sequential order. This ensures no solution can bypass QA environments or approval processes.

Understanding pipelines triggers and actions

Each step of a pipeline deployment triggers a real-time event at the beginning and completion of the step for which you can initiate custom logic. Additional triggers are produced when gated extensions are enabled. These correspond to the custom step inserted when an extension is enabled on the pipeline stage.

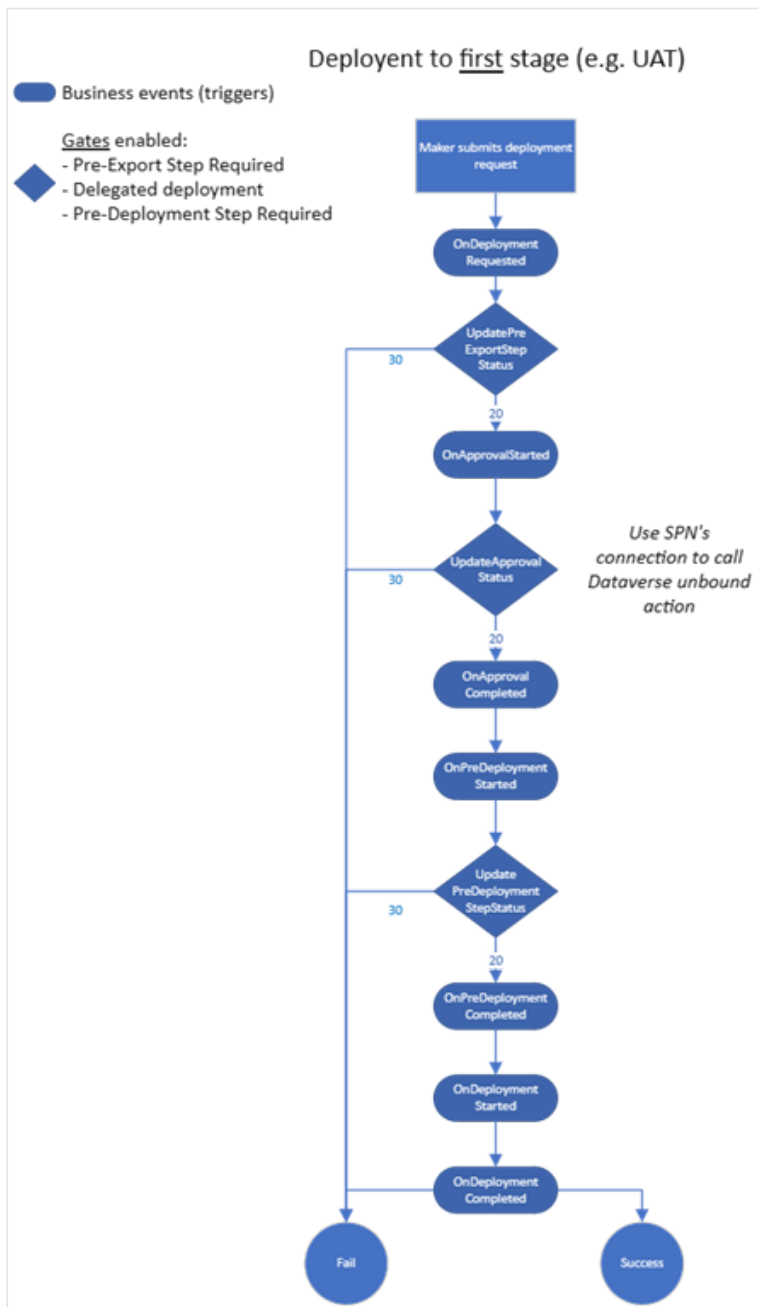
The tables here indicate triggers and actions required for each extension. Output parameters from each trigger can be used within subsequent steps of a cloud flow.

 Expand table

Gated extension	Step started trigger	Step completed trigger	Unbound action	Connection to use
Pre-export step required	OnDeploymentRequested	N/A	UpdatePreExportStepStatus	Any identity with access to update the deployment stage run record
Is delegated deployment	OnApprovalStarted	OnDeploymentCompleted	Create a connection as the service principal or pipeline stage owner as configured on the pipeline stage. The pipeline stage owner must be an owner of the service principal in Microsoft Entra ID.	
Predeployment step required	OnPreDeploymentStarted	OnPreDeploymentCompleted	UpdatePreDeploymentStepStatus	Any identity with access to update the deployment stage run record

Note

OnDeploymentRequested triggers for all deployments.



Triggers

Triggers are available in Power Automate cloud flows within the pipelines host environment under the **When an action is performed** trigger of the Dataverse connector.

- **Catalog:** Microsoft Dataverse Common.
- **Category:** Power Platform Pipelines.
- **Table name:** (None).
- **Action name:** Select an option from the list. Pipelines provide the following custom actions that produce a trigger in Power Automate cloud flows. Approval and OnPreDeployment* related events only trigger when the corresponding extension is enabled.

OnDeploymentRequested

OnApprovalStarted

OnApprovalCompleted

OnPreDeploymentStarted

OnPreDeploymentCompleted

OnDeploymentStarted

OnDeploymentCompleted

Here's an example screenshot of a trigger.

When an action is performed

* Catalog: Microsoft Dataverse Common

* Category: Power Platform Pipelines

* Table name: (none) Show options

* Action name: OnDeploymentRequested

- OnDeploymentCompleted
- OnDeploymentRequested
- OnDeploymentStarted
- OnPreDeploymentCompleted
- OnPreDeploymentStarted
- Enter custom value

Trigger conditions

Power Automate [trigger conditions](#) allow you to customize the trigger so that the flow runs only when certain conditions are met. For example, you might need to run different flows and associated business logic within different pipelines or pipeline stages. This also helps keep flow runs and Power platform requests consumption low.

Settings for 'OnPreDeploymentRequested'

Custom Tracking Id
Set the tracking id for the run. For split-on this tracking id is for the initiating request.

Tracking Id

Secure Inputs
Secure inputs of the operation.
Secure Inputs: Off

Secure Outputs
Secure outputs of the operation and references of output properties.
Secure Outputs: Off

Retry Policy
A retry policy applies to intermittent failures, characterized as HTTP status codes 408, 429, and 5xx, in addition to any connectivity exceptions. The default is an exponential interval policy set to retry 4 times.
Type: Default

Concurrency Control
Limit number of concurrent runs of the flow, or leave it off to run as many as possible at the same time. Concurrency control changes the way new runs are queued. It cannot be undone once enabled.
Limit: Off

Trigger Conditions
Specify one or more expressions which must be true for the trigger to fire.
@equals(triggerOutputs()[?body/OutputParameters/DeploymentPipelineName]...)

+ Add

Done Cancel

Trigger a flow for a specific pipeline

Triggers a flow run for all stages of a pipeline (Contoso Pipeline). In this example, replace 'Contoso Pipeline' with the name of your pipeline:

```
@equals(triggerOutputs()?['body/OutputParameters/DeploymentPipelineName'], 'Contoso Pipeline')
```

Trigger a flow for a specific stage in a pipeline

Triggers a flow for a deployment stage with the stage name *equal* to 'Contoso UAT':

```
@equals(triggerOutputs()?['body/OutputParameters/DeploymentStageName'], 'Contoso UAT')
```

Trigger a flow for specific stages in a pipeline

Triggers a flow for deployment stages with the stage name *containing* 'QA':

```
@contains(triggerOutputs()?['body/OutputParameters/DeploymentStageName'], 'QA')
```

Learn more about expressions used in conditions: [Use expressions in conditions to check multiple values.](#)

Actions

After running the desired logic in cloud flows, use the **Perform an unbound action** in conjunction with the corresponding action for each gated extension.

These parameters are exposed across the actions for the corresponding gated extensions.

- UpdatePreExportStepStatus, UpdateApprovalStatus, and PreDeploymentStepStatus:
 - **10** is the pending status set by the system.
 - **20** for completing the step.
 - **30** for rejecting the step. The deployment won't proceed and status will be set to failed. You can also add both maker facing and admin facing comments to indicate the reason for rejection.
- ApprovalComments and predeployment comments:
 - Comments that are visible to the maker within pipelines run history. Intended for approvers to share comments with the requesting maker. For example, why their deployment was rejected or information about company specific processes.
- PreExportProperties and ApprovalProperties:
 - Admins can store information or custom parameters here without it surfacing to makers - for example a link to the flow run or approval, or other pertinent data. The intent is to provide flexibility and simplify custom reporting for deployment related data.

Samples

Download and then import the desired managed solution into your pipelines host environment. These samples can be customized to fit the unique needs of your organization.

Pipelines extensibility samples

This download contains sample cloud flows for using the above triggers and actions with pipelines. More information: [Download sample solution](#) [↗](#)

Delegated deployment samples

This download contains sample cloud flows for managing approvals and sharing deployed canvas apps and flows within the target environment. More information: [Download sample solution](#) ↗

Next steps

[Run pipelines in Power Platform](#)

Related content

[What are cloud flows?](#)

[Use Dataverse connector with cloud flows](#)

Deploy pipelines as a service principal or pipeline owner

Article • 10/10/2024

Delegated deployments can be run as a service principal or pipeline stage owner. When enabled, the pipeline stage deploys as the delegate (service principal or pipeline stage owner) instead of the requesting maker.

Deploy with a service principal

Prerequisites

- A Microsoft Entra user account. If you don't already have one, you can [Create an account for free](#) .
- One of the following Microsoft Entra roles: Cloud Application Administrator, or Application Administrator.
- You must be an owner of the enterprise application (service principal) in Microsoft Entra ID.

For a delegated deployment with a service principal, follow these steps.

1. Create an enterprise application (service principal) in Microsoft Entra ID.

Important

Anyone enabling or modifying service principal configurations in pipelines must be an owner of the enterprise application (service principal) in Microsoft Entra ID.

2. Add the enterprise application as a server-to-server (S2S) user in your pipelines host environment and each target environment it deploys to.
3. Assign the Deployment Pipeline Administrator security role to the S2S user within the pipelines host, and System Administrator security role within target environments. Lower permission security roles can't deploy plug-ins and other code components.
4. Choose (check) **Is delegated deployment** on a pipeline stage, select **Service Principal**, and enter the Client ID. Select **Save**.

5. Optionally, **Allow sharing requests** so that deployment requestors can specify which security groups can access deployed objects within the target environment. Sharing requests are part of the deployment request and can be approved or rejected.

Important

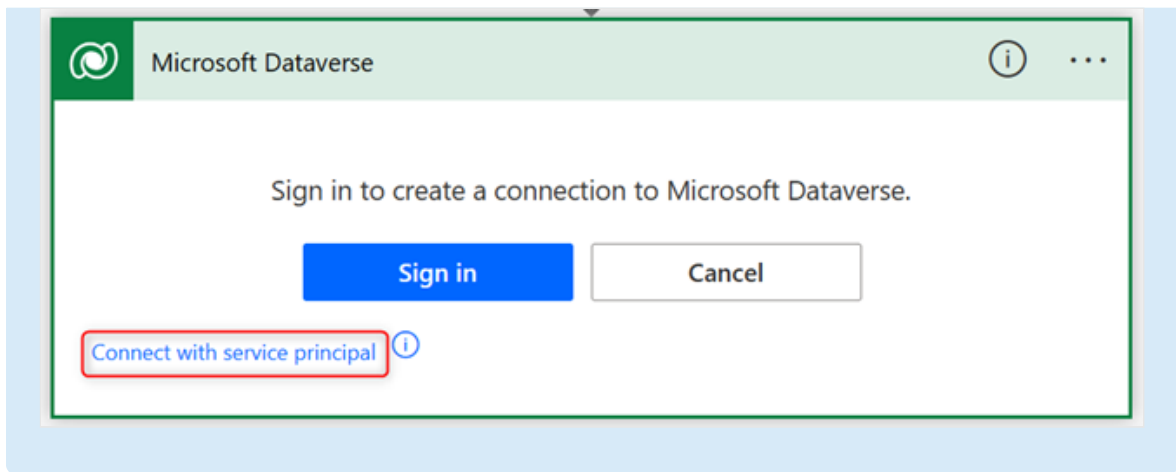
Deployment approvers are responsible for carefully reviewing sharing and security role information. When a deployment is approved, pipelines automatically assigns permissions using the deploying service principal's identity.

>

1. Create a cloud flow within the pipelines host environment. Alternative systems can be integrated using pipelines' Microsoft Dataverse APIs.
2. Select the **OnApprovalStarted** trigger.
3. Add steps for your desired custom logic.
4. Insert an approval step. Use Dynamic content for sending deployment request information to the approver(s).
5. Insert a condition.
6. Create a Dataverse connection for the service principal. You need a client ID and secret.
7. Add Dataverse **Perform an unbound action** using the settings shown here.
Action Name: UpdateApprovalStatus ApprovalComments: Insert Dynamic Content.
Comments are visible to the requestor of the deployment. ApprovalStatus: 20 = approved, 30 = rejected ApprovalProperties: Insert Dynamic Content. Admin information accessible from within the pipelines host.

Important

The UpdateApprovalStatus action must use the service principal's connection.

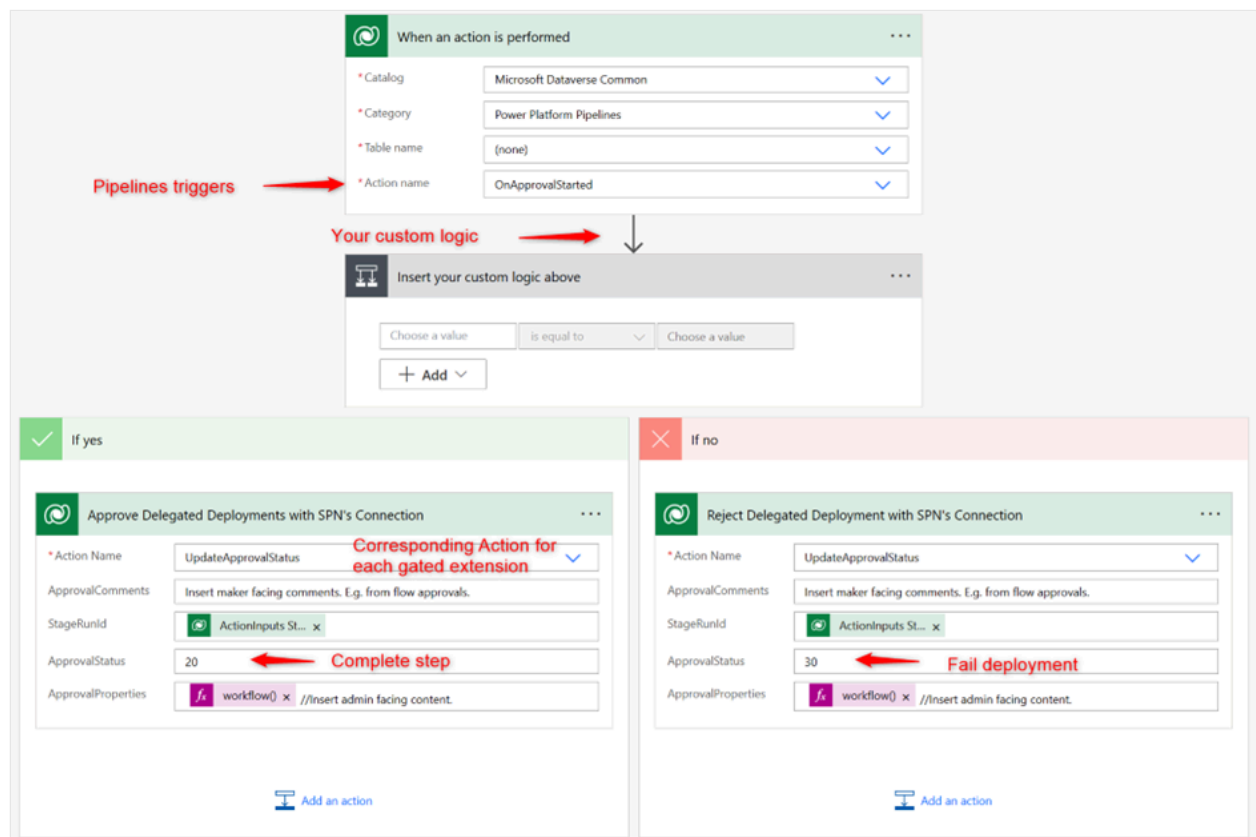


💡 Tip

To improve the debug experience, select ApprovalProperties and insert workflow() from the dynamic content menu. This links the flow run to the pipeline stage run (run history).

8. Save, and then test the pipeline.

Here's a screenshot of a canonical approval flow.



Deploy as the pipeline stage owner

Regular users, including those used as service accounts, can also serve as delegates. Configuration is more straightforward when compared to service principals, but solutions containing connection references for oAuth connections can't be deployed.

To deploy as the pipeline stage owner, follow these steps.

1. Assign the Deployment Pipeline Administrator security role to the pipeline stage owner within the pipelines host, and assign System Administrator security role within target environments.

Lower permission security roles can't deploy plug-ins and other code components.

2. Sign in as the pipeline stage owner. Only the owner can enable or modify these settings. Team ownership isn't allowed.

3. Select **Is delegated deployment** on a pipeline stage, and select **Stage Owner**.

- The pipeline stage owner's identity is used for all deployments to this stage.
- Similarly, this identity must be used to approve deployments.

4. Create a cloud flow in a solution within the pipelines host environment.

- a. Select the **OnApprovalStarted** trigger.
- b. Insert actions as desired. For example, an approval.
- c. Add Dataverse **Perform an unbound action**.

Action Name: UpdateApprovalStatus (20 = completed, 30 = rejected)

Delegated deployment samples

Important

The functionality provided in these samples is now supported natively in the product, but these samples might provide insights into extending native sharing functionality.

This download contains sample cloud flows for managing approvals and sharing deployed canvas apps and flows within the target environment. [Download sample solution](#) 

Download and import the managed solution to your pipelines host environment. The solution can then be customized to fit the needs of your organization.

Frequently asked questions

How can makers access deployed objects within target environments?

Sharing during deployment is a native capability of delegated deployments with service principals. It avoids admins needing to manually assign security roles and share deployed apps, flows, Copilots, and so on, within the Power Platform admin center. Instead, admins only need to approve the deployment request and sharing is carried out automatically by the system.

Which object types can be shared during deployment?

Currently, security roles, canvas apps, and cloud flows are supported. Copilot sharing might also be available depending on your region.

Can I update sharing when new versions are deployed?

Sharing is available the first time an object is deployed to the target environment. Sharing cannot be updated when new versions are deployed. Be sure to select an appropriate security group during the first deployment. Manage ongoing access via security groups.

Which permissions are assigned for canvas apps and flows?

Pipelines assigns the minimum privileges required to run apps and flows. If higher privileges are desired, pipelines can be extended. We recommend you enable the 'Block unmanaged customizations' feature when assigning higher permissions.

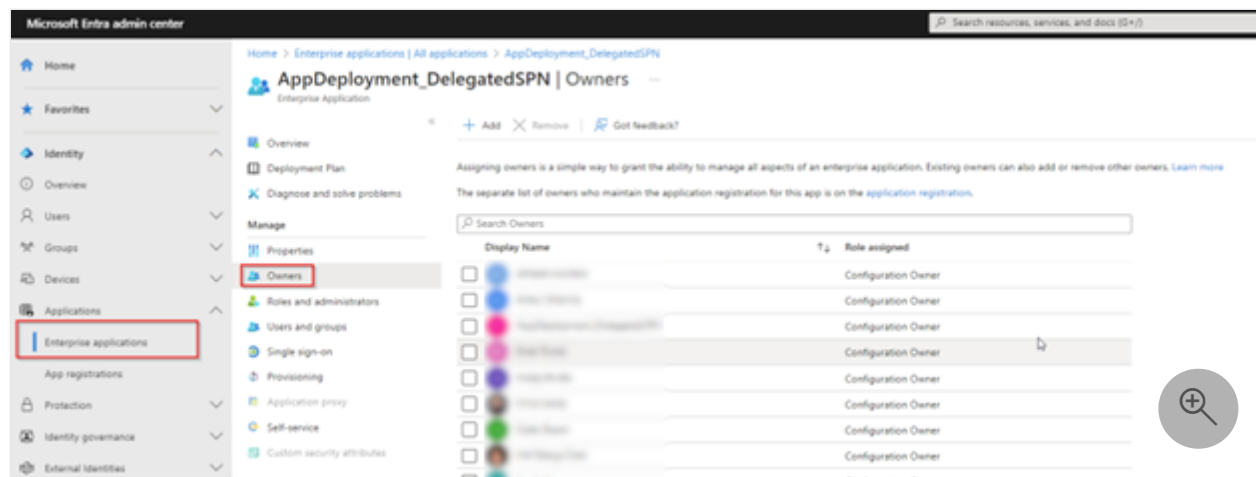
Can makers share with individual users?

Not currently. We recommend managing individual user access via security groups after the first deployment of the object.

I'm getting an error **The deployment stage isn't an owner of the service principal (<AppId>). Only owners of the service principal may use it for delegated deployments.**

Ensure you're the owner of the Enterprise Application (Service Principal) in Microsoft Entra ID (formerly Azure AD). You might only be the owner of the App Registration, and

not the Enterprise Application.



For stage owner based delegated deployments, why can't I assign another user as the deployer?

For security reasons, you must sign in as the user that will be set as the pipeline stage owner. This prevents adding a nonconsenting user as the deployer.

For stage owner based delegated deployments, can I use a custom DeploymentSettings.json file?

Not currently within the maker experience.

Why are my delegated deployments stuck in a pending state?

All delegated deployments are pending until approved. Ensure your admin has configured a Power Automate approval flow or other automation, that it's working properly, and that the deployment was approved.

Who owns deployed solution objects?

The deploying identity. For delegated deployments, the owner is the delegated service principal or pipeline stage owner.

Can I add custom approval steps?

Yes. For example, Power Automate approvals can be customized to meet the needs of your organization. You might also integrate other approval systems.

Why do I have to own the service principal?

This is enforced for security reasons. You might also create pipelines using a service account and add the same service account as the owner. Another option is assigning the service principal (application user) as the owner of the pipeline stage and as an owner of itself (Enterprise application) in Microsoft Entra. However, assigning pipeline stage ownership to an application must be done via the Dataverse API in the pipelines host.

I'm getting an error *Delegated deployments of type 'ServicePrincipal' may only be approved or rejected by the Service Principal configured in the deployment stage.*

Ensure the Dataverse custom action `UpdateApprovalStatus` is called by the service principal. If using Power Automate approvals, ensure this action is configured to use the delegate service principal's connection.

I'm getting an error *Delegated deployments of type 'Owner' may only be approved or rejected by the owner of the deployment stage.*

Ensure the Dataverse custom action `UpdateApprovalStatus` is called by the pipeline stage owner. If using Power Automate approvals, ensure this action is configured to use the delegate pipeline stage owner's connection.

I'm getting an error in my approval flow *Unable to find approval status attribute for stage run record.*

This happens when the approval status isn't yet in the pending state. Make sure this is a delegated deployment and you're using the `OnApprovalStarted` trigger in your approval flow.

Can I use different service principals for different pipelines and stages?

Yes.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Extend pipelines using GitHub Actions

Article • 04/16/2025

This article demonstrates using [GitHub Actions](#) and Power Automate cloud flows for extending pipelines in Power Platform. When a pipeline deployment is submitted, a cloud flow triggers the GitHub workflow to download, unpack, and commit the artifact's source code to a GitHub branch.

Workflow details

The workflow is triggered via a `workflow_dispatch` event. The workflow runs on `ubuntu-latest` and has the `contents: write` permission to be able to commit changes to the GitHub repository branch.

The workflow consists of the following steps:

1. `actions/checkout@v3`: Checks out the repository.
2. `create new branch if specified`: Creates a new branch if a `target_branch` is specified in the inputs.
3. `download solution from artifact`: Downloads the solution from the artifact created by pipelines.
4. `unpack solution`: Unpacks the solution.
5. `commit changes`: Commits changes to the existing or new branch.
6. `push to branch`: Pushes the committed changes to the source branch.

Workflow inputs

The following workflow inputs are required or optional:

- `artifact_url` (required): The URL of the Microsoft Dataverse row (record) ID for the artifact created by the pipelines.
- `solution_name` (required): Name of the solution in the Dataverse environment.
- `source_branch` (required): Branch for the solution commit.
- `target_branch` (optional): Branch to create for the solution commit. If not specified, the `source_branch` is used.
- `commit_message` (required): Message to provide for the commit.

Workflow secrets

The following secrets are required to connect to Dataverse using an Application User configured in Dataverse and in Microsoft Entra ID (AD). Configure these secrets in the GitHub repository settings.

- `CLIENT_ID`: The client ID of the registered Microsoft Entra application.
- `TENANT_ID`: The tenant ID of the Microsoft Entra directory associated with the Microsoft Entra application.
- `CLIENT_SECRET`: The client secret of the registered Microsoft Entra application.

For more information, go to [Creating and using encrypted secrets](#) and [Create an application user](#).

Workflow code

Listed below is the GitHub Actions workflow code.

makefile

```
name: Download, unpack and commit the solution to git
run-name: Getting ${ github.event.inputs.solution_name } from pipelines host
environment and committing
on:
  workflow_dispatch:
    inputs:
      artifact_url:
        description: "The url of the Dataverse record ID for the artifact created
by the pipelines (Example: https://[your-
env].crm.dynamics.com/api/data/v9.0/deploymentartifacts([your-artifact-
id])/artifactfile/$value)."
        required: true
      solution_name:
        description: "Name of the Solution in Dataverse environment"
        required: true
      user_name:
        description: "User name for the commit"
        required: true
      source_branch:
        description: "Branch for the solution commit"
        required: true
      target_branch:
        description: "Branch to create for the solution commit"
        required: false
      commit_message:
        description: "Message to provide for the commit"
        required: true
permissions:
  contents: write
jobs:
  export-unpack-commit:
```

```
runs-on: ubuntu-latest
```

```
steps:
```

- uses: actions/checkout@v3
with:
ref: \${github.event.inputs.source_branch }

```
# Commit changes to the existing or new branch
```

- name: create new branch if specified
shell: pwsh
run: |
if('\${github.event.inputs.target_branch }' -ne '') {
git checkout -b \${github.event.inputs.target_branch } \${github.event.inputs.source_branch }
}

```
# Export the solution from the artifact created by pipelines
```

- name: download solution from artifact
env:
CLIENT_ID: \${secrets.CLIENT_ID}
TENANT_ID: \${secrets.TENANT_ID}
CLIENT_SECRET: \${secrets.CLIENT_SECRET}
shell: pwsh
run: |
\$aadHost = "login.microsoftonline.com"
\$url = "\${github.event.inputs.artifact_url }"
\$options = [System.StringSplitOptions]::RemoveEmptyEntries
\$dataverseHost = \$url.Split("://", \$options)[1].Split("/")[0]

\$body = @{client_id = \$env:CLIENT_ID; client_secret =
\$env:CLIENT_SECRET; grant_type = "client_credentials"; scope =
"https://\$dataverseHost/.default"; }
\$OAuthReq = Invoke-RestMethod -Method Post -Uri
"https://\$aadHost/\$env:TENANT_ID/oauth2/v2.0/token" -Body \$body
\$spnToken = \$OAuthReq.access_token
\$headers = New-Object "System.Collections.Generic.Dictionary[[String],
[String]]"
\$headers.Add("Authorization", "Bearer \$spnToken")
\$headers.Add("Content-Type", "application/json")

Download the managed solution
\$response = Invoke-RestMethod "\${github.event.inputs.artifact_url
}" -Method 'GET' -Headers \$headers
\$bytes = [Convert]::FromBase64String(\$response.value)
[IO.File]::WriteAllBytes("\${github.event.inputs.solution_name
}_managed.zip", \$bytes)

Download the unmanaged solution (for now we will need to use string
manipulation to get the unmanaged solution URL, until the API provides this value)
\$unmanaged_artifact_url = "\${github.event.inputs.artifact_url
}".Replace("artifactfile", "artifactfileunmanaged")
\$response = Invoke-RestMethod "\$unmanaged_artifact_url" -Method 'GET'
-Headers \$headers
\$bytes = [Convert]::FromBase64String(\$response.value)
[IO.File]::WriteAllBytes("\${github.event.inputs.solution_name

```

}}.zip", $bytes)

# Unpack the solution
- name: unpack solution
  uses: microsoft/powerplatform-actions/unpack-solution@v0
  with:
    solution-file: "${{ github.event.inputs.solution_name }}.zip"
    solution-folder: "${{ github.event.repository.name }}"
    solution-type: 'Both'
    process-canvas-apps: false
    overwrite-files: true

# Commit changes to the existing or new branch
- name: commit changes
  shell: pwsh
  run: |
    rm -rf ${{ github.event.inputs.solution_name }}.zip
    rm -rf ${{ github.event.inputs.solution_name }}_managed.zip
    git config user.name ${{ github.event.inputs.user_name }}
    git pull
    git add --all
    git commit -am "${{ github.event.inputs.commit_message }}" --allow-empty

# Push the committed changes to the source branch
- name: push to branch
  shell: pwsh
  run: |
    if('${{ github.event.inputs.target_branch }}' -ne '') {
      git push origin ${{ github.event.inputs.target_branch }}
    } else {
      git push origin ${{ github.event.inputs.source_branch }}
    }

```

ⓘ Note

The Dataverse web API used to download the solution artifact has a maximum file size limit of 16 MB.

Example Power Automate flow

To call a GitHub workflow, you can create a Power Automate flow that is triggered when a deployment request is made in Dataverse. The flow can be configured to pass the required inputs to the GitHub workflow. For more information on how to create a Power Automate flow, go to [Create a flow](#).

Flow details

The flow triggers when the `OnDeploymentRequested` action is run in Dataverse. The flow calls the HTTP connector to trigger the GitHub workflow. The flow passes the required inputs to the GitHub workflow. Include the following inputs in the request body:

- `artifact_url`: URL of the Dataverse solution artifact created by the pipelines.
- `solution_name`: Name of the solution in the Dataverse environment.
- `user_name`: User name for the commit.
- `source_branch`: Source branch for the solution commit.
- `target_branch`: Branch to create for the solution commit.
- `commit_message`: Message to provide for the commit.

The values passed into the `artifact_url`, `solution_name`, and `user_name` are pulled from the outputs of the action that triggered the pipeline. The `commit_message` is pulled from the deployment stage run row in Dataverse.

- `artifact_url`: `@{triggerOutputs()?['body/OutputParameters/ArtifactFileDownloadLink']}`
- `solution_name`: `@{triggerOutputs()?['body/OutputParameters/ArtifactName']}`
- `user_name`: `@{triggerOutputs()?['body/OutputParameters/DeployAsUser']}`
- `commit_message`: `@{outputs('Retrieve_the_Deployment_Stage_Run')?['body/deploymentnotes']}`

The flow also uses a personal access token (PAT) to authenticate with GitHub. For more information on how to create a GitHub personal access token, go to [Creating a personal access token](#). The PAT is passed in the `Authorization` header of the HTTP request.

Update the following values in the Flow:

- `[GitHub Personal Access Token]` - Replace with your GitHub personal access token.
- `[GitHub Organization]` - Replace with your GitHub organization name.
- `[GitHub Repository]` - Replace with your GitHub repository name.
- `[GitHub Workflow YAML File]` - Replace with your GitHub workflow YAML file name.
- `[Source Branch]` - Replace with the Git branch to commit the solution.
- `[Target Branch]` - Replace with the Git branch to create for the solution commit. `Target Branch` is optional. If you don't specify a target branch, then your solution is committed to the `Source Branch`.


When a Deployment is Requested

* Catalog
Microsoft Dataverse Common

* Category
Power Platform Pipelines

* Table name
(none)

* Action name
OnDeploymentRequested




Retrieve the Deployment Stage Run

* Table name
Deployment Stage Runs

* Row ID
 ActionInputs St...

[Show advanced options](#)




Call the GitHub Workflow

* Method
POST

* URI
https://api.github.com/repos/[GitHub Organization]/[GitHub Repository]/actions/workflows/[GitHub Workflow YAML File]/dispatches

Headers

Authorization	Bearer [GitHub Personal Access Token]		
Enter key	Enter value		

Queries

Enter key	Enter value	
-----------	-------------	---

Body

```

{
  "ref": "main",
  "inputs": {
    "artifact_url": "
```

[Quickstart for GitHub Actions](#) ↗

[Extend pipelines in Power Platform](#)

[What are cloud flows?](#)

Run pipelines in Power Platform

Article • 01/30/2025

Pipelines automate solution deployments between Power Platform environments and facilitate healthy application management practices with minimal effort.

Prerequisites

- One or more pipelines must already be created and associated with the environment that's used for development.
- The development environment must have Microsoft Dataverse or Dataverse plus Dynamics 365 customer engagement apps.
- You must have access to run a pipeline. More information: [Grant access to edit or run pipelines](#)
- You must have privileges to import solutions to the target environments associated with a pipeline.
- The **Power Platform Pipelines** application must be installed in your pipeline host environment. More information: [Install the pipelines application in your host environment](#)

For more information about these prerequisites, go to [Set up pipelines](#).

Run a pipeline

1. Sign in to a Power Platform environment using Power Apps (make.powerapps.com) or Power Automate (make.powerautomate.com) and select your development environment.
2. To deploy a solution using a pipeline, go to **Solutions** and create or select an unmanaged solution to deploy.
3. From the **Solutions** area, choose between two options to include the solution in the pipeline:
 - Select **Pipelines** from the left navigation pane.
 - Select **Overview** from the left navigation pane, and then select **Deploy** on the command bar.
4. Select the stage to deploy to, such as *Deploy to Test*, select **Deploy here**, and the deployment pane appears on the right.
5. Choose to deploy **Now** or schedule for **Later**, and then select **Next** on the right pane. This initiates validation of the solution against the test environment. This

validation can also be referred to as preflight checks. Missing dependencies and other common issues are checked that might cause a deployment to fail.

6. If connection references or environment variables are present, you're prompted to provide these (just as you would when manually importing solutions).
7. Review the summary of the deployment and optionally add deployment notes.
8. Select **Deploy**. This initiates an automated deployment to the target environment.

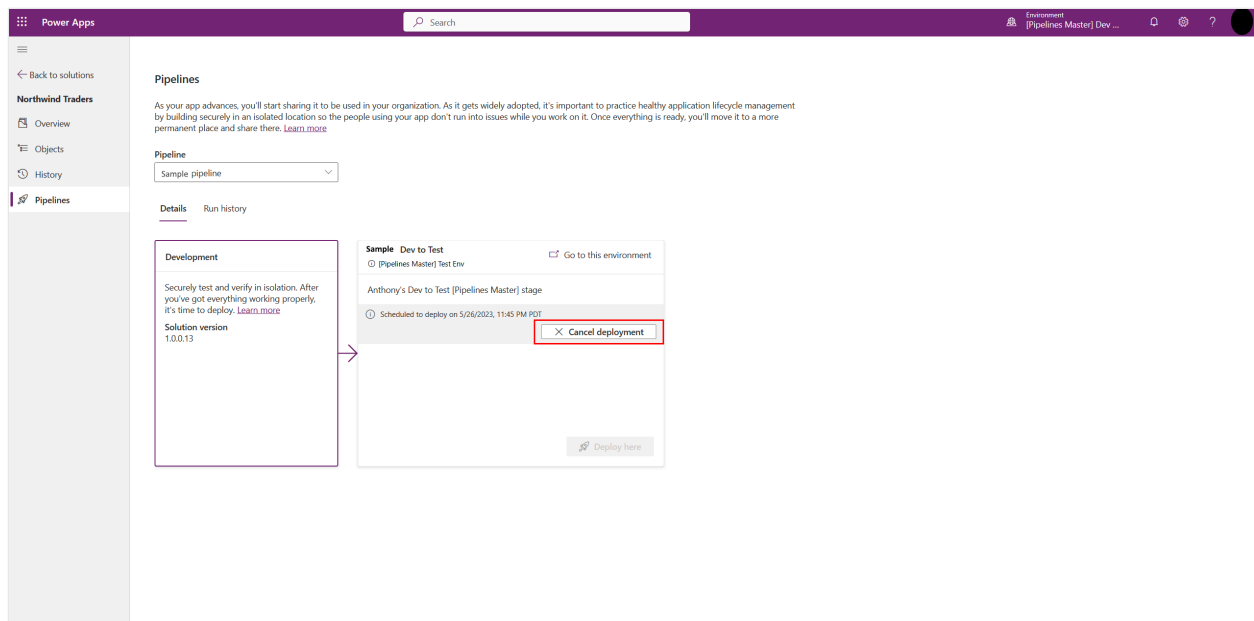
ⓘ Note

- Pipelines aren't visible within the default solution, managed solutions, or target environments.
- You must complete the deployment stages in order. For example, you can't deploy version 1.0.0.1 to production before it has been deployed to test. After you deploy to test, the same solution that was deployed will then be deployed to production, even if afterward you made changes to the solution without incrementing the version.
- A message stating your request to deploy here is pending, which means your admin attached [background processes or approvals](#) that run before your deployment can proceed.

Cancel a scheduled deployment

If you have a scheduled deployment, you can cancel it through three different methods:

- In the pipeline **Details** section where you began your deployment, there's an option to **Cancel deployment** before the scheduled deployment time.
- In **Run history**, selecting ... on a scheduled deployment displays a **Cancel deployment** option.
- In the *Information* pane, select a deployment record in **Run history**, and then select **Cancel deployment** under the *Status* of a scheduled deployment.



Change the time of a scheduled deployment as a pipeline admin

In the Deployment Pipeline Configuration app, perform the following steps:

1. Navigate to **Run history** under *Deployments*.
2. Select the record for the scheduled deployment that you want to change.
3. Change the **Scheduled Time** (shown in UTC, which might differ from your time zone) as desired.

Monitor pipeline deployments

The **Pipelines** page in the **Solutions** area displays all deployment activity for the current pipeline and solution.

Select a pipeline, then select **Run history** to view more detail and error information if there was a failure.

Power Apps

Search

Environ

Back to solutions

1

Overview

Objects

History

Pipelines

Pipelines

Preview

As your app advances, you'll start sharing it to be used in your organization. As it gets widely adopted, it's important to practice healthy application lifecycle management by building securely in an isolated location so the people using your app don't run into issues while you work on it. Once everything is ready, you'll move it to a more permanent place and share there. [Learn more](#)

Pipeline

ABC Pipeline

Details

Run history

Start time	End time	Deployed by	Stage	Destination	Status
Aug 22, 2022 9:56 PM (14 minutes ago)	Aug 22, 2022 9:58 PM (13 minutes ago)		Production stage	caburk Pipelines Prod	Succeeded
Aug 22, 2022 9:50 PM (9 minutes ago)	Aug 22, 2022 9:55 PM (5 minutes ago)		Test stage	caburk Pipelines Test	Succeeded

Information

Details

Stage

Production stage

Source

Pipeline Dev

Destination

Pipelines Prod

Start time

Aug 22, 2022 9:56 PM (14 minutes ago)

End time

Aug 22, 2022 9:58 PM (13 minutes ago)

Version

1.0.0.1

Deployed by

Operation

Deploy

Sub operation

None

Status

Succeeded

More details

Validation results

Passed

Failure details

Operation details

Related articles

[Solution concepts](#)

Feedback

Was this page helpful?



[Provide product feedback](#)

Associate a pipeline with development environments in a group (preview)

Article • 05/14/2025

[This article is prerelease documentation and is subject to change.]

The default deployment pipeline rule for environment groups is designed to mitigate steps required to implement standardized application lifecycle management (ALM) processes at scale. These rules ensure consistent application of pipeline settings and configurations across an entire group, simplifying governance and compliance for your organization.

Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#) , and are available before an official release so that customers can get early access and provide feedback.

Prerequisites to configure the default deployment pipeline rule

- You must be a **Tenant Administrator** to access [environment groups](#) in the Power Platform admin center.
- The Power Platform Pipelines package must be installed in at least one environment and you must have a [pipelines host](#) configured.
- At least one pipeline must be configured in the desired pipelines host.

Important

- Ensure that all users of the pipeline have access to the selected pipelines host. The easiest way to do this is to create a Microsoft Entra team on the pipelines host that maps to a Microsoft Entra security group containing the makers. This ensures users can obtain just-in-time (JIT) access to the pipelines host and be part of the 'default team,' thus gaining the 'Deployment Pipeline User' role.
- For the purposes of the default deployment pipeline rule, "development" environments are defined as **developer** or **sandbox** environments. The chosen pipeline will only be associated with environments of these types. This

requirement prevents production environments from being unintentionally specified as development environments in the pipelines host. Environments can only be specified as development or target environments in a pipelines host, they can't be both.

- All environments used in pipelines must have a Microsoft Dataverse database.
- All target environments used in a pipeline must be enabled as [Managed Environments](#).

For more information about environment group rules, go to [environment group rules](#).

Configure the default deployment pipeline rule for a group

1. Within an [environment group](#) in the Power Platform admin center, select **Rules**.
2. Select **Default deployment pipeline** from the list of rules.
3. In the first dropdown list, select a **Pipeline Host**.
4. Once a host is selected, select a **Pipeline** to associate with all development environments in the group and to share with all users on the default team of the pipelines host environment.



5. Select **Save**.
6. You should now observe an asterisk next to the rule, indicating that it has unpublished changes. Select **Publish rules** in the command bar.

The default deployment pipeline rule is now configured for development and sandbox environments in the group. Wait up to 10 minutes for the rule to apply.

Related content

- Environment Groups
- Managed Environments
- Set up pipelines in Power Platform
- Extend pipelines in Power Platform
- Run pipelines in Power Platform

Enable cross-geo solution deployments for pipelines in Power Platform

Article • 04/30/2024

Administrators can enable solution deployments via pipelines across geographical [regions](#). This provides flexibility for organizations that are distributed across geographical regions, making it easy to centrally administer global deployments within a single management plane.

Prerequisites to enable cross-geo solution deployments

- You must be a **System Administrator** or a **Deployment Pipeline Administrator** to access the Deployment Pipeline Configuration app *and* have **Write** privileges on the **Organization Setting (organizationsetting)** table in the host environment to enable this setting.

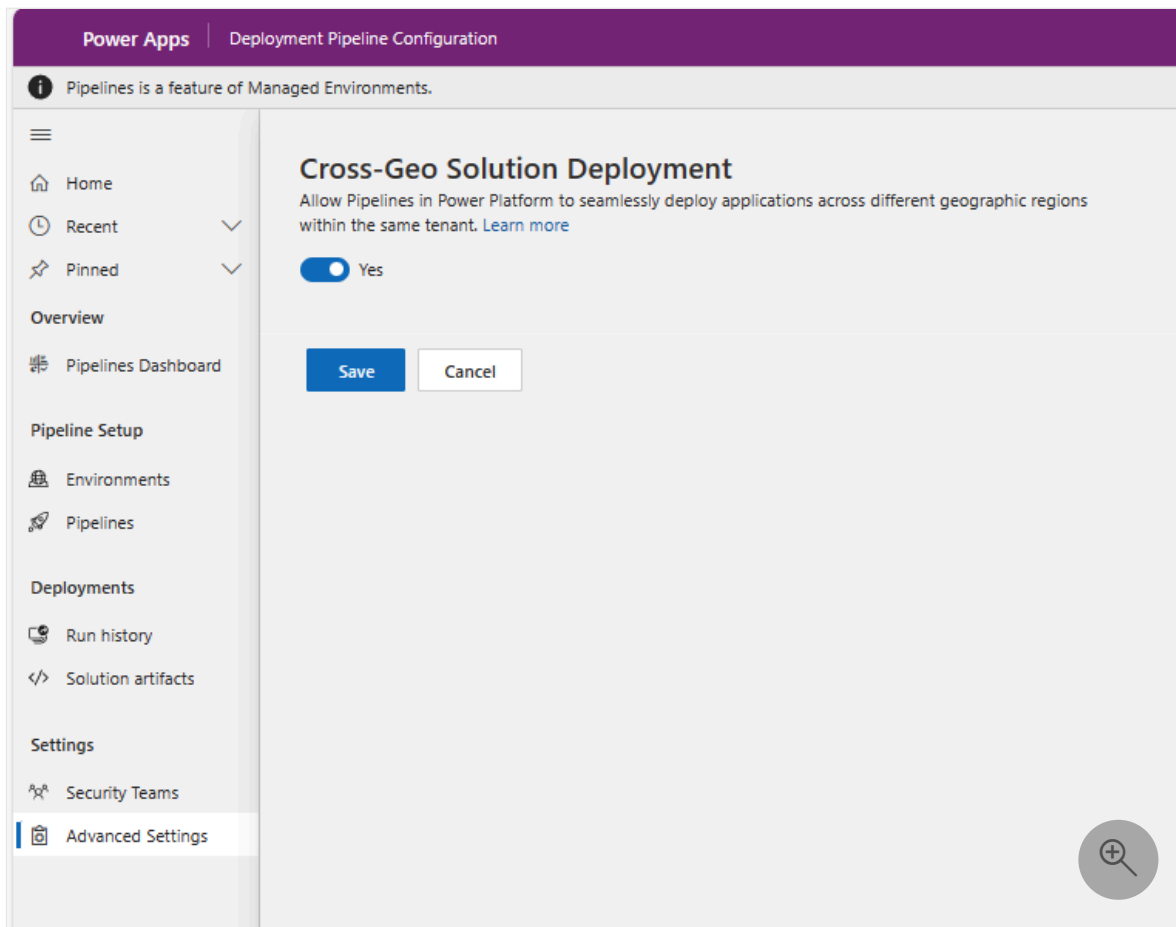
Important

This setting enables data to be shared across geographical regions within in your tenant.

- All environments used in pipelines must have a Microsoft Dataverse database.
- All target environments used in a pipeline must be enabled as [Managed Environments](#).

Accessing and enabling the cross-geo setting for pipelines

1. Go to the Deployment Pipeline Configuration app by selecting **Manage pipelines** on the **Pipelines** page within a solution in Power Apps.
2. Select **Advanced Settings** from the left navigation pane.
3. Enable or disable the **Cross-Geo Solution Deployment** setting with the toggle.
4. Select **Save**.



Next steps

- [Set up pipelines in Power Platform](#)
- [Extend pipelines in Power Platform](#)
- [Run pipelines in Power Platform](#)

Feedback

Was this page helpful?

Yes

No

[Provide product feedback](#)

Import solutions from a pipelines host

Article • 10/11/2024

When selecting **Import solution** from the command bar on the **Solutions** page in Power Apps (make.powerapps.com), you can choose **Pipelines** as a source if the current environment is associated with a pipelines host.

Import a solution

Environment
-US-Preview-Dev1

Import a solution from *

☐ This device

☒ Pipelines

Show shared solution deployments
☐

Unmanaged

Managed

Filter by name:

Display name	Version	Created Date	Source Enviro... ↓	Owner	Name
Anthony's solution	1.0.0.1	Mar 27, 2024 3:49 PM...	MS Personal Product...	Anthony	Anthonyssolution
Anthony Test App	1.0.0.1	Feb 15, 2024 11:03 A...	asheehi-US-Preview-...	Anthony	AnthonyTestApp

Next

Cancel

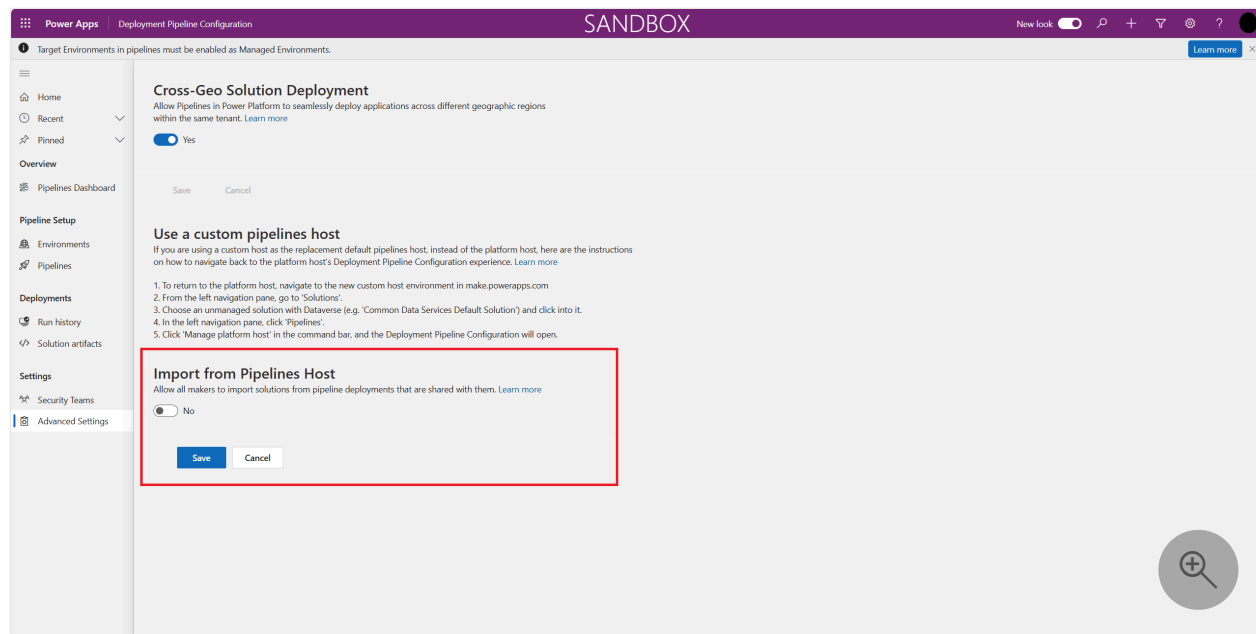
Select a solution from the list

The list of solutions is generated by reading the deployment artifacts that you own as the deployer, separating managed solutions and unmanaged solutions as both are

stored in the host. Users can toggle between **Unmanaged** and **Managed** to choose the appropriate solution type they would like to import.

Show shared solution deployments

By default, all users aside from admins (who can always view all deployments in their tenant) are able to see their deployed solutions only. However, if enabled in **Advanced Settings** in the pipelines host's Deployment Pipeline Configuration app, all users can switch the **Show shared solution deployments** toggle to view all shared deployments, which, by default, includes all deployments in the tenant.



See also

[Set up pipelines in Power Platform](#)

[Create a pipeline using a custom pipelines host](#)

Feedback

Was this page helpful?

☒ Yes

☐ No

[Provide product feedback](#)

Set a default pipelines host instead of the platform host

Article • 10/01/2024

By default, the platform host is used to help jumpstart easy application lifecycle management (ALM) in all organizations. However, if you have a custom pipelines host you would rather use by default for the environments in your tenant, use this setting to replace it.

Prerequisites for enabling the setting

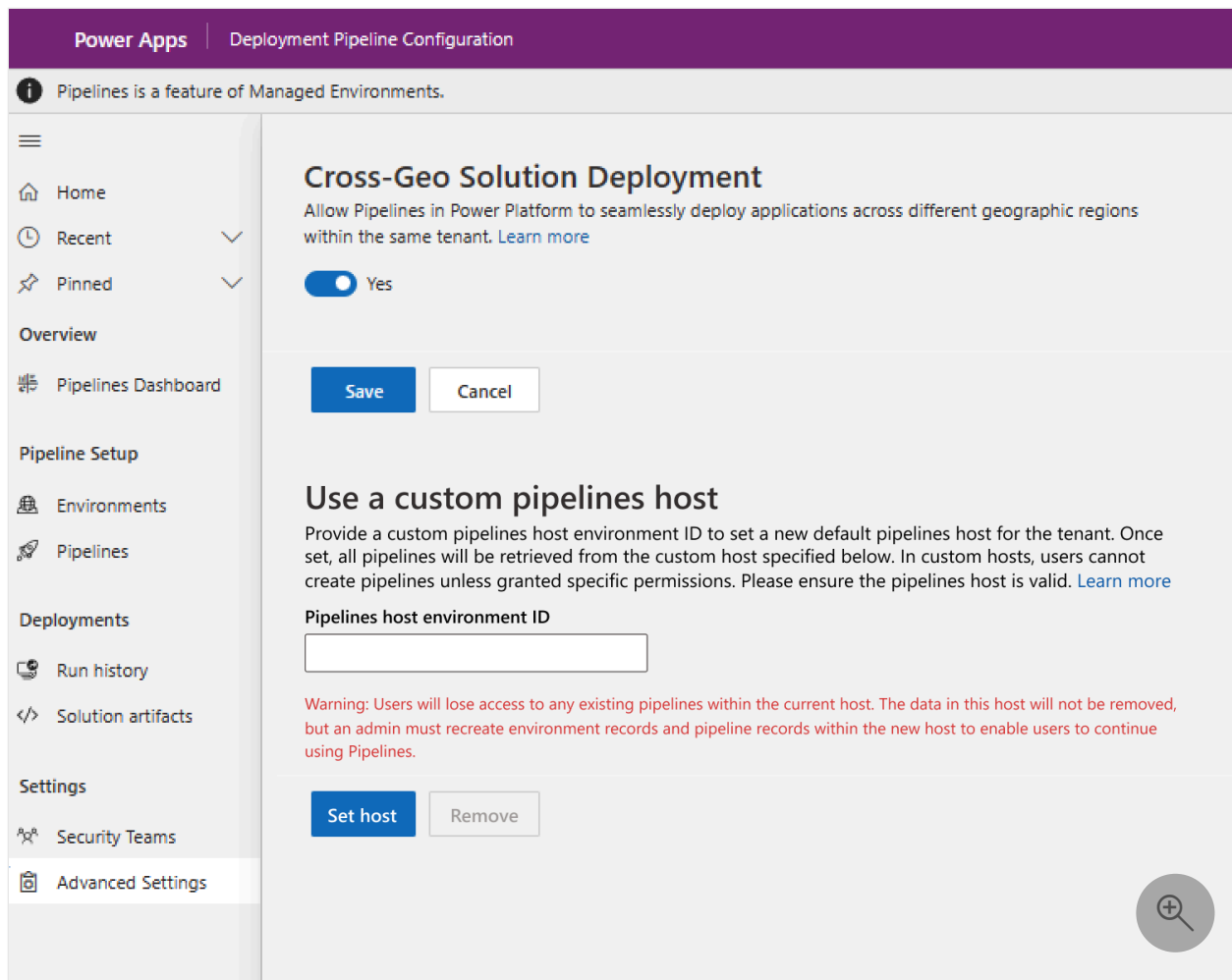
- You must be a Power Platform admin for your tenant.
- You must have a valid custom pipelines host with the **Power Platform Pipelines** package installed. More information: [Set up pipelines](#).

Enable the "Use a custom pipelines host" setting from the platform host

1. Go to the Deployment Pipeline Configuration app by selecting **Manage pipelines** on the **Pipelines** page within a solution in Power Apps.
2. Select **Advanced Settings** from the left navigation pane.
3. Enter the environment ID for the custom host you wish to use.
4. Select **Set Host**.

Important

After you enable the setting, users lose access to any existing pipelines within the current host. The data in this host is not removed, but an admin must re-create environment records and pipeline records within the new host to enable users to continue using pipelines. Pipelines and permissions to access them must be shared to users in the new host so they can view and deploy those pipelines.



Return to the platform host's deployment pipeline configuration experience

If you have configured the setting and are using a custom host as the replacement default pipelines host, instead of the platform host, here are instructions on how to navigate back to the platform host's Deployment Pipeline Configuration experience.

1. To return to the platform host, navigate to the new custom host environment in Power Apps (make.powerapps).
2. On the left navigation pane, select **Solutions**.
3. Open an unmanaged solution with Microsoft Dataverse, such as the Common Data Services Default Solution.
4. On the left navigation pane, select **Pipelines**, and then select **Manage platform host** on the command bar.

The platform host's Deployment Pipeline Configuration opens.

Remove or replace an existing default custom pipelines host

When in the platform host's Deployment Pipeline Configuration experience, select **Advanced Settings** on the left navigation pane.

- To replace: In the **Use a custom pipelines host** setting, replace the **Pipelines host environment ID**, and then select **Save**.
- To remove: In the **Use a custom pipelines host** setting, select **Remove**, and then confirm.

Frequently Asked Questions

I receive the error: EnvironmentNotFound - The environment '{Environment ID}' could not be found in the tenant '{Tenant ID}'

This error indicates either following situation:

- You don't have the necessary permissions to access the pipelines host. In this case, an admin must provide the necessary privileges to use pipelines
- The pipelines host is deleted so it can't be found.

See also

[Run pipelines in Power Platform](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Redeploy past solution versions

Article • 03/15/2024

With pipelines in Power Platform, makers and admins have the power to redeploy past Microsoft Dataverse solution versions from previous successful deployments.

Prerequisites

- One or more pipelines must already be created and associated with the environment that's used for development.
- The development environment must have Dataverse or Dataverse plus Dynamics 365 customer engagement apps.
- You must have access to run a pipeline. More information: [Grant access to edit or run pipelines](#)
- You must have privileges to import solutions to the target environments associated with a pipeline.
- The **Power Platform Pipelines** package version **9.1.24023.240222117** or later version must be installed in your pipeline host environment. More information: [Install the pipelines application in your host environment](#)
- The setting for **Allow redeployments of older versions** must be enabled in the pipeline settings.

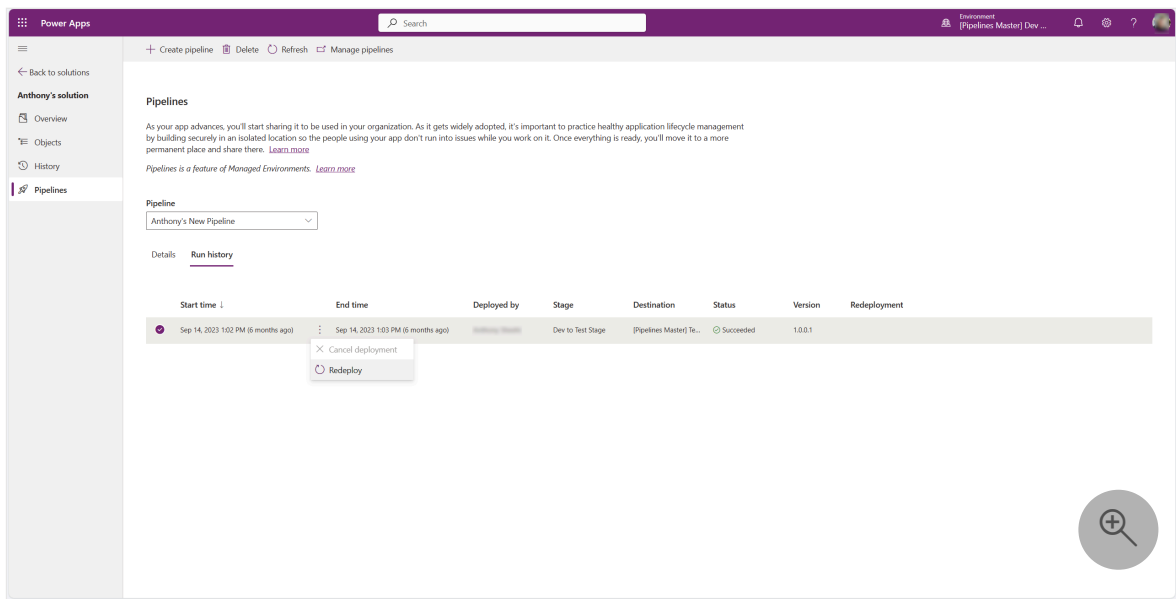
For more information about these prerequisites, go to [Set up pipelines](#).

Initiate a redeployment

Warning

Redeploying overwrites the most recent version and any data within that version. This is an irreversible operation, and the data lost is unrecoverable. Updates, including new tables, will also be removed.

1. From the **Pipelines** page, choose the pipeline in the dropdown list that you wish to redeploy.
2. Select **Run history**.
3. Find the record of the successful deployment that you wish to redeploy, select the vertical ellipses next to the start time, and then select **Redeploy**.



4. Read and acknowledge the warning, and then select **Redeploy** to continue with the usual deployment configuration process.

See also

[Solution concepts](#)

Feedback

Was this page helpful?

Yes

No

[Provide product feedback](#)

Use Copilot to generate deployment notes in pipelines

Article • 01/06/2025

Makers spend time and energy noting what is included in their deployed solution to give enough context for any admin approving (triggered via [extensibility](#)) their deployment requests or viewing their pipeline deployment record. With AI-generated deployment notes, in just seconds, a solution summary can be generated for the maker and pre-fill the deployment notes field, giving anyone viewing the deployment request or record enough context to understand what the solution does and what it contains.

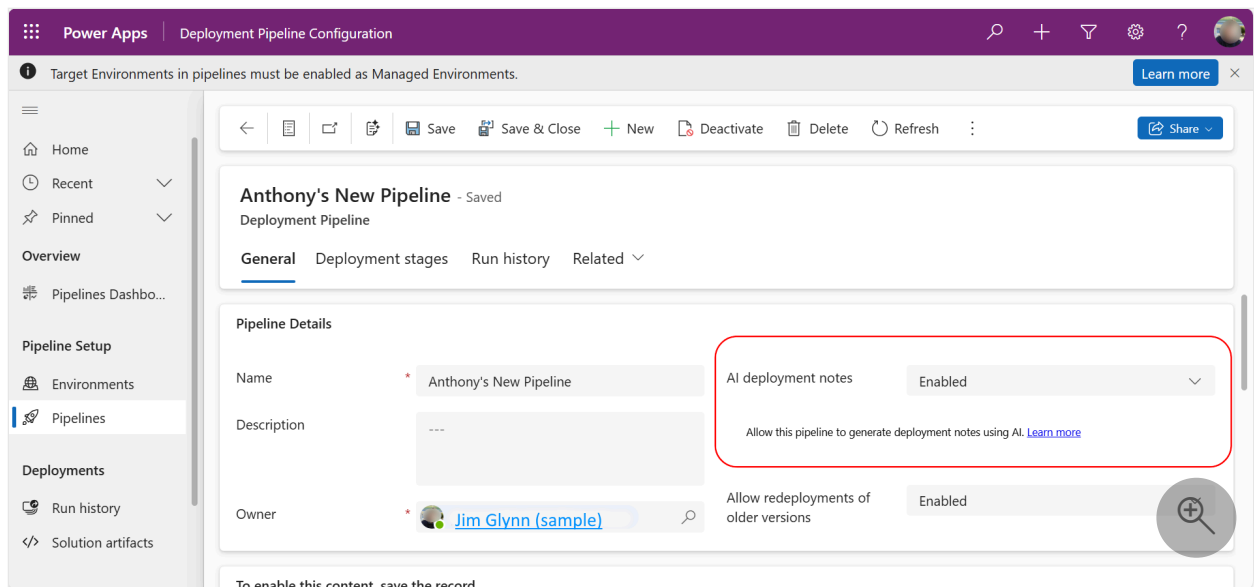
When you deploy solutions in a pipeline in Power Platform, you can [deploy with notes](#).

Important

- This capability is powered by [Azure OpenAI Service](#).
- Learn more with our [FAQ for Copilot data security and privacy in Dynamics 365](#).
- This capability is in process of rolling out, and may not be available in your region yet.
- This capability may be subject to usage limits or capacity throttling.

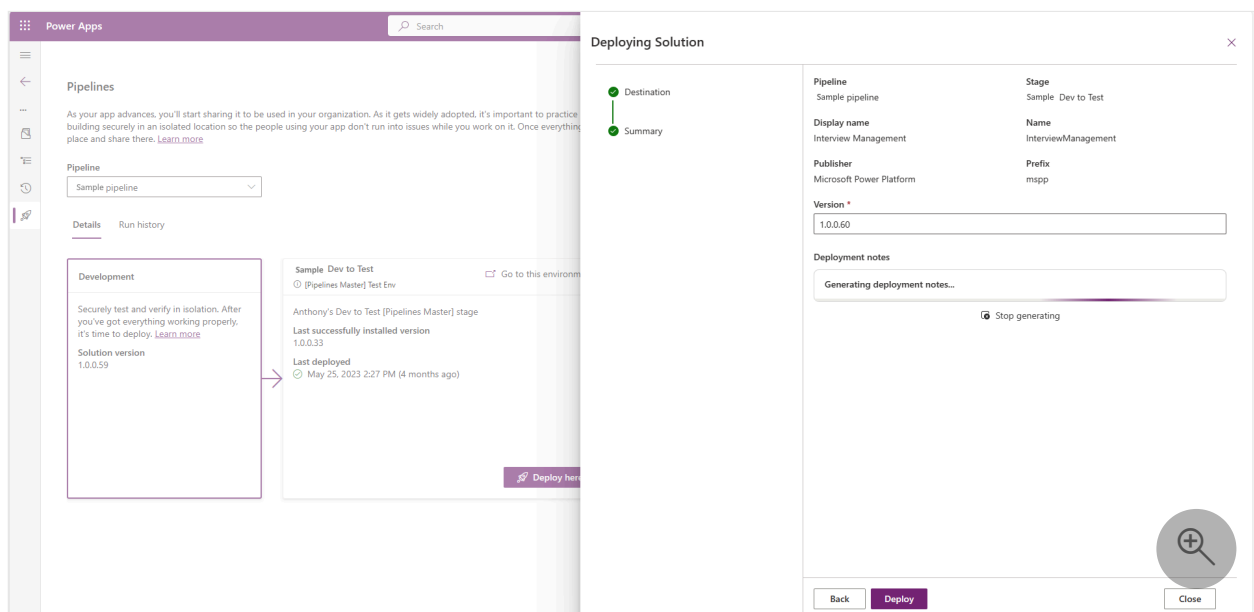
Prerequisites

- One or more pipelines must already be created and associated with the environment that's used for development.
- You must have access to run a pipeline. More information: [Grant access to edit or run pipelines](#)
- The setting for **AI deployment notes** must be set to **Enabled** in the **Deployment Pipeline Configuration** app for each pipeline you want to use this feature with.



Deploy a solution with Copilot generated deployment notes

Solutions can become complex, often containing dozens of components that are difficult to partition and summarize. With pipeline deployments, admins and deployment approvers have to open the solutions themselves to gain an understanding of what's included unless that information is provided in the deployment notes. By helping makers generate useful solution descriptions with AI, Copilot saves valuable time and provides better, more accurate information than makers could within the generation time. These descriptions are generated for you, pre-filling the deployment notes field.



Disable AI-generated deployment notes

By default, AI-generated deployment notes are enabled for pipelines in Power Platform. To disable the feature for a pipeline, follow these steps:

1. Play the **Deployment Pipeline Configuration** app in the pipelines host environment.
2. Select **Pipelines** in the left navigation bar.
3. Open the pipeline that you want to disable AI for.
4. Set "AI deployment notes" to **Disabled** via the dropdown list.
5. Go **Back to solutions** in the maker experience, if still running, and refresh to ensure the change was made.

See also

[FAQ for using Copilot to generate deployment notes in pipelines](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Block unmanaged customizations in Dataverse

Article • 07/16/2024

This article describes how you as a Power Platform administrator can enable the **Block unmanaged customizations** environment setting in the Power Platform admin center for your Microsoft Dataverse environments. Unmanaged customizations reside at the top layer for a component and subsequently define the runtime behavior of the component. In most situations, you don't want unmanaged customizations determining the behavior of your components.

This environment setting allows system administrators to prevent unmanaged customizations in Microsoft Dataverse environments. By blocking unmanaged customizations, you ensure the changes to the environment are only coming from approved application lifecycle management (ALM) processes that have sufficient safety, visibility, and auditing. Block unmanaged customizations continues to allow solution operations that are needed to transport the unmanaged customizations to development environments while keeping the environment unmanaged customization free by removing the unmanaged customizations. This feature is helpful in reducing unmanaged customizations in production environments.

When the setting is enabled, these operations are blocked for the environment:

- Import of unmanaged solutions into the environment is blocked.
- Creation of new solution components like apps, tables, and forms are blocked.
- Adding unmanaged changes to existing managed components are blocked.
- Any unmanaged changes result in an error returned to the user, such as: "This environment doesn't allow unmanaged customizations. This was a choice made by your admin, and certain actions won't be available or will be view only."

The following operations can be completed even with the block unmanaged customizations setting enabled:

- Change of environment variable, enabling or disabling components. For example, turning Power Automate flows on or off, assigning ownership, or sharing of table records.
- Solutions operations like remove [Remove an unmanaged layer](#).
- Creation and export of unmanaged solutions.
- Review flow run history to monitor and understand errors.
- Run the flow to test it.

Enable block unmanaged customizations

This feature can be enabled or disabled from the **Features** area of the environment settings for environments in the Power Platform admin center. By default, this setting is disabled. More information: [Block unmanaged customizations](#)

Known limitations

These features have limitations and might not work as expected in an environment where block unmanaged customizations is enabled. Using these features can result in this error: "This environment doesn't allow unmanaged customizations. This was a choice made by your admin, and certain actions won't be available or will be view only."

If you must use one of the below apps or features in an environment where block unmanaged customizations is enabled, we recommend you disable the setting.

 Expand table

App or feature	Limitation
Power Automate desktop flows	Hosted machine groups don't work.
Dynamics 365 Resource Scheduling Optimization	Install or upgrade don't work.
Automatic record creation	Activation, deactivation, and editing of automatic record creation doesn't work.
Legacy workflows	Enable and disable of legacy workflows doesn't work.
Field Service	Enabling or adjusting the enhanced autonumbering doesn't work. However, default autonumbering continues to work.
Connected Field Service	Install doesn't work.
Field Service Mobile	Install of geofencing solution doesn't work. Update doesn't work when relationships created during initial install are deleted.
Resource Scheduling	Enable of scheduling table and selection option to auto create relationships doesn't work.
Sales Accelerator	Enabling Sales Accelerator or modifying settings, enabling leads data hygiene, creating or editing sequences, configuring work assignment, creating or editing segments, don't work.

App or feature	Limitation
Dataflows	Creation of a new dataflow, republish an existing dataflow, change refresh settings, change incremental refresh, and importing dataflows don't work.
Dynamics 365 Customer Insights - Journeys (real-time journeys)	Journeys can't be created and published. Engagement assets such as triggers can't be created and published. Channel assets such as emails, forms, SMS, and push notifications can't be created and published.
Omnichannel	Install or upgrade don't work.
Service Level Agreement (SLA)	Activation, deactivation, and editing of service level agreement doesn't work.
Attachments	File attachments to appointments or emails using the <code>activitymimeattachment</code> table fail.

FAQ for block unmanaged customizations

Can this be used to give users read-only access to flow run history in test and production environments?

Yes, this is one of the key scenarios that prompted this feature. When unmanaged customizations are blocked, responsible users can be made a co-owner on a managed flow in a test or production environment. Co-owners can review flow run history, using make.powerautomate.com or via APIs or PowerShell, to watch for errors and understand those errors. Those users won't be able to make changes to the managed component, so it's effectively read-only to them. They're able to turn the flow on and off if needed, as well as to run the flow for use or testing purposes. Co-owners of a flow can then use that increased visibility and knowledge to adjust the flows in a development environment and request to have them redeployed to the test or production environment.

See also

[Solution layers](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

ALM for developers

Article • 02/05/2025

The articles in this section describe how you as a developer can use available APIs, tools, and other resources to implement application lifecycle management (ALM) using Microsoft Power Platform.

We'll start off talking a little about team development and then dive into Azure DevOps and available build tools.

To learn more about key ALM concepts and working with solutions, go to [Overview of application lifecycle management](#) and watch the following video.

<https://learn-video.azurefd.net/vod/player?ev=powerful-devs-conference&session=how-to-do-alm-with-power-platform&locale=en-us&embedUrl=%2Fpower-platform%2Falm%2Falm-for-developers> ↗

Team development

When we refer to *team development*, we're talking about multiple developers collaborating in the same environment versus multiple developers working on the same application and sharing source code. For team development, it's a good practice to use tooling or processes to achieve developer isolation.

More information: [Scenario 5: Supporting team development](#)

Conflict management

For team development, the goal is to avoid conflict when making changes to shared code. With a source control system, branching and merging help to avoid change conflicts and keep team members from affecting each other with partially completed work in a shared repo.

Another method is to use strategies to avoid conflict. For example, you can have only one person at a time work on a complex component, to avoid a merge conflict that a source control system might not automatically be able to resolve.

Working with complex components

What are complex components? Examples include forms, canvas apps, flows, and workflows.

Coordinate efforts with other team members to avoid having more than one developer work on the same form or component at a time. If you do have multiple developers working on the same canvas app, have them work on separate components to avoid conflict.

See also

[Microsoft Power Platform Build Tools for Azure DevOps](#)
[Power Platform developer docs](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

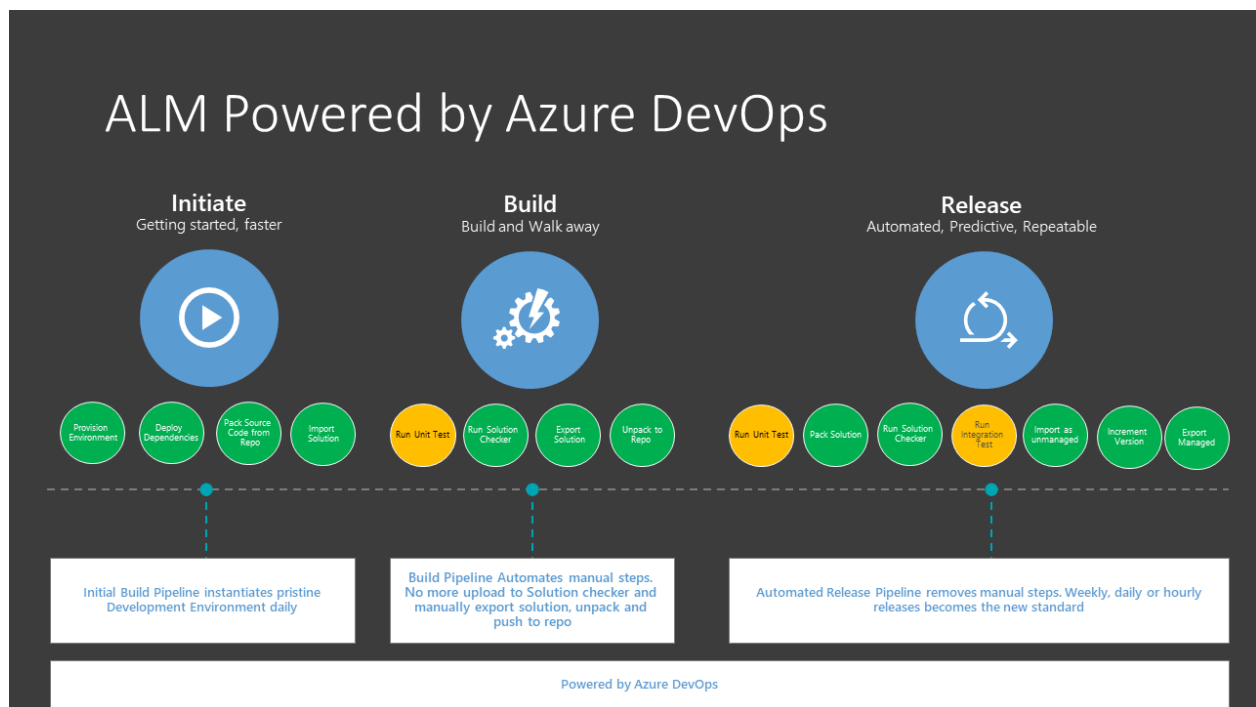
Microsoft Power Platform Build Tools for Azure DevOps

Article • 10/07/2024

Use Microsoft Power Platform Build Tools to automate common build and deployment tasks related to apps built on Microsoft Power Platform. These tasks include:

- Synchronizing solution metadata (also known as solutions) that contains the various platform components such as:
 - Customer engagement apps: Dynamics 365 Sales, Customer Service, Field Service, Marketing, and Project Service Automation
 - Canvas apps
 - Model-driven apps
 - UI flows
 - Virtual agents
 - AI Builder models
 - Connectors between development environments and source control
- Generating build artifacts
- Deploying to downstream environments
- Provisioning or deprovisioning environments
- Performing static analysis checks against solutions by using the Power Apps checker service

Use Microsoft Power Platform Build Tools tasks with any other available Azure DevOps tasks to compose your build and release pipelines. Pipelines that teams commonly put in place include Initiate, Export from Dev, Build, and Release.



ⓘ Note

Microsoft Power Platform Build Tools are supported only for a Microsoft Dataverse environment with a database. More information: [Create an environment with a database](#)

Microsoft Power Platform Build Tools are now available for use in **GCC** and **GCC High** regions.

What are Microsoft Power Platform Build Tools?

Microsoft Power Platform Build Tools are a collection of Power Platform specific Azure DevOps build tasks that eliminate the need to manually download custom tooling and scripts to manage the application lifecycle of apps built on Microsoft Power Platform.

You can use the tasks individually, such as importing a solution into a downstream environment, or together in a pipeline to orchestrate a scenario such as *generate a build artifact, deploy to test, or harvest maker changes*. The build tasks can largely be categorized into four types:

- Helper
- Quality check
- Solution
- Environment management

For more information about the available tasks, see [Microsoft Power Platform Build Tools tasks](#).

What is Power Platform Build Tools version 2.0?

In addition to what Power Platform Build Tools provide, version 2.0 of Power Platform Build Tools is Power Platform CLI based.

Power Platform Build Tools version 1.0 is based on PowerShell. Power Platform Build Tools 2.0 is the version that will be serviced and newer features added.

For previous versions of Power Platform Build Tools, we'll do critical security updates as needed. We would highly recommend that customers move to Power Platform Build Tools version 2.0 at their earliest.

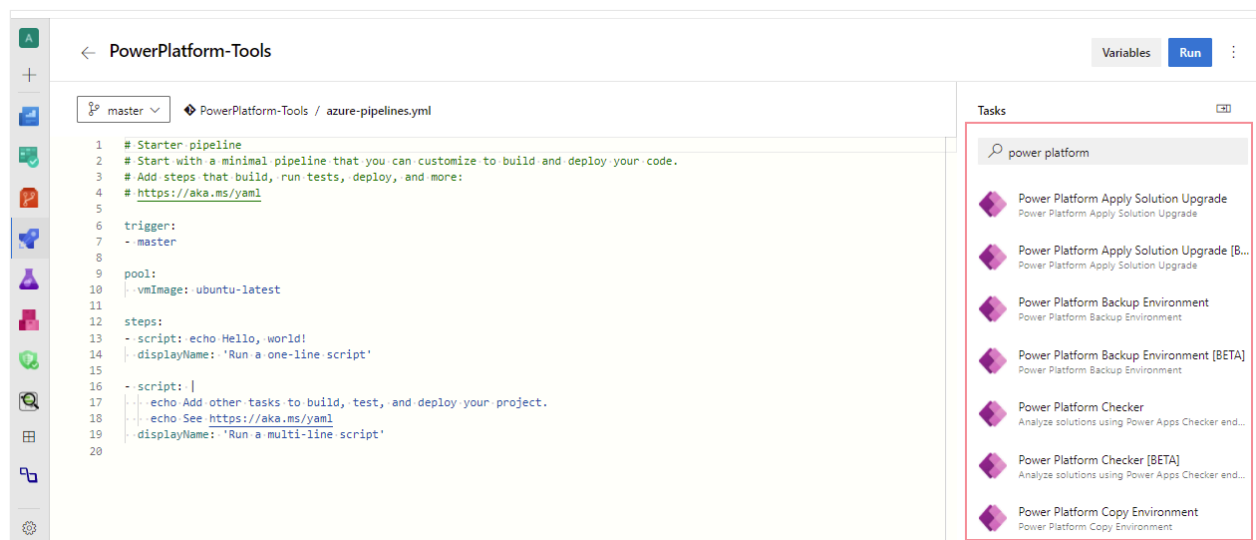
Important

You cannot mix and match task versions for Power Platform Build Tools in your build pipeline.

Get Microsoft Power Platform Build Tools

Microsoft Power Platform Build Tools can be installed into your Azure DevOps organization from [Azure Marketplace](#).

After installation, all tasks included in the Microsoft Power Platform Build Tools will be available to add into any new or existing pipeline. You can find them by searching for "Power Platform".



Connection to environments

To interact with the Microsoft Power Platform environment, you must establish a connection so the build tool tasks can perform the required actions. Two types of connections are available:

 Expand table

Connection type	Description
Service Principal via Workload Identity Federation (recommended)	Uses service principal based authentication via federated credentials. Recommended for tenants which require multi-factor authentication.
Service principal and client secret	Uses service principal based authentication.
Username/password	A generic service connection with username and password. Does not support users requiring multi-factor authentication.

Configure service connections using a service principal

To configure a connection using service principal, you must first have a Power Platform profile authenticated with the required permissions. You must add a Microsoft Entra ID application and associated application user to the Dataverse environment.

Create service principal and client secret

Use the [pac admin create-service-principal](#) command to create and configure the service principal to be used with the Microsoft Power Platform Build Tools tasks.

This command first registers an Application object and corresponding Service Principal Name (SPN) in Microsoft Entra ID. Then it adds the application as an administrator user to the Microsoft Power Platform tenant.

This command has two parameters:

 Expand table

name	short name	Required	Description
environment	env	Yes	The ID or URL of the environment to add an application user to.
role	r	No	Name or ID of security role to be applied to application user. The default value is: 'System Administrator'. Solution Checker requires a role with prvAppendmsdyn_analysisjob privilege to append to the msdyn_analysisjob table. The System Administrator role has this privilege by default.

You can use it like this:

PowerShell

```
C:\> pac admin create-service-principal --environment <environment id>
```

When successful, four columns are displayed:

- Power Platform TenantId
- Application ID
- Client Secret (in clear text)
- Expiration

For example:

PowerShell

```
PS C:\>pac admin create-service-principal --environment d3fcc479-0122-e7af-9965-bde57f69ee1d
Connected as admin@M365x57236726.onmicrosoft.com
Successfully assigned user adde6d52-9582-4932-a43a-beca5d182301 to
environment d3fcc479-0122-e7af-9965-bde57f69ee1d with security role System
Administrator
Tenant ID                               Application ID
Client Secret                           Expiration
2b0463ed-efd7-419d-927d-a9dca49d899c  adde6d52-9582-4932-a43a-beca5d182301
beY8Q~JBZ~CBDgIKKBjbZ3g6BofKzoZkYj23Hbf 7/31/2024 4:27:03 PM
```

Use the information displayed to configure the Power Platform service connection.

Important

Keep the client secret safe and secure. Once the command prompt is cleared, you cannot retrieve the same client secret again.

Configure environment with the Application ID

You must add the Application ID as an Application User in the Microsoft Power Platform environment you're connecting to. There are two ways you can do this.

- You can review this article: [Application user creation](#).
- You can use Power Platform CLI. For example:

PowerShell

```
PS C:\>pac admin assign-user --user "adde6d52-9582-4932-a43a-beca5d182301" -  
-role "System administrator" --environment d3fcc479-0122-e7af-9965-  
bde57f69ee1d --application-user  
Connected to... SnDemoDev  
Connected as admin@M365x57236726.onmicrosoft.com  
Successfully assigned user adde6d52-9582-4932-a43a-beca5d182301 to  
environment d3fcc479-0122-e7af-9965-bde57f69ee1d with security role System  
Administrator
```

Ensure that the added Application User has the system administrator role assigned (available from "Manage Roles" in the security settings for the application user).

Frequently asked questions (FAQs)

Will my old Microsoft Power Platform Build Tool pipelines continue to work?

The older pipelines continue to work, but newer features won't be added to the older tasks. If you want to exercise the newer features, then we would highly recommend that you change the version of your task in the pipeline.

Why did you base the new task on Power Platform CLI instead of PowerShell?

The Power Platform CLI tasks are multi-platform with great flexibility including running on linux runners. These are the same tasks that are available with Power Platform Actions for GitHub. You now have a consistent experience between Azure DevOps and GitHub.

Do the Microsoft Power Platform Build Tools only work for Power Apps?

The build tools work for both canvas and model-driven apps, Microsoft CoPilot Studio virtual agents, UI Flows and traditional flows, AI Builder, custom connectors and dataflows, all of which can now be added to a solution. This list also includes customer engagement

apps (Dynamics 365 Sales, Customer Service, Field Service, Marketing, and Project Service Automation). Separate build tasks are available for Finance and Operations applications.

I had previously installed the preview of the Build Tools - can I upgrade from the preview of Power Apps Build Tools to Power Platform Build Tools?

You can't upgrade from the preview version as we had to introduce some breaking changes in the Generally Available release. To move from the preview version, you have to install the Microsoft Power Platform Build Tools and either rebuild your pipelines, or reconfigure your existing pipelines to use the new Build Tools tasks. You must also create new Service connections as well.

Can I include flow and canvas apps?

Yes, flows and canvas apps are solution aware so if these components are added to your solution, they can participate in the lifecycle of your app. However, some steps still require manual configurations. The need for manual configuration will be addressed later this year when we introduce environment variables and connectors. A list of current limitations are available here: [Known limitations](#).

How much do the Microsoft Power Platform Build Tools cost?

The build tools are available at no cost. However, a valid subscription to Azure DevOps is required to utilize the Build Tools. More information is available [Pricing for Azure DevOps](#).

I can see the extension, but why don't I have an option to install it?

*If you don't see the **install** option, then you most likely lack the necessary install privileges in your Azure DevOps organization. More info available [Manage extension permissions](#).*

How can developers use the results of the Checker task?

The output of the Checker task is a [Sarif file](#) and both VS Code and Visual Studio extensions are available for viewing and taking action on Sarif files.

See Also

[Power Platform Build Tools for Azure DevOps release notes](#)

[Build tool tasks](#)

[Microsoft Power Platform Build Tools labs](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

Microsoft Power Platform Build Tools tasks

Article • 12/03/2024

The available build tasks are described in the following sections. Afterwards, we showcase some example Azure Pipelines making use of these tasks. For information about the build tools and how to download them, see [Microsoft Power Platform Build Tools for Azure Pipelines](#).

Helper task

The available helper tasks are described next.

Power Platform Tool Installer

This task is required to be added once before any other Power Platform Build Tools tasks in build and release pipelines. This task installs a set of Power Platform–specific tools required by the agent to run the Microsoft Power Platform build tasks. This task doesn't require any more configuration when added. This task contains parameters for the specific versions of each of the tools that are being installed.

To stay up to date with the tool versions over time, make sure these parameters correspond to the versions of the tools that are required for the pipeline to run properly.

YAML snippet (Installer)

yml

```
# Installs default Power Platform Build Tools
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.tool-
  installer.PowerPlatformToolInstaller@2
  displayName: 'Power Platform Tool Installer'
```

yml

```
# Installs specific versions of the Power Platform Build Tools
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.tool-
  installer.PowerPlatformToolInstaller@2
  displayName: 'Power Platform Tool Installer'
  inputs:
```

DefaultVersion: false
XrmToolingPackageDeploymentVersion: 3.3.0.928

Parameters (Installer)

 Expand table

Parameters	Description
DefaultVersion Use default tool versions	Set to true to use the default version of all tools, otherwise false . Required (and false) when any tool versions are specified.
PowerAppsAdminVersion XrmToolingPackageDeploymentVersion MicrosoftPowerAppsCheckerVersion CrmSdkCoreToolsVersion Tool version	The specific version of the tool to use.

Power Platform WhoAmI

Verifies a Power Platform environment service connection by connecting and making a WhoAmI request. This task can be useful to include early in the pipeline, to verify connectivity before processing begins.

YAML snippet (WhoAmI)

yml

```
# Verifies an environment service connection
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.whoami.PowerPlatformWhoAmi@2
  displayName: 'Power Platform WhoAmI'

  inputs:
  # Service Principal/client secret (supports MFA)
  authenticationType: PowerPlatformSPN
  PowerPlatformSPN: 'My service connection'
```

yml

```
# Verifies an environment service connection
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.whoami.PowerPlatformWhoAmi@2
  displayName: 'Power Platform WhoAmI'
```

```
inputs:
# Username/password (no MFA support)
PowerPlatformEnvironment: 'My service connection'
```

Parameters (WhoAml)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Optional) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection. More information: see BuildTools.EnvironmentUrl under Power Platform Create Environment
PowerPlatformEnvironment Power Platform environment URL	The service endpoint for the environment to connect to. Defined under Service Connections in Project Settings . More information: see BuildTools.EnvironmentUrl under Power Platform Create Environment
PowerPlatformSPN Power Platform Service Principal	The service endpoint for the environment to connect to. Defined under Service Connections in Project Settings .

Quality check

In the next section are the available tasks for checking the quality of a solution.

Power Platform Checker

This task runs a static analysis check on your solutions against a set of best-practice rules to identify any problematic patterns that you might inadvertently introduced when building your solution.

YAML snippet (Checker)

```
yml

# Static analysis check of your solution
- task: microsoft-IsvExpTools.PowerPlatform-
  BuildTools.checker.PowerPlatformChecker@2
  displayName: 'Power Platform Checker '
```

```
inputs:
  PowerPlatformSPN: 'Dataverse service connection'
  RuleSet: '0ad12346-e108-40b8-a956-9a8f95ea18c9'
```

yml

```
# Static analysis check of your solution
- task: microsoft-IsvExpTools.PowerPlatform-
  BuildTools.checker.PowerPlatformChecker@2
  displayName: 'Power Platform Checker '
  inputs:
    PowerPlatformSPN: 'Dataverse service connection'
    UseDefaultPACheckerEndpoint: false
    CustomPACheckerEndpoint: 'https://japan.api.advisor.powerapps.com/'
    FileLocation: sasUriFile
    FilesToAnalyzeSasUri: 'SAS URI'
    FilesToAnalyze: '**\*.zip'
    FilesToExclude: '**\*.tzip'
    RulesToOverride: 'JSON array'
    RuleSet: '0ad12346-e108-40b8-a956-9a8f95ea18c9'
```

Parameters (Checker)

 Expand table

Parameters	Description
<code>PowerPlatformSPN</code> Service Connection	(Required) A connection to a licensed Microsoft Power Platform environment is required to use the Power Platform checker. Service connections are defined in Service Connections under Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment Note: Service Principal and username/password authentication methods are available for the checker task. For more information on how to configure service principals to be used with this task, see Configure service principal connections for Power Platform environments .
<code>UseDefaultPACheckerEndpoint</code> Use default Power Platform Checker endpoint	By default (true), the geographic location of the checker service uses the same geography as the environment you connect to.
<code>CustomPACheckerEndpoint</code> Custom PAC checker endpoint	Required when <code>UseDefaultPACheckerEndpoint</code> is false . You may specify another geo to use, for example <code>https://japan.api.advisor.powerapps.com</code> . For a list of

Parameters	Description
	available geographies, see Use the Power Platform Checker API .
FileLocation Location of files to analyze	Required when referencing a file from a shared access signature (SAS) URL <code>sasUriFile</code>. Note: It's important to reference an exported solution file and not the unpacked source files in your repository. Both managed and unmanaged solution files can be analyzed.
FilesToAnalyzeSasUri SAS files to analyze	Required when <code>FileLocation</code> is set to <code>sasUriFile</code>. Enter the SAS URI. You can add more than one SAS URI through a comma (,) or semi-colon (;) separated list.
FilesToAnalyze Local files to analyze	Required when SAS files aren't analyzed. Specify the path and file name of the zip files to analyze. Wildcards can be used. For example, enter <code>***.zip</code> for all zip files in all subfolders.
FilesToExclude Local files to exclude	Specify the names of files to be excluded from the analysis. If more than one, provide through a comma (,) or semi-colon (;) separated list. This list can include a full file name or a name with leading or trailing wildcards, such as <code>*jquery</code> or <code>form.js</code>
RulesToOverride Rules to override	A JSON array containing rules and levels to override. Accepted values for <code>OverrideLevel</code> are: Critical, High, Medium, Low, Informational. Example: <code>[{"Id":"meta-remove-dup-reg","OverrideLevel":"Medium"}, {"Id":"il-avoid-specialized-update-ops","OverrideLevel":"Medium"}]</code>
RuleSet Rule set	(Required) Specify which rule set to apply. The following two rule sets are available: • Solution checker: This rule set is the same one that is run from the Power Apps maker portal. • AppSource: The extended rule set that is used to certify an application before it can be published to AppSource.
ErrorLevel Error Level	Combined with the error, threshold parameter defines the severity of errors and warnings that are allowed. Supported threshold values are <code><level>IssueCount</code> where <code>level</code>=Critical, High, Medium, Low, and Informational.
ErrorThreshold Error threshold	Defines the number of errors (<code>>=0</code>) of a specified level that are allowed for the checker to pass the solutions

Parameters	Description
	being checked.
<code>FailOnPowerAppsCheckerAnalysisError</code> Fail on error	When true , fail if the Power Apps Checker analysis is returned as Failed or FinishedWithErrors.
<code>ArtifactDestinationName</code> DevOps artifact name	Specify the Azure Artifacts name for the checker .sarif file.

Solution tasks

This set of tasks can automate solution actions. The environment tasks outlined later in this section that create, copy, or restore an environment overwrite the service connections with the newly created environments. These tasks make it possible to perform solution tasks against environments that are created on demand.

Power Platform Import Solution

Imports a solution into a target environment.

YAML snippet (Import)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.import-
  solution.PowerPlatformImportSolution@2
  displayName: 'Power Platform Import Solution '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
    SolutionInputFile: 'C:\Public\Contoso_sample_1_0_0_1_managed.zip'
    HoldingSolution: true
    OverwriteUnmanagedCustomizations: true
    SkipProductUpdateDependencies: true
    ConvertToManaged: true
```

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.import-
  solution.PowerPlatformImportSolution@2
  displayName: 'Power Platform Import Solution '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
```

```
SolutionInputFile: 'C:\Public\Contoso_sample_1_0_0_1_managed.zip'
AsyncOperation: true
MaxAsyncWaitTime: 60
PublishWorkflows: false
```

Parameters (Import)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to import the solution into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to import the solution into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
SolutionInputFile Solution input file	(Required) The path and file name of the solution .zip file to import into the target environment (for example, <code>\$(Build.ArtifactStagingDirectory)\$(SolutionName).zip</code>). Note: Variables give you a convenient way to get key bits of data into various parts of your pipeline. See Use predefined variables for a comprehensive list.
HoldingSolution Import as a holding solution	An advance parameter (true false) used when a solution needs to be upgraded. This parameter hosts the solution in Dataverse but does not upgrade the solution until the Apply Solution Upgrade task is run.
OverwriteUnmanagedCustomizations Overwrite un-managed customizations	Specify whether to overwrite un-managed customizations (true false).

Parameters	Description
<code>SkipProductUpdateDependencies</code> Skip product update dependencies	Specify whether the enforcement of dependencies related to product updates should be skipped (true false).
<code>ConvertToManaged</code> Convert to managed	Specify whether to import as a managed solution (true false).
<code>AsyncOperation</code> Asynchronous import	If selected (true), the import operation will be performed asynchronously. This is recommended for larger solutions as this task will automatically timeout after 4 minutes otherwise. Selecting asynchronous will poll and wait until <code>MaxAsyncWaitTime</code> has been reached (true false).
<code>MaxAsyncWaitTime</code> Maximum wait time	Maximum wait time in minutes for the asynchronous operation; default is 60 min (1 hr), same as Azure DevOps default for tasks.
<code>PublishWorkflows</code> Activate processes after import	Specify whether any processes (workflows) in the solution should be activated after import (true false).
<code>UseDeploymentSettingsFile</code> Use deployment settings file	Connection references and environment variable values can be set using a deployment settings file (true false).
<code>DeploymentSettingsFile</code> Deployment settings file	(Required when <code>UseDeploymentSettingsFile</code> = true) The path and file name of the deployment settings file.

Power Platform Add Solution Component

Adds a solution component to an unmanaged solution.

YAML snippet (Add)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.add-solution-component.PowerPlatformAddSolutionComponent@2
  displayName: 'Power Platform Add Solution Component '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    SolutionName: 'SampleSolution '
    Component: contact
    ComponentType: 1
    AddRequiredComponents: false
```

Parameters (Add)

 Expand table

Parameters	Description
ComponentId ID of the solution component	The schema name or ID of the component to add to the target solution.
ComponentType Power Platform environment URL	The value that represents the solution component that you are adding. See Use predefined component types for a comprehensive list of component values.
SolutionUniqueName Name of the solution	Unique name of the solution.
AddRequiredComponents Any components required from other solution dependent on the solution component	(Optional) Indicates whether other solution components that are required by the solution component that you are adding should also be added to the unmanaged solution.
Environment Environment URL or ID	(Optional) Environment URL or ID of the target environment.

Power Platform Apply Solution Upgrade

Upgrades a solution that has been imported as a holding solution.

YAML snippet (Upgrade)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.apply-solution-upgrade.PowerPlatformApplySolutionUpgrade@2
  displayName: 'Power Platform Apply Solution Upgrade '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
    SolutionName: 'Contoso_sample'
    AsyncOperation: false
```

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.apply-solution-upgrade.PowerPlatformApplySolutionUpgrade@2
  displayName: 'Power Platform Apply Solution Upgrade '
```

```
inputs:
  authenticationType: PowerPlatformSPN
  PowerPlatformSPN: 'Dataverse service connection '
  SolutionName: 'Contoso_sample'
  MaxAsyncWaitTime: 45
```

Parameters (Upgrade)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to upgrade the solution into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to upgrade the solution into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
SolutionName Solution name	(Required) The name of the solution to apply the upgrade. Always use the solution <i>Name</i> not its <i>Display Name</i> .
AsyncOperation Asynchronous upgrade	If selected (true), the upgrade operation will be performed as an asynchronous batch job. Selecting asynchronous will poll and wait until <code>MaxAsyncWaitTime</code> has been reached.
MaxAsyncWaitTime Maximum wait time	Maximum wait time in minutes for the asynchronous operation; default is 60 min (1 hr), same as Azure DevOps default for tasks.

Note

Variables give you a convenient way to get key bits of data into various parts of your pipeline. See [Use predefined variables](#) for a comprehensive list. You can pre-populate connection reference and environment variables information for the target environment while importing a solution using a deployment settings file.

More information: [Pre-populate connection references and environment variables for automated deployments](#)

Power Platform Export Solution

Exports a solution from a source environment.

YAML snippet (Export)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.export-
  solution.PowerPlatformExportSolution@2
  displayName: 'Power Platform Export Solution '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
    SolutionName: 'Contoso_sample'
    SolutionOutputFile: 'C:\Public\Contoso_sample_1_0_0_1_managed.zip'
    Managed: true
    MaxAsyncWaitTime: 120
```

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.export-
  solution.PowerPlatformExportSolution@2
  displayName: 'Power Platform Export Solution '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
    SolutionName: 'Contoso_sample'
    SolutionOutputFile: 'C:\Public\Contoso_sample_1_0_0_1_managed.zip'
    Managed: true
    MaxAsyncWaitTime: 120
    ExportAutoNumberingSettings: true
    ExportCalendarSettings: true
    ExportCustomizationSettings: true
    ExportEmailTrackingSettings: true
    ExportGeneralSettings: true
    ExportIsvConfig: true
    ExportMarketingSettings: true
    ExportOutlookSynchronizationSettings: true
    ExportRelationshipRoles: true
    ExportSales: true
```

Parameters (Export)

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformEnvironment</code> Power Platform environment URL	The service endpoint that you want to upgrade the solution into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint that you want to upgrade the solution into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
<code>SolutionName</code> Solution name	(Required) The name of the solution to export. Always use the solution <i>Name</i> not its <i>Display Name</i> .
<code>SolutionOutputFile</code> Solution output file	(Required) The path and file name of the solution.zip file to export the source environment to (for example, <code>\$(Build.ArtifactStagingDirectory)\$(SolutionName).zip</code>). Note: Variables give you a convenient way to get key bits of data into various parts of your pipeline. See Use predefined variables for a comprehensive list.
<code>AsyncOperation</code> Asynchronous export	If selected (true), the export operation will be performed as an asynchronous batch job. Selecting asynchronous will poll and wait until <code>MaxAsyncWaitTime</code> has been reached.
<code>MaxAsyncWaitTime</code> Maximum wait time	Maximum wait time in minutes for the asynchronous operation; default is 60 min (1 hr), same as Azure DevOps default for tasks.
<code>Managed</code> Export as managed	If selected (true), export the solution as a managed solution; otherwise export as an unmanaged solution.

Parameters	Description
<code>ExportAutoNumberingSettings</code> Export auto-numbering settings	Export auto-numbering settings (true false).
<code>ExportCalendarSettings</code> Export calendar settings	Export calendar settings (true false).
<code>ExportCustomizationSettings</code> Export customization settings	Export customization settings (true false).
<code>ExportEmailTrackingSettings</code> Export email tracking settings	Export email tracking settings (true false).
<code>ExportGeneralSettings</code> Export general settings	Export general settings (true false).
<code>ExportIsvConfig</code> Export ISV configuration	Export ISV configuration (true false).
<code>ExportMarketingSettings</code> Export marketing settings	Export marketing settings (true false).
<code>ExportOutlookSynchronizationSettings</code> Export Outlook sync settings	Export Outlook synchronization settings (true false).
<code>ExportRelationshipRoles</code> Export relationship roles	Export relationship roles (true false).
<code>ExportSales</code> Exports sales	Exports sales (true false).

Power Platform Unpack Solution

Takes a compressed solution file and decomposes it into multiple XML files so that these files can be more easily read and managed by a source control system.

YAML snippet (Unpack)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.unpack-
  solution.PowerPlatformUnpackSolution@2
  displayName: 'Power Platform Unpack Solution '
  inputs:
    SolutionInputFile: 'C:\Public\Contoso_sample_1_0_0_1_managed.zip'
```

```
SolutionTargetFolder: 'C:\Public'  
SolutionType: Both
```

Parameters (Unpack)

 Expand table

Parameters	Description
<code>SolutionInputFile</code> Solution input file	(Required) The path and file name of the solution.zip file to unpack.
<code>SolutionTargetFolder</code> Target folder to unpack solution	(Required) The path and target folder you want to unpack the solution into.
<code>SolutionType</code> Type of solution	(Required) The type of solution you want to unpack. Options include: Unmanaged (recommended), Managed , and Both .

Power Platform Pack Solution

Packs a solution represented in source control into a solution.zip file that can be imported into another environment.

YAML snippet (Pack)

```
yml  
  
steps:  
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.pack-  
  solution.PowerPlatformPackSolution@2  
  displayName: 'Power Platform Pack Solution '  
  inputs:  
    SolutionSourceFolder: 'C:\Public'  
    SolutionOutputFile: 'Contoso_sample_1_0_0_1_managed.zip'  
    SolutionType: Managed
```

Parameters (Pack)

 Expand table

Parameters	Description
<code>SolutionOutputFile</code> Solution output file	(Required) The path and file name of the solution.zip file to pack the solution into.
<code>SolutionSourceFolder</code> Source folder of solution to pack	(Required) The path and source folder of the solution to pack.
<code>SolutionType</code> Type of solution	(Required) The type of solution you want to pack. Options include: Managed (recommended), Unmanaged , and Both .

Power Platform Delete Solution

Deletes a solution in the target environment.

YAML snippet (Delete)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.delete-
  solution.PowerPlatformDeleteSolution@2
  displayName: 'Power Platform Delete Solution '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
    SolutionName: 'Contoso_sample'
```

Parameters (Delete)

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformEnvironment</code> Power Platform environment URL	The service endpoint that you want to delete the solution (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment

Parameters	Description
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to delete the solution (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
SolutionName Solution name	(Required) The name of the solution to delete. Always use the solution <i>Name</i> not its <i>Display Name</i> .

Power Platform Publish Customizations

Publishes all customizations in an environment.

YAML snippet (Publish)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.publish-
  customizations.PowerPlatformPublishCustomizations@2
  displayName: 'Power Platform Publish Customizations '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
```

Parameters (Publish)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to publish the customizations (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment

Parameters	Description
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to publish the customizations (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment

Power Platform Set Solution Version

Updates the version of a solution.

YAML snippet (Version)

yml
<pre> steps: - task: microsoft-IsvExpTools.PowerPlatform-BuildTools.set-solution-version.PowerPlatformSetSolutionVersion@2 displayName: 'Power Platform Set Solution Version ' inputs: authenticationType: PowerPlatformSPN PowerPlatformSPN: 'Dataverse service connection ' SolutionName: 'Contoso_sample' SolutionVersionNumber: 1.0.0.0 </pre>

Parameters (Version)

[Expand table](#)

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to set the solution version (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
PowerPlatformSPN Power Platform Service	The service endpoint that you want to set the solution version (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined

Parameters	Description
Principal	under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
<code>SolutionName</code> Solution name	(Required) The name of the solution to set the version for. Always use the solution <i>Name</i> not its <i>Display Name</i> .
<code>SolutionVersionNumber</code> Solution version number	(Required) Version number you want to set.

While version number can be hardcoded in the pipeline, it is recommended to use an Azure DevOps pipeline variable like `BuildId`. This provides options to define the exact shape of version number under the "Options" tab, for example:

```
$(Year:yyyy)-$(Month:MM)-$(Day:dd)-$(rev:rr)-3
```

This definition can then be used in the Set Solution Version task by setting the Version Number property with: `$(Build.BuildId)` instead of hard coding 20200824.0.0.2.

Alternatively a powershell inline task script `$(Get-Date -Format yyyy.MM.dd.HH:mm)` output set to empty variable named `SolutionVersion` as `Write-Host ("##vso[task.setvariable variable=SolutionVersion;]$version")`, Set Solution Version as `$(SolutionVersion)`.

Power Platform Set Connection Variables

Sets `BuildTools.*` variables to provide custom script tasks access to use the service connection as a single source of truth.

YAML snippet (SetConnectionVariables)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.set-connection-variables.PowerPlatformSetConnectionVariables@2
  displayName: 'Power Platform Set Connection Variables '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
  timeoutInMinutes: 2
  retryCountOnTaskFailure: 5
```

Parameters (SetConnectionVariables)

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint that you want to set connection variables for. Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
<code>timeoutInMinutes</code> Timeout	Specifies the maximum time, in minutes, that a task is allowed to execute before being canceled by server. A zero value indicates an infinite timeout.
<code>retryCountOnTaskFailure</code> Number of retries if task failed	Specifies the number of retries for this task that will happen in case of task failure. Not supported for agent-less tasks.
<code>ApplicationId</code> Application Id for login	(Required for username/password) Application Id to use for login (See additional note below).
<code>RedirectUri</code> Redirect URI of the specified app	(Required for username/password) Redirect URI of the specified app. Needed when specifying an App Id. (See additional note below).

Note

When using the username/password authentication type, the Application Id and Redirect Uri will be required. This [authenticate oauth](#) document has an explanation of why you need to create an app registration to authenticate with Dataverse. When you connect using username/password authentication, the connection behaves as an application which is connecting on behalf of a user to Dataverse.

Power Platform Deploy Package

Deploys a package to an environment. Deploying a [package](#) as opposed to a single solution file provides an option to deploy multiple solutions, data, and code into an environment.

YAML snippet (Deploy)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.deploy-
package.PowerPlatformDeployPackage@2
  displayName: 'Power Platform Deploy Package '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
    PackageFile: 'C:\Users\Public\package.dll'
    MaxAsyncWaitTime: 120
```

Parameters (Deploy)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to deploy the package into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to deploy the package into (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type. More information: see <code>BuildTools.EnvironmentUrl</code> under Power Platform Create Environment
PackageFile Package file	(Required) The path and file name of the package file assembly (.dll).
MaxAsyncWaitTime Maximum wait time	Maximum wait time in minutes for the asynchronous operation; default is 60 min (1 hr), same as Azure DevOps default for tasks.

Environment management tasks

Automate common Environment Lifecycle Management (ELM) tasks.

Power Platform Create Environment

Creates a new environment. Creating a new environment also automatically creates `BuildTools.EnvironmentUrl`.

📘 Important

When set, `BuildTools.EnvironmentUrl` will be used as the **default service connection** for subsequent tasks in the pipeline. Each task described in this article only uses the endpoint from the service connection when `BuildTools.EnvironmentUrl` is not set.

A new environment can only be provisioned if your license or capacity allows for the creation of additional environments. For more information on how to view capacity see [Capacity page details](#).

YAML snippet (Create-env)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.create-
  environment.PowerPlatformCreateEnvironment@2
  displayName: 'Power Platform Create Environment '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
    DisplayName: 'First Coffee test'
    DomainName: firstcoffee
```

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.create-
  environment.PowerPlatformCreateEnvironment@2
  displayName: 'Power Platform Create Environment '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
    DisplayName: 'First Coffee prod'
    EnvironmentSku: Production
    AppsTemplate: 'D365_CustomerService,D365_FieldService'
    LocationName: canada
    LanguageName: 1036
    CurrencyName: ALL
    DomainName: firstcoffee
```

Parameters (Create-env)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to create the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to create the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
DisplayName Display name	(Required) The display name of the environment created.
LocationName Deployment region	(Required) The region that the environment should be created in.
EnvironmentSku Environment type	(Required) The type of instance to deploy. Options are Sandbox , Production , Trial , and SubscriptionBasedTrial .
AppsTemplate Apps	For a non-trial environment type, the supported apps are D365_CustomerService, D365_FieldService, D365_ProjectServiceAutomation, and D365_Sales.
CurrencyName Currency	(Required) Base currency for the environment created. The currency cannot be updated after the environment is created.
LanguageName Language	(Required) The base language in the environment.
DomainName Domain Name	(Required) This is the environment-specific string that forms part of the URL. For example, for an environment with the following URL: <code>https://powerappsbuildtasks.crm.dynamics.com/</code>, the domain name would be 'powerappsbuildtasks'. Note: If you enter a domain name that's already in use, the task appends a numeric value to the domain name, starting with 0. For the example above, the URL might become <code>https://powerappsbuildtasks0.crm.dynamics.com/</code>.

Power Platform Delete Environment

Deletes an environment.

YAML snippet (Delete-env)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.delete-
  environment.PowerPlatformDeleteEnvironment@2
  displayName: 'Power Platform Delete Environment '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
```

Parameters (Delete-env)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to delete the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to delete the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.

Power Platform Assign User

Assign a user to a target environment with specified security role.

YAML snippet (AssignUser-env)

yml

```

steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.assign-
  user.PowerPlatformAssignUser@2
  displayName: 'Power Platform Assign User '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    User: EnvironmentUsername
    Role: 'Environment Maker'
    ApplicationUser: true

```

Parameters (AssignUser-env)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformSPN Power Platform Service Principal	The service endpoint for the environment that you want to assign the user to (for example, https://powerappsbuildtools.crm.dynamics.com). Defined under Service Connections in Project Settings using the Power Platform connection type.
User Power Platform user name	Microsoft Entra object ID or user's principal name to assign to the target environment.
Role Security role name or ID	Security role name or ID to be assigned to the user.
ApplicationUser Power Platform application user name	Specifies whether the input user is an application user (true false).

Power Platform Reset Environment

Resets an environment.

YAML snippet (Reset-env)

```
yml
```

```

steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.reset-
environment.PowerPlatformResetEnvironment@2
  displayName: 'Power Platform Reset Environment '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    CurrencyName: ALL
    Purpose: 'Reset to PointInTime'
    AppsTemplate: 'D365_CustomerService,D365_FieldService'

```

Parameters (Reset-env)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint that you want to reset the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
PowerPlatformSPN Power Platform Service Principal	The service endpoint that you want to reset the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.

Power Platform Backup Environment

Backs up an environment.

YAML snippet (Backup-env)

yml

```

steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.backup-
environment.PowerPlatformBackupEnvironment@2
  displayName: 'Power Platform Backup Environment '
  inputs:

```

```
PowerPlatformEnvironment: 'My service connection'
BackupLabel: 'Full backup - $(Build.BuildNumber)'
```

Parameters (Backup-env)

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformEnvironment</code> Power Platform environment URL	The service endpoint that you want to back up the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint that you want to back up the environment (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
<code>BackupLabel</code> Backup label	(Required) The label to be assigned to the backup.

Power Platform Copy Environment

Copies an environment to a target environment. Two types of copies are available: full and minimal. A *Full* copy includes both data and solution metadata (customizations), whereas a *minimal* copy only includes solution metadata and not the actual data.

YAML snippet (Copy-env)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.copy-
  environment.PowerPlatformCopyEnvironment@2
  displayName: 'Power Platform Copy Environment '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
    TargetEnvironmentUrl: 'https://contoso-test.crm.dynamics.com'
```

yml

```

steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.copy-
  environment.PowerPlatformCopyEnvironment@2
  displayName: 'Power Platform Copy Environment '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: 'Dataverse service connection '
    TargetEnvironmentUrl: 'https://contoso-test.crm.dynamics.com'
    CopyType: MinimalCopy
    OverrideFriendlyName: true
    FriendlyName: 'Contoso Test'
    SkipAuditData: true

```

Parameters (Copy-env)

[Expand table](#)

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint for the source environment that you want to copy from (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
PowerPlatformSPN Power Platform Service Principal	The service endpoint for the source environment that you want to copy from (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
TargetEnvironmentUrl Target environment URL	(Required) The URL for the target environment that you want to copy to.
CopyType Copy type	The type of copy to perform: FullCopy or MinimalCopy
OverrideFriendlyName Override friendly name	Change the target environment's friendly name to another name (true false).
FriendlyName Friendly name	The friendly name of the target environment.
SkipAuditData	Whether to skip audit data during copy operation. (true false).

Parameters	Description
SkipAuditData	

Power Platform Restore Environment

Restores an environment to a given backup.

YAML snippet (Restore-env)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.restore-
  environment.PowerPlatformRestoreEnvironment@2
  displayName: 'Power Platform Restore Environment '
  inputs:
    PowerPlatformEnvironment: 'My service connection'
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    TargetEnvironmentUrl: 'https://contoso-test.crm.dynamics.com'
    RestoreLatestBackup: false
    RestoreTimeStamp: '12/01/2022 09:00'
    FriendlyName: 'Contoso Test'
```

Parameters (Restore-env)

[Expand table](#)

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformEnvironment Power Platform environment URL	The service endpoint for the source environment that you want to restore from (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
PowerPlatformSPN Power Platform Service Principal	The service endpoint for the source environment that you want to restore from (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.

Parameters	Description
<code>RestoreLatestBackup</code> Latest backup to restore	Whether to restore latest backup or provide <code>RestoreTimeStamp</code> (true false).
<code>RestoreTimeStamp</code> DateTime of the backup	DateTime of the backup in 'mm/dd/yyyy hh:mm' format OR string 'latest'.
<code>TargetEnvironmentUrl</code> Target environment URL	(Required) The URL for the target environment that you want to restore.
<code>FriendlyName</code> Name of the restored environment	(Optional) Name of the restored environment.
<code>DisableAdminMode</code> Disable admin mode	Whether to disable administration mode (true false).

Export Dataverse Data

Export data from an environment with the provided schema.

YAML snippet (exportdata-env)

yml
<pre> steps: - task: microsoft-IsvExpTools.PowerPlatform-BuildTools.export- data.PowerPlatformExportData@2 displayName: 'Power Platform Export Data ' inputs: authenticationType: PowerPlatformSPN PowerPlatformSPN: Build SchemaFile: 'C:\tempschema.xml' DataFile: exportdata.zip Overwrite: true </pre>

Parameters (exportdata-env)

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.

Parameters	Description
PowerPlatformSPN Power Platform Service Principal	The service endpoint for the environment that you want to export data from (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service Connections in Project Settings using the Power Platform connection type.
SchemaFile Schema XML file name	Schema XML file name. It can be created using the Configuration Migration tool.
DataFile Data file name	File name for data to export (zip file).
Overwrite Content overwrite	Power Pages website content to overwrite (true false).

Import Dataverse Data

Import data to an environment.

YAML snippet (importdata-env)

yml
<pre> steps: - task: microsoft-IsvExpTools.PowerPlatform-BuildTools.import- data.PowerPlatformImportData@2 displayName: 'Power Platform Import Data ' inputs: authenticationType: PowerPlatformSPN PowerPlatformSPN: Build DataFile: 'C:\Data\Import.zip' </pre>

Parameters (importdata-env)

 Expand table

Parameters	Description
authenticationType Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
PowerPlatformSPN Power Platform	The service endpoint for the environment that you want to import data (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined

Parameters	Description
Service Principal	under Service Connections in Project Settings using the Power Platform connection type.
<code>DataFile</code> Data file name	File name for compressed zip) data file to import, or the folder with data-schema.xml and data.xml to be imported.

Power Pages management tasks

Tasks for working with a Power Pages website.

Power Platform Download PAPortal

Download Power Pages website content from the current Dataverse environment.

YAML snippet (download-paportal)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.download-paportal.PowerPlatformDownloadPaportal@2
  displayName: 'Power Platform Download PAPortal '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    DownloadPath: 'C:\portals'
    WebsiteId: 'f88b70cc-580b-4f1a-87c3-41debefeb902'
    Overwrite: true
```

Parameters (download-paportal)

[Expand table](#)

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint for the environment that you want to download content from (for example, <code>https://powerappsbuildtools.crm.dynamics.com</code>). Defined under Service

Parameters	Description
	Connections in Project Settings using the Power Platform connection type.
<code>WebsiteId</code> Portal website ID	Power Pages website ID to download.
<code>Overwrite</code> Content overwrite	Power Pages website content to overwrite (true false).
<code>ModelVersion</code> Site Data Model	Indicates if the site data to be uploaded will use the standard (1) or enhanced data model (2). Default value is '1'.

Power Platform Upload PAPortal

Upload Power Pages website content to the current Dataverse environment.

YAML snippet (upload-paportal)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools.upload-
  paportal.PowerPlatformUploadPaportal@2
  displayName: 'Power Platform Upload PAPortal '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    UploadPath: 'C:\portals\starter-portal'
```

Parameters (upload-paportal)

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint for the environment that you want upload content to (for example, https://powerappsbuildtools.crm.dynamics.com). Defined under Service Connections in Project Settings using the Power Platform connection type.

Parameters	Description
<code>UploadPath</code> Upload content path	Path from where the Power Pages website content will be uploaded.
<code>ModelVersion</code> Site Data Model	Indicates if the site data to be uploaded will use the standard (1) or enhanced data model (2). Default value is '1'.

Catalog for Power Platform tasks (preview)

These tasks perform actions against the catalog in Power Platform and include the following.

ⓘ Note

Currently, catalog tasks for Power Platform are a **preview** release.

Install catalog item

Install a catalog item to the target environment.

[Expand table](#)

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint for the environment that you want upload content to (for example, https://powerappsbuildtools.crm.dynamics.com). Defined under Service Connections in Project Settings using the Power Platform connection type.
<code>Environment</code> Target environment URL	(Required) Environment url this task targets.
<code>CatalogItemId</code> Catalog Item ID to be installed	(Required) Catalog item to be installed on the target environment.
<code>TargetEnvironmentUrl</code> Target environment URL	(Required) The Url of the target environment for catalog item installation into (for example, " https://YourOrg.crm.dynamics.com ").

Parameters	Description
<code>PollStatus</code> Check poll status	(Optional) Poll to check the status of your request (true false).

Parameters (catalog-install)

yml
<pre> steps: - task: microsoft-IsvExpTools.PowerPlatform-BuildTools-DEV.install-catalog.PowerPlatformInstallCatalog@2 displayName: 'Power Platform Install Catalog ' inputs: authenticationType: PowerPlatformSPN PowerPlatformSPN: Build Environment: '\$(BuildTools.EnvironmentUrl)' CatalogItemId: '00000000-0000-0000-0000-000000000001' TargetEnvironmentUrl: 'https://test.crm.dynamics.com/' PollStatus: true </pre>

Submit Catalog approval

Submit catalog approval request.

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint for the environment that you want upload content to (for example, " https://powerappsbuildtools.crm.dynamics.com "). Defined under Service Connections in Project Settings using the Power Platform connection type.
<code>Environment</code> Target environment URL	(Required) Environment url this task targets.
<code>CatalogSubmissionFile</code> Catalog submission file	(Required) Specify the path and file name of the catalog submission file.
<code>UsePackageSolutionZipFile</code> File type	File type: package or solution zip file.

Parameters	Description
<code>SolutionZipFile</code> Solution zip file name	Specify the path and file name of the solution zip file.
<code>PollStatus</code> Check poll status	(Optional) Poll to check the status of your request (true false).

Parameters (catalog-submit)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools-DEV.submit-catalog.PowerPlatformSubmitCatalog@2
  displayName: 'Power Platform Submit Catalog '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    Environment: '$(BuildTools.EnvironmentUrl)'
    CatalogSubmissionFile: submission.json
    UsePackageSolutionZipFile: true
    FileType: SolutionZipFile
    SolutionZipFile: solution.zip
    PollStatus: true
```

Catalog request status

Get the status of the catalog install/submit request.

 Expand table

Parameters	Description
<code>authenticationType</code> Type of authentication	(Required for SPN) Specify either PowerPlatformEnvironment for a username/password connection or PowerPlatformSPN for a Service Principal/client secret connection.
<code>PowerPlatformSPN</code> Power Platform Service Principal	The service endpoint for the environment that you want upload content to (for example, " https://powerappsbuildtools.crm.dynamics.com "). Defined under Service Connections in Project Settings using the Power Platform connection type.
<code>Environment</code> Target environment URL	(Required) Environment url this task targets.

Parameters	Description
TrackingId Request tracking ID	(Required) Request tracking ID.
RequestType Request type	(Required) Request type. (Values: Install Submit).

Parameters (catalog-status)

yml

```
steps:
- task: microsoft-IsvExpTools.PowerPlatform-BuildTools-DEV.catalog-
  status.PowerPlatformCatalogStatus@2
  displayName: 'Power Platform Catalog Status '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: Build
    Environment: '$(BuildTools.EnvironmentUrl)'
    TrackingId: '00000000-0000-0000-0000-000000000002'
    RequestType: Submit
```

Build and release pipelines

Now that we've identified what can be done using the build tools, let's see how you might apply these tools to your build and release pipelines. A conceptual overview is shown below. Let's view some details of the pipeline implementation using the build tool tasks in the sub-sections that follow.

To learn more about creating these pipelines and actually do hands-on pipeline authoring using the Microsoft Power Platform Build Tools, complete the [build tools labs](#), which you can download from GitHub.

More information about Azure Pipelines: [Use Azure Pipelines](#)

Build pipeline: Export a solution from a development environment (DEV)

The following figure shows the build tool tasks that you might add to a pipeline that exports a solution from a development environment.

Pipeline

Build pipeline

...

≡

Get sources

▶ HelloDevOps

🔗 master

Agent job 1

Run on agent

+

🔧

Power Platform Tool Installer

Power Platform Tool Installer

🔧

Power Platform Export Solution

Power Platform Export Solution

🔧

Power Platform Unpack Solution

Power Platform Unpack Solution

>_

Commit Solution to repo

Command line

Build pipeline: Build a managed solution

The following figure shows the build tool tasks that you might add to a pipeline that builds a managed solution.

Pipeline

Build pipeline

...

≡

Get sources

▶ HelloDevOps

🔗 master

Agent job 1

Run on agent

+

🔧

Power Platform Tool Installer

Power Platform Tool Installer

🔧

Power Platform Pack Solution

Power Platform Pack Solution

🔧

Power Platform Import Solution

Power Platform Import Solution

🔧

Power Platform Export Solution

Power Platform Export Solution

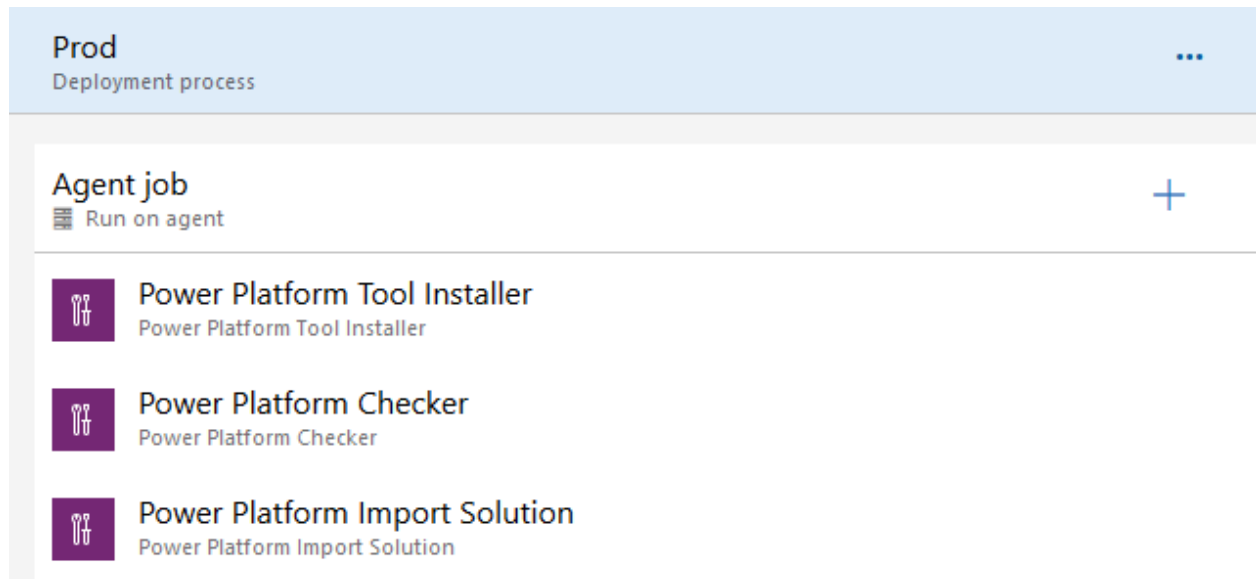
⬆

Publish Artifact: drop

Publish build artifacts

Release pipeline: Deploy to a production environment (PROD)

The following figure shows the build tool tasks that you might add to a pipeline that deploys to a production environment.



See Also

[Microsoft Power Platform Build Tools for Azure DevOps](#)

Feedback

Was this page helpful?

Yes

No

[Provide product feedback](#)

Pre-populate connection references and environment variables for automated deployments

Article • 02/25/2022

Connection references and *environment variables* enable you to interactively specify the connection details and configuration settings specific to the target environment where your app or solution is deployed.

More information:

- [Connection reference overview](#)
- [Environment variables overview](#)

After importing a solution containing connection reference and environment variable information, you are prompted to provide information specific to your environment in the UI. However, entering this information does not work well for fully automated Continuous Integration/ Continuous Delivery (CI/CD) scenarios.

To enable a fully automated deployment, you can now pre-populate the connection reference and environment variable information specific to the target environment so that you don't have to interactively provide it after importing a solution.

Deployment settings file

To pre-populate the connection reference and environment variable information for your deployment, use the deployment settings file (JSON) to store the information, and pass it as a parameter when importing the solution using Power Platform Build Tools. You can store the JSON file in your source control system to update and manage as required for your organization.

Below is an example of the deployment settings file:

JSON

```
{
  "EnvironmentVariables": [
    {
      "SchemaName": "tst_Deployment_env",
      "Value": ""
    },
    {
```

```

    "SchemaName": "tst_EnvironmentType",
    "Value": ""
  },
],
"ConnectionReferences": [
  {
    "LogicalName":
    "tst_sharedtst5fcreateuserandjob5ffeb85c4c63870282_b4cc7",
    "ConnectionId": "",
    "ConnectorId": "/providers/Microsoft.PowerApps/apis/shared_tst-
5fcreateuserandjob-5ff805fab2693f57dc"
  },
  {
    "LogicalName": "tst_SharepointSiteURL",
    "ConnectionId": "",
    "ConnectorId":
    "/providers/Microsoft.PowerApps/apis/shared_sharepointonline"
  },
  {
    "LogicalName": "tst_AzureDevopsConnRef",
    "ConnectionId": "",
    "ConnectorId":
    "/providers/Microsoft.PowerApps/apis/shared_visualstudioteamservices"
  },
  {
    "LogicalName": "tst_GHConn",
    "ConnectionId": "",
    "ConnectorId": "/providers/Microsoft.PowerApps/apis/shared_github"
  }
]
}

```

In the above example JSON file, the values shown as "" are missing and will need to be populated. We will address doing that later in this article.

Step 1: Generate the deployment settings file

The deployment setting file can be generated using the [Power Platform CLI](#). You can generate the file while exporting or cloning the solution.

Option 1: Generate deployment settings file using create-settings property

Use the `create-settings` property with Power Platform CLI:

```
C:\> pac solution create-settings --solution-zip <solution_zip_file_path> --settings-file <settings_file_name>
```

This command will generate a JSON file as shown below.

```
PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo> pac solution create-settings --solution-zip .\mytestsoln.zip --settings-file DeploymentSettingsDev.json
Extracting Dataverse connection references and/or environment variables from: C:\Users\kartikka\Documents\GitHub\conn-reff-demo\mytestsoln.zip
Deployment settings file created: C:\Users\kartikka\Documents\GitHub\conn-reff-demo\DeploymentSettingsDev.json
PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo>
```

In the JSON file, few values are empty in the `ConnectionReferences` section. These values need to be gathered after creating them in the target environment.

```
{
  "EnvironmentVariables": [
    {
      "SchemaName": "tyt_Deploymentenv",
      "Value": ""
    },
    {
      "SchemaName": "tyt_Deploymentsite",
      "Value": ""
    }
  ],
  "ConnectionReferences": [
    {
      "LogicalName": "tyt_AzureDevOpsConnRef",
      "ConnectionId": "",
      "ConnectorId": "/providers/Microsoft.PowerApps/apis/shared_visualstudioteamservices"
    },
    {
      "LogicalName": "tyt_GHConn",
      "ConnectionId": "",
      "ConnectorId": "/providers/Microsoft.PowerApps/apis/shared_github"
    }
  ]
}
```

Option 2: Generate deployment settings file by cloning the solution

A project solution clone is needed for this step because it renders the original solution into a buildable format. After you have cloned a solution using Power Platform CLI, you get the following directory structure created on your computer:

```
PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo> pac solution list
Connected to...mytestenv
Listing all Solutions from the current Dataverse Organization...

  Index      Unique Name      Friendly Name
  Version
  [1]        Cr0f8cf          Common Data Services Default Solution
  1.0.0.0
  [2]        mytestsoln      mytestsoln
  1.0.0.0
  [3]        [REDACTED]      VehicleQualitySolution      Vehicle Quality Solution
  1.3.0.0

PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo> pac solution clone --name mytestsoln
```

Proceed to create the settings file in the context of the current folder and populate the value of the settings file as shown below.

```
PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo> cd .\mytestsoln\
PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo\mytestsoln> dir

Directory: C:\Users\kartikka\Documents\GitHub\conn-reff-demo\mytestsoln

Mode                LastWriteTime         Length Name
----                -
d----             8/19/2021 10:37 AM             src
-a---             8/17/2021  8:51 AM             129 .gitignore
-a---             8/19/2021 10:37 AM            2475 mytestsoln.cdsproj

PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo\mytestsoln> pac solution create-settings --solution-folder .\ --setting
gs-file .\DeploymentSettingsTest.json
Extracting Dataverse connection references and/or environment variables from: C:\Users\kartikka\Documents\GitHub\conn-reff-d
emo\mytestsoln\src
Deployment settings file created: C:\Users\kartikka\Documents\GitHub\conn-reff-demo\mytestsoln\DeploymentSettingsTest.json
PS C:\Users\kartikka\Documents\GitHub\conn-reff-demo\mytestsoln>
```

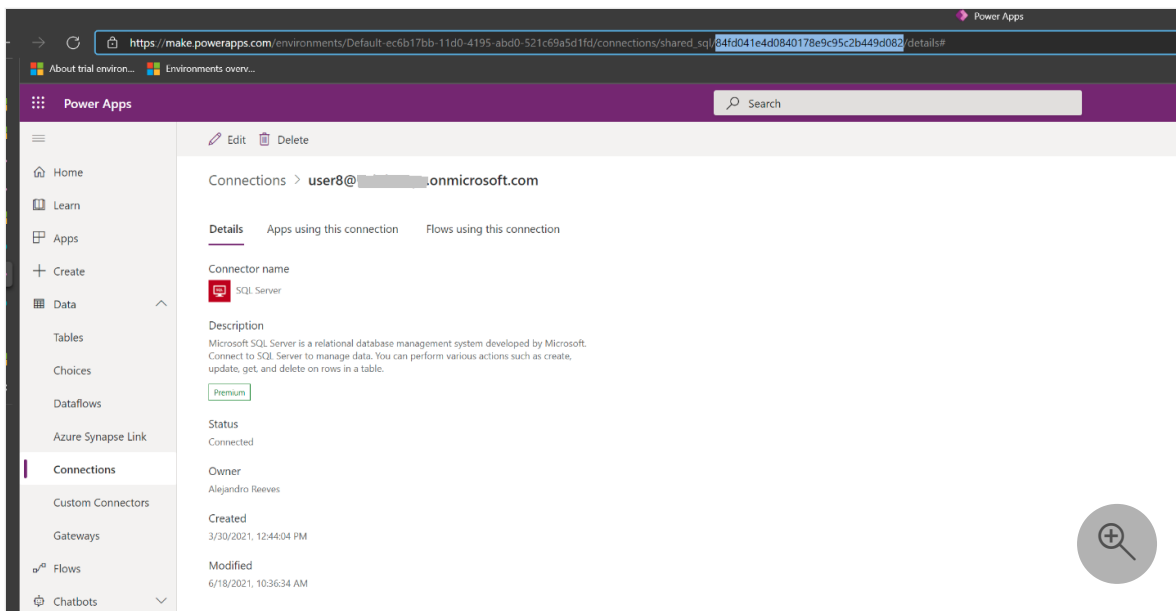
Step 2: Get the connection reference and environment variable information

To populate the deployment settings file, you will need to obtain the connection reference and environment variable information of the target solution.

Get the connection reference information

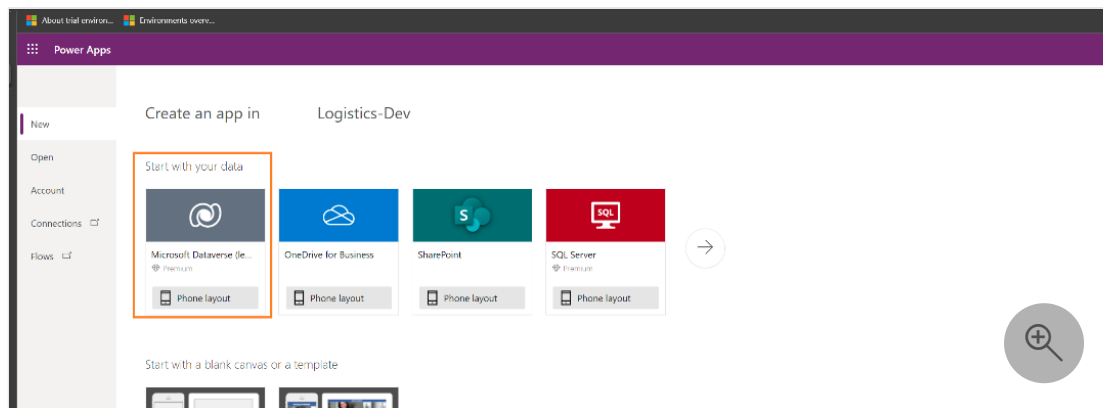
To get the connection ID of the target environment, use one of the following ways:

- Sign in to [Power Apps](#) and select your target environment. In the left navigation pane, select **Data > Connections**, select the connection you want to get the connection ID for and then look at the URL to get the connection ID.

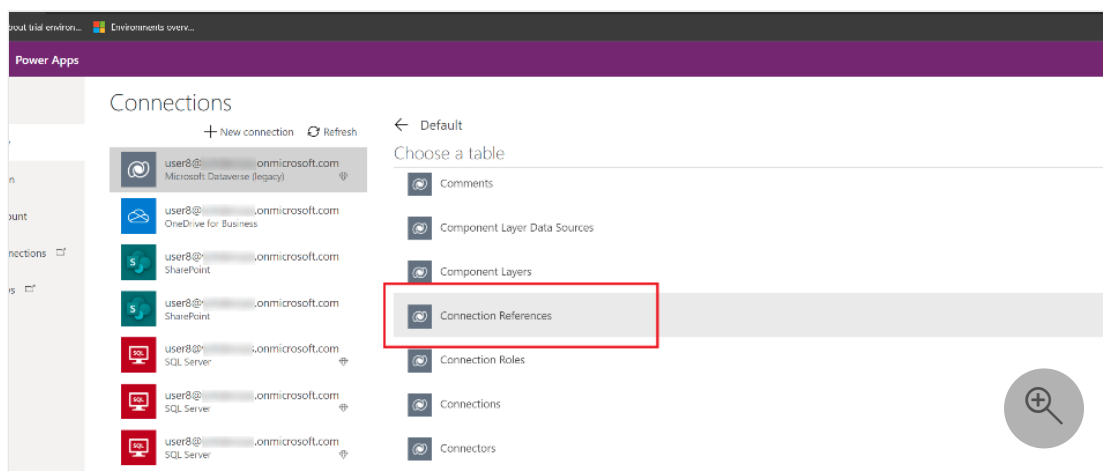


- Create a canvas app on the connection reference entity. The steps are as follows:

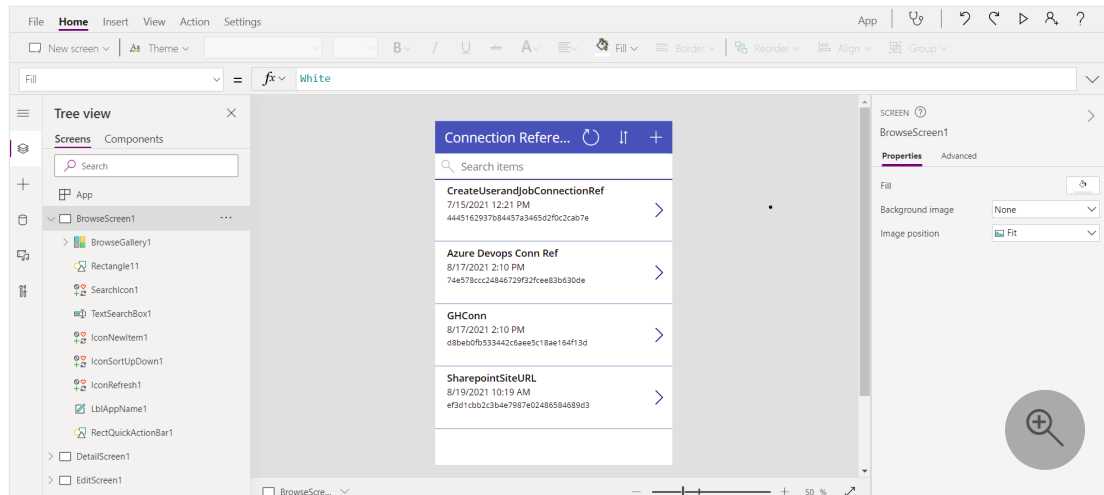
1. Sign in to [Power Apps](#) and select your target environment.
2. In the left navigation pane, select **Apps**, and then select **New app** > **Canvas**.
3. Select Dataverse as your data source.



4. Select the **Connection References** table and select **Connect**.



5. This will create a gallery application that will list out all the connections and their connection IDs within the environment.



Get the environment variable information

To get the values of the environment variable in the target environment, sign in to [Power Apps](#), select the target environment, and right-click on the ellipsis and choose to edit. This will provide the information needed to populate the deployment settings file (the underlined values are the values needed for the file):

Edit Deploymentsite



Use this variable to store information about an app or flow. Its values can be updated as it moves to different environments. [Learn more](#)

Display name *

Name * 

Description

Data Type *

 Text



Default Value 

Current Value

Override the default value by setting the current value for your environment.

 New value

Save

Cancel

Otherwise, you can provide the appropriate value for the target environment based on your knowledge of the target environment.

Step 3: Update the values in the deployment settings file

Manually edit the deployment settings file (JSON) to add the connection and environment variable information appropriately. Below is an example settings file with the (previously) missing values added.

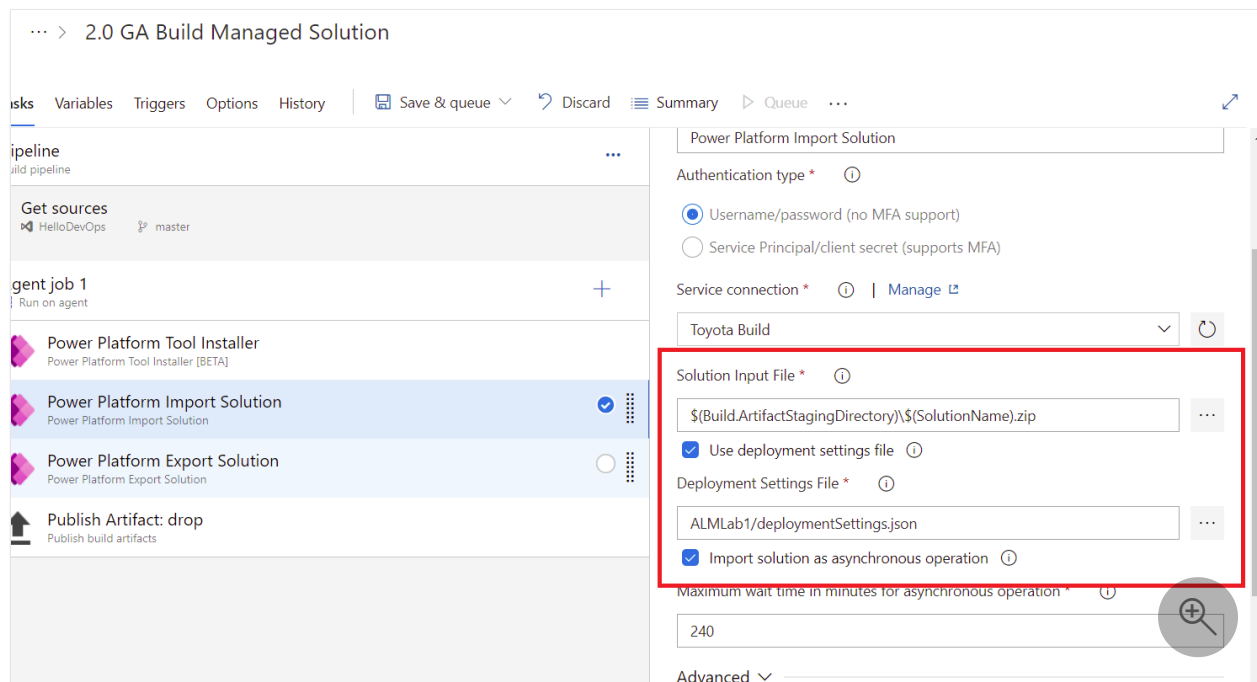
JSON

```
{
  "EnvironmentVariables": [
    {
      "SchemaName": "tst_Deployment_env",
      "Value": "Test"
    },
    {
      "SchemaName": "tst_EnvironmentType",
      "Value": "UAT"
    }
  ],
  "ConnectionReferences": [
    {
      "LogicalName":
        "tst_sharedtst5fcreateuserandjob5ffeb85c4c63870282_b4cc7",
      "ConnectionId": "4445162937b84457a3465d2f0c2cab7e",
      "ConnectorId": "/providers/Microsoft.PowerApps/apis/shared_tst-5fcreateuserandjob-5ff805fab2693f57dc"
    },
    {
      "LogicalName": "tst_SharepointSiteURL",
      "ConnectionId": "ef3d1cbb2c3b4e7987e02486584689d3",
      "ConnectorId":
        "/providers/Microsoft.PowerApps/apis/shared_sharepointonline"
    },
    {
      "LogicalName": "tst_AzureDevopsConnRef",
      "ConnectionId": "74e578ccc24846729f32fcee83b630de",
      "ConnectorId":
        "/providers/Microsoft.PowerApps/apis/shared_visualstudioteamservices"
    },
    {
      "LogicalName": "tst_GHConn",
      "ConnectionId": "d8beb0fb533442c6aee5c18ae164f13d",
      "ConnectorId": "/providers/Microsoft.PowerApps/apis/shared_github"
    }
  ]
}
```

Step 4: Use the deployment settings file for Build Tools tasks

Pass the deployment settings file as parameter when importing the solution into the target environment. This will create the required connection reference and environments with appropriate values upon import without having the user to interactively specify the value.

When using the [Import Solution task](#) in Power Platform Build Tools, select **Use deployment settings file** and provide the path to the deployment settings file. Next, initiate the pipeline.



See also

[What is Microsoft Power Platform CLI?](#)

[Microsoft Power Platform Build Tools tasks](#)

GitHub Actions for Microsoft Power Platform

Article • 07/08/2024

[GitHub Actions](#) enable developers to build automated software development lifecycle workflows. With [GitHub Actions for Microsoft Power Platform](#), you can create workflows in your repository to build, test, package, release, and deploy apps; perform automation; and manage bots and other components built on Microsoft Power Platform.

GitHub Actions for Microsoft Power Platform include the following capabilities:

- Importing and exporting application metadata (also known as solutions) that contain various platform components such as canvas apps, model-driven apps, desktop flows, Microsoft CoPilot Studio virtual agent chatbots, AI Builder models, customer engagement apps (Dynamics 365 Sales, Dynamics 365 Customer Service, Dynamics 365 Field Service, Dynamics 365 Marketing, and Dynamics 365 Project Service Automation), and connectors between development environments and source control.
- Deploying to downstream environments.
- Provisioning or de-provisioning environments
- Performing static analysis checks against solutions by using [Power Apps solution checker](#).

You can use GitHub Actions for Microsoft Power Platform along with any other available GitHub Actions to compose your build and release workflows. Workflows that teams commonly put in place include provisioning development environments, exporting from a development environment to source control, generating builds, and releasing apps.

GitHub Actions for Microsoft Power Platform are available at

<https://github.com/marketplace/actions/powerplatform-actions>.

Important

GitHub Actions for Microsoft Power Platform are supported only for a Microsoft Dataverse environment with a database. More information: [Create an environment with a database](#)

Key concepts

GitHub Actions enable you to create custom software development lifecycle workflows directly in your GitHub repository. For an overview of GitHub Actions and core concepts, review the following articles:

- [About GitHub Actions](#) ↗
- [Core concepts](#) ↗
- [About packaging with GitHub Actions](#) ↗

What are GitHub Actions for Microsoft Power Platform?

GitHub Actions for Microsoft Power Platform is a collection of Microsoft Power Platform–specific GitHub Actions that eliminate the need to manually download custom tooling and scripts to manage the application lifecycle of apps built on Microsoft Power Platform. The tasks can be used individually, such as importing a solution into a downstream environment, or used together in a workflow to orchestrate a scenario such as "generate a build artifact," "deploy to test," or "harvest maker changes." The build tasks can largely be categorized into four types:

- Helper
- Quality check
- Solution
- Environment management

For more information about individual tasks, go to [GitHub Actions for Microsoft Power Platform](#).

Get GitHub Actions for Microsoft Power Platform

You can use GitHub Actions for Microsoft Power Platform by adding the actions in your workflow definition file (.yml). Sample workflow definitions are available from the [GitHub Actions lab](#) ↗.

Connection to environments

To interact with a Dataverse environment, a secret must be created that enables the various GitHub Actions to perform the required task. Two types of connections are available:

- Username/password: Configured as a generic service connection with username and password. Username/password authentication doesn't support multifactor authentication.
- Service principal and client secret: This connection type uses service principal-based authentication and supports multifactor authentication. Service principal authentication

Available runners

GitHub Actions for Microsoft Power Platform can run on both Microsoft Windows agents and Linux agents.

Frequently asked questions

How do I get started with GitHub Actions?

Tutorial: [Get started with GitHub Actions](#) is available right now for you try out. The tutorials show how to use service principal authentication and standard username/password authentication.

How do I get GitHub Actions for Microsoft Power Platform working in my repository?

Go to the [GitHub Marketplace for Actions](#) and search for Power Platform. When you arrive on the page, select the green button to instantiate the actions into your repository.

Marketplace / Actions / powerplatform-actions



GitHub Action
powerplatform-actions
0.4.0 Latest version

Power Platform Actions

This repo provides multiple [GitHub Actions for the Power Platform](#); for more general info on [GitHub Actions](#). Each action wraps the existing [Power Platform CLI](#).

Detailed documentation on GitHub actions for Power Platform is available [here](#).

Sample workflows and detailed instructions are available in our [GitHub actions lab repo](#) but if you are already familiar with GitHub actions and Power Platform solutions, simply add below to your existing workflows; also add the secret `MYPASSWORD` to your repository's 'Settings' | 'Secrets'

```
jobs:
  build:
    runs-on: windows-latest # alternate runner OS is: ubuntu-latest
```

Choose a version

✓ 0.4.0

GA of powerplatform-actions

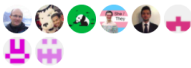
Use latest version

GitHub has verified that this action was created by **microsoft**.
[Learn more about verified Actions.](#)

Stars

☆ Star 53

Contributors



Categories

Do GitHub Actions only work for Power Apps?

GitHub Actions work for both canvas and model-driven apps, Microsoft CoPilot Studio virtual agents, UI flows and traditional flows, AI Builder, custom connectors, and dataflows, all of which can now be added to a solution. Also included are customer engagement apps.

Can I include flow and canvas apps?

Yes, flows and canvas apps are solution-aware, so if these components are added to your solution they can participate in the lifecycle of your app. However, some steps still require manual configuration, which will be addressed later this year when we introduce environment variables and connectors. For a list of current limitations, go to [Known limitations](#).

How much does GitHub Actions for Microsoft Power Platform cost?

GitHub Actions are available at no cost. However, a valid GitHub subscription is required to use the actions on GitHub. To get started, 2,000 action minutes/month are available for free. More information: [GitHub pricing](#) 

Can I use GitHub Actions for Microsoft Power Platform with Power Pages?

Yes. You can upload portal data and use the deployment profile to customize the deployment parameters.

See also

[GitHub Actions for Power Platform release notes](#) 

[Available GitHub Actions Hands on Lab](#) 

[Available GitHub Actions](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Available GitHub Actions for Microsoft Power Platform development

Article • 04/03/2024

[GitHub Actions for Microsoft Power Platform](#) are described in the following sections. In addition, sample GitHub workflows are shown as well. For more information about GitHub Actions and how to download them, go to [GitHub Actions for Microsoft Power Platform](#).

Configure credentials to use with GitHub Actions with Microsoft Power Platform

Many of the actions require you to connect to a Microsoft Dataverse environment. You can add service principal or user credentials as secrets in your GitHub repository and then use them in your workflows.

- For details on how to set up secrets in GitHub, see [Encrypted secrets](#)
- For details how to set up service principal authentication for Microsoft Power Platform, see [DevOps Build tools](#)

Once configured, you can call the Service Principal from within your Action scripts.

Parameters to define within your GitHub Action Script as [Environment Variables](#):

- Application ID such as: `WF_APPLICATION_ID:<your application id>`
- Tenant ID such as: `WF_TENANT_ID:<your tenant id>`

The client Secret must be added and stored as a GitHub Secret, and will be referenced from within the workflow using a parameter like: `client_secret:`

```
${{secrets.CLIENT_SECRET_GITHUB_ACTIONS}}
```

Helper tasks

The available helper task is described below.

actions-install

When a GitHub Actions workflow forces an install of Power Platform CLI in the context of a runner, and timeout error occurs then you must use version 1 (@v1) with an additional action (`actions-install`) as below.

❗ Important

- Using version 1 (@v1) might lead to updates to existing GitHub actions that can result in current workflows being updated.
- You must add the **Install Power Platform Tools** task as a first task in your workflow prior to any other GitHub Actions for Power Platform.

```
jobs:
  builds:
    runs-on: windows-latest    # alternate runner OS is: ubuntu-latest

    steps:
      - name: Install Power Platform Tools
        uses: microsoft/powerplatform-actions/actions-install@v1

      - name: Export Solution
        uses: microsoft/powerplatform-actions/export-solution@v1
        with:
          environment-url: 'https://myenv.crm.dynamics.com'
          user-name: 'me@myenv.onmicrosoft.com'
          password-secret: '${{ secrets.MYPASSWORD }}'
          solution-name: aSolution
          solution-output-file: 'aSolution.zip'
          working-directory: 'out'
```

You can find additional samples of GitHub Actions at [Power Platform Actions](#).

whoAmI

Verifies the service connection by connecting to the service and sending a `WhoAmI` [SDK/Web API] request. This task can be useful to include early in your GitHub workflow, to verify connectivity before processing begins.

[Expand table](#)

Parameter	Description
environment-url	The URL for the environment you're connecting to.
user-name	The username of the account you're using to connect with.
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Solution tasks

These tasks perform actions against solutions and include the following.

import-solution

Imports a solution into a target environment.

 Expand table

Parameter	Description
environment-url	(Required) The URL for the target environment that you want to import the solution into (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for the account you're using to connect with.
solution-file	(Required) The path and name of the solution file you want to import.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

export-solution

Exports a solution from a source environment.

 Expand table

Parameter	Description
environment-url	(Required) The URL for the environment that you want to export the solution from (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
solution-name	(Required) The name of the solution to export. Always use the solution's <i>name</i> , not its <i>display name</i> .
solution-output-file	(Required) The path and name of the solution.zip file to export the source environment to.
managed	(Required) Set to true to export as a managed solution; the default (false) is to export as an unmanaged solution.

unpack-solution

Takes a compressed solution file and decomposes it into multiple XML files so these files can be more easily read and managed by a source control system.

 Expand table

Parameter	Description
solution-file	(Required) The path and file name of the solution.zip file to unpack.
solution-folder	(Required) The path and target folder you want to unpack the solution into.

Parameter	Description
solution-type	(Required) The type of solution you want to unpack. Options include Unmanaged (recommended), Managed , and Both .

pack-solution

Packs a solution represented in source control into a solution.zip file that can be imported into another environment.

 Expand table

Parameter	Description
solution-file	(Required) The path and file name of the solution.zip file to pack the solution into (for example, out/CI/ALMLab.zip).
solution-folder	(Required) The path and source folder of the solution to pack.
solution-type	(Optional) The type of solution to pack. Options include Unmanaged (recommended), Managed , and Both .

publish-solution

Publishes the solution customizations.

 Expand table

Parameter	Description
environment-url	(Required) The URL for the environment that you want to publish the solution into (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for the account you're using to connect with.
solution-file	(Required) The path and name of the solution file you want to import.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.

Parameter	Description
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

clone-solution

Clones the solution for a given environment.

 Expand table

Parameter	Description
environment-url	(Required) The URL for the environment that you want to clone the solution from (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for the account you're using to connect with.
solution-file	(Required) The path and name of the solution file you want to import.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
solution-name	(Required) The path and name of the solution.zip file needed to clone (for example, <code>out/CI/ALMLab.zip</code>).
solution-version	Version of the solution to clone.
target-folder	Target folder to place the extracted solution into. (for example, <code>Git repository\target-solution-folder</code>).
Working-directory	Temporary folder for work in progress artifacts needed for cloning the solution. default: <code>root of the repository</code>

check-solution

Checks the solution file to detect inconsistencies.

[Expand table](#)

Parameter	Description
environment-url	(Required) The URL for the environment that you want to clone the solution from (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for the account you're using to connect with.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
path	(Required) The path and name of the solution file you want to check.
geo	Which geo location of the Microsoft Power Platform Checker service to use. Default value is 'united states'.
rule-level-override	Path to file a containing a JSON array of rules and their levels. Accepted values are: Critical, High, Low, and Informational. Example: <code>[{"Id":"meta-remove-dup-reg","OverrideLevel":"Medium"}, {"Id":"il-avoid-specialized-update-ops","OverrideLevel":"Medium"}]</code>
checker-logs-artifact-name	The name of the artifact folder for which Microsoft Power Platform checker logs will be uploaded. Default value is 'CheckSolutionLogs'.

upgrade-solution

Provides the ability to upgrade the solution.

[Expand table](#)

Parameter	Description
environment-url	(Required) The URL for the environment that you want to clone the solution from (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for

Parameter	Description
	the account you're using to connect with.
solution-file	(Required) The path and name of the solution file you want to import.
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
solution-name	(Required) Name of the solution to upgrade.
async	Upgrades the solution asynchronously.
max-async-wait-time	Maximum asynchronous wait time in minutes. Default value is 60 minutes.

Package tasks

These tasks perform actions against packages and include the following.

deploy-package

Provides the ability to deploy a package dll or a zip file with a package.

ⓘ Note

This action is only supported on a Windows.

[Expand table](#)

Parameter	Description
environment-url	(Required) The URL for the environment that you want to clone the solution from (for example, <code>https://YourOrg.crm.dynamics.com</code>).
user-name	(Required) If you're using username/password authentication, the username of the account you're using to connect with.
password-secret	(Required) If you're using username/password authentication, the password for the account you're using to connect with.

Parameter	Description
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
package	(Required) The path to the package dll or zip file with a package.

Portal tasks

These tasks perform the following actions against Power Pages.

upload-paportal

Uploads data to Power Pages.

 Expand table

Parameter	Description
environment-url	(Required) The URL for the environment that you want to import the solution into (for example, <code>https://YourOrg.crm.dynamics.com</code>).
app-id	The application ID to authenticate with. This parameter is required when authenticating with Service Principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is required when authenticating with Service Principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
upload-path	Path where the website content is stored (alias: -p).
deployment-profile	Upload portal data with environment details defined through profile variables in deployment-profiles/[profile-name].deployment.yaml file.
model-version	Indicates if the site data to be uploaded will use the standard (1) or enhanced data model (2). Default value is '1'.

Learn more about [Microsoft Power Platform CLI support for Power Pages](#).

Catalog tasks (preview)

These tasks perform actions against the catalog in Power Platform and include the following.

Note

Currently, catalog tasks for Power Platform are a **preview** release.

install

Install a catalog item to the target environment.

[Expand table](#)

Parameter	Description
catalog-item-id	(Required) Catalog item to be installed on the target environment.
target-url	(Required) The Url of the target environment for the catalog item installation into (for example, "https://YourOrg.crm.dynamics.com").
settings	(Required) Runtime package settings for the installation framework to execute. The format of the string must be <code>key=value key=value</code> .
target-version	Target version to install. If left empty, the published version is selected.
poll-status	Poll to check the status of your request.

submit

List all published catalog items from the current Dataverse environment (organization).

[Expand table](#)

Parameter	Description
path	(Required) Path to catalog submission document.
package-zip	Path to package zip file.
solution-zip	Path to solution zip file.
poll-status	Poll to check status of your request.

status

Get status of the catalog install/submit request.

 Expand table

Parameter	Description
tracking-id	(Required) Request tracking id.
type	(Required) Request type (Values: Install, Submit)

GitHub workflow authoring

To learn more about composing GitHub workflows using GitHub actions, complete the [GitHub Actions for Microsoft Power Platform labs](#).

More information: [About GitHub Actions](#)

See Also

[GitHub Actions for Microsoft Power Platform](#)

Available GitHub Actions for Microsoft Power Platform administration

Article • 02/15/2022

This article provides information about GitHub Actions that are available to administer Microsoft Power Platform.

Configure credentials to use within your GitHub workflows

Many of the actions require you to connect to a Microsoft Dataverse environment. You can add service principal or user credentials as secrets in your GitHub repository and then use them in the workflow.

- For details about how to set up secrets in GitHub, go to [Using encrypted secrets in a workflow](#).
- For details about how to set up service principal authentication for Microsoft Power Platform, go to [Configure service connections using a service principal](#). After it's configured properly, you can call the service principal from within your action scripts.

Define the following parameters within your GitHub Actions script as [environment variables](#):

- Application ID: `WF_APPLICATION_ID:<your application id>`
- Tenant ID: `WF_TENANT_ID:<your tenant id>`

The client secret will be stored as a GitHub secret, as described in [Encrypted secrets](#), and will be referenced from within the action script by using a parameter like `client secret: ${secrets.CLIENT_SECRET_GITHUB_ACTIONS}`.

Administrative tasks

The available administrative tasks are explained below.

Create an environment

Parameter	Description
-----------	-------------

Parameter	Description
user-name	The username of the account you're using to connect with.
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .
name	The name of the environment that you're going to create.
region	The name of the region where your environment will be created. The default value is <code>unitedstates</code> .
type	The type of environment (Trial, Sandbox, Production, SubscriptionBasedTrial). More information: Trial environments
currency	The currency to use for the environment. The default value is <code>USD</code> .
language	The language to use for the environment. The default value is <code>English</code> .
templates	The templates that need to be deployed to the environment. These are passed as comma-separated values.
domain	The domain name of the environment URL. For example, <code>https://{contoso}0.crm.dynamics.com</code>

The output will be the URL of the new environment.

Copy an environment

Parameter	Description
source-url	The source URL of the environment to copy. For example, <code>https://source-env.crm.dynamics.com</code>
target-url	The target URL of the environment to copy. For example, <code>https://target-copy-env.crm.dynamics.com</code>
user-name	The username of the account you're using to connect with.

Parameter	Description
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Back up an environment

Parameter	Description
environment-url	The URL of the environment that needs to be backed up. For example, <code>https://env-to-backup.crm.dynamics.com</code>
backup-label	A meaningful name to use as a label for the backup of the environment.
user-name	The username of the account you're using to connect with.
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Restore an environment from a backup

Parameter	Description
source-url	The source URL of the environment to restore. For example, <code>https://env-backup-source.crm.dynamics.com</code>
target-url	The target URL of the environment to be restored to. For example, <code>https://env-target-restore.crm.dynamics.com</code>
user-name	The username of the account you're using to connect with.

Parameter	Description
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Reset an environment

Parameter	Description
environment-url	The URL of the environment that needs to be reset. For Example, <code>https://env-to-reset.crm.dynamics.com</code>
user-name	The username of the account you're using to connect with.
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Delete an environment

Parameter	Description
environment-url	The URL of the environment that needs to be deleted. For example, <code>https://env-to-delete.crm.dynamics.com</code>
user-name	The username of the account you're using to connect with.
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.

Parameter	Description
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Helper tasks

The available helper tasks are described below.

WhoAmI

Verifies the service connection by connecting to the service and sending a `WhoAmI` [SDK/Web API] request. This task can be useful to include early in your DevOps pipeline, to verify connectivity before processing begins.

Parameter	Description
environment-url	The URL for the environment you're connecting to.
user-name	The username of the account you're using to connect with.
password-secret	The password for <i>user-name</i> . GitHub passwords are defined in Settings under Secrets . You can't retrieve a secret after it has been defined and saved.
app-id	The application ID to authenticate with. This parameter is <i>required</i> when authenticating with service principal credentials.
client-secret	The client secret used to authenticate the GitHub pipeline. This parameter is <i>required</i> when authenticating with service principal credentials.
tenant-id	The tenant ID when authenticating with <code>app-id</code> and <code>client-secret</code> .

Build and release pipeline authoring

To learn about creating multiple-action workflows and pipelines through hands-on authoring by using GitHub Actions for Microsoft Power Platform, complete the [GitHub Actions for Microsoft Power Platform labs](#) .

More information: [About GitHub Actions](#) .

See also

Tutorial: Get started with GitHub Actions for Microsoft Power Platform

Article • 12/28/2023

This three part tutorial will give you an opportunity to get hands on with best practices to automate building and deploying your app using GitHub Actions for Power Platform. The first two tutorials are all about setting up required environments and creating a solution to later use with GitHub Actions. If you are experienced with creating environments and solutions, you can follow the Tip below and skip to the third tutorial to begin using GitHub Actions for Power Platform.

- ✓ Create three Microsoft Dataverse environments in your tenant
- ✓ **(Highly recommended)** Create a service principal and provide the appropriate permissions
- ✓ Create a model-driven app
- ✓ Export and deploy your app using application lifecycle management (ALM) automation

Tip

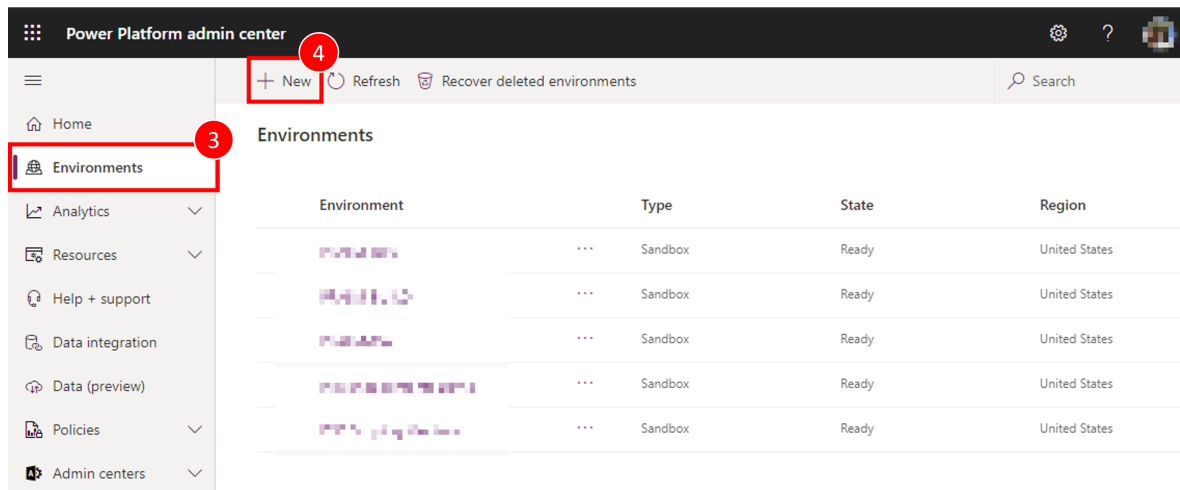
If you are already familiar with the concept of multiple Dataverse environments as well as how to use solutions to package your app, simply download and use the sample **ALMLab solution** [↗](#) and then skip to the **last tutorial** of this series.

Let's get started with tutorial #1 and create three Dataverse environments.

Create required environments

You will need to create, or have access to, three Dataverse environments in your demo or customer tenant. To create these environments, follow the instructions below. Otherwise, proceed to the end of this tutorial for the next steps.

1. Sign in to the [Power Platform admin center](#) [↗](#) with credentials that provide access to a tenant with a minimum 3-GB available capacity (required to create the three environments).
2. Select **Environments** in the navigation area.
3. Select + **New** to create your first new environment.



4. The first environment should be named "Your Name – dev", set the region to "United States (default)", set the Environment type to **Sandbox** (if available), if not use "Trial".

- Ensure the *Create a database for this environment* radio toggle is set to **Yes**

New environment



 This operation is subject to [capacity constraints](#)

Name *

ALM - Test

Region *

Preview (United States) - Default



A local region can provide quicker data access

Type  *

Sandbox



Purpose

Power Platform Actions demo

Create a database for this environment? 



Yes

Next

Cancel

5. Click Next.

6. Set the Language and currency as preferred and set the "**Deploy sample apps and data?**" radio button to **Yes**, then click **Save**

← Add database

×

i

This operation is subject to [capacity constraints](#)

Language *

English

▼

Default language for user interfaces in this environment

URL

A unique domain name will be generated.
Click [here](#) to enter a custom domain

Currency *

USD (\$)

▼

Reports will use this currency

Enable Dynamics 365 apps?

In addition to Power Apps. [Learn more](#)

No

Deploy sample apps and data?

Yes

Security group

Restrict environment access to people in this security group. Otherwise, everyone can access. [Learn more](#)

+

Select

Save

Cancel

7. Your development environment has been created, follow steps 2 – 7 above to create a second environment called "Your Name – build" , and then finally, create a third environment called "Your Name – prod"

Now you have the environments that we will need for this and ready to begin the next modules of this Hands-on lab.

Create the service principal account and give it rights to the environments created

1. You will need to create an application registration within Microsoft Entra ID. Do this for all DEV/BUILD/PROD environments used. More information: [Tutorial:](#)

Register an app with Microsoft Entra ID

2. Upon creation of the application registration, please note and save the Directory (tenant) ID and the Application (client) ID of the application.

The screenshot shows the 'Overview' page of an application registration in the Microsoft Entra ID portal. The left navigation pane includes sections like Overview, Quickstart, Integration assistant, Manage (with sub-items: Branding, Authentication, Certificates & secrets, Token configuration, API permissions, Expose an API, App roles, Owners, Roles and administrators | Preview, Manifest), and Support + Troubleshooting. The main content area displays 'Essentials' with the following details:

- Display name: App-wrkdevops.onmicrosoft.com-20200601-085313
- Application (client) ID: f6c (highlighted with a red box)
- Object ID: 05 (highlighted with a red box)
- Directory (tenant) ID: ec6b176b-11d0-4195-abd0-521c69a5d1fd (highlighted with a red box)
- Supported account types: My organization only

Client credentials are listed as:

- Add a certificate or secret
- 0 web, 0 spa, 1 public client
- Add an Application ID URI
- Managed application in L... : App-wrkdevops.onmicrosoft.com-20200601-085313

A notice at the bottom states: 'Starting June 30th, 2020 we will no longer add any new features to Azure Active Directory Authentication Library (ADAL) and Azure AD Graph. We will continue to provide technical support and security updates but we will no longer provide feature updates. Applications will need to be upgraded to...'

3. On the navigation panel of the **Overview** page, select **API permissions**.
4. Choose **+ Add a permission**, and in the **Microsoft APIs** tab, Choose **Dynamics CRM**.
5. In the **Request API permissions** form, select **Delegated permissions**, check **user_impersonation**, and then choose **Add permissions**.
6. From the **Request API permissions** form, choose **PowerApps Runtime Service**, select **Delegated permissions**, check **user_impersonation**, and then choose **Add permissions**.
7. From the **Request API permissions** form, choose **APIs my organization uses**, search for "PowerApps-Advisor" using the search field, select **PowerApps-Advisor** in the results list, select **Delegated permissions**, check **Analysis.All** rights, and then choose **Add permissions**.

The screenshot shows the 'API permissions' page for the application. The left navigation pane is the same as the previous screenshot, with 'API permissions' selected. The main content area shows a list of configured permissions:

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission ✓ Grant admin consent for TEST_TEST_Microsoft

API / Permissions name	Type	Description	Admin consent requ...	Status
▼ Dynamics CRM (1)				
user_impersonation	Delegated	Access Common Data Service as organization users	No	...
▼ Microsoft Graph (1)				
User.Read	Delegated	Sign in and read user profile	No	...
▼ PowerApps Runtime Service (1)				
user_impersonation	Delegated	Common Data Service	No	...
▼ PowerApps-Advisor (2)				
Analysis.All	Delegated	Analysis.All	No	...

8. Next, proceed to create a client secret, in the navigation panel, select **Certificates & secrets**.
9. Below **Client secrets**, select **+ New client secret**.
10. In the form, enter a description and select **Add**. Record the secret string, you will not be able view the secret again once you leave the form.

Home > TEST_TEST_Microsoft > App-wrkdevops.onmicrosoft.com-20200601-085313

 **App-wrkdevops.onmicrosoft.com-20200601-085313** | Certificates & secrets

Search (Ctrl+/) << Got feedback?

Overview
Quickstart
Integration assistant

Manage

Branding
Authentication
Certificates & secrets
Token configuration
API permissions
Expose an API
App roles
Owners
Roles and administrators | Preview
Manifest

Support + Troubleshooting

Credentials enable confidential applications to identify themselves to the authentication service when receiving tokens at a web addressable location (scheme). For a higher level of assurance, we recommend using a certificate (instead of a client secret) as a credential.

Certificates

Certificates can be used as secrets to prove the application's identity when requesting a token. Also can be referred to as public keys.

[Upload certificate](#)

Thumbprint	Start date	Expires	Certificate ID
No certificates have been added for this application.			

Client secrets

A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

[+ New client secret](#)

Description	Expires	Value	Secret ID
-------------	---------	-------	-----------

Application user creation

In order for the GitHub workflow to deploy solutions as part of a CI/CD pipeline an "Application user" needs to be given access to the environment. An "Application user" represents an unlicensed user that is authenticated using the application registration completed in the prior steps.

1. Sign in to [Power Platform admin center](#).
2. Go to **Environments** > open the environment you want > **Settings** > **Users + permissions** > **Application users**.
3. Select **+ New app user**. A panel will open on the right hand side of the screen.
4. Select **+ Add an app**. A list of all the application registrations in your Microsoft Entra tenant is shown. Proceed to select the application name from the list of registered apps.
5. Under **Business unit**, in the drop down box, select your environment as the business unit.

- Under **Security roles**, select **System administrator**, and then select **create**. This will allow the service principal access to the environment.

Create a new app user



App^{*}

App-wrkdevops.onmicrosoft.com-20210216-150945



Business unit^{*}

orgc6432c71



Security roles(1)



System Administrator

Now that you have created the service principal, you can use either the service principal or the standard username and password for your GitHub Workflow.

Important

If you have multi-factor authentication (MFA) enabled, service principal authentication is the authentication method you want to use.

Next steps

See Also

[Automate your workflow from idea to production](#)

Tutorial: Build a model-driven app for deployment using GitHub Actions for Microsoft Power Platform

Article • 07/07/2022

In this tutorial, you will be creating a simple model-driven app to deploy using GitHub Actions for Microsoft Power Platform in the next tutorial.

- ✓ Create a model-driven app

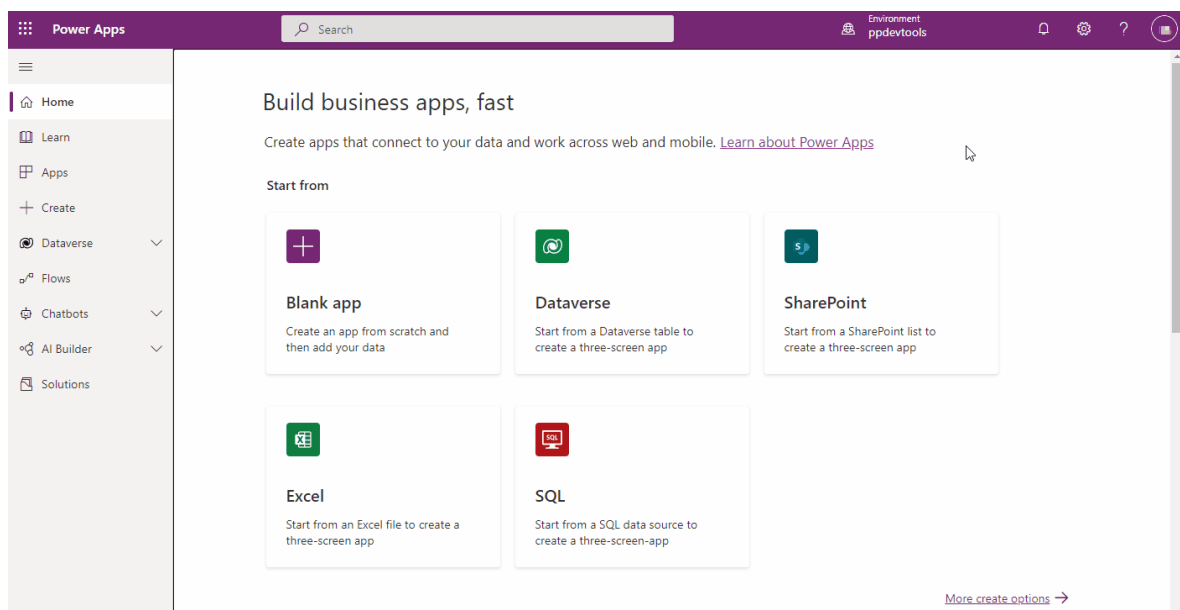
In the next tutorial, you will learn how to:

- ✓ Export and deploy your app using application lifecycle management (ALM) automation

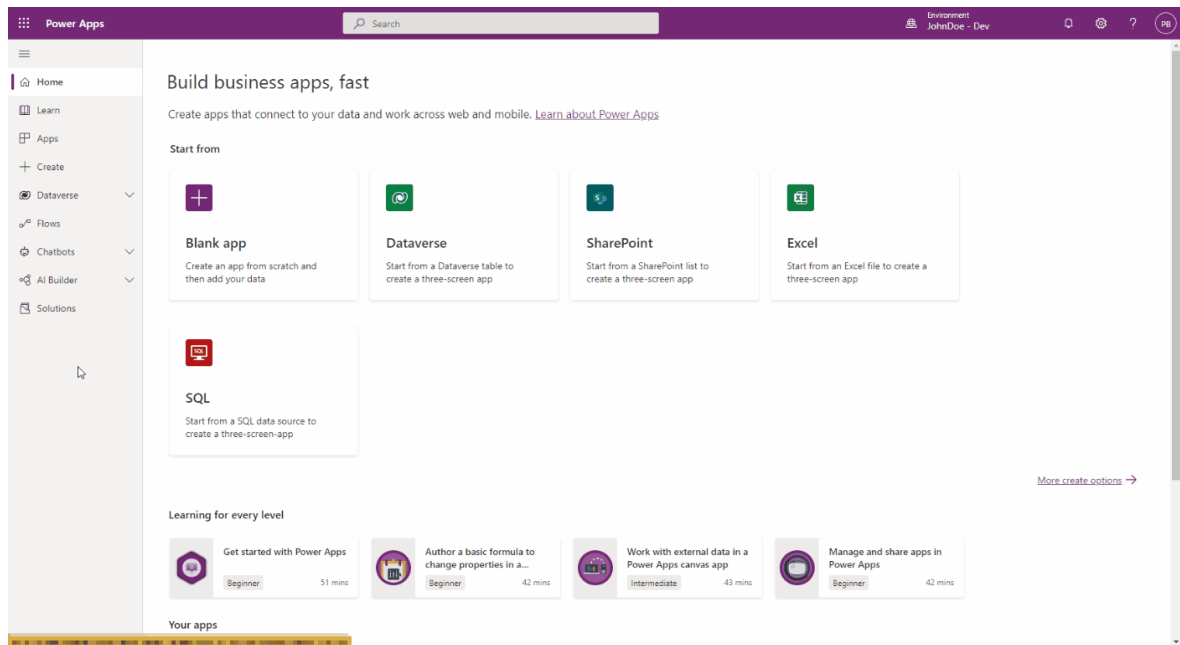
Build a model-driven app

Follow the steps below to build a model-driven app.

1. In your browser, navigate to <https://make.powerapps.com> and sign in with your credentials. Click the environment selector dropdown in the header and select your development environment.



2. Click the **Solutions** area in the left navigation, and then click the **New solution** button to create a new solution.



3. In the side panel that appears, enter a name for the application and then click the **Add Publisher** option.

ⓘ Note

The solution publisher specifies who developed the app, so you should always create a solution publisher name that is meaningful. Furthermore, the solution publisher includes a prefix, which helps you distinguish system components or components introduced by others and is also a mechanism to help avoid naming collisions. This allows for solutions from different publishers to be installed in an environment with minimal conflicts.

New solution



Display name *

Name *

Publisher *

+ New publisher

Version *

More options 

4. For the purposes of this tutorial, enter 'ALMLab' for the *display name*, *name*, and *prefix*, and then choose **Save and Close**.

New publisher



Publishers indicate who developed associated solutions. [Learn more](#)

Properties Contact

Display name *

ALMLab

Name *

almlab

Description

Prefix *

almlab

Choice value prefix *

81435

Preview of new object name

almlab_Object

Save

Cancel

5. On the new solution panel, select the publisher that you just created and **click Create** to create a new unmanaged solution in the environment.

New solution ✕

Display name *

Name *

Publisher *

✎

+ New publisher

Version *

[More options](#) ▼

Create Cancel

6. In the solutions list, select the solution you just created and click the **Edit** button.

⋮ Power Apps

☰

Home

Learn

Apps

Create

Data

Flows

AI Builder

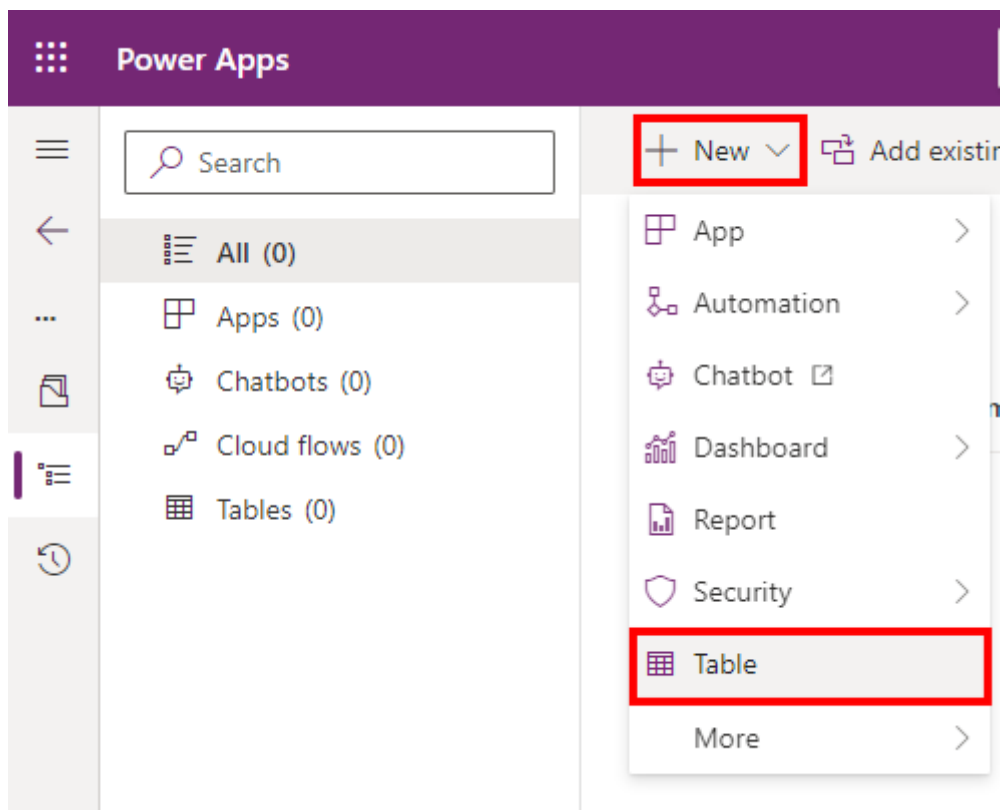
Solutions

+ New solution ✎ Edit 🗑 Delete ➡ Export 📦 Show dependencies

Solutions

Display name	Name
✓ ALMLab	⋮ ALMLab

7. Your new solution will be empty, and you need to add components to it. In this lab we will create a custom table. Click the **+ New** dropdown from the top navigation and select **Table**



8. Enter a **display name**, plural name will be generated for you. Click **Save** to create the table.

New table



Use tables to hold and organize your data. Previously called entities

[Learn more](#)

Properties

Primary column

Display name *

Time Off Request

Plural name *

Time Off Requests

Description

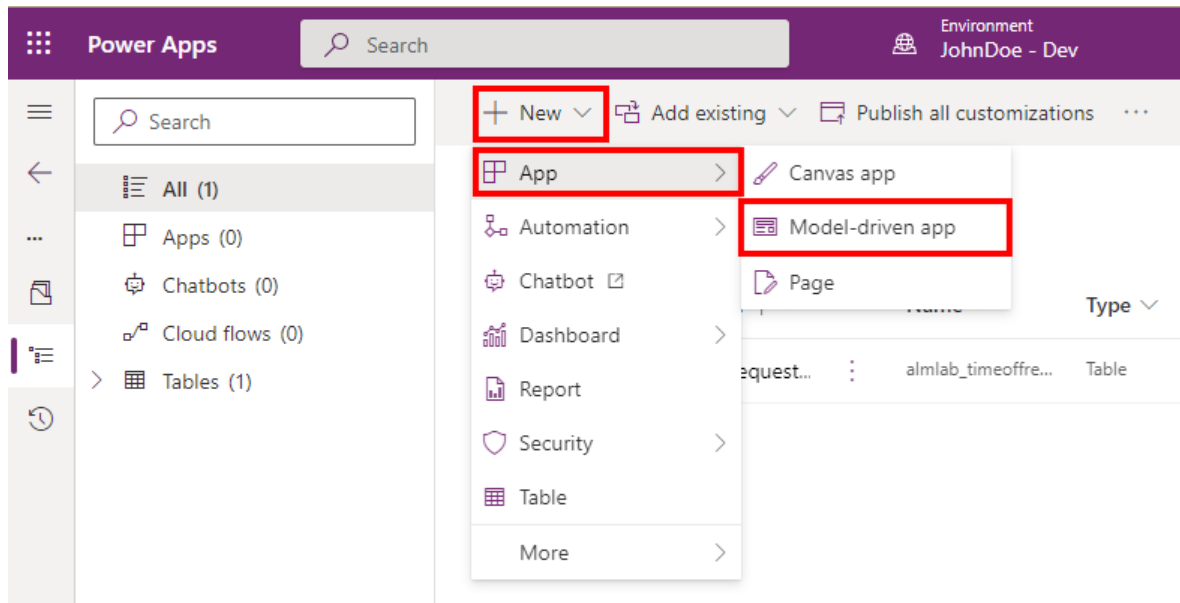
☐ Enable attachments (including notes and files) ¹

[Advanced options](#) ▾

Save

Cancel

9. Once your table is created, select the solution name again to go back to the solution view to add another component.
10. Click the + **New** dropdown, then **App**, and finally click **Model-driven app**. If you get a pop-up asking to select the creating experience choose *Modern app designer*



11. Enter an app name, then click the **Create** button

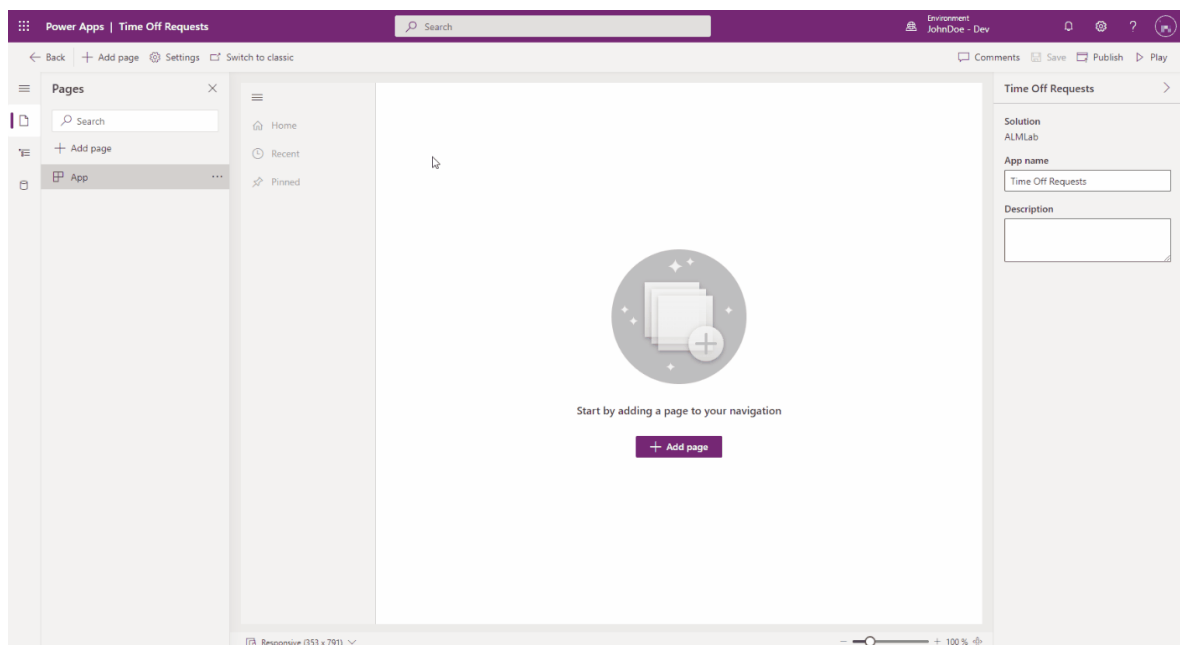
New model-driven app

Name *

Description

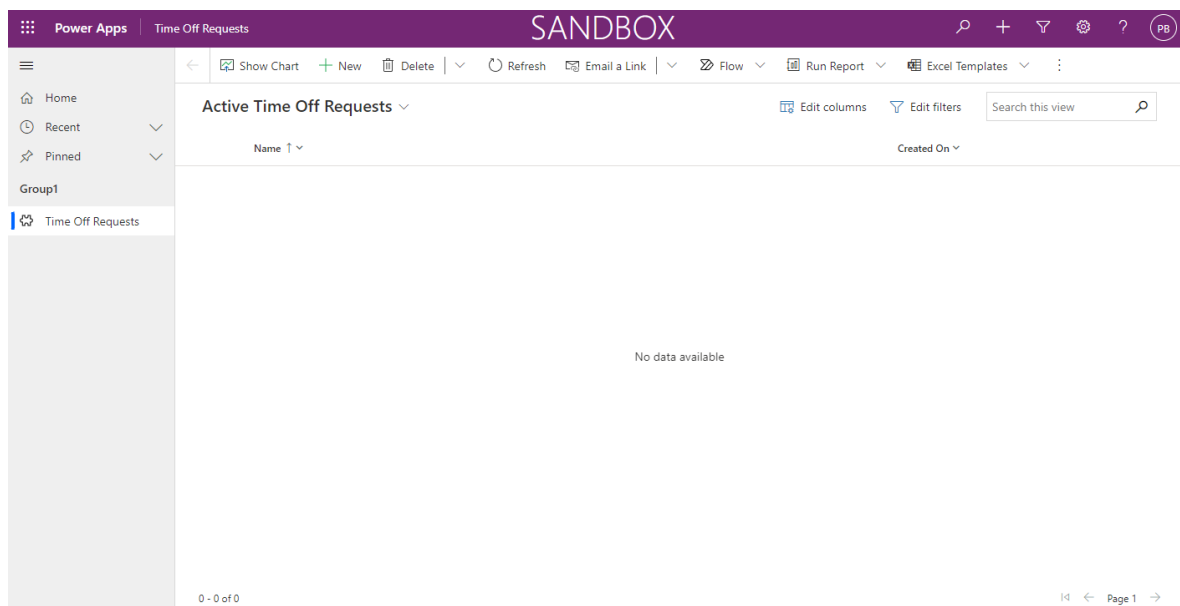
CreateBack

12. In the application designer, click + **Add page**, choose **Table based view and form**, then click **Next**. On the next screen **search by name for the table you previously created**. Check mark the selection and click **Add**
 - Ensure that *Show in navigation* is checked



13. Click **Publish**, once the publish action is complete click **Play**.

14. This will take you to the application so that you can see how it looks. You can use the application and close the tab when you are satisfied.



Next steps

See Also

[Automate your workflow from idea to production](#)

Tutorial: Automate solution deployment using GitHub Actions for Microsoft Power Platform

Article • 10/06/2022

In this tutorial, you will learn how to:

- ✓ Create a new GitHub repository
- ✓ Create two GitHub workflows using GitHub Actions for Microsoft Power Platform

The workflows can automatically export your app (as an unmanaged solution) from a development environment, generate a build artifact (managed solution), and deploy the app into your production environment. This tutorial uses the [ALMLab solution](#) you built and the environments you set up in previous tutorials.

Related tutorials: [Get started](#), and [Build a model-driven app](#).

Create a GitHub Account

1. Go to <https://github.com> and click **Sign up** or **Start a free trial** (or sign in if you have an existing account).

Username *

Email address *

Password *

Make sure it's at least 15 characters OR at least 8 characters including a number and a lowercase letter.
[Learn more.](#)

Email preferences

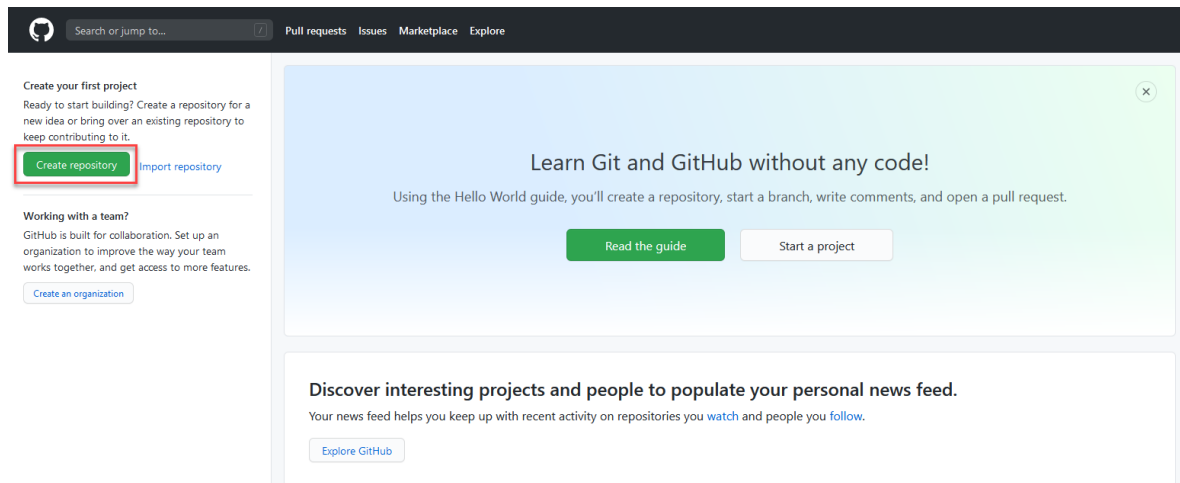
☐ Send me occasional product updates, announcements, and offers.

Verify your account



Create account

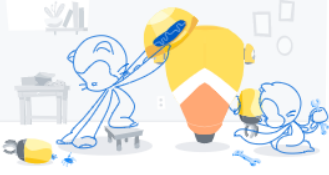
2. After you have created your account, create a repository by selecting **Create repository** or **New**.



You might see the following alternative landing screen:

What do you want to do first?

Every developer needs to configure their environment, so let's get your GitHub experience optimized for you.



Start a new project
Start a new repository or bring over an existing repository to keep contributing to it.

Create a repository



Collaborate with your team
Improve the way your team works together and get access to more features with an organization.

Create an organization



Learn how to use GitHub
Get started with an "Introduction to GitHub" course in our Learning Lab.

Start Learning

3. Create your new repository and name it 'poweractionslab'. Make sure you select **Add a README file** to initiate the repo and choose **Create repository**.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?

[Import a repository.](#)

Repository template

Start your repository with a template repository's contents.

No template ▾

Owner *



Repository name *

poweractionslab ✓

Great repository names are [poweractionslab is available.](#) for inspiration? How about [upgraded-tribble?](#)

Description (optional)

☒ Public



Anyone on the internet can see this repository. You choose who can commit.

☐ Private



You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☒ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▾

Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

License: MIT License ▾

This will set [main](#) as the default branch. Change the default name in your [settings](#).

Grant your Marketplace apps access to this repository

You are subscribed to 1 Marketplace app

☒  Azure Pipelines
Continuously build, test, and deploy to any platform and cloud

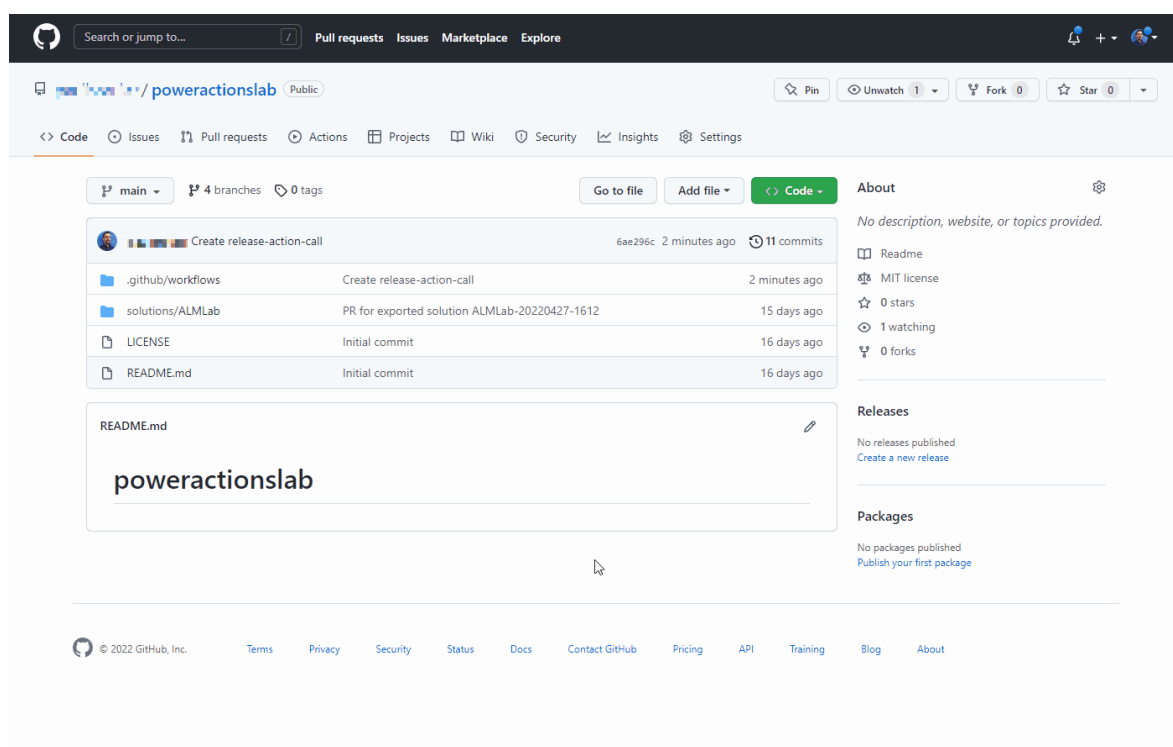
 You are creating a public repository in your personal account.

Create repository

Creating a new secret for Service Principal Authentication

1. Navigate to your repository and click **Settings**, then expand **Secrets**, and then click **Actions**.
2. On the *Secrets* page, name the secret 'PowerPlatformSPN'. Use the client secret from the application registration created in Azure Active Directory and enter it into

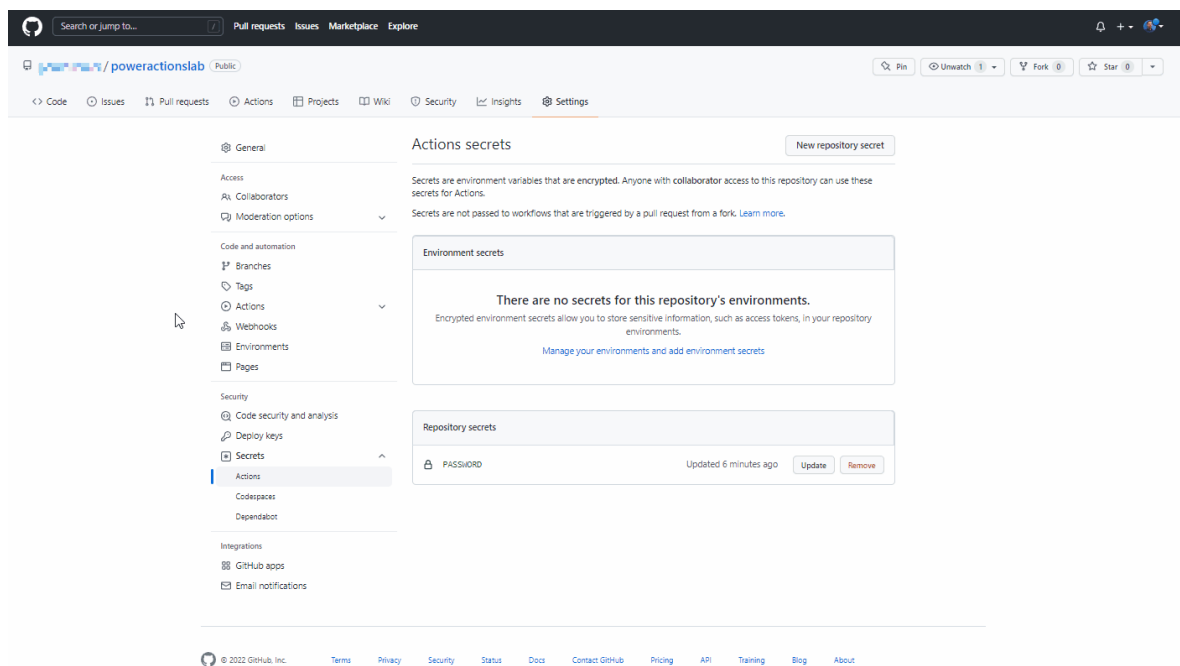
the **Value** field, and then select **Add secret**. The client secret will be referenced in the YML files used to define the GitHub workflows later in this lab.



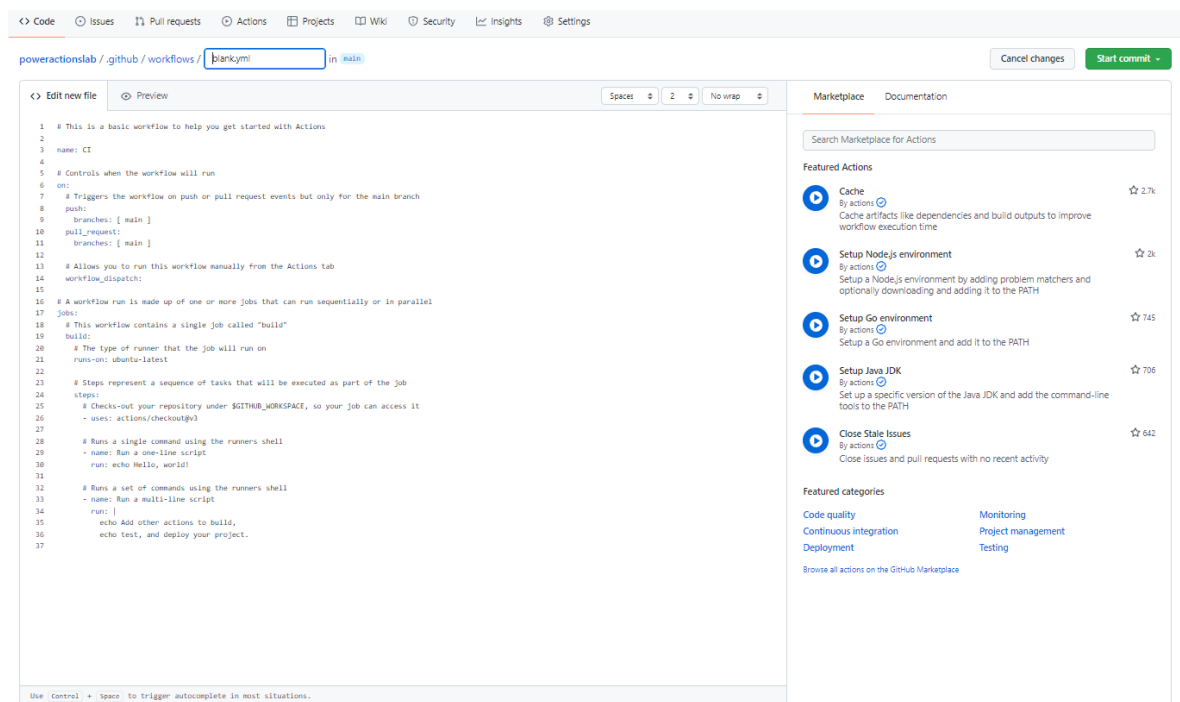
The client secret is now securely stored as a GitHub secret.

Create a workflow to export and unpack the solution file to a new branch

1. click on **Actions** and click **set up a workflow yourself** or click **Configure** in the *Simple workflow* box under the *suggested for this repository* section.



- This will start a new YAML file with a basic workflow to help you get started with GitHub actions.



- Delete the pre-created content, paste the content from the [export-and-branch-solution-with-spn-auth.yml](#) file, and then rename the file to 'export-and-branch-solution.yml'.

```

1 name: export-and-branch-solution
2 # Export solution from DEV environment
3 # unpack it and prepare, commit and push a git branch with the changes
4
5 on:
6   workflow_dispatch:
7   inputs:
8     # Change this value
9     solution_name:
10      description: 'name of the solution to worked on from Power Platform'
11      required: true
12      default: ALMLab
13    #Do Not change these values
14    solution_exported_folder:
15      description: 'folder name for staging the exported solution *do not change*'
16      required: true
17      default: out/exported/
18    solution_folder:
19      description: 'staging the unpacked solution folder before check-in *do not change*'
20      required: true
21      default: out/solutions/
22    solution_target_folder:
23      description: 'folder name to be created and checked in *do not change*'
24      required: true
25      default: solutions/
26  env:
27    #edit your values here
28    ENVIRONMENT_URL: '<ENVIRONMENTURL>'
29    CLIENT_ID: '<APPID>'
30    TENANT_ID: '<TENANT ID>|'
31
32 -jobs-

```

Use Control + Space to trigger autocomplete in most situations.

4. Update `<ENVIRONMENTURL>` with the URL for the development environment you want to export from (for example: `https://poweractionsdev.crm.dynamics.com`).

5. Update `<APPID>` and `<TENANT ID>` with your values.

- If you are using credentials, paste the [export-and-branch-solution.yml](#) file instead of the *export-and-branch-solution-with-spn-auth.yml* file contents. Update `<USERNAME>` with the username you are using to connect to the environment.

```

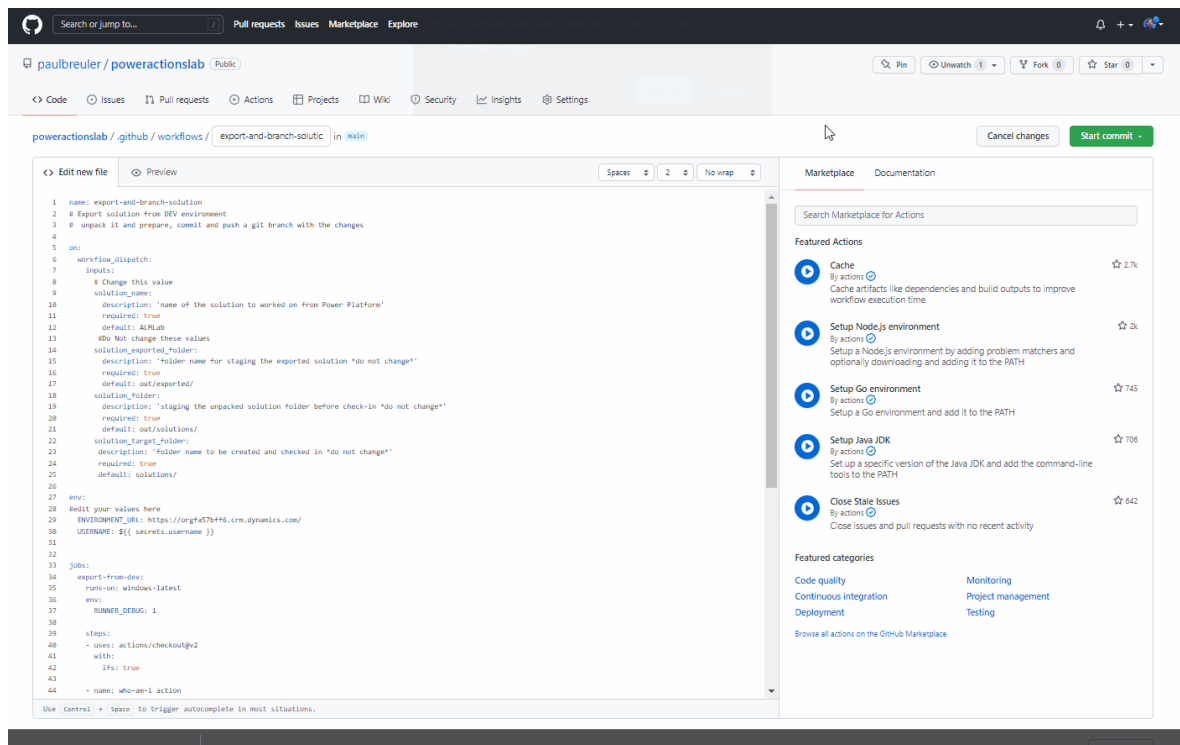
75 lines (65 sloc) | 2.6 KB
1 name: export-and-branch-solution
2 # Export solution from DEV environment
3 # unpack it and prepare, commit and push a git branch with the changes
4
5 on:
6   workflow_dispatch:
7   inputs:
8     # Change this value
9     solution_name:
10      description: 'name of the solution to worked on from Power Platform'
11      required: true
12      default: ALMLab
13    #Do Not change these values
14    solution_exported_folder:
15      description: 'folder name for staging the exported solution *do not change*'
16      required: true
17      default: out/exported/
18    solution_folder:
19      description: 'staging the unpacked solution folder before check-in *do not change*'
20      required: true
21      default: out/solutions/
22    solution_target_folder:
23      description: 'folder name to be created and checked in *do not change*'
24      required: true
25      default: solutions/
26  env:
27    #edit your values here
28    ENVIRONMENT_URL: '<ENVIRONMENTURL>'
29    USERNAME: '<USERNAME>'
30
31

```

💡 Tip

If you are not familiar with GitHub Actions and want to learn more check out the official documentation at <https://docs.github.com/en/actions>.

6. You are now ready to commit your changes. Select **Start commit**, type **Create export.yml** in the title field, and then add a description (optional). Next, click **Commit new file**.

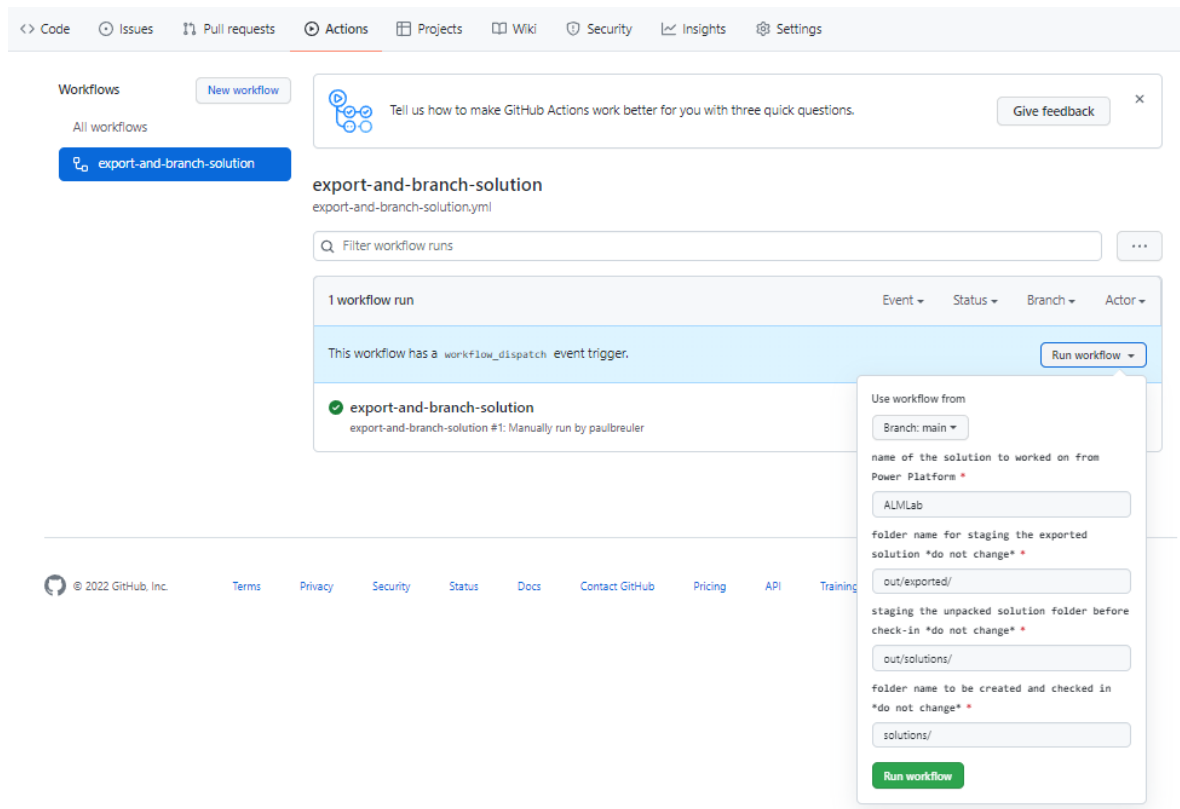


Congratulations, you have just created your first GitHub workflow using the following actions:

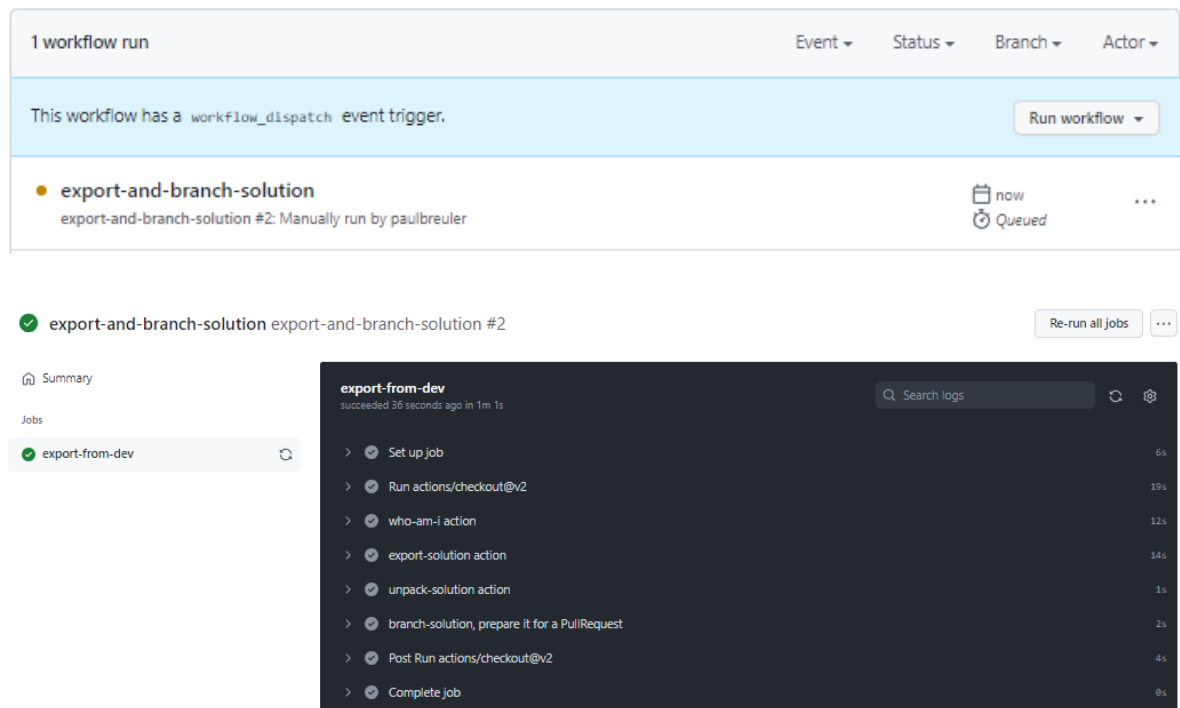
- **Who Am I:** Ensures that you can successfully connect to the environment you are exporting from.
- **Export Solution:** Exports the solution file from your development environment.
- **Unpack Solution:** The solution file that is exported from the server is a compressed (zip) file with consolidated configuration files. These initial files are not suitable for source code management as they are not structured to make it feasible for source code management systems to properly do differencing on the files and capture the changes you want to commit to source control. You need to 'unpack' the solution files to make them suitable for source control storage and processing.
- **Branch Solution:** Creates a new branch to store the exported solution.

Test the export and unpack workflow

1. Next, test that the workflow runs. Navigate to **Actions**, **Run workflow**, and choose **Run workflow**. If you have a different solution name than 'ALMLab' then change the value here but leave the other values as is.

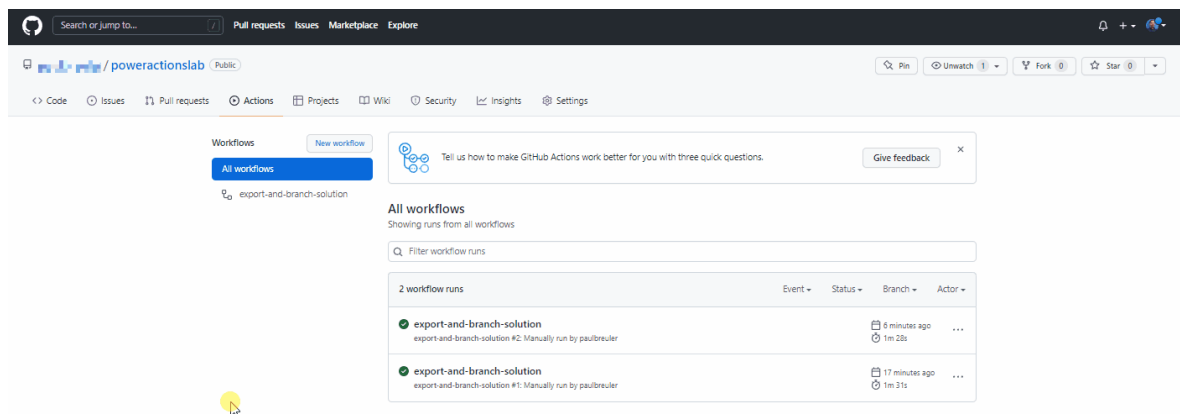


- After 5–10 seconds the workflow will start, and you can select the running workflow to monitor progress.



- After the workflow has completed, validate that a new branch has been created with the solution unpacked to the solutions/ALMLab folder. **Navigate to the Code tab and expand the *branches* drop-down.**
- Select the branch that was created by the action.

5. Validate that the solutions/ALMLab folder has been created in the new branch and then create a Pull request to merge the changes into the main branch. **Click *Contribute*** and in the flyout **click *Open Pull request***.
6. On the *Open a Pull request* screen, add a title and description, as desired, then **click *Create pull request***.
7. The screen will update showing the newly create pull request. As the pull request is created confirmation will be provided showing that our branch has no conflict with the main branch. This confirmation means that the changes can be merged into the main branch automatically. **Click *Merge pull request*** and then **click *Confirm merge***. Optionally, click delete branch to clean up the now defunct branch.



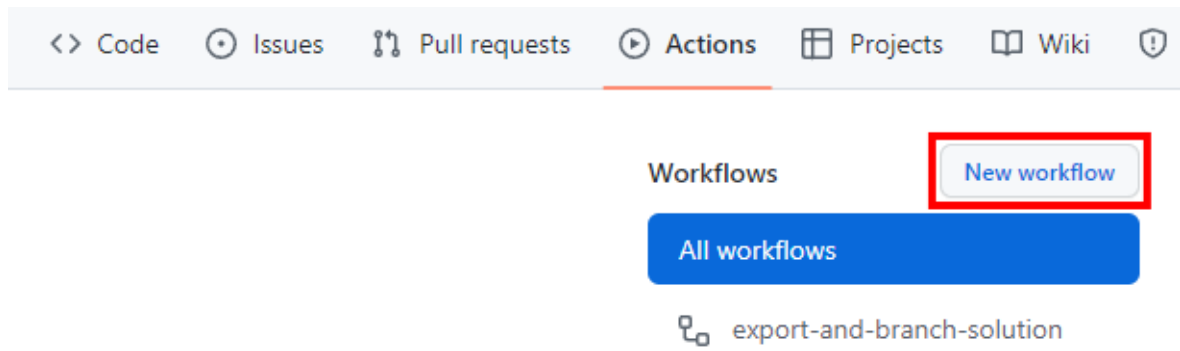
8. Navigate back to the default (main) branch and validate the solution is now available there as well.

Create a reusable workflow to generate a build artifact and import to production

In this section, we will create an additional workflow that:

- Creates a managed solution and publishes it as a GitHub artifact
- Imports the build artifact into the production environment

1. Navigate to **Actions** and select **New workflow**.



2. Chose **setup** a workflow yourself.

Choose a workflow template

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow template to get started.

Skip this and [set up a workflow yourself →](#)

3. Rename the title of the workflow to 'release-solution-to-prod-with-inputs' and copy the content from the [release-solution-to-prod-with-inputs.yml](#) file and paste it into the **Edit new file** screen.

poweractionslab / .github / workflows / [release-solution-to-prod-](#) in main

```
1 name: release-solution-to-prod-reusable
2 # Reusable workflow
3 # convert solution to managed (using a build PowerPlatform environment for the conversion)
4 # upload the solution to the GitHub artifacts and deploy to the PROD environment
5 on:
6   workflow_call:
7     inputs:
8       #Do Not change these values
9       #Values are set by the caller
10      #caller sample: release-action-call.yml
11      solution_name:
12        description: 'The solution name.'
13        type: string
14        default: ALMLab
15      solution_shipping_folder:
16        description: 'folder name for staging the exported solution *do not change*'
17        type: string
18        default: out/ship/
19      solution_outbound_folder:
20        description: 'staging the unpacked solution folder before check-in *do not change*'
21        type: string
22        default: out/solutions/
23      solution_source_folder:
24        description: 'folder name to be created and checked in *do not change*'
25        type: string
26        default: solutions/
27      solution_release_folder:
28        description: 'folder where the released binaries are going to be hosted *do not change*'
29        type: string
30        default: out/release
31      BUILD_ENVIRONMENT_URL:
32        description: 'Build environment url'
33
```

4. Commit the changes. Choose **Start commit** and then add a title and description (optional). Next, select **Commit new file**.

Cancel changes

Start commit ▾

Commit new file

Create release-solution-to-prod-with-inputs.yml

Add an optional extended description...

☒ Commit directly to the `main` branch.

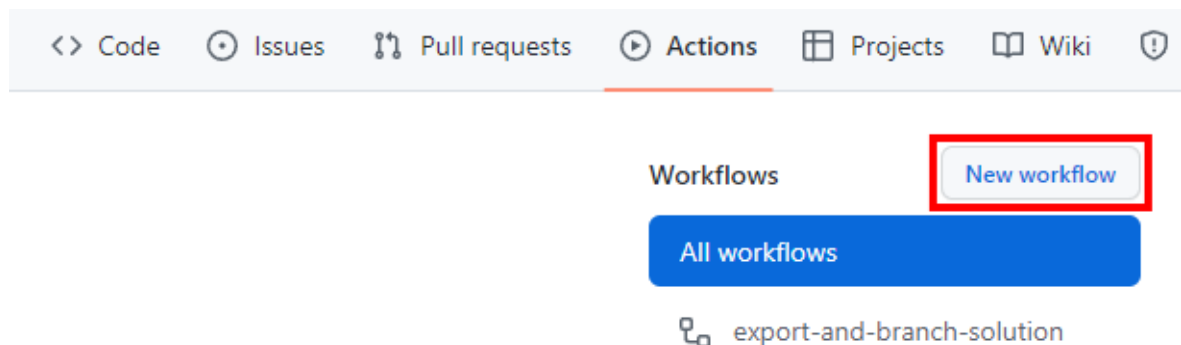
☐ Create a new branch for this commit and start a pull request. [Learn more about pull requests.](#)

Commit new file

Call the reusable workflow on the release event

In this section, we will call the reusable workflow on the [release event](#).

1. Navigate to **Actions** and select **New workflow**.



2. Choose **setup a workflow yourself**.

Choose a workflow template

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow template to get started.

Skip this and [set up a workflow yourself →](#)

3. Rename the title of the workflow to 'release-action-call' and copy the content from the [release-action-call.yml](#) file and paste it into the **Edit new file** screen.

powerplatform-actions-lab / sample-workflows / release-action-call.yml in main

<> Edit file

Preview changes

```
1 name: Release action
2 # Call the reusable workflow release-solution-with-inputs.yml
3 # Release your solution to prod when you create a new release.
4
5 on:
6   release:
7     types: [created]
8
9 jobs:
10   Release-solution-ALMLab:
11     uses: ../github/workflows/release-solution-with-inputs.yml
12     with:
13       #You can specify the solution name here
14       solution_name: ALMLab
15       #Update your values here
16       BUILD_ENVIRONMENT_URL: <BUILD_ENVIRONMENT>
17       PRODUCTION_ENVIRONMENT_URL: <PROD_ENVIRONMENT>
18       CLIENT_ID: <APP_ID>
19       TENANT_ID: <TENANT_ID>
20     secrets:
21       envSecret: ${ secrets.PowerPlatformSPN }
22
```

4. Update the following variables in the new workflow file:

- Update `<BUILD_ENVIRONMENT>` with the URL for the build environment you are using to generate the managed solution. For example:
`https://poweractionsbuild.crm.dynamics.com.`
- Update `<PROD_ENVIRONMENT>` with the URL for the production environment you are deploying to. For example:
<https://poweractionsprod.crm.dynamics.com>.
- Update `<APP_ID>` with the Application (Client) ID that can be found in the [App registrations blade of the Microsoft Azure Portal](#) by clicking into the registration created previously in this tutorial.
- Update `<TENANT_ID>` with the Directory (tenant) ID that can be found in the [App registrations blade of the Microsoft Azure Portal](#) by clicking into the registration created previously in this tutorial.

5. Commit the changes. Choose **Start commit** and then add a title (optional) and description (optional). Next, select **Commit new file**.

Cancel changes

Start commit ▾

Commit new file

Create release-action-call

Add an optional extended description...

☒ Commit directly to the `main` branch.

☐ Create a new branch for this commit and start a pull request. [Learn more about pull requests.](#)

Commit new file

Test the release to production workflow

You are now ready to test the last workflow. This workflow is triggered when a new release is deployed to production.

1. Navigate to Releases.

The screenshot shows the GitHub repository page for `poweractionslab`. The repository is public and has 2 branches and 1 tag. The file list shows the following structure:

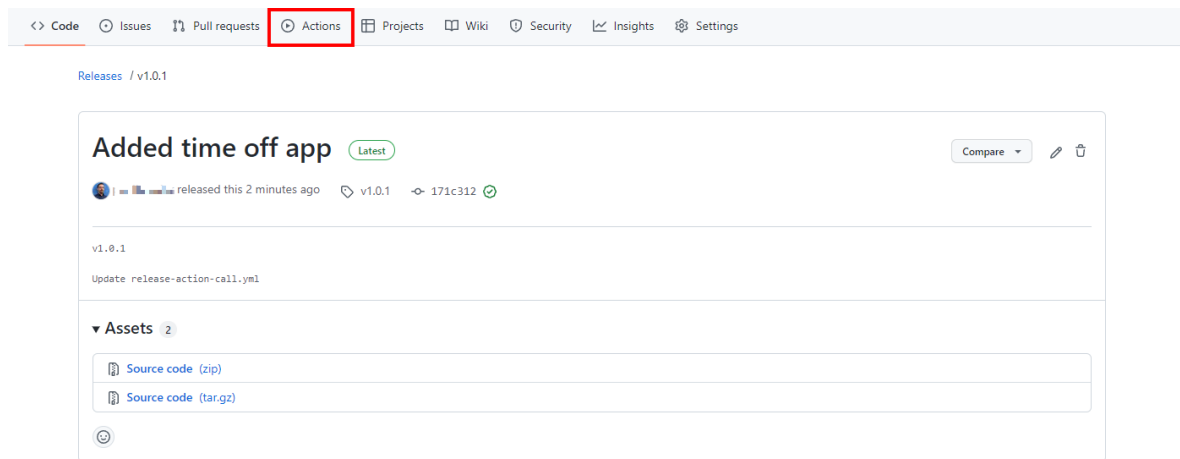
File	Commit	Time
<code>.github/workflows</code>	Update export-and-branch-solution-with-spn-auth.yml	17 minutes ago
<code>solutions/ALMLab</code>	PR for exported solution ALMLab-20220427-1612	15 days ago
<code>LICENSE</code>	Initial commit	16 days ago
<code>README.md</code>	Initial commit	16 days ago

The `README.md` file is open, showing the text `poweractionslab`. The right sidebar shows the `Releases` section with `1 tags` and a `Create a new release` button. A yellow cursor points to the `Create a new release` button.

2. Select Draft a new release.

3. Add a release tag, a title, and choose **Publish release**.

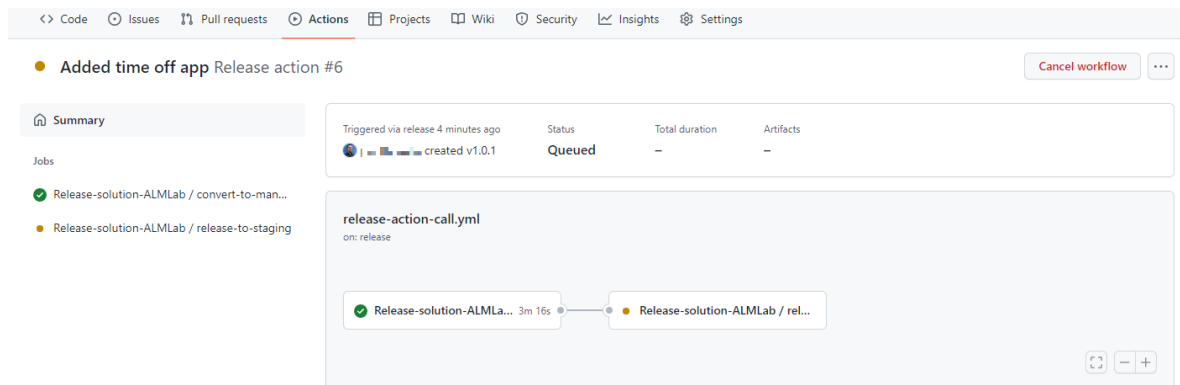
4. Select **Actions** to view the running workflow.



5. Choose the running workflow to view the actions as they run.



6. Wait for each action to complete.

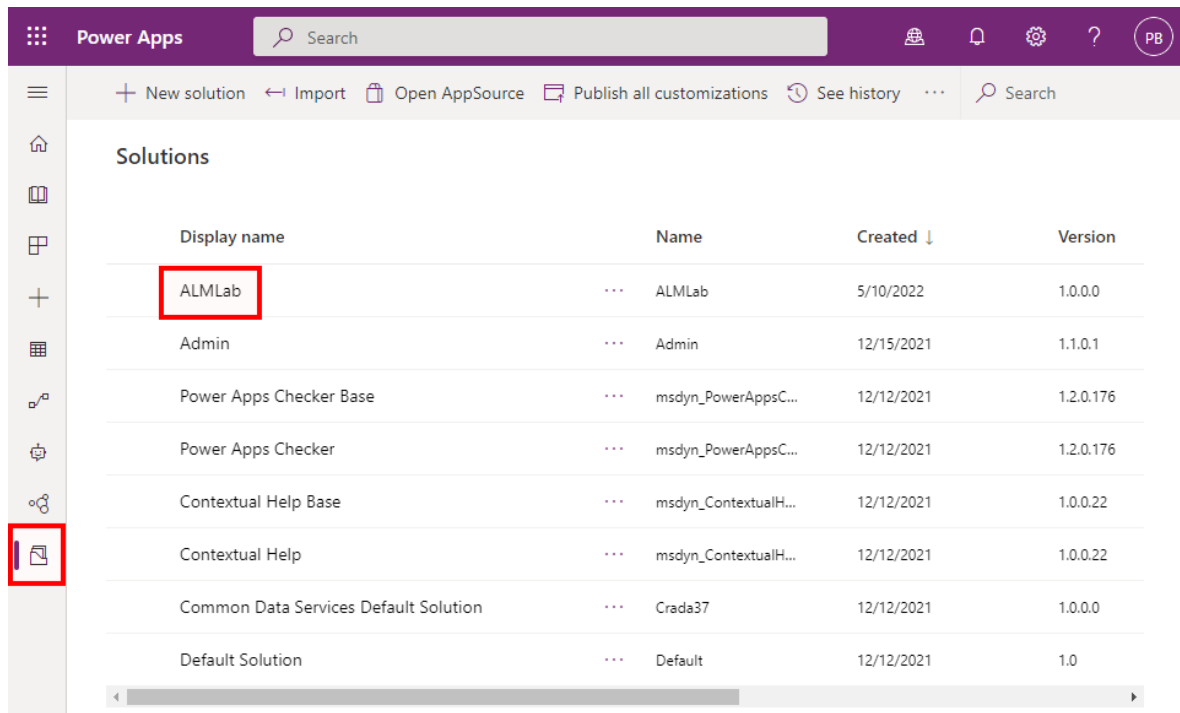


7. After the workflow has completed, log into your production environment and validate that the solution has been deployed as a managed solution.

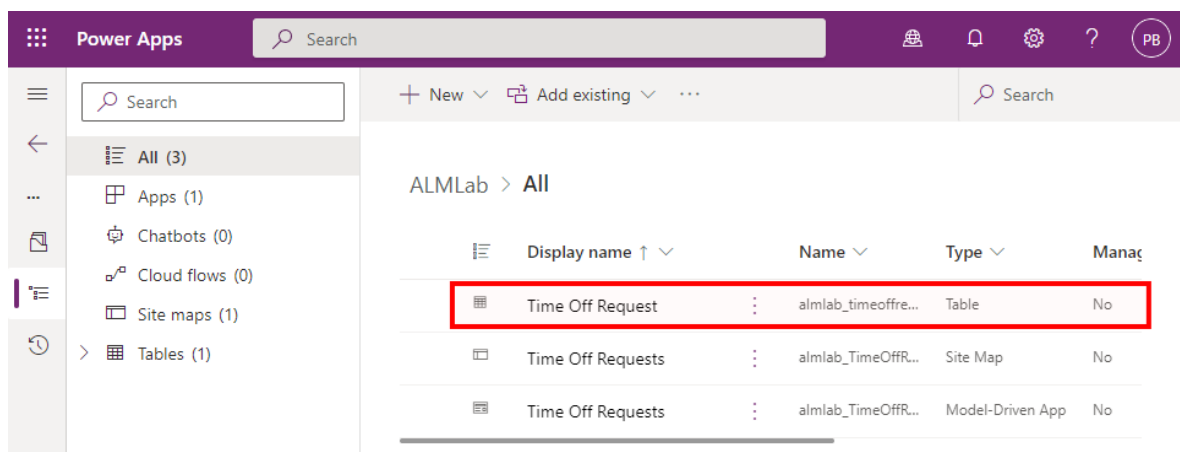
Deploy the update and review changes before production release

We will now test the end-to-end process and then see how we can view and validate changes to an app before it is deployed to production.

1. Navigate to the ALMLab solution in your development environment and choose **Edit** or click the solutions display name.



2. Select and view the **Time off Request** table.



3. Click + **Add column** and create the new column as shown in the figure below.

Power Apps Search

ALMLab > Tables > Time Off Request

Columns Relationships Business rules

Display name ↑	Name ↓
Created By	createdt...
Created By (Delegate)	createdt...
Created On	createdt...
Import Sequence Nu...	imports...
Modified By	modifie...
Modified By (Delegate)	modifie...
Modified On	modifie...
I Primary Name Column	almlab_...
Owner	ownerid...
Owning Business Unit	owningt...
Owning Team	owningt...
Owning User	owningt...
Record Created On	overrid...
Status	stateco...
Status Reason	statusc...
Time Off Request	almlab_...
Time Zone Rule Versi...	timezon...
UTC Conversion Time...	utconve...
Version Number	version...

Approved ×

Display name *

Approved

Name * ⓘ

almlab_ Approved

Data type * ⓘ

Yes/No

Items

☒ Yes

☐ No

Default value

No

Required * ⓘ

Optional

☒ Searchable ⓘ

Calculated or Rollup ⓘ **+ Add** ▾

Description ⓘ

Advanced options ▾

Done Cancel

Updated Field values:**

- Display name: Approved
- Data type: Yes/No

4. Click **Done**.

5. Click **Save Table**

The screenshot shows the Power Apps interface for configuring a table named 'Time Off Request'. The left sidebar shows the navigation pane with 'Tables (1)' selected. The main area displays the table configuration with columns and their properties. The 'Save Table' button is highlighted with a red box.

Display name	Name	Data type	Type	Custom
Approved	almlab_appro...	Yes/No	Custom	✓
Created By	createdby	Lookup	Standard	✓
Created By (Delegate)	createdonbeh...	Lookup	Standard	✓
Created On	createdon	Date an...	Standard	✓
Import Sequence Nu...	importsequen...	Whole ...	Standard	✓
Modified By	modifiedby	Lookup	Standard	✓
Modified By (Delegate)	modifiedonbe...	Lookup	Standard	✓
Modified On	modifiedon	Date an...	Standard	✓
I Primary Name Column	almlab_name	Text	Custom	✓
Owner	ownerid	Owner	Standard	✓
Owning Business Unit	owningbusine...	Lookup	Standard	✓
Owning Team	owningteam	Lookup	Standard	✓
Owning User	owninguser	Lookup	Standard	✓
Record Created On	overriddencre...	Date Only	Standard	✓
Status	statecode	Choice	Standard	✓
Status Reason	statuscode	Choice	Standard	✓
Time Off Request	almlab_timeof...	Unique ...	Standard	✓

6. Navigate back to your GitHub repositories **Actions** tab, choose **Run workflow**, and select the **Run workflow** button.

The screenshot shows the GitHub Actions interface for the 'export-and-branch-solution' workflow. The 'Run workflow' button is highlighted with a red box.

2 workflow runs

This workflow has a workflow_dispatch event trigger.

Run workflow

Use workflow from

Branch: main

name of the solution to worked on from Power Platform *

ALMLab

folder name for staging the exported solution *do not change* *

out/exported/

staging the unpacked solution folder before check-in *do not change* *

out/solutions/

folder name to be created and checked in *do not change* *

solutions/

Run workflow

7. After 5–10 seconds, the workflow will start and you can click on the running workflow to monitor its progress.

The screenshot shows the GitHub Actions interface. At the top, it says "2 workflow runs" with filters for Event, Status, Branch, and Actor. Below this, a message states "This workflow has a workflow_dispatch event trigger." with a "Run workflow" button. A list of runs shows "export-and-branch-solution" as "export-and-branch-solution #3: Manually run by [user]" with a status of "Queued".

Below the runs list, the "Summary" tab is selected, showing a job named "export-from-dev". The "Jobs" section lists "export-from-dev". The workflow log for "export-from-dev" is displayed, showing it started 52s ago. The log includes the following steps:

- Set up job (43s)
- Run actions/checkout@v2 (9s)

The log for "Run actions/checkout@v2" shows the following details:

```
1 Run actions/checkout@v2
2 with:
3   lfs: true
4   repository: [redacted]/poweractionslab
5   token: ***
6   ssh-strict: true
7   persist-credentials: true
8   clean: true
9   fetch-depth: 1
10  submodules: false
11  set-safe-directory: true
12  env:
13    ENVIRONMENT_URL: https://orp36dcab52.crm.dynamics.com/
14    CLIENT_ID: db972f39-0fe5-42f6-a1c6-5f9e1b28be43
15    TENANT_ID: 1557f771-4c8e-4dbd-8b80-dd00a8e833e
16    RUNNER_DEBUG: 1
17  Syncing repository: [redacted]/poweractionslab
18  Getting Git version info
19  Working directory is 'D:\a\poweractionslab\poweractionslab'
20  "C:\Program Files\Git\bin\git.exe" version
21  git version 2.35.1.windows.2
22  "C:\Program Files\Git\bin\git.exe" lfs version
23  who-am-i action
24  export-solution action
25  unpack-solution action
26  branch-solution, prepare it for a PullRequest
27  Post Run actions/checkout@v2
```

8. After the workflow completes, navigate to the new branch by selecting **Code** and then **Branches**.

The screenshot shows the GitHub repository view for "poweractionslab". The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below the navigation bar, the "main" branch is selected, showing "2 branches" and "4 tags". A "Go to file" button is visible.

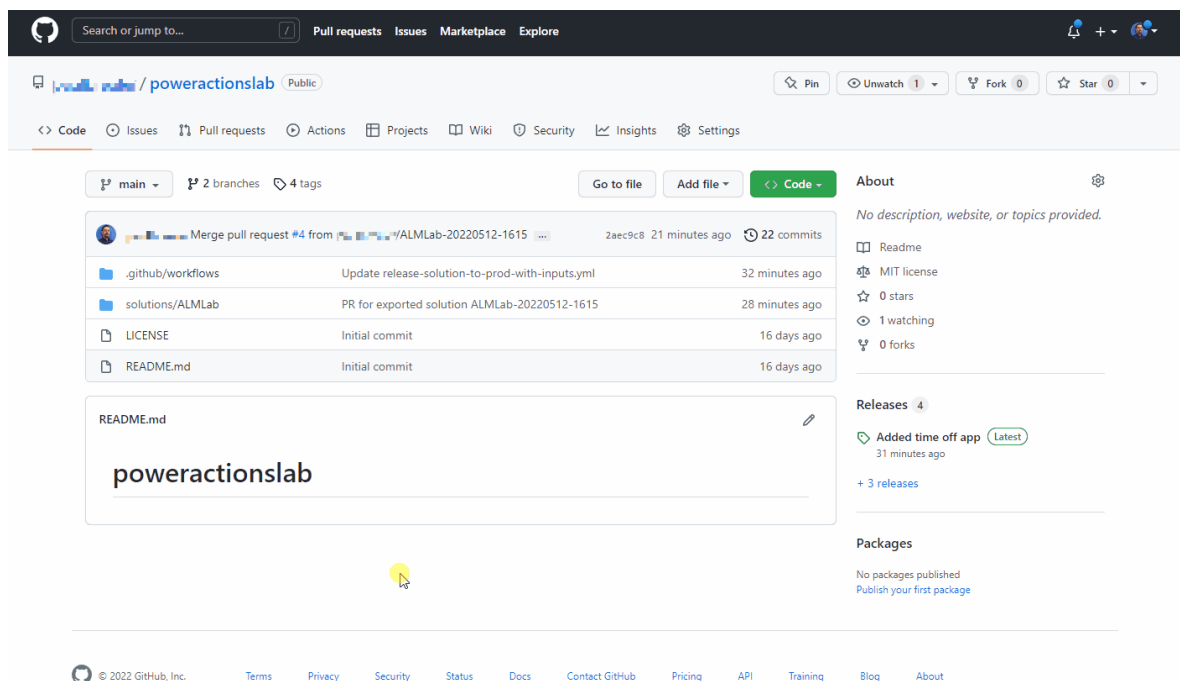
The repository content is displayed as a table of files and folders:

File/Folder	Description	Commit Date
.github/workflows	Update release-solution-to-prod-with-inputs.yml	5 minutes ago
solutions/ALMLab	PR for exported solution ALMLab-20220427-1612	15 days ago
LICENSE	Initial commit	16 days ago
README.md	Initial commit	16 days ago

Below the table, the "README.md" file is shown, containing the text "poweractionslab".

9. Select the branch that was created by the expand **contribute** and click **Open pull request**.

10. Add a title (optional) and then click **Create pull request**.
11. After the content updates, click the **Files changed** tab.
12. Notice that the changes to the solution are highlighted in green to indicate that this section of the file was added when compared to the same file in the main branch.
13. Navigate back to the **Conversation** tab. Select **Pull requests** and then select the pull request previously created.
14. On the **Pull request** page, select **Squash and merge** to merge the updated solution file into your main branch, optionally delete that is now merged into main.



15. Follow the steps in the [Test the release to production workflow](#) section to create a new release and validate that the updated solution has been deployed to your production environment.

Congratulations, you have successfully setup a sample CI/CD workflow using GitHub actions!

See Also

[Automate your workflow from idea to production](#)

Power Apps component framework

Article • 04/13/2022

To be accessible by Power Apps makers, components in the Power Apps component framework must be packaged in a solution, exported, and then imported into a Power Apps environment with Dataverse. The following sections describe how to do this.

For more information about using ALM techniques with code components see [Code components application lifecycle management \(ALM\)](#).

Package and deploy a code component

This section describes how to import code components into Microsoft Dataverse so that the components are available to Power Apps makers.

After implementing the code components by using the Power Platform CLI, the next step is to pack all the code component elements into a solution file and import the solution file into Dataverse so that you can see the code components in the maker runtime experience.

To create and import a solution file:

1. Create a new folder in the folder that has the cdsproj file, and name it **Solutions** (or any name of your choice) by using the CLI command `mkdir Solutions`. Navigate to the directory by using the command `cd Solutions`.
2. Create a new solution project by using the following command. The solution project is used for bundling the code component into a solution zip (compressed) file that's used for importing into Dataverse.

.NET CLI

```
pac solution init --publisher-name \<enter your publisher name\>
--publisher-prefix \<enter your publisher prefix\>
```

⚠ Note

The publisher-name and publisher-prefix values must be unique to your environment. More information: [Solution publisher](#) and [Solution publisher prefix](#)

3. After the new solution project is created, refer the **Solutions** folder to the location where the created sample component is located. You can add the reference by using the command shown below. This reference informs the solution project about which code components should be added during the build. You can add references to multiple components in a single solution project.

.NET CLI

```
pac solution add-reference --path \<path to your Power Apps component  
framework project\>  
```.dotnetcli
```

4. To generate a zip file from the solution project, go to your solution project directory and build the project by using the following command. This command uses the MSBuild program to build the solution project by pulling down the NuGet dependencies as part of the restore. Only use `/restore` the first time the solution project is built. For every build after that, you can run the command `msbuild`.

.NET CLI

```
msbuild /t:build /restore
```

#### Tip

- If *MSBuild* 15.9.\* is not in the path, open Developer Command Prompt for Visual Studio 2017 to run the `msbuild` commands.
- Building the solution in the *debug* configuration generates an unmanaged solution package. A managed solution package is generated by building the solution in *release* configuration. These settings can be overridden by specifying the `SolutionPackageType` property in the `cdsproj` file.
- You can set the `msbuild` configuration to **Release** to issue a production build. Example: `msbuild /p:configuration=Release`
- If you encounter an error that says "Ambiguous project name" when running the `msbuild` command on your solution, ensure that your solution name and project name aren't the same.

5. The generated solution files are located in the \bin\debug\ (or \bin\release) folder after the build is successful.
6. You can use the [Microsoft Power Platform Build Tools](#) to automate importing the solution into a Dataverse environment; otherwise, you can manually [import the solution into Dataverse](#) by using the web portal.

## Additional tasks that you can do with the framework and solutions

Below are links to additional common tasks that you can do when working with the framework and solutions.

- [Create a solution project based on an existing solution in Dataverse](#)
- [Create a plug-in project and add a reference to it in your solution](#)
- [Remove components from a solution](#)

### See also

[Plug-ins](#)

# Plug-ins

Article • 10/14/2023

A solution is used to package and deploy plug-ins and custom workflow activities to other environments. For example, the sequence below defines a simplistic development and deployment sequence.

1. Create a [custom publisher](#) and [unmanaged](#) solution in your DEV environment.
2. Write one or more [plug-ins](#) or [custom workflow activities](#).
3. [Register](#) the plug-ins or custom workflow activities in the unmanaged solution you created in step 1.
4. [Export](#) the unmanaged solution as a managed solution.
5. Import the managed solution into another environment (that is, TEST or PROD).

In the real world, you debug the code in the TEST environment, go back and update the unmanaged solution with the revised code, and export to a managed solution. Along the way you use revision control to manage the code updates and solution versions. For more information about revision control and versioning of solutions, see [Source control](#).

When planning your solution design, consider whether you'll place your custom code and other customizations (customized entities, forms, views, and so on) in the same solution or you'll divide these customizations among multiple solutions, where one solution contains the custom code and another solution contains the other customizations (customized entities, forms, views, and so on).

## Tip

Start with a custom publisher and unmanaged solution, and then develop and test the plug-in or custom workflow activity in that solution. We recommend against developing a plug-in or custom workflow activity in the default solution and then adding it to a custom solution.

## Register a plug-in or custom workflow activity in a custom unmanaged solution

After you've created a custom publisher and unmanaged solution, and have written the custom code, you're ready to register the code in the unmanaged solution and begin

testing it.

## Register a custom workflow activity assembly

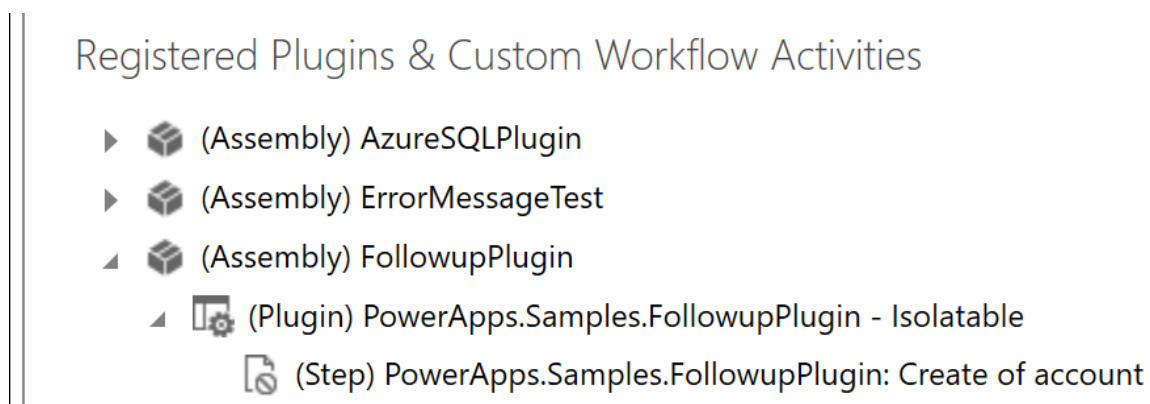
To distribute a custom workflow activity in a solution, you must add the registered assembly that contains it to an unmanaged solution. First, [register](#) the custom workflow assembly, and then add the assembly to a solution by following these steps.

1. Create a new solution in the Power Apps [maker portal](#) , or use an existing solution. To create a new solution, select **Solutions** > **New solution**, and then enter the required information for your solution.
2. With **Solutions** selected in the left navigation pane, select the solution name in the list, and then select **Add existing** > **Other** > **Plug-in assembly**.
3. Search for the compiled custom workflow assembly by name.
4. Select the custom workflow activity assembly, and then select **Add**.

## Register a plug-in assembly and step

The procedure to register a plug-in is similar to registering a custom workflow activity assembly, except you must also register one or more *steps* which identify the conditions under which the plug-in should be executed by Microsoft Dataverse.

To begin, follow these instructions to [register a plug-in and step](#) by using the Plug-in Registration tool. Next, we'll create a solution and then add the plug-in assembly and step to the solution by using the modern maker interface. The example "Followup" plug-in and step registration we'll use is shown in the following illustration.



Let's get started adding those components to our solution.

1. Create a new solution in the Power Apps [maker portal](#) , or use an existing solution. To create a new solution, select **Solutions** > **New solution**, and enter the



required information for your solution.

2. With **Solutions** selected in the left navigation panel, select the solution name in the list, and then select **Add existing** > **Other** > **Plug-in assembly**.
3. Search for the compiled plug-in assembly by name.
4. Select the plug-in assembly, and then select **Add**.

## Add existing plug-in assemblies

Select plug-in assemblies from other solutions or plug-in assemblies that aren't in solutions yet. Add also add them to Common Data Service.

1 plug-in assembly selected

	Name	Version
✓	FollowupPlugin	1.0.0.0

5. Add a step to the solution by selecting **Add existing** > **Other** > **Plug-in step**.

### 💡 Tip

In the Plug-in Registration tool, a step is called a **step**. In the classic interface **Solution Explorer**, a step is called an **Sdk message processing step**. In the modern maker interface, a step is called an **Plug-in step**.

6. Search for the registered step, select it, and then select **Add**.

## Add existing plug-in steps

Select plug-in steps from other solutions or plug-in steps that aren't in solutions yet. Adding plug-in steps that aren't already in solution to Common Data Service.

1 plug-in step selected

🔍 Plug-in step ▾

✓	Name	SDK mess...	Event han...	Primary o...	Execution...
✓	PowerApps.Samples.FollowupPlugin: Create of account	Create	PowerApps.Sam	4608	Asynchronous

The resulting solution with the two components is shown in the following illustration.

Display name ▾	Name	Type ▾	Managed...	Modified
 FollowupPlugin	...	FollowupPlugin	Plugin assembly 	10 min ago
PowerApps.Samples.FollowupPlug	...	PowerApps.Samples.F	Sdk message 	9 min ago

It's possible to add the step to the solution before (or without) adding the related plug-in assembly. To add the assembly, select the option (...) menu next to the plug-in step name, select **Add required components**, and then select **OK**. This automatically adds the related plug-in assembly. Don't add the plug-in assembly to the solution if you intend to provide that assembly in another solution that the current solution will be dependent on.

Note that removing the plug-in assembly from the solution won't remove any steps that depend on it. You must remove those steps individually.

## Delete a plug-in from a solution by updating the assembly

Sometimes you may need to delete a plug-in from a solution, however you cannot do so with a solution upgrade. Instead, you can delete an assembly with a solution upgrade, and end up with the desired result.

The details of this workaround are described in these instructions.

1. Start by updating the "major" version number of the assembly which has the target plug-in types removed.

After this version change, you will not be able to update the assembly using the Plug-in Registration tool (PRT). However, you will be able to register the assembly as a new assembly side-by-side with the old assembly.

2. Register your new assembly side by side with the old assembly using the PRT.
3. Migrate any existing steps for the old plug-in types to the corresponding plug-in types in the new assembly.
4. Delete the old assembly.
5. Export the unmanaged solution as a managed solution with a new solution version number.

6. Import the new managed solution, as an upgrade, into an environment where the old assembly is registered. The solution upgrade should delete the old assembly, and the unwanted types with it.

## See also

[Update a solution](#)

[Assembly versioning](#)

[Web resources](#)

# Web resources

Article • 02/15/2022

Solutions can be used to package and deploy web resources to other environments. To manually add a web resource to a solution, follow these steps.

1. Open the unmanaged solution you want to add the web resource to by using [Power Apps](#) .
2. Select **Add existing** > **Other** > **Web resource**.
3. Search for the web resource by name.
4. Select the web resource, and then select **Add**

You can also add web resources to a solution by using code. Use the `SolutionUniqueName` optional parameter to associate a web resource with a specific solution when the web resource is created. This is demonstrated in this sample: [Import files as web resources](#).

## See also

[Configuration Migration and Package Deployer tools](#)

# Use managed properties

Article • 07/15/2022

You can control which of your managed solution components are customizable by using managed properties. By default, all custom solution components are customizable. Each solution component has a **Can be customized** (`IsCustomizable`) property. As long as this property value is set to true, more properties specific to the type of solution component can be specified. If you set the `IsCustomizable.Value` property to false, after the solution is installed as a managed solution the solution component will not be customizable.

Managed properties ensure that only a solution from the same publisher will be able to change the component. Managed properties will only affect managed components and does not force this in the development environments where the component is still unmanaged. The use of the `IsCustomized` managed property is intended to ensure that no other solution layer from any other publisher, and no active customizations can be done on the component once it is installed as a managed solution.

The following table lists some managed properties for a subset of available solution components.

Component	Display Name	Property
Entity	Can be customized	<code>IsCustomizable.Value</code>
Entity	Display name can be modified	<code>IsRenameable.Value</code>
Entity	Can be related entity in relationship	<code>CanBeRelatedEntityInRelationship.Value</code> (Read Only)
Entity	Can be primary entity in relationship	<code>CanBePrimaryEntityInRelationship.Value</code> (Read Only)
Entity	Can be in many-to-many relationship	<code>CanBeInManyToMany.Value</code> (Read Only)

Component	Display Name	Property
Entity	New forms can be created	<a href="#">CanCreateForms</a> .Value
Entity	New charts can be created	<a href="#">CanCreateCharts</a> .Value
Entity	New views can be created	<a href="#">CanCreateViews</a> .Value
Entity	Can change any other entity properties not represented by a managed property	<a href="#">CanModifyAdditionalSettings</a> .Value
Entity	Can create attributes	<a href="#">CanCreateAttributes</a> .Value
Field (Attribute)	Can be customized	<a href="#">IsCustomizable</a> .Value
Field (Attribute)	Display name can be modified	<a href="#">IsRenameable</a> .Value
Field (Attribute)	Can change requirement level	<a href="#">RequiredLevel</a> .CanBeChanged  Note:  <a href="#">RequiredLevel</a> is the only managed property to use the <a href="#">CanBeChanged</a> property.

Component	Display Name	Property
Field (Attribute)	Can change any other attribute properties not represented by a managed property	<a href="#">CanModifyAdditionalSettings</a> .Value
Entity Relationship	Can be customized	<a href="#">IsCustomizable</a> .Value
Form	Can be customized	SystemForm.IsCustomizable.Value
Chart	Can be customized	SavedQueryVisualization.IsCustomizable.Value
View	Can be customized	SavedQuery.IsCustomizable.Value
Option Set	Can be customized	<a href="#">IsCustomizable</a> .Value
Web Resource	Can be customized	WebResource.IsCustomizable.Value
Workflow	Can be customized	Workflow.IsCustomizable.Value
Workflow	Is Custom Processing Step Allowed For Other Publishers	Workflow.IsCustomProcessingStepAllowedForOtherPublishers.Value
Assembly	Can be customized	SdkMessageProcessingStep.IsCustomizable.Value
Assembly Registration	Can be customized	ServiceEndpoint.IsCustomizable.Value
E-mail Template	Can be customized	Template.IsCustomizable.Value
KB Article Template	Can be customized	KbArticleTemplate.IsCustomizable.Value

Component	Display Name	Property
Contract Template	Can be customized	<code>ContractTemplate.IsCustomizable.Value</code>
Mail Merge Template	Can be customized	<code>MailMergeTemplate.IsCustomizable.Value</code>
Dashboard	Can be customized	<code>SystemForm.IsCustomizable.Value</code>
Security Roles	Can be customized	<code>Role.IsCustomizable.Value</code>
System Form	Can be deleted	<code>CanBeDeleted.Value</code>
System Query	Can be deleted	<code>CanBeDeleted.Value</code>

## Workflow Is Custom Processing Step Allowed For Other Publishers

This managed property controls whether plug-in steps registered by 3rd parties for messages created by custom process actions will run. The default value is `false`, which means plug-in steps registered which do not use the same solution publisher will not run. When this is `true`, the publisher of the custom process action allows registered plug-in registration steps to run.

## Update managed properties

After you release your managed solution, you may decide that you want to change the managed properties. You can only change managed properties to make them less restrictive. For example, after your initial release you can decide to allow customization of an entity.

You update managed properties for your solution by releasing an update to your solution with the changed managed properties. Your managed solution can only be updated by another managed solution associated with the same publisher record as the original managed solution. If your update includes a change in managed properties to make them more restrictive, those managed property changes will be ignored but other changes in the update will be applied.



Because the original publisher is a requirement to update managed properties for a managed solution, any unmanaged solution cannot be associated with a publisher that has been used to install a managed solution.

#### ⓘ Note

This means you will not be able to develop an update for your solution by using an organization where your managed solution is installed.

## Check managed properties

Use [IsComponentCustomizableRequest](#) to check whether a solution component is customizable. Alternatively, you can check the solution component properties but you must consider that the ultimate determination of the meaning depends on the values of several properties. Each solution component has an `IsCustomizable` property. When a solution component is installed as part of a managed solution, the `IsManaged` property will be true. Managed properties are only enforced for managed solutions. When checking managed properties to determine if an individual solution component is customizable, you must check both the `IsCustomizable` and `IsManaged` properties. A solution component where `IsCustomizable` is false and `IsManaged` is false, is customizable.

Entity and attribute have more managed properties in addition to `IsCustomizable`. These managed properties are not updated if `IsCustomizable` is set to false. This means that in addition to checking the individual managed property, you must also check the `IsCustomizable` property to see if the managed property is being enforced.

## See also

[Managed properties](#)

# Dependency tracking for solution components

Article • 02/07/2025

Solutions are made of solution components. You use the **Solutions** area in Microsoft Dataverse to create or add solution components. You can perform these actions programmatically by using the [AddSolutionComponentRequest](#) message or any messages that create or update solution components that include a `SolutionUniqueName` parameter.

Solution components often depend on other solution components. You can't delete any solution component that has dependencies on another solution component. For example, a customized ribbon typically requires image or script web resources to display icons and perform actions using scripts. As long as the customized ribbon is in the solution, the specific web resources it uses are required. Before you can delete the web resources, you must remove references to them in the customized ribbon. These solution component dependencies can be viewed in the application by selecting **Show Dependencies**.

This article describes the types of solution components you can include in your solutions and how they depend on each other.

## All solution components

The complete list of available solutions component types is located in the system `componenttype` global choice column. The supported range of values for this property is available by including the file `OptionSets.cs` or `OptionSets.vb` in your project. However, many of the solution component types listed there are for internal use only and the list doesn't provide information about the relationships between solution components.

## Solution component dependencies

Solution component dependencies help make sure you have a reliable experience working with solutions. They prevent actions that you normally perform from unintentionally breaking customizations defined in a solution. These dependencies are what allow a managed solution to be installed and uninstalled simply by importing or deleting a solution.

The solutions framework automatically tracks dependencies for solution components. Every operation on a solution component automatically calculates any dependencies to other components in the system. The dependency information is used to maintain the integrity of the system and prevent operations that could lead to an inconsistent state.

As a result of dependency tracking the following behaviors are enforced:

- Deletion of a component is prevented if another component in the system depends on it.
- Export of a solution warns the user if there are any missing components that could cause failure when importing that solution in another environment.

Warnings during export can be ignored if the solution developer intends that the solution is only to be installed in an environment where dependent components are expected to exist. For example, when you create a solution that is designed to be installed over a preinstalled "base" solution.

- Import of a solution fails if all required components aren't included in the solution and also don't exist in the target environment.
- Additionally, when you import a managed solution all required components must match the package type of the solution. A component in a managed solution can only depend on another managed component.

There are three types of solution component dependencies:

- Solution internal. Internal dependencies are managed by Dataverse. These dependencies exist when a particular solution component can't exist without another solution component.
- Published. Published dependencies are created when two solution components are related to each other and then published. To remove this type of dependency, the association must be removed and the tables published again.
- Unpublished. Unpublished dependencies apply to the unpublished version of a publishable solution component that is being updated. After the solution component is published, it becomes a published dependency.

Solution internal dependencies are dependencies where actions with a solution component require an action for another solution component. For example, if you delete a table, you should expect that all the table attributes are deleted with it. Any table relationships with other tables are also deleted.

However, an internal dependency might lead to a published dependency and still require manual intervention. For example, if you include a lookup field on a table form

and then delete the primary table in the relationship, you can't complete that deletion until you remove the lookup column from the related table form and then publish the form.

When you perform actions programmatically with solutions, you can use messages related to the `Dependency` table. Go to [Dependency Entity](#) for messages you can use to identify dependencies that might exist before you delete a component or uninstall a solution.

## Common solution components

These are the solution components displayed in the application and the components that you work with directly when adding or removing solution components using the solution page. Each of the other types of solution components depend on one or more of these solution components to exist.

- [Application ribbons](#)
- [Entity](#)
- [Report](#)
- [Article template](#)
- [Column security profile](#)
- [SDK message processing step](#)
- [Connection role](#)
- [Mail merge template](#)
- [Security role](#)
- [Contract template](#)
- [Option set](#)
- [Service endpoint](#)
- [Dashboard or entity form](#)
- [Plug-in assembly](#)
- [Site map](#)
- [Email template](#)
- [Process](#)
- [Web resource](#)

## Application ribbons (RibbonCustomization)

Ribbon customizations for the application ribbon and table ribbon templates. Application ribbons don't include definitions of ribbons at the table or form level.

Custom application ribbons frequently have published dependencies on web resources. Web resources are used to define ribbon button icons and JavaScript functions to control when ribbon elements are displayed or what actions are performed when a particular ribbon control is used. Dependencies are only created when the ribbon definitions use the `$webresource:` directive to associate the web resource to the ribbon. More information: [\\$webresource directive](#)

## Article template (KBArticleTemplate)

Template that contains the standard attributes of an article. There's always an internal dependency between the article template and the KbArticle table.

## Connection role (ConnectionRole)

Role describing a relationship between a two records. Each connection role defines what types of entity records can be linked using the connection role. This creates a published dependency between the connection role and the table.

## Contract template (ContractTemplate)

Template that contains the standard attributes of a contract. There's always an internal dependency between the contract template and the contract table.

## Dashboard or entity form (SystemForm)

System form table records are used to define dashboards and table forms. When a `SystemForm` is used as a table form there's an internal dependency on the table. When a `SystemForm` is used as a dashboard there are no internal dependencies. Both table forms and dashboards commonly have published dependencies related to their content. A table form might have lookup columns that depend on a table relationship. Both dashboards and table forms can contain charts or subgrids that create a published dependency on a view, which then has an internal dependency on a table. A published dependency on web resources can be created because of content displayed within the dashboard or form or when a form contains JavaScript libraries. Table forms have published dependencies on any attributes that are displayed as columns in the form.

## Email template (EmailTemplate)

Template that contains the standard attributes of an email message. An email template typically includes columns that insert data from specified table attributes. An email

template can be linked to a specific table when it's created so there can be an internal dependency on the table. A global email template isn't associated with a specific table, but it might have published dependencies on table attributes used to provide data. A process (workflow) frequently is configured to send an email using an email template creating a published dependency with the workflow.

## **Entity (Entity)**

Tables, formerly known as entities, are the primary structure used to model and manage data in Dataverse. Charts, forms, table relationships, views, and attributes associated with a table are deleted automatically when the table is deleted because of the internal dependencies between them. Tables frequently have published dependencies with processes, dashboards, and email templates.

## **Column security profile (FieldSecurityProfile)**

Profile that defines the access level for secured attributes.

## **Mail merge template (MailMergeTemplate)**

Template that contains the standard attributes of a mail merge document. A mail merge template has a published dependency on the table it's associated with.

## **Option set (OptionSet)**

A choice column, formerly known as an option set, defines a set of options. A picklist attribute uses a choice to define the options provided. Several picklist attributes might use a global choice so that the options they provide are always the same and can be maintained in one place. A published dependency occurs when a picklist attribute references a global choice. You can't delete a global choice that is being used by a picklist attribute.

## **Plug-in assembly (PluginAssembly)**

Assembly that contains one or more plug-in types. Plug-ins are registered to events that are typically associated with a table. This creates a published dependency.

## **Process (Workflow)**

Set of logical rules that define the steps necessary to automate a specific business process, task, or set of actions to be performed. Processes provide a wide range of actions that create published dependencies on any other solution component referenced by the process. Each process also has a published dependency on the table it's associated with.

## Report (Report)

Data summary in an easy-to-read layout. A report has published dependencies on any table or attribute data included in the report. Each report must also be associated with a Report category creating an internal dependency on a solution component called Report Related Category (ReportCategory). Reports can be configured to be subreports creating a published dependency with the parent report.

## SDK message processing step (SDKMessageProcessingStep)

Stage in the execution pipeline that a plug-in is to execute.

## Security role (Role)

Grouping of security privileges. Users are assigned security roles that authorize their access to the Dataverse system. Table forms can be associated with specific security roles to control who can view the form. This creates a published dependency between the security role and the form.

### Note

Only security roles from the environment business unit can be added to a solution.  
Only a user with read access to those security roles can add them to a solution.

## Service endpoint (ServiceEndpoint)

Service endpoint that can be contacted.

## Site map (SiteMap)

XML data used to control the application navigation pane. The site map might be linked to display an HTML web resource or an icon in the site map might use an image web

resource. When the `$webresource:` directive is used to establish these associations, a published dependency is created. More information: [\\$webresource directive](#)

## Web resource (WebResource)

Data equivalent to files used to extend a model-driven app or in web development. Web resources provide client-side components that are used to provide custom user interface elements. Web resources might have published dependencies with table forms, ribbons, and the site map. When the `$webresource:` directive is used to establish associations in a ribbon or the site map, a published dependency is created. For more information, go to [\\$webresource directive](#).

### Note

Web resources might depend on other web resources based on relative links. For example, an HTML web resource might use a CSS or script web resource. A Silverlight web resource displayed outside of a table form or chart must have an HTML web resource to host it. These dependencies aren't tracked as solution dependencies.

## See also

- [Solution concepts](#)
- [Removing dependencies](#)
- [Environment strategy](#)
- [Work with solutions using the SDK APIs](#)
- [Solution Entity Reference](#)
- [SolutionComponent Entity Reference](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 



# Check for solution component dependencies

Article • 02/16/2023

When you edit solutions you may find that you can't delete a solution component because it has a published dependency with another solution component. Or, you may not be able to uninstall a managed solution because one of the components in the managed solution has been used in a customization in another unmanaged solution.

The following table lists the messages that you can use to retrieve data about solution component dependencies.

Message	Description
<a href="#">RetrieveDependentComponentsRequest</a> <a href="#">RetrieveDependentComponents function</a>	Returns a list of dependencies for solution components that directly depend on a solution component.   For example, when you use this message for a global option set solution component, dependency records for solution components representing any option set attributes that reference the global option set solution component are returned.   When you use this message for the solution component record for the account entity, dependency records for all of the solution components representing attributes, views, and forms used for that entity are returned.
<a href="#">RetrieveRequiredComponentsRequest</a> <a href="#">RetrieveRequiredComponents function</a>	Returns a list of the dependencies for solution components that another solution component directly depends on. This message provides the reverse of the <code>RetrieveDependentComponentsRequest</code> message.
<a href="#">RetrieveDependenciesForDeleteRequest</a> <a href="#">RetrieveDependenciesForDelete function</a>	Returns a list of all the dependencies for solution components that could prevent deleting a solution component.
<a href="#">RetrieveDependenciesForUninstallRequest</a> <a href="#">RetrieveDependenciesForUninstall function</a>	Returns a list of all the dependencies for solution components that could prevent uninstalling a managed solution.

# Create solutions that support multiple languages

Article • 02/15/2022

Microsoft Dataverse supports multiple languages. If you want your solution to be installed for organizations that include different base languages or that have multiple languages provisioned, take this into account when planning your solution. The following table lists tactics to use along with solution components to include in a solution that supports multiple languages.

Tactic	Solution component type
<a href="#">String (RESX) web resources</a>	Web Resources
<a href="#">Embedded labels</a>	Application Navigation (SiteMap) Ribbons
<a href="#">Export and import translations</a>	Attributes Charts Dashboards Entity Entity Relationships Forms Messages Option Sets Views
<a href="#">Localization in base language strings</a>	Contract Templates Connection Roles Processes (Workflow) Security Roles Field Security Profiles
<a href="#">Localization not required</a>	SDK Message Processing Steps Service Endpoints
<a href="#">Separate component for each language</a>	Article Templates Email Templates Mail Merge Templates Reports Dialogs
<a href="#">Use XML web resources as language resources</a>	Plug-in Assemblies

The following sections provide additional details for each tactic.

# String (RESX) web resources

With string (RESX) web resources added with Dataverse developers have a more robust option to create web resources that support multiple languages. More information [String \(RESX\) web resources](#).

## Embedded labels

Each of the solution components that use this tactic requires that any localized text be included in the solution component.

## Ribbons

When a language pack is installed, the application ribbon automatically displays localized text for all of the default text in the ribbon. System labels are defined in a `ResourceId` attribute value that is for internal use only. When you add your own text you should use the `<LocLabels>` element to provide localized text for the languages you support. More information: [Use Localized Labels with Ribbons](#)

## SiteMap

When a language pack is installed, the default text in the application navigation bar automatically displays localized text. To override the default text or to provide your own text, use the `<Titles>` element. The `Titles` element should contain a `<Title>` element that contains localized text for any languages your solution supports. If a `Title` element isn't available for the user's preferred language, the title that corresponds to the organization base language is displayed.

The `<SubArea>` element allows for passing the user's language preference by using the `userId` parameter so that content that is the target of the `SubArea.Url` attribute can be aware of the user's language preference and adjust accordingly. More information: [Passing Parameters to a URL Using SiteMap](#)

## Export and import translations

Localizable labels for the solution components in the following table can be exported for localization.

Entities	Attributes	Relationships
----------	------------	---------------

Global Option Sets	Entity Messages	Entity Forms
Entity Views (SavedQuery)	Charts	Dashboards

## Translating labels and display strings

You can only perform customizations in the application by using the base language. Therefore, when you want to provide localized labels and display strings for these customizations, you must export the text of the labels so that they can be localized for any other languages enabled for the organization. Use the following steps:

1. Ensure that the organization that you're working on has all the MUI packs installed and languages provisioned for languages you want to provide translations for.
2. Create your solution and modify the components.
3. After you have finished developing your solution use the "Export Translations" functionality. This generates a Office Excel spreadsheet (CrmTranslations.xml) that contains all the labels that need translation.
4. In the spreadsheet, provide the corresponding translations.
5. Import translations back into the same Dataverse organization using the "Import Translations" functionality and publish your changes.
6. The next time the solution is exported it carries all the translations that you provided.

When a solution is imported, labels for languages that aren't available in the target system are discarded and a warning is logged.

If labels for the base language of the target system are not provided in the solution package, the labels of the base language of the source are used instead. For example, if you import a solution that contains labels for English and French with English as the base language, but the target system has Japanese and French with Japanese as the base language, English labels are used instead of Japanese labels. The base languages labels cannot be **null** or empty.

## Exporting translations

Before you export translations you must first install the language packs and provision all the languages you want to have localized. You can export the translations in the web

application or by using the [ExportTranslationRequest](#) message. For more information, see [Export Customized Entity and Field Text for Translation](#).

## Translating text

When you open the CrmTranslations.xml file in Office Excel you will see the three worksheets listed in the following table.

Worksheet	Description
Information	Displays information about the organization and solution that the labels and strings were exported from.
Display Strings	Display strings that represent the text of any messages associated with a metadata component. This table includes error messages and strings used for system ribbon elements.
Localized Labels	Displays all of the text for any metadata component labels.

You can send this file to a linguistic expert, translation agency, or localization firm. They will need to provide localized strings for any of the empty cells.

### Note

For custom entities there are some common labels that are shared with system entities, such as **Created On** or **Created By**. Because you have already installed and provisioned the languages, if you export languages for the default solution you may be able to match some labels in your custom entities with localized text for identical labels used by other entities. This can reduce the localization costs and improve consistency.

After the text in the worksheets has been localized, add both the CrmTranslations.xml and [Content\_Types].xml files to a single compressed .zip file. You can now import this file.

If you prefer to work with the exported files programmatically as an XML document, see [Word, Excel, and PowerPoint Standards Support](#) for information about the schemas these files use.

## Importing translated text

### Important

You can only import translated text back into the same organization it was exported from.

After you have exported the customized entity or attribute text and had it translated, you can import the translated text strings in the web application by using the [ImportTranslationRequest](#) message. The file that you import must be a compressed file that contains the `CrmTranslations.xml` and the `[Content_Types].xml` file at the root. For more information, see [Import Translated Entity and Field Text](#).

After you import the completed translations, customized text appears for users who work in the languages that you had the text translated into.

### Note

Dataverse cannot import translated text that is over 500 characters long. If any of the items in your translation file are longer than 500 characters, the import process fails. If the import process fails, review the line in the file that caused the failure, reduce the number of characters, and try to import again.

Because customization is supported only in the base language, you may be working in Dataverse with the base language set as your language preference. To verify that the translated text appears, you must change your language preference for the Dataverse user interface. To perform additional customization work, you must change back to the base language.

## Localization in base language strings

Some solution components do not support multiple languages. These components include names or text that can only be meaningful in a specific language. If you create a solution for a specific language, define these solution components for the intended organization base language.

If you need to support multiple languages, one tactic is to include localization within the base language strings. For example, if you have a Connection Role named "Friend" and you need to support English, Spanish, and German, you might use the text "Friend (Amigo / Freund)" as the name of the connection role. Due to issues of the length of the text there are limitations on how many languages can be supported using this tactic.

Some solution components in this group are only visible to administrators. Because customization of the system can only be done in the organization base language it is not necessary to provide multiple language versions. **Security Roles** and **Field Security Profile** components belong to this group.

**Contract Templates** provide a description of a type of service contract. These require text for the **Name** and **Abbreviation** fields. You should consider using names and abbreviations that are unique and appropriate for all users of the organization.

**Connection Roles** rely on a person selecting descriptive Connection Role categories and names. Because these may be relatively short, it is recommended that you include localization in base language strings.

**Processes (Workflows)** that are initiated for events can work well as long as they do not need to update records with text that is to be localized. It is possible to use a workflow assembly so that logic that might apply to localized text could use the same strategy as plug-in assemblies ([Use XML Web Resources as Language Resources](#)).

**On-Demand workflows** require a name so that people can select them. In addition to including localization within the name of the on-demand workflow, another tactic is to create multiple workflows with localized names that each call the same child process. However, all users will see the complete list of on-demand workflows, not just the ones in their preferred user interface language.

## Localization not required

**SDK Message Processing Step** and **Service Endpoint** solution components do not expose localizable text to users. If it is important that these components have names and descriptions that correspond to the organization's base language, you can create and export a managed solution with names and descriptions in that language.

## Separate component for each language

The following solution components each may contain a considerable amount of text to be localized:

- Article Templates
- Email Templates
- Mail Merge Templates
- Reports

- Dialogs

For these types of solution components, the recommended tactic is to create separate components for each language. This means that you typically create a base managed solution that contains your core solution components and then a separate managed solution that contains these solution components for each language. After customers install the base solution, they can install the managed solutions for the languages they have provisioned for the organization.

Unlike **Processes (Workflows)**, you can create **Dialogs** that will reflect the user's current language preference settings and display the dialogs only to users of that language.

## Create a localized dialog box

1. Install the appropriate language pack and provision the language.

For more information, see [Language Pack Installation Instructions](#).

2. Change your personal options to specify the **User Interface Language** for the language you want for the dialog.
3. Navigate to **Settings** and, in the **Process Center** group, select **Processes**.
4. Click **New** and create the dialog in the language that you specified.
5. After you have created the dialog, change your personal options to specify the organization base language.
6. While using the organization base language you can navigate to the **Solutions** area in **Settings** and add the localized dialog as part of a solution.

The dialog created in the other language will only be displayed to users who view Dataverse using that language.

## Use XML web resources as language resources

Plug-in assembly solution components can send messages to an end user by throwing an [InvalidPluginExecutionException](#) as well as creating and updating records. Unlike Silverlight web resources, plug-ins cannot use resource files.

When a plug-in requires localized text, you can use an XML web resource to store the localized strings so the plug-in can access them when needed. The structure of the XML is your option, but you may want to follow the structure used by ASP.NET Resource



(.resx) files to create separate XML web resources for each language. For example, the following is an XML web resource named **localizedString.en\_US** that follows the pattern used by .resx files.

XML

```
<root>
 <data name="ErrorMessage">
 <value>There was an error completing this action. Please try again.
 </value>
</data>
 <data name="Welcome">
 <value>Welcome</value>
 </data>
</root>
```

The following code shows how a localized message can be passed back in a plug-in to display a message to a user. It is for the pre-validation stage of a **Delete** event for the **Account** entity:

C#

```
protected void ExecutePreValidateAccountDelete(LocalPluginContext
localContext)
{
 if (localContext == null)
 {
 throw new ArgumentNullException("localContext");
 }
 int OrgLanguage =
RetrieveOrganizationBaseLanguageCode(localContext.OrganizationService);
 int UserLanguage =
RetrieveUserUILanguageCode(localContext.OrganizationService,
localContext.PluginExecutionContext.InitiatingUserId);
 String fallBackResourceFile = "";
 switch (OrgLanguage)
 {
 case 1033:
 fallBackResourceFile = "new_localizedStrings.en_US";
 break;
 case 1041:
 fallBackResourceFile = "new_localizedStrings.ja_JP";
 break;
 case 1031:
 fallBackResourceFile = "new_localizedStrings.de_DE";
 break;
 case 1036:
 fallBackResourceFile = "new_localizedStrings.fr_FR";
 break;
 case 1034:
 fallBackResourceFile = "new_localizedStrings.es_ES";
```

```

 break;
 case 1049:
 fallbackResourceFile = "new_localizedStrings.ru_RU";
 break;
 default:
 fallbackResourceFile = "new_localizedStrings.en_US";
 break;
}
String ResourceFile = "";
switch (UserLanguage)
{
 case 1033:
 ResourceFile = "new_localizedStrings.en_US";
 break;
 case 1041:
 ResourceFile = "new_localizedStrings.ja_JP";
 break;
 case 1031:
 ResourceFile = "new_localizedStrings.de_DE";
 break;
 case 1036:
 ResourceFile = "new_localizedStrings.fr_FR";
 break;
 case 1034:
 ResourceFile = "new_localizedStrings.es_ES";
 break;
 case 1049:
 ResourceFile = "new_localizedStrings.ru_RU";
 break;
 default:
 ResourceFile = fallbackResourceFile;
 break;
}
XmlDocument messages = RetrieveXmlWebResourceByName(localContext,
ResourceFile);
String message = RetrieveLocalizedStringFromWebResource(localContext,
messages, "ErrorMessage");
throw new InvalidPluginExecutionException(message);
}
protected static int
RetrieveOrganizationBaseLanguageCode(IOrganizationService service)
{
 QueryExpression organizationEntityQuery = new
QueryExpression("organization");
 organizationEntityQuery.ColumnSet.AddColumn("languagecode");
 EntityCollection organizationEntities =
service.RetrieveMultiple(organizationEntityQuery);
 return (int)organizationEntities[0].Attributes["languagecode"];
}
protected static int RetrieveUserUILanguageCode(IOrganizationService
service, Guid userId)
{
 QueryExpression userSettingsQuery = new QueryExpression("usersettings");
 userSettingsQuery.ColumnSet.AddColumns("uilanguageid", "systemuserid");
 userSettingsQuery.Criteria.AddCondition("systemuserid",

```

```

ConditionOperator.Equal, userId);
 EntityCollection userSettings =
service.RetrieveMultiple(userSettingsQuery);
 if (userSettings.Entities.Count > 0)
 {
 return (int)userSettings.Entities[0]["uilanguageid"];
 }
 return 0;
}
protected static XmlDocument
RetrieveXmlWebResourceByName(LocalPluginContext context, string
webresourceSchemaName)
{
 context.TracingService.Trace("Begin:RetrieveXmlWebResourceByName,
webresourceSchemaName={0}", webresourceSchemaName);
 QueryExpression webresourceQuery = new QueryExpression("webresource");
 webresourceQuery.ColumnSet.AddColumn("content");
 webresourceQuery.Criteria.AddCondition("name", ConditionOperator.Equal,
webresourceSchemaName);
 EntityCollection webresources =
context.OrganizationService.RetrieveMultiple(webresourceQuery);
 context.TracingService.Trace("Webresources Returned from server. Count=
{0}", webresources.Entities.Count);
 if (webresources.Entities.Count > 0)
 {
 byte[] bytes = Convert.FromBase64String((string)webresources.Entities[0]
["content"]);
 // The bytes would contain the ByteOrderMask. Encoding.UTF8.GetString()
does not remove the BOM.
 // Stream Reader auto detects the BOM and removes it on the text
 XmlDocument document = new XmlDocument();
 document.XmlResolver = null;
 using (MemoryStream ms = new MemoryStream(bytes))
 {
 using (StreamReader sr = new StreamReader(ms))
 {
 document.Load(sr);
 }
 }
 context.TracingService.Trace("End:RetrieveXmlWebResourceByName ,
webresourceSchemaName={0}", webresourceSchemaName);
 return document;
 }
 else
 {
 context.TracingService.Trace("{0} Webresource missing. Reinstall the
solution", webresourceSchemaName);
 throw new InvalidPluginExecutionException(String.Format("Unable to
locate the web resource {0}.", webresourceSchemaName));
 return null;
 // This line never reached
 }
}
protected static string
RetrieveLocalizedStringFromWebResource(LocalPluginContext context,

```

```
XmlDocument resource, string resourceId)
{
 XmlNode valueNode =
resource.SelectSingleNode(string.Format(CultureInfo.InvariantCulture,
"./root/data[@name='{0}']/value", resourceId));
 if (valueNode != null)
 {
 return valueNode.InnerText;
 }
 else
 {
 context.TracingService.Trace("No Node Found for {0} ", resourceId);
 throw new InvalidPluginExecutionException(String.Format("ResourceID {0}
was not found.", resourceId));
 }
}
```

## See also

[Localize product property values](#)

# Configuration Migration tool

Article • 01/30/2025

The Configuration Migration tool is used to transport configuration and test data from one environment to another. It provides the means to capture such data, use that data in source control, and use that data to automate testing. Don't just rely on an environment for isolation: use source control instead.

Test data is data that's necessary to run your tests (that is, sample data). Configuration capture can't be automated.

## Tip

You can also automate running the Configuration Migration tool with PowerShell by using the [Microsoft.Xrm.Tooling.ConfigurationMigration](#)  module.

For more information about using the tool, including how to download it, go to [Move configuration data across environments and organizations with the Configuration Migration tool](#).

## Related articles

[Package Deployer tool](#)

[Solution Packager tool](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# Create packages for the Package Deployer tool

Article • 06/10/2024

Package Deployer lets administrators deploy packages on Microsoft Dataverse instances. A Package Deployer *package* can consist of any or all of the following:

- One or more Dataverse solution files.
- Flat files or exported configuration data file from the Configuration Migration tool. For more information about the tool, see [Move configuration data across instances and organizations with the Configuration Migration tool](#).
- Custom code that can run before, while, or after the package is deployed to the Dataverse instance.
- HTML content specific to the package that can display at the beginning and end of the deployment process. This content can be useful to provide a description of the solutions and files that are deployed in the package.

## ⓘ Note

There is another package type called a *plug-in package*. That kind of package is for plug-in dependent assemblies and has no relationship with Package Deployer packages.

## Prerequisites

- Ensure that you have all the solution and other files ready that you want to include in the package.
- Visual Studio 2019 or later, or Visual Studio Code.

## Process overview

To create a Package Deployer package, perform the following steps.

- ✓ Create a Visual Studio or MSBuild project
- ✓ Add solutions and other files to the project
- ✓ Update provided HTML files (optional)
- ✓ Specify configuration values for the package
- ✓ Define custom code for the package

- ✓ Build and deploy the package

These steps are described in detail in this article.

## Create a package project

The first step is to create a Visual Studio or MSBuild project for the package. To do that, you must have one of two available tool extensions installed on your development computer. If using Visual Studio Code, install [Microsoft Power Platform CLI](#). Otherwise, if using Visual Studio 2019 or later, install [Power Platform tools for Visual Studio](#).

Select the appropriate tab below to find out how to create a project using the desired tool extension. Both tools output the project in a similar format.

### Power Platform CLI

Run the [pac package init](#) command to create the initial package. More information: [pac package](#)

Bash

```
pac package init help
pac package init --outputDirectory DeploymentPackage
```

The resulting CLI output contains the folders and files shown below. The "DeploymentPackage" folder name was used here as an example.

Bash

```
C:\.
├── DeploymentPackage
│ ├── DeploymentPackage.csproj
│ └── PackageImportExtension.cs
└── PkgAssets
 ├── ImportConfig.xml
 └── manifest.ppkg.json
```

In the created project, find the ImportConfig.xml configuration file in the PkgAssets folder and the PackageImportExtension.cs file. You'll modify these files as described later in this article.

## Add package files

After you have created a package project, you can begin adding solutions and other files to that project.

#### Power Platform CLI

When using the CLI, you can add external packages, solutions, and references to your package project using one of the **add** subcommands. Enter `pac package help` to see the list of subcommands. Let's add a solution to our package.

Bash

```
> pac package add-solution help

Commands:
Usage: pac package add-solution --path [--import-order] [--skip-validation]
[--publish-workflows-activate-plugins] [--overwrite-unmanaged-customizations]
[--import-mode] [--missing-dependency-behavior] [--dependency-overrides]

> cd .\DeploymentPackage\
> pac package add-solution --path ..\TestSolution_1_0_0_1_managed.zip

The item was added successfully.
```

## Configure the package

1. Define the package configuration by adding information about your package in the **ImportConfig.xml** file in the project. Open the file for editing. The following list provides information about each parameter and node in the config file.

`installsampleddata`

**True** or **false**. If **true**, installs sample data to Dataverse instance. This data is the same sample data that you can install from **Settings > Data Management** area in Dataverse.

`waitforsampledatatoinstall`

**True** or **false**. If **true**, and if `installsampleddata` is also set to **true**, waits for sample data to install before deploying the package.

ⓘ **Note**



Ensure that you set `installsampledata` to `true` if you are setting `waitforsampledatatoinstall` to `true`.

#### `agentdesktopzipfile`

File name of the zip file to unpack. If you specify a .zip file name here, it adds a screen during the package deployment process that prompts you to select a location where you want to unpack the contents of the file.

This attribute is commonly used for creating packages for Unified Service Desk for Dynamics 365. For information about Unified Service Desk, see [Administration Guide for Unified Service Desk 3.0](#).

#### `agentdesktopexename`

Name of the .exe or .msi file in the zip file or a URL to be invoked at the end of the deployment process.

This attribute is commonly used for creating packages for Unified Service Desk.

#### `crmmigdataimportfile`

File name of the default configuration data file (.zip) exported using the Configuration Migration tool.

- You can also import a localized version of the configuration data file based on the locale ID (LCID) specified using new runtime settings while running the package deployer. Use the `<cmtdatafile>` node (explained later) to specify the localized versions of the configuration data file in a package and then use the `OverrideConfigurationDataFileLanguage` method (explained later) to specify the logic for importing the configuration data file based on the locale ID specified using the runtime settings. You can't import more than one configuration data file using a package at a time.
- For Dataverse (on-premises), if your configuration data file contains user information, and both the source and target Dataverse instances are on the same Active Directory Domain, user information is imported to the target Dataverse instance. To import user information to a Dataverse (on-premises) instance on a different domain, you must include the user map file (.xml) generated using the Configuration Migration tool in your project, and specify it along with the configuration data file using the `usermapfilename` attribute in the `<cmtdatafile>` node explained later. User information can't be imported to Dataverse instances.

#### `<solutions>` node

Contains an array of `<configsolutionfile>` nodes that describe the solutions

to import. The order of the solutions under this node indicates the order in which the solutions will be imported on the target Dataverse instance.

`<configsolutionfile>` node

Use this node under the `<solutions>` node to specify the individual solutions and the following information for each solution to be imported:

- `solutionpackagefilename`: Specify the .zip file name of your solution. Required.
- `overwriteunmanagedcustomizations`: Specify whether to overwrite any unmanaged customizations when importing a solution that already exists in the target Dynamics 365 instance. This attribute is optional, and if you don't specify this attribute, by default the unmanaged customizations in the existing solution are maintained on the target Dynamics 365 instance.
- `publishworkflowsandactivateplugins`: Specify whether to publish workflows and activate plug-ins in the target Dynamics 365 instance after the solution is imported. This attribute is optional, and if you don't specify not specify this attribute, by default the workflows are published and plug-ins are activated after the solution is imported on the target Dynamics 365 instance.

You can add multiple solution file names in a package by adding as many `<configsolutionfile>` nodes. For example, if you want three solution files to be imported, add them as shown below:

XML

```
<solutions>
<configsolutionfile
solutionpackagefilename="SampleSolutionOne_1_0_managed.zip"
overwriteunmanagedcustomizations="false"
publishworkflowsandactivateplugins="true"/>
<configsolutionfile
solutionpackagefilename="SampleSolutionTwo_1_0_managed.zip"
overwriteunmanagedcustomizations="false"
publishworkflowsandactivateplugins="true"/>
<configsolutionfile
solutionpackagefilename="SampleSolutionThree_1_0_managed.zip" />
</solutions>
```

`<filestoimport>` node

Contains an array of `<configimportfile>` and `<zipimportdetails>` nodes that are used to describe individual files and zip files respectively to be imported.

`<configimportfile>` node

Use this node under the `<configimportfile>` node to describe a file to be imported to Dataverse. You can add multiple files in a package by adding as many `<configimportfile>` nodes.

#### XML

```
<filestoimport>
<configimportfile filename="File.csv"
filetype="CSV"
associatedmap="FileMap"
importtoentity="FileEntity"
datadelimiter=""
fielddelimiter="comma"
enableduplicatedetection="true"
isfirstrowheader="true"
isrecordownerateam="false"
owneruser=""
waitforimporttocomplete="true" />
<configimportfile filename="File.zip"
filetype="ZIP"
associatedmap="FileMapName"
importtoentity="FileEntity"
datadelimiter=""
fielddelimiter="comma"
enableduplicatedetection="true"
isfirstrowheader="true"
isrecordownerateam="false"
owneruser=""
waitforimporttocomplete="true"/>

</filestoimport>
```

Below is a list of supported attributes:

[Expand table](#)

Attribute	Description
<code>filename</code>	Name of the file that contains the import data. If the file is a .zip file, a <code>&lt;zipimportdetails&gt;</code> node must be present with a <code>&lt;zipimportdetail&gt;</code> node for each file in the .zip file.
<code>filetype</code>	This value can be csv, xml, or zip.
<code>associatedmap</code>	Name of the Dataverse import data map to use with this file. If blank, attempts to use the system determined import data map name for this file.

Attribute	Description
<code>importttoentity</code>	Can be the name of the exe in the zip file, a URL, or an .msi file to provide a link to invoke at the end of the process.
<code>datadelimiter</code>	Name of the data delimiter used in the import file. Valid values are single quote or double quotes.
<code>fielddelimiter</code>	Name of the field delimiter used in the import file. Valid values are comma or colon, or single quote.
<code>enableduplicatedetection</code>	Indicates whether to enable duplicate detections rules on data import. Valid values are <b>true</b> or <b>false</b> .
<code>isfirstrowheader</code>	Used to denote that the first row of the import file contains the field names. Valid values are <code>true</code> or <code>false</code> .
<code>isrecordownerateam</code>	Indicates whether the owner of the record on import should be a team. Valid values are <code>true</code> or <code>false</code> .
<code>owneruser</code>	Indicates the user ID that should own the records. The default value is the currently logged in user.
<code>waitforimporttocomplete</code>	If <code>true</code> , the system waits for the import to complete before proceeding. If <code>false</code> , it queues the jobs and moves on.

`<zipimportdetails>` node

This node contains an array of `<zipimportdetail>` nodes that describe the files included in a zip file that is used to import to Dynamics 365.

`<zipimportdetail>` node

Use this node under the `<zipimportdetails>` node to provide information about an individual file in a .zip file that is specified in the `<configimportfile>` node.

XML

```

<filestoimport>
...
...
<zipimportdetails>
<zipimportdetail filename="subfile1.csv" filetype="csv"
importttoentity="account" />
<zipimportdetail filename="subfile2.csv" filetype="csv"
importttoentity="contact" />
</zipimportdetails>
</filestoimport>

```

Supported attributes are listed below:

Attribute	Description
filename	Name of the file that contains the import data.
filetype	This value can be csv or xml.
importtoentity	Can be the name of the exe in the zip file, a url, or an .msi file to provide a link to invoke at the end of the process.

#### `<filesmapstoimport>` node

This node contains an array of `<configimportmapfile>` nodes to import. The order of the map files in this node indicates the order in which they're imported. For information about data maps, see [Create data maps for import](#).

#### `<configimportmapfile>` node

Use this node under the `<filesmapstoimport>` node to provide information about an individual map file to import in Dataverse.

XML

```
<filesmapstoimport>
 <configimportmapfile filename="FileMap.xml" />
</filesmapstoimport>
```

#### `<cmtdatafiles>` node

This node contains an array of `<cmtdatafile>` nodes that contains localized version of the configuration data file to be imported.

#### `<cmtdatafile>` node

Use this node under the `<cmtdatafiles>` node to specify the localized configuration data files along with locale ID (required) and user information map file (optional). For example:

XML

```
<cmtdatafiles>
 <cmtdatafile filename="data_1033.zip" lcid="1033"
 usermapfilename="UserMap.xml" />
 <cmtdatafile filename="data_1041.zip" lcid="1041" usermapfilename="" />
</cmtdatafiles>
```

You can define your custom logic in the `OverrideConfigurationDataFileLanguage` method (explained later) to import a localized configuration data file instead of the

default one (specified in crmmigdataimportfile) based on the locale ID (LCID) value specified using the runtime settings (explained later).

## 2. Select **Save All**.

The following xml represents the contents of a sample `ImportConfig.xml` file.

XML

```
<?xml version="1.0" encoding="utf-16"?>
<configdatastorage xmlns:xsi="https://www.w3.org/2001/XMLSchema-
instance"
xmlns:xsd="https://www.w3.org/2001/XMLSchema"
installsampleddata="true"
waitforsampledatatoinstall="true"
agentdesktopzipfile=""
agentdesktopexename=""
crmmigdataimportfile="data_1033.zip">
<solutions>
<configsolutionfile
solutionpackagefilename="SampleSolutionOne_1_0_managed.zip"
overwriteunmanagedcustomizations="false"
publishworkflowsandactivateplugins="true"/>
<configsolutionfile
solutionpackagefilename="SampleSolutionTwo_1_0_managed.zip"
overwriteunmanagedcustomizations="false"
publishworkflowsandactivateplugins="true"/>
<configsolutionfile
solutionpackagefilename="SampleSolutionThree_1_0_managed.zip" />
</solutions>
<filestoimport>
<configimportfile filename="SampleOption.csv"
filetype="CSV"
associatedmap="SampleOption"
importtoentity="sample_option"
datadelimiter=""
fielddelimiter="comma"
enableduplicatedetection="true"
isfirstrowheader="true"
isrecordownerateam="false"
owneruser=""
waitforimporttocomplete="false"/>
<configimportfile filename="File.zip"
filetype="ZIP"
associatedmap="FileMapName"
importtoentity="FileEntity"
datadelimiter=""
fielddelimiter="comma"
enableduplicatedetection="true"
isfirstrowheader="true"
isrecordownerateam="false"
owneruser=""
waitforimporttocomplete="true"/>
```

```

<zipimportdetails>
<zipimportdetail filename="subfile1.csv"
filetype="csv"
importtoentity="account" />
<zipimportdetail filename="subfile2.csv"
filetype="csv"
importtoentity="contact" />
</zipimportdetails>
</filestoimport>
<filesmapstoimport>
<configimportmapfile filename="SampleOption.xml" />
</filesmapstoimport>
<cmtdatafiles>
<cmtdatafile filename="data_1033.zip"
lcid="1033"
usermapfilename="UserMap.xml" />
<cmtdatafile filename="data_1041.zip"
lcid="1041"
usermapfilename="" />
</cmtdatafiles>
</configdatastorage>

```

## Add custom code

You can add custom code that executes before, during, and after the package is imported into an environment. To do so, follow these instructions.

1. Edit the **PackageTemplate.cs** (or **PackageImportExtension.cs**) file in the project's root folder.
2. In the C# file, you can:
  - a. Enter custom code to execute when the package is initialized in the override method definition of `InitializeCustomExtension`.

This method can be used to let users use the runtime parameters while running a package. As a developer, you can add support for any runtime parameter to your package by using the [RuntimeSettings](#) property as long as you have code to process it based on the user input.

For example, the following sample code enables a runtime parameter called `SkipChecks` for the package that has two possible values: true or false. The sample code checks if the user has specified any runtime parameters while running Package Deployer (either by using the command line or PowerShell), and then accordingly processes the information. If no runtime parameter is

specified by the user while running the package, the value of the `RuntimeSettings` property will be null.

C#

```
public override void InitializeCustomExtension()
{
 // Do nothing.

 // Validate the state of the runtime settings object.
 if (RuntimeSettings != null)
 {
 PackageLog.Log(string.Format("Runtime Settings populated. Count = {0}", RuntimeSettings.Count));
 foreach (var setting in RuntimeSettings)
 {
 PackageLog.Log(string.Format("Key={0} | Value={1}", setting.Key, setting.Value.ToString()));
 }

 // Check to see if skip checks is present.
 if (RuntimeSettings.ContainsKey("SkipChecks"))
 {
 bool bSkipChecks = false;
 if (bool.TryParse((string)RuntimeSettings["SkipChecks"], out bSkipChecks))
 {
 OverrideDataImportSafetyChecks = bSkipChecks;
 }
 }
 else
 {
 PackageLog.Log("Runtime Settings not populated");
 }
 }
}
```

This code enables the administrator use the command line or the [Import-CrmPackage](#) cmdlet to specify whether to skip the safety checks while running the Package Deployer tool to import the package. More information: [Deploy packages using Package Deployer and Windows PowerShell](#)

- b. Enter custom code to execute before the solutions are imported in the override method definition of `PreSolutionImport` to specify whether to maintain or overwrite customizations while updating the specified solution in a target Dataverse instance, and whether to automatically activate plug-ins and workflows.
- c. Use the override method definition of `RunSolutionUpgradeMigrationStep` to perform data transformation or upgrade between two versions of a solution. This method is called only if the solution you're importing is already present in the target Dataverse instance.



This function expects the following parameters:

 Expand table

Parameter	Description
<code>solutionName</code>	Name of the solution
<code>oldVersion</code>	Version number of the old solution
<code>newVersion</code>	Version number of the new solution
<code>oldSolutionId</code>	GUID of the old solution.
<code>newSolutionId</code>	GUID of the new solution.

d. Enter custom code to execute before the solution import completes in the override definition of the `BeforeImportStage` method. The sample data and some flat files for solutions specified in the `ImportConfig.xml` file are imported before the solution import completes.

e. Override the currently selected language for configuration data import using the override method definition of `OverrideConfigurationDataFileLanguage`. If the specified locale ID (LCID) of the specified language isn't found in the list of available languages in the package, the default data file is imported.

You specify the available languages for the configuration data in the `<cmtdatafiles>` node in the `ImportConfig.xml` file. The default configuration data import file is specified in the `crmmigdataimportfile` attribute in the `ImportConfig.xml` file.

Skipping data checks (`OverrideDataImportSafetyChecks = true`) can be effective here if you're sure that the target Dataverse instance doesn't contain any data.

f. Enter custom code to execute after the import completes in the override definition of `AfterPrimaryImport` method. The remaining flat files that weren't imported earlier, before the solution import started, are imported now.

g. Change the default name of your package folder to the package name that you want. To do so, rename the `PkgFolder` (or `PkgAssets`) folder in the **Solution Explorer** pane, and then edit the return value under the `GetImportPackageDataFolderName` property.

C#

```

public override string GetImportPackageDataFolderName
{
 get
 {
 // WARNING this value directly correlates to the folder name in the
 // Solution Explorer where the ImportConfig.xml and sub content is
 // located.
 // Changing this name requires that you also change the correlating
 // name in the Solution Explorer
 return "PkgFolder";
 }
}

```

- h. Change the package name by editing the return value under the `GetNameOfImport` property.

```

C#

public override string GetNameOfImport(bool plural)
{
 return "Package Short Name";
}

```

This returned value is the name of your package that appears on the package selection page in the Dynamics 365 Package Deployer wizard.

- i. Change the package description by editing the return value under the `GetImportPackageDescriptionText` property.

```

C#

public override string GetImportPackageDescriptionText
{
 get { return "Package Description"; }
}

```

This returned value is the package description that appears alongside the package name on the package selection page in the Package Deployer wizard.

- j. Change the package long name by editing the return value under the `GetLongNameOfImport` property.

```

C#

```

```
public override string GetLongNameOfImport
{
 get { return "Package Long Name"; }
}
```

The package long name appears on the next page after you have selected the package to install.

3. Additionally, the following function and variables are available to the package:

[Expand table](#)

Name	Type	Description
<a href="#">CreateProgressItem(String)</a>	Function	Used to create a new progress item in the user interface (UI).
<a href="#">RaiseUpdateEvent(String, ProgressPanelItemStatus)</a>	Function	Used to update the progress created by the call to <a href="#">CreateProgressItem(String)</a>.   <a href="#">ProgressPanelItemStatus</a> is an enum with the following values:   Working = 0  Complete = 1  Failed = 2  Warning = 3  Unknown = 4
<a href="#">RaiseFailEvent(String, Exception)</a>	Function	Used to fail the current status import with an exception message.
<a href="#">IsRoleAssociatedWithTeam(Guid, Guid)</a>	Function	Used to determine if a role is associated with a specified team.
<a href="#">IsWorkflowActive(Guid)</a>	Function	Used to determine if a specified workflow is active.
<a href="#">PackageLog</a>	Class Pointer	A pointer to the initialized logging interface for the package. This interface is used by a package to log messages and exceptions to the package log file.
<a href="#">RootControlDispatcher</a>	Property	A dispatcher interface used to allow your control to render its own UI during package deployment. Use this interface to wrap any UI elements or commands. It's

Name	Type	Description
		important to check this variable for null values before using it as it might not be set to a value.
<a href="#">CrmSvc</a>	Property	A pointer to <a href="#">CrmServiceClient</a> class that allows for a package to address Dynamics 365 from within the package. Use this pointer to execute SDK methods and other actions in the overridden methods.
<a href="#">DataImportBypass</a>	Property	Specify whether Dynamics 365 Package Deployer skips all data import operations such as importing Dataverse sample data, flat file data, and data exported from the Configuration Migration tool. Specify true or false. Default is <code>false</code> .
<a href="#">OverrideDataImportSafetyChecks</a>	Property	Specify whether Dynamics 365 Package Deployer bypasses some of its safety checks, which helps in improving the import performance. Specify <code>true</code> or <code>false</code>. Default is <code>false</code>.   You should set this property to <code>true</code> only if the target Dataverse instance doesn't contain any data.

4. Save your project. The next step is to build the package.

## Build and deploy

The following sections describe how to build and deploy a package.

### Build

Building your package is described below depending on which tool you're using.

#### Power Platform CLI

To build a package created with the CLI, you could load the `.csproj` file into Visual Studio, but instead we're going to use the `dotnet` command and MSBuild. The example below assumes the working directory contains the `*.csproj` file.

```
Bash
```

```
> dotnet publish
```

```
DeploymentPackage ->
```

```
C:\Users\peter\Downloads\DeploymentPackage\bin\Debug\DeploymentPackage.1.0.0.pdpkg.zip
```

You can optionally look at the details of the built package.

```
Bash
```

```
> pac package show --package
```

```
.\bin\Debug\DeploymentPackage.1.0.0.pdpkg.zip
```

Your package is made of the following files under the `<Project>\Bin\Debug` folder.

- **<PackageName> folder:** The folder name is the same as the one you changed for your package folder name in step 2.g of this section [Add custom code](#). This folder contains all solutions, configuration data, flat files, and the contents for your package.

#### ⓘ Note

You may see a .NET folder (e.g, net472) containing a pdpublish folder. Your DLL and other project files are in that pdpublish folder.

- **<PackageName>.dll:** The assembly contains the custom code for your package. By default, the name of the assembly is the same as your project name.

## Deploy

After you create a package, you can deploy it on the Dataverse instance by using the Package Deployer tool, Windows PowerShell, or a CLI command.

- To deploy using the Package Deployer tool, first download the tool as described in [Dataverse development tools](#). Next, follow the detailed information on package deployment in the article [Deploy packages using Package Deployer or Windows PowerShell](#).
- To deploy using the CLI, use the `pac package deploy` command.

```
Bash
```

```
> pac package deploy --package
.\bin\Debug\DeploymentPackage.1.0.0.pdpkg.zip
```

#### ⓘ Note

To deploy a package to a target environment using the CLI, you must first set up an authentication profile and select an organization. More information: [pac auth create](#), [pac org select](#)

## Best practices

Listed below are a few best practice tips to follow when working with Package Deployer packages.

### Creating packages

When creating packages, developers must:

- *Ensure that package assemblies are signed.*

### Deploying packages

When deploying packages, Dataverse administrators must:

- *Insist on signed package assemblies* so that you can track an assembly back to its source.
- *Test the package on a preproduction instance*, preferably a mirror image of the production instance, before running it on a production instance.
- *Back up the production instance* before deploying the package.

## See also

[Solution Packager tool](#)

---

## Feedback

Was this page helpful?

👍 Yes

👎 No

[Provide product feedback](#)

# ImportConfig Reference

Article • 06/13/2024

The ImportConfig.xml file used with package deployer contains information about the solutions and data to import. The following is an example:

XML

```
<?xml version="1.0" encoding="utf-16"?>
<configdatastorage
 installsamplerdata="true"
 waitforsamplerdatatoinstall="true"
 agentdesktopzipfile=""
 agentdesktopexename=""
 crmmigdataimportfile="data_1033.zip">
 <solutions>
 <configsolutionfile
 solutionpackagefilename="SampleSolutionOne_1_0_managed.zip"
 overwriteunmanagedcustomizations="false"
 publishworkflowsandactivateplugins="true" />
 <configsolutionfile
 solutionpackagefilename="SampleSolutionTwo_1_0_managed.zip"
 overwriteunmanagedcustomizations="false"
 publishworkflowsandactivateplugins="true" />
 <configsolutionfile
 solutionpackagefilename="SampleSolutionThree_1_0_managed.zip" />
 </solutions>
 <filestoimport>
 <configimportfile filename="SampleOption.csv"
 filetype="CSV"
 associatedmap="SampleOption"
 importtoentity="sample_option"
 datadelimiter=""
 fielddelimiter="comma"
 enableduplicatedetection="true"
 isfirstrowheader="true"
 isrecordownerateam="false"
 owneruser=""
 waitforimporttocomplete="false" />
 <configimportfile filename="File.zip"
 filetype="ZIP"
 associatedmap="FileMapName"
 importtoentity="FileEntity"
 datadelimiter=""
 fielddelimiter="comma"
 enableduplicatedetection="true"
 isfirstrowheader="true"
 isrecordownerateam="false"
 owneruser=""
 waitforimporttocomplete="true" />
 </filestoimport>
 <zipimportdetails>
```



```

 <zipimportdetail filename="subfile1.csv"
 filetype="csv"
 importtoentity="account" />
 <zipimportdetail filename="subfile2.csv"
 filetype="csv"
 importtoentity="contact" />
 </zipimportdetails>
</filestoimport>
<filesmapstoimport>
 <configimportmapfile filename="SampleOption.xml" />
</filesmapstoimport>
<cmtdatafiles>
 <cmtdatafile filename="data_1033.zip"
 lcid="1033"
 usermapfilename="UserMap.xml" />
 <cmtdatafile filename="data_1041.zip"
 lcid="1041"
 usermapfilename="" />
</cmtdatafiles>
</configdatastorage>

```

The following sections provide details about the valid XML elements and attributes for this document.

## configdatastorage element

`configdatastorage` is the root element.

## configdatastorage attributes

All of these attributes are optional.

[Expand table](#)

Name	Description
<code>installsampleddata</code>	Whether to install sample data. This is the same sample data installed using these other methods:      - <a href="#">Add or remove sample data</a>     - <a href="#">Add and remove sample data with code</a>
<code>waitforsampledatatoinstall</code>	If <code>true</code> , and if <code>installsampleddata</code> is also set to <code>true</code> , waits for sample data to install before deploying the package. Don't set this to <code>true</code> when <code>installsampleddata</code> is <code>false</code> .
<code>agentdesktopzipfile</code>	File name of the zip file to unpack. If you specify a .zip file name here, it adds a screen during the package deployment process that prompts you to select a location where you want to unpack the

Name	Description
	contents of the file.   This attribute is commonly used for creating packages for <a href="#">Unified Service Desk for Dynamics 365</a>.
<code>agentdesktopexename</code>	Name of the .exe or .msi file in the zip file or a URL to be invoked at the end of the deployment process.   This attribute is commonly used for creating packages for <a href="#">Unified Service Desk for Dynamics 365</a>.
<code>crmmigdataimportfile</code>	File name of the default configuration data file (.zip) exported using the Configuration Migration tool.

## configdatastorage elements

These are the valid child elements:

 Expand table

Element	Descriptions
<code>solutions</code>	Contains 0 or many <a href="#">configsolutionfile elements</a> that describe solutions to import.
<code>filestoimport</code>	Contains       - 1 or many <a href="#">configimportfile elements</a>     - 0 or many <a href="#">zipimportdetails element</a> that describe individual files and zip files to be imported.
<code>filesmapstoimport</code>	Contains 1 or many <a href="#">configimportmapfile element</a> . The order of the map files in this element indicates the order in which they're imported. <a href="#">Learn more about creating data maps for import</a>
<code>cmtdatafiles</code>	Contains 1 or many <a href="#">cmtdatafile elements</a> that contain localized version of the configuration data file to be imported.

## `configsolutionfile` element

Describes solutions to import in the `configdatastorage.solutions` element.

## configsolutionfile attributes

[Expand table](#)

Name	Description
<code>solutionpackagefilename</code>	<b>Required</b> Specify the .zip file name of your solution.
<code>overwriteunmanagedcustomizations</code>	<b>Optional</b> Whether to overwrite any unmanaged customizations when importing a solution that already exists in the target environment. If you don't specify this behavior, the unmanaged customizations in the existing solution are maintained on the target environment.
<code>publishworkflowsandactivateplugins</code>	<b>Optional</b> Whether to publish workflows and activate plug-ins in the target environment after the solution is imported.If you don't specify this behavior, the workflow are published and plug-ins activated.

## `configimportfile` element

Describes files to import in the `configdatastorage.filestoimport` element.

### `configimportfile` attributes

[Expand table](#)

Name	Description
<code>filename</code>	Name of the file that contains the import data.
<code>filetype</code>	This value can be <code>csv</code> , <code>xml</code> , or <code>zip</code> . If the file is a .zip file, a <a href="#">zipimportdetails element</a> must be present with a <a href="#">zipimportdetail element</a> for each file in the .zip file.
<code>associatedmap</code>	Name of the Dataverse import data map to use with this file. If blank, Dataverse attempts to use the system determined import data map name for this file.
<code>importtoentity</code>	Can be the name of the exe in the zip file, a URL, or an .msi file to provide a link to invoke at the end of the process.
<code>datadelimiter</code>	Name of the data delimiter used in the import file. Valid values are single quote or double quotes.
<code>fielddelimiter</code>	Name of the field delimiter used in the import file. Valid values are comma or colon, or single quote.
<code>enableduplicatedetection</code>	Whether to enable duplicate detections rules on data import. Valid

Name	Description
	values are <code>true</code> or <code>false</code> .
<code>isfirstrowheader</code>	Whether the first row of the import file contains the field names. Valid values are <code>true</code> or <code>false</code> .
<code>isrecordownerateam</code>	Whether the owner of the record on import should be a team. Valid values are <code>true</code> or <code>false</code> .
<code>owneruser</code>	The user ID that should own the records. The default value is the currently logged in user.
<code>waitforimporttocomplete</code>	If <code>true</code> , the system waits for the import to complete before proceeding. If <code>false</code> , it queues the jobs and moves on.

## zipimportdetails element

Describes zip files to import in the `configdatastorage.filestoimport` element when the `configimportfile.filetype` is `zip`.

Contains 1 or more [zipimportdetail elements](#) that provide information about an individual file within the zip file specified in the [configimportfile element](#).

## zipimportdetail element

Provide information about an individual file within the zip file specified in the [configimportfile element](#).

## zipimportdetail attributes

 Expand table

Name	Description
<code>filename</code>	Name of the file that contains the import data.
<code>filetype</code>	This value can be <code>csv</code> or <code>xml</code> .
<code>importtoentity</code>	Can be the name of the exe in the zip file, a url, or an .msi file to provide a link to invoke at the end of the process.

## configimportmapfile element

Contains information about an individual map file to import in Dataverse.

## configimportmapfile attributes

 Expand table

Name	Description
<code>filename</code>	The name of the file containing the mapping data.

## cmtdatafile element

Specifies the localized configuration data files along with locale ID (required) and user information map file (optional).

## cmtdatafile example

XML

```
<cmtdatafiles>
 <cmtdatafile filename="data_1033.zip"
 lcid="1033"
 usermapfilename="UserMap.xml" />
 <cmtdatafile filename="data_1041.zip"
 lcid="1041"
 usermapfilename="" />
</cmtdatafiles>
```

## cmtdatafile attributes

 Expand table

Name	Description
<code>filename</code>	The name of the file containing the mapping data.
<code>lcid</code>	The string representation of the LCID language code identifier.
<code>usermapfilename</code>	For Dynamics 365 (on-premises) only. Contains the user map file (.xml) generated using the Configuration Migration tool in your project. This file is required to import user information to a Dynamics 365 (on-premises) instance on a different domain.

# Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# SolutionPackager tool

Article • 05/01/2024

SolutionPackager is a tool that can reversibly decompose a Microsoft Dataverse compressed solution file into multiple XML files and other files. You can then easily manage these files by using a source control system. The following sections show you how to run the tool and how to use the tool with managed and unmanaged solutions.

## Important

The SolutionPackager tool is no longer the recommended way to unpack and pack solutions. The capabilities of the SolutionPackager tool have been incorporated into the [Power Platform CLI](#). The [pac solution](#) command has a number of verbs including [unpack](#), [pack](#), [clone](#), and [sync](#) that incorporate the same underlying capabilities of the SolutionPackager tool.

## Where to find the SolutionPackager tool

The SolutionPackager tool is distributed as part of the [Microsoft.CrmSdk.CoreTools](#)  NuGet package. To install the program, follow these steps.

1. Download the NuGet package.
2. Rename the package filename extension from .nupkg to .zip.
3. Extract the contents of the compressed (zip) file.

Find the SolutionPackager.exe executable in the <extracted-folder-name>/contents/bin/coretools folder. Run the program from the coretools folder or add that folder to your PATH.

## SolutionPackager command-line arguments

SolutionPackager is a command-line tool that can be invoked with the parameters identified in the following table.

 Expand table

Argument	Description
/action: {Extract Pack}	Required. The action to perform. The action can be either to extract a solution .zip file to a folder, or to pack a folder into a

Argument	Description
	.zip file.
/zipfile: <file path>	Required. The path and name of a solution .zip file. When extracting, the file must exist and be readable. When packing, the file is replaced.
/folder: <folder path>	Required. The path to a folder. When extracting, this folder is created and populated with component files. When packing, this folder must already exist and contain previously extracted component files.
/packagetype: {Unmanaged Managed Both}	Optional. The type of package to process. The default value is Unmanaged. This argument may be omitted in most occasions because the package type can be read from inside the .zip file or component files. When extracting and Both is specified, managed and unmanaged solution .zip files must be present and are processed into a single folder. When packing and Both is specified, managed and unmanaged solution .zip files are produced from one folder. For more information, see the section on working with managed and unmanaged solutions later in this article.
/allowWrite:{Yes No}	Optional. The default value is Yes. This argument is used only during an extraction. When /allowWrite:No is specified, the tool performs all operations but is prevented from writing or deleting any files. The extract operation can be safely assessed without overwriting or deleting any existing files.
/allowDelete:{Yes No Prompt}	Optional. The default value is Prompt. This argument is used only during an extraction. When /allowDelete:Yes is specified, any files present in the folder specified by the /folder parameter that aren't expected are automatically deleted. When /allowDelete:No is specified, no deletes occur. When /allowDelete:Prompt is specified, the user is prompted through the console to allow or deny all delete operations. If /allowWrite:No is specified, no deletes occur even if /allowDelete:Yes is also specified.
/clobber	Optional. This argument is used only during an extraction. When /clobber is specified, files that have the read-only attribute set are overwritten or deleted. When not specified, files with the read-only attribute aren't overwritten or deleted.
/errorlevel: {Off Error Warning Info Verbose}	Optional. The default value is Info. This argument indicates the level of logging information to output.
/map: <file path>	Optional. The path and name of an .xml file containing file mapping directives. When used during an extraction, files typically read from inside the folder specified by the /folder



Argument	Description
	parameter are read from alternate locations as specified in the mapping file. During a pack operation, files that match the directives aren't written.
/nologo	Optional. Suppress the banner at runtime.
/log: <file path>	Optional. A path and name to a log file. If the file already exists, new logging information is appended to the file.
@ <file path>	Optional. A path and name to a file that contains command-line arguments for the tool.
/sourceLoc: <string>	Optional. This argument generates a template resource file, and is valid only on extract.   Possible values are <code>auto</code> or an LCID/ISO code for the language you want to export. When this argument is used, the string resources from the given locale are extracted as a neutral .resx file. If <code>auto</code> or just the long or short form of the switch is specified, the base locale or the solution is used. You can use the short form of the command: <code>/src</code>.
/localize	Optional. Extract or merge all string resources into .resx files. You can use the short form of the command: <code>/loc</code>. The localize option supports shared components for .resx files. More information: <a href="#">Using RESX web resources</a>

## Use the /map command argument

The following discussion details the use of the /map argument to the SolutionPackager tool.

Files that are built in an automated build system, such as .xap Silverlight files and plug-in assemblies, are typically not checked into source control. Web resources may already be present in source control in locations that aren't directly compatible with the SolutionPackager tool. By including the /map parameter, the SolutionPackager tool can be directed to read and package such files from alternate locations and not from inside the Extract folder as it would typically be done. The /map parameter must specify the name and path to an XML file containing mapping directives. Those directives instruct the SolutionPackager to match files by their name and path, and indicate the alternate location to find the matched file. The following information applies to all directives equally.

- Multiple directives may be listed including those directives that match identical files. Directives listed early in the file take precedence over directives listed later.
- If a file is matched to any directive, it must be found in at least one alternative location. If no matching alternatives are found, the SolutionPackager issues an error.
- Folder and file paths may be absolute or relative. Relative paths are always evaluated from the folder specified by the /folder parameter.
- Environment variables may be specified by using a %variable% syntax.
- A folder wildcard "\*\*\*" may be used to mean "in any subfolder". It can only be used as the final part of a path, for example: "c:\folderA\\*\*\*".
- File name wildcards may be used only in the forms "\*.ext" or "\*.\*". No other pattern is supported.

The three types of directives mappings are described here, along with an example that shows you how to use them.

## Folder mapping

The following information provides detailed information on folder mapping.

### Xml Format

```
<Folder map="folderA" to="folderB" />
```

### Description

File paths that match "folderA" are switched to "folderB".

- The hierarchy of subfolders under each must exactly match.
- Folder wildcards aren't supported.
- No file names may be specified.

### Examples

XML

```
<Folder map="folderA" to="folderB" />
<Folder map="folderA\folderB" to="..\..\folderC\" />
<Folder map="WebResources\subFolder" to="%base%\WebResources" />
```

# File To file mapping

The following information provides more details on file-to-file mapping.

## Xml Format

```
<FileToFile map="path\filename.ext" to="path\filename.ext" />
```

## Description

Any file matching the `map` parameter is read from the name and path specified in the `to` parameter.

For the `map` parameter:

- A file name must be specified. The path is optional. If no path is specified, files from any folder may be matched.
- File name wildcards aren't supported.
- The folder wildcard is supported.

For the `to` parameter:

- A file name and path must be specified.
- The file name may differ from the name in the `map` parameter.
- File name wildcards aren't supported.
- The folder wildcard is supported.

## Examples

XML

```
<FileToFile map="assembly.dll" to="c:\path\folder\assembly.dll" />
<FileToFile map="PluginAssemblies**\this.dll"
to="..\..\Plugins**\that.dll" />
<FileToFile map="Webresrouces\ardvark.jpg"
to="%SRCBASE%\CrmPackage\WebResources\JPG format\ardvark.jpg" />
```

XML

```
<FileToFile
map="pluginpackages\cr886_PluginPackageTest\package\cr886_PluginPackageTest.
```

```
nupkg"
 to="myplg\bin\Debug\myplg.1.0.0.nupkg" />
```

In the above NuGet package example, cr886\_PluginPackageTest.nupkg is not overwritten if the file already exists in the specified location.

## File to path mapping

The following provides detailed information on file-to-path mapping.

### Xml Format

```
<FileToPath map="path\filename.ext" to="path" />
```

### Description

Any file matching the `map` parameter is read from the path specified in the `to` parameter.

For the `map` parameter:

- A file name must be specified. The path is optional. If no path is specified, files from any folder may be matched.
- File name wildcards are supported.
- The folder wildcard is supported.

For the `to` parameter:

- A path must be specified.
- The folder wildcard is supported.
- A file name must not be specified.

### Examples

#### XML

```
<FileToPath map="assembly.dll" to="c:\path\folder" />
<FileToPath map="PluginAssemblies**\this.dll" to="..\..\Plugins\bin**"
/>
<FileToPath map="*.jpg" to="%SRCBASE%\CrmPackage\WebResources\JPG format\"
/>
<FileToPath map="*.*)" to="..\..\%ARCH%\%TYPE%\drop" />
```

## Example mapping

The following XML code sample shows a complete mapping file that enables the SolutionPackager tool to read any web resource and the two default generated assemblies from a Developer Toolkit project named CRMDevToolkitSample.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<Mapping>
 <!-- Match specific named files to an alternate folder -->
 <FileToFile map="CRMDevToolkitSamplePlugins.dll"
to="..\..\Plugins\bin**\CRMDevToolkitSample.plugins.dll" />
 <FileToFile map="CRMDevToolkitSampleWorkflow.dll"
to="..\..\Workflow\bin**\CRMDevToolkitSample.Workflow.dll" />
 <!-- Match any file in and under WebResources to an alternate set of
subfolders -->
 <FileToPath map="WebResources*.*"
to="..\..\CrmPackage\WebResources**" />
 <FileToPath map="WebResources***.*"
to="..\..\CrmPackage\WebResources**" />
</Mapping>
```

## Managed and unmanaged solutions

A Dataverse compressed solution (.zip) file can be exported in one of two types as shown here.

### Managed solution

A completed solution ready to be imported into an organization. Once imported, components can't be added or removed, although they can optionally allow further customization. This is recommended when development of the solution is complete.

### Unmanaged solution

An open solution with no restrictions on what can be added, removed, or modified. This is recommended during development of a solution.

The format of a compressed solution file will be different based on its type, either managed or unmanaged. The SolutionPackager can process compressed solution files of either type. However, the tool can't convert one type to another. The only way to convert solution files to a different type, for example from unmanaged to managed, is by importing the unmanaged solution .zip file into a Dataverse server and then exporting the solution as a managed solution.

The SolutionPackager can process unmanaged and managed solution .zip files as a combined set via the /PackageType:Both parameter. To perform this operation, it is necessary to export your solution twice as each type, naming the .zip files as follows.

[Expand table](#)

Unmanaged .zip file: AnyName.zip	Managed .zip file: AnyName_managed.zip
----------------------------------	----------------------------------------

The tool will assume the presence of the managed zip file in the same folder as the unmanaged file and extract both files into a single folder preserving the differences where managed and unmanaged components exist.

After a solution has been extracted as both unmanaged and managed, it is possible from that single folder to pack both, or each type individually, using the /PackageType parameter to specify which type to create. When specifying both files, two .zip files will be produced using the naming convention as above. If the /PackageType parameter is missing when packing from a dual managed and unmanaged folder, the default is to produce a single unmanaged .zip file.

## Troubleshooting

If you use Visual Studio to edit resource files created by the solution packager, you may receive a message when you repack similar to this: “Failed to determine version id of the resource file <filename>.resx the resource file must be exported from the solutionpackager.exe tool in order to be used as part of the pack process.” This happens because Visual Studio replaces the resource file’s metadata tags with data tags.

## Workaround

1. Open the resource file in your favorite text editor and locate and update the following tags:

XML

```
<data name="Source LCID" xml:space="preserve">
<data name="Source file" xml:space="preserve">
<data name="Source package type" xml:space="preserve">
<data name="SolutionPackager Version" mimetype="application/x-
microsoft.net.object.binary.base64">
```

2. Change the node name from `<data>` to `<metadata>`.

For example, this string:

XML

```
<data name="Source LCID" xml:space="preserve">
 <value>1033</value>
</data>
```

Changes to:

XML

```
<metadata name="Source LCID" xml:space="preserve">
 <value>1033</value>
</metadata>
```

This allows the solution packager to read and import the resource file. This problem has only been observed when using the Visual Studio Resource editor.

## See also

[Use Source Control with Solution Files](#)

[Solution concepts](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# Source control with solution files

Article • 02/07/2025

The Solution Packager tool can be used with any source control system. After a solution .zip file is extracted to a folder, add and submit the files to your source control system. These files can then be synchronized on another computer where they can be packed into a new identical solution .zip file.

An important aspect when using extracted component files in source control is that adding all the files into source control might cause unnecessary duplication. Go to the [Solution Component File Reference](#) to discover which files are generated for each component type and which files are recommended for use in source control.

As further customizations and changes are necessary for the solution, developers should edit or customize components through existing means, export again to create a .zip file, and extract the compressed solution file into the same folder.

## Important

Except for the sections described in [When to edit the customizations file](#), manual editing of extracted component files and .zip files isn't supported.

When the Solution Packager tool extracts the component files, it doesn't overwrite existing component files of the same name if the file contents are identical. In addition, the tool honors the read-only attribute on component files producing a warning in the console window that particular files weren't written. This protection enables the user to check out, from source control, the minimal set of files that are changing. The `/clobber` parameter can be used to override and cause read-only files to be written or deleted. The `/allowWrite` parameter can be used to assess what impact an extract operation has without actually causing any files to be written or deleted. Use of the `/allowWrite` parameter with verbose logging is effective.

After the extract operation is completed with the minimal set of files checked out from source control, a developer can submit the changed files back into source control, as is done with any other type of source file.

## Team development

When there are multiple developers working on the same solution component, a conflict might arise where changes from two developers result in changes to a single file. This



occurrence is minimized by decomposing each individually editable component or subcomponent into a distinct file. Consider the following example.

1. Developer A and B are both working on the same solution.
2. On independent computers, they both get the latest sources of the solution from source control, pack, and import an unmanaged solution .zip file into independent Microsoft Dataverse organizations.
3. Developer A customizes the "Active Contacts" system view and the main form for the Contact entity.
4. Developer B customizes the main form for the Account entity and changes the "Contact Lookup View."
5. Both developers export an unmanaged solution .zip file and extract.
  - a. Developer A will need to check out one file for the Contact main form, and one file for the "Active Contacts" view.
  - b. Developer B will need to check out one file for the Account main form, and one file for the "Contact Lookup View."
6. Both developers might submit, in any order, as their respective changes touched separate files.
7. After both submissions are complete, they can repeat step #2 and then continue to make further changes in their independent organizations. They each have both sets of changes, with no overwrites of their own work.

The previous example works only when there are changes to separate files. It's inevitable that independent customizations require changes within a single file. Based on the example shown previously, consider that developer B customized the "Active Contacts" view while developer A was also customizing it. In this new example, the order of events becomes important. The correct process to reconcile this predicament, written out in full, is described here.

1. Developer A and B are both working on the same solution.
2. On independent computers, they both get the latest sources of the solution from source control, pack, and import an unmanaged solution .zip file into independent organizations.
3. Developer A customizes the "Active Contacts" system view and the main form for the Contact table.

4. Developer B customizes the main form for the Account table and changes the "Active Contacts."
5. Both developers export an unmanaged solution .zip file and extract.
  - a. Developer A will need to check out one file for the Contact main form, and one file for the "Active Contacts" view.
  - b. Developer B will need to check out one file for the Account main form, and one file for the "Active Contacts" view.
6. Developer A is ready first.
  - a. Before developer A submits to source control, they must get latest sources to ensure no prior check-ins conflict with their changes.
  - b. There are no conflicts so developer A is able to submit.
7. Developer B is ready next following developer A.
  - a. Before developer B submits, they must get the latest sources to ensure no prior check-ins conflict with their changes.
  - b. There's a conflict because the file for "Active Contacts" has been modified since developer B last retrieved the latest sources.
  - c. Developer B must reconcile the conflict. It's possible the capabilities of the source control system in use can aide this process; otherwise the following choices are all viable.
    - i. Developer B, through source control history, if available, can observe that the developer A made the prior change. Through direct communication they can discuss each change. Then developer B only has to update the organization with the agreed resolution. Developer B then exports, extracts, and overwrites the conflicting file and submits.
    - ii. Allow source control to overwrite the local file. Developer B packs the solution and imports it into their organization, then assesses the state of the view and recustomizes it as necessary. Next, developer B might export, extract, and overwrite the conflicting file.
    - iii. If the prior change is deemed unnecessary, developer B allows their copy of the file to overwrite the version in source control and submits.

Whether working on a shared environment or independent environment, team development of Dataverse solutions requires those actively working on a common

solution to be aware of the work of others. The Solution Packager tool doesn't fully remove this need but it does enable easy merging of nonconflicting changes at the source control level, and it proactively highlights the concise components where conflicts arise.

The next sections are the generic processes to effectively use the Solution Packager tool in source control when developing with teams. These work equally with independent environments or shared development environments, though with shared environments the export and extract naturally includes all changes present within the solution, not just those made by the developer performing the export. Similarly, when importing a solution .zip file the natural behavior to overwrite all components occurs.

## Create a solution

This procedure identifies the typical steps used when you first create a solution.

1. In a clean environment with Dataverse, create a solution, and then add or create components as necessary.
2. When you're ready to check in, follow these steps.
  - a. Export the unmanaged solution.
  - b. Using the Solution Packager tool, extract the solution into component files.
  - c. From those extracted component files, add the necessary files to source control.
  - d. Submit these changes to source control.

## Modify a solution

The following procedure identifies the typical steps used when modifying an existing solution.

1. Synchronize or get the latest solution component file sources.
2. Using the Solution Packager tool, pack component files into an unmanaged solution .zip file.
3. Import the unmanaged solution file into an environment.
4. Customize and edit the solution as necessary.
5. When you're ready to check the changes into source control, follow these steps.

- a. Export the unmanaged solution.
- b. Using the Solution Packager tool, extract the exported solution into component files.
- c. Synchronize or get the latest sources from source control.
- d. Reconcile if any conflicts exist.
- e. Submit the changes to source control.

Steps 2 and 3 must be done before further customizations occur in the development organization. Within step 5, step b must be completed before step c.

## See also

[Solution Component File Reference \(SolutionPackager\)](#)  
[SolutionPackager tool](#)

---

## Feedback

Was this page helpful?



[Provide product feedback](#) 

# Use the Power Apps checker web API

Article • 05/01/2024

The Power Apps checker web API provides a mechanism to run static analysis checks against customizations and extensions to the Microsoft Dataverse platform. It's available for makers and developers to perform rich static analysis checks on their solutions against a set of best practice rules to quickly identify problematic patterns. The service provides the logic for the [solution checker feature](#) in the Power Apps maker [portal](#) and is included as part of the automation for [applications submitted to AppSource](#). Interacting with the service directly in this manner allows for analysis of solutions that are included as part of on-premises (all supported versions) and online environments.

For information about using the checker service from PowerShell code, refer to [Work with solutions using PowerShell](#).

## ⓘ Note

- Use of Power Apps checker does not guarantee that a solution import will be successful. The static analysis checks performed against the solution do not know the configured state of the destination environment and import success may be dependent on other solutions or configurations in the environment.

## Alternative approaches

Before reading through the details of how to interact at the lowest level with the web APIs, consider using our PowerShell module, `Microsoft.PowerApps.Checker.PowerShell`, instead. It's a fully supported tool that is available in the [PowerShell Gallery](#). The current restriction is that it does require Windows PowerShell. If unable to meet this requirement, then interacting with the APIs directly is the best approach.

## Getting started

It's important to note that a solution analysis can result in a long running process. It can typically take sixty (60) seconds to upwards of five (5) minutes depending on a various factors, such as number, size, and complexity of customizations and code. The analysis flow is multi-step and asynchronous beginning with initiating an analysis job with the status API being used to query for job completion. An example flow for an analysis is as follows:

1. Obtain an OAuth token
2. Call upload (for each file in parallel)
3. Call analyze (initiates the analysis job)
4. Call status until finished (looping with a pause in between calls until the end is signaled or thresholds are met)
5. Download the result(s) from the provided SAS URI

A few variations are:

- Include a lookup of the ruleset or rules as a pre-step. However, it would be slightly faster to pass in a configured or hard coded ruleset ID. It is recommended that you use a ruleset that meets your needs.
- You can opt to not use the upload mechanism (see the upload for limitations).

You need to determine the following requirements:

- [Which geography?](#)
- [Which version?](#)
- [Which rulesets and rules?](#)
- [What is your tenant ID?](#)

Refer to the following articles for documentation on the individual APIs:

[Retrieve the list of rulesets](#)

[Retrieve the list of rules](#)

[Upload a file](#)

[Invoke analysis](#)

[Check for analysis status](#)

## Determine a geography

When you interact with the Power Apps checker service, files are temporarily stored in Azure along with the reports that are generated. By using a geography specific API, you can control where the data is stored. Requests to a geography endpoint are routed to a regional instance based on best performance (latency to the requestor). Once a request enters a regional service instance, all processing and persisted data remains within that particular region. Certain API responses return regional instance URLs for subsequent requests once an analysis job is routed to a specific region. Each geography may have a different version of the service deployed at any given point in time. Use of different service versions is due to the multi-stage safe deployment process, which ensures full version compatibility. Thus, the same geography should be used for each API call in the

analysis lifecycle and may reduce overall execution time as the data may not have to travel as far over the wire. The following are the available geographies:

 Expand table

Azure datacenter	Name	Geography	Base URI
Public	Preview	United States	unitedstatesfirstrelease.api.advisor.powerapps.com
Public	Production	United States	unitedstates.api.advisor.powerapps.com
Public	Production	Europe	europe.api.advisor.powerapps.com
Public	Production	Asia	asia.api.advisor.powerapps.com
Public	Production	Australia	australia.api.advisor.powerapps.com
Public	Production	Japan	japan.api.advisor.powerapps.com
Public	Production	India	india.api.advisor.powerapps.com
Public	Production	Canada	canada.api.advisor.powerapps.com
Public	Production	South America	southamerica.api.advisor.powerapps.com
Public	Production	United Kingdom	unitedkingdom.api.advisor.powerapps.com
Public	Production	France	france.api.advisor.powerapps.com
Public	Production	Germany	germany.api.advisor.powerapps.com
Public	Production	United Arab Emirates	unitedarabemirates.api.advisor.powerapps.com
Public	Production	Switzerland	switzerland.api.advisor.powerapps.com
Public	Production	South Africa	southafrica.api.advisor.powerapps.com
Public	Production	Korea	korea.api.advisor.powerapps.com
Public	Production	Norway	norway.api.advisor.powerapps.com
Public	Production	Singapore	singapore.api.advisor.powerapps.com
Public	Production	Sweden	sweden.api.advisor.powerapps.com
Public	Production	US Government	gov.api.advisor.powerapps.us
Public	Production	US Government L4	high.api.advisor.powerapps.us

Azure datacenter	Name	Geography	Base URI
Public	Production	US Government L5 (DOD)	mil.api.advisor.appsplatform.us
Public	Production	China operated by 21Vianet	china.api.advisor.powerapps.cn

### ⓘ Note

You may choose to use the preview geography to incorporate the latest features and changes earlier. However, note that the preview uses United States Azure regions only.

## Versioning

While not required, it's recommended to include the api-version query string parameter with the desired API version. The current API version is 2.0 for rulesets and rules and 1.0 for all other requests. For example, the following ruleset is an HTTP request specifying to use the 2.0 API version:

```
https://unitedstatesfirstrelease.api.advisor.powerapps.com/api/ruleset?api-version=2.0
```

If not provided, the latest API version is used by default. Using an explicit version number is recommended as the version is incremented if breaking changes are introduced. If the version number is specified in a request, backward compatibility support in later (numerically greater) versions will be maintained.

## Rulesets and rules

Power Apps checker requires a list of rules when run. These rules can be provided in the form of individual rules or a grouping of rules, referred to as a *ruleset*. A ruleset is a convenient way to specify a group of rules instead of having to specify each rule individually. For example, the solution checker feature uses a ruleset named *Solution Checker*. As new rules are added or removed, the service includes these changes automatically without requiring any change by the consuming application. If you require that the list of rules not change automatically as described above, then the rules can be specified individually. Rulesets can have one or more rules with no limit. A rule can be in



no or multiple rulesets. You can get a list of all rulesets by calling the API as follows:

`[Geographical URL]/api/ruleset`. This endpoint now requires authentication.

## Solution checker ruleset

The solution checker ruleset contains a set of impactful rules that have limited chances for false positives. If running analysis against an existing solution, it's recommended that you start with this ruleset. This ruleset is used by the [solution checker feature](#).

## AppSource certification ruleset

When publishing applications on AppSource, you must get your application certified. [Applications published on AppSource](#) are required to meet a high quality standard. The AppSource certification ruleset contains the rules that are part of the solution checker ruleset, plus other rules to ensure only high quality applications are published on the store. Some of AppSource certification rules are more prone to false positives and may require more attention to resolve.

## Find your tenant ID

The ID of your tenant is needed to interact with the APIs that require a token. Refer to [this article](#) for details on how to obtain the tenant ID. You can also use PowerShell commands to retrieve the tenant ID. The following example applies the cmdlets in the [AzureAD module](#).

PowerShell

```
Login to Microsoft Entra ID as your user
Connect-AzureAD

Establish your tenant ID
$tenantId = (Get-AzureADTenantDetail).ObjectId
```

The tenant ID is the value of the `ObjectId` property that is returned from `Get-AzureADTenantDetail`. You may also see it after logging in using the `Connect-AzureAD` cmdlet in the cmdlet output. In this case, it will be named `TenantId`.

## Authentication and authorization

Querying for rules and rulesets don't require an OAuth token, but all of the other APIs do require the token. The APIs do support authorization discovery by calling any of the

APIs that require a token. The response is an unauthorized HTTP status code of 401 with a WWW-Authenticate header, the authorization URI, and the resource ID. You should also provide your tenant ID in the `x-ms-tenant-id` header. Refer to [Power Apps Checker authentication and authorization](#) for more information. The following is an example of the response header returned from an API request:

HTTP

```
WWW-Authenticate →Bearer
authorization_uri="https://login.microsoftonline.com/0082fff7-33c5-44c9-920c-c2009943fd1e", resource_id="https://api.advisor.powerapps.com/"
```

Once you have this information, you can choose to use the Microsoft Authentication Library (MSAL) or some other mechanism to acquire the token. The following is an example of how this can be done using C# and the [MSAL .NET](#) library:

C#

```
// Substitute your own environment URL here.
string resource = "https://<env-name>.api.<region>.dynamics.com";

// Example Microsoft Entra app registration.
// For your custom apps, you will need to register them with Microsoft Entra ID yourself.
// See https://docs.microsoft.com/powerapps/developer/data-platform/walkthrough-register-app-azure-active-directory
var clientId = "51f81489-12ee-4a9e-aaae-a2591f45987d";
var redirectUri = "http://localhost"; // Loopback required for the interactive login.

var authBuilder = PublicClientApplicationBuilder.Create(clientId)
 .WithAuthority(AadAuthorityAudience.AzureAdMultipleOrgs)
 .WithRedirectUri(redirectUri)
 .Build();
var scope = resource + "/.default";
string[] scopes = { scope };

AuthenticationResult tokenResult =
 await authBuilder.AcquireTokenInteractive(scopes).ExecuteAsync();
```

For the full working code, see the Web API [QuickStart sample](#).

Once you have acquired the token, it's advised that you provide the same token to subsequent calls in the request lifecycle. However, more requests may warrant a new token be acquired for security reasons.

## Transport security

For best-in-class encryption, the checker service only supports communications using Transport Layer Security (TLS) 1.2 and greater. For guidance on .NET best practices around TLS, refer to [Transport Layer Security \(TLS\) best practices with the .NET Framework](#).

## Report format

The result of the solution analysis is a zip file containing one or more reports in a standardized JSON format. The report format is based on static analysis results referred to as Static Analysis Results Interchange Format (SARIF). There are tools available to view and interact with SARIF documents. Refer to this [web site](#) for details. The service uses version two of the [OASIS standard](#).

## See also

[Retrieve the list of rulesets](#)

[Retrieve the list of rules](#)

[Upload a file](#)

[Invoke analysis](#)

[Check for analysis status](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)

# Invoke analysis

Article • 02/17/2022

Initiating an analysis job is done by submitting a `POST` request to the `analyze` route. Analysis can be a long running process that usually lasts longer than a minute. The API first does some basic validation, initiates the request on the backend by submitting a job, and then responds with a status code of 202 and a `Location` header or with the appropriate error details. The `Location` header value is a URL that can be used to check on the status of the request and to obtain the URL(s) of the result(s). There are various options through the `POST` action to tailor the job based on your criteria, such as the list of rules or rulesets, files to exclude from the analysis, and more. You can initiate the analysis using the following `[Geographical URL]/api/analyze?api-version=1.0`.

## ⓘ Note

It is recommended to wait between 15 to 60 seconds between status checks. Analysis usually takes between 1 to 5 minutes to run. This API does require an OAuth token.

## Headers

Name	Type	Expected value	Required?
Authorization	string	The OAuth 1 bearer token with Azure Active Directory (AAD) Application ID claim.	yes
x-ms-tenant-id	GUID	The ID of the tenant for the application.	yes
x-ms-correlation-id	GUID	The Identifier for the analysis run. You should provide the same ID for the entire execution (upload, analyze, status).	yes
Accept	object	<code>application/json</code> , <code>application/x-ms-sarif-v2</code>	yes
Accept-Language	string	The language code or codes (e.g., en-US). The default is en-US. If multiple languages are provided, the first will be the primary. However, all translations (if the language is supported) will be included.	no

## Body

## Commonly used options:

Property	Type	Expected value	Required?
sasUriList	array of strings	A list of URIs that provides the service access to download a single solution, a zip file containing multiple solution files, or a package.	Yes
ruleSets	array of custom	0 or more	No
ruleSets.id	guid	The ID of the ruleset, which can be found by querying the ruleset API.	No, but this is usually what you would want to use. You must use either this or ruleCodes.
ruleCodes.code	string	The ID of the desired rule, which can be found by querying the ruleset and rule APIs.	No, you must use either this or ruleSets.
fileExclusions	array of strings	A list of file names or file name patterns to exclude. Support exists for using "*" as a wildcard in the beginning and/or end of a file name ( e.g., *jquery.dll and *jquery*).	No

## Expected responses

HTTP status code	Scenario	Result
202	Request for analysis was accepted and the status check URI was returned in the <code>Location</code> header	No result body
400	A non-zip file was sent, incorrect parameters, or a file was included with a virus	No result body
409	A request with a duplicate <code>x-ms-correlation-id</code> header value was sent	No result body

## Expected response headers

Name	Type	Expected value	Required?
------	------	----------------	-----------

Name	Type	Expected value	Required?
Location	Uri	URL to use in querying for the current status and to obtain the results	yes

## Example: initiate an analysis

This is an example of initiating an analysis job with the *AppSource Certification* ruleset, a single file, and excluding files that contain the text *jquery* and *json* in the name.

### Request

HTTP

```
POST [Geographical URI]/api/analyze?api-version=1.0
Accept: application/json
Content-Type: application/json; charset=utf-8
x-ms-correlation-id: 9E378E56-6F35-41E9-BF8B-C0CC88E2B832
x-ms-tenant-id: F2E60E49-CB87-4C24-8D4F-908813B22506

{
 "ruleSets": [{
 "id": "0ad12346-e108-40b8-a956-9a8f95ea18c9"
 }],
 "sasUriList":
 ["https://testenvfakelocation.blob.core.windows.net/mySolution.zip"],
 "fileExclusions": ["*jquery*", "*json*"]
}
```

### Response

HTTP

```
HTTP/1.1 202 Accepted
Content-Type: application/json; charset=utf-8
Location: [Geographical URI]/api/status/9E378E56-6F35-41E9-BF8B-C0CC88E2B832&api-version=1.0
```

## See also

[Use the Power Apps checker web API](#)

[Retrieve the list of rulesets](#)

[Retrieve the list of rules](#)

[Upload a file](#)

[Check for analysis status](#)

# Check for analysis status

Article • 02/17/2022

A URL is returned as part of the `Location` header in response to a request to the `analyze` API. It is to be used to query via HTTP `GET` for the analysis job's status. When the analysis job is finished the response body will include the URL or list of URLs in which the results output can be downloaded. Keep calling this URI until an HTTP status code of 200 is returned. While the job is still running, an HTTP status code of 202 will be returned with the `Location` header containing this same URI that was returned from `analyze`. Once a 200 response is returned, the `resultFileUri` property will include the single or list of downloadable locations of the output, which is contained in a zip file. A [Static Analysis Results Interchange Format \(SARIF\) V2](#) formatted file is included within this zip download that is a `JSON` formatted file containing the results of the analysis. The response body will contain an `IssueSummary` object that contains a summary of the count of issues found.

## ⓘ Note

It is recommended to wait between 15 to 60 seconds between status checks. Analysis usually takes between 1 to 5 minutes to run. This API does require an OAuth token that must be a token for the same client application that initiated the analysis job.

## Headers

Name	Type	Expected value	Required?
Authorization	string	The OAuth 1 bearer token with AAD Application ID claim.	yes
x-ms-tenant-id	GUID	The ID of the tenant for the application.	yes
x-ms-correlation-id	GUID	The identifier for the analysis run. You should provide the same Id for the entire execution (upload, analyze, status)	yes

## Expected responses

HTTP status code	Scenario	Result
------------------	----------	--------

HTTP status code	Scenario	Result
200	One or more results were found	See the example below. One result will be returned.
202	Still processing	See the example below. One result will be returned.
403	Forbidden	The requestor is not the same as the originator of the request for analysis.
404	Not found	Unable to find the analysis request with the reference provided in the URL.

## Expected response headers

Name	Type	Expected value	Required?
Location	uri	URI to use in querying for the current status and to obtain the results	yes

## Expected response body

The following table outlines the structure of the response for each request (HTTP 200 or 202 response only).

Property	Type	Expected value	Required?
privacyPolicy	string	The URI of the privacy policy.	Yes
progress	int	A value ranging from 0-100 percentage complete, where 10 means that processing is approximately 10% complete.	Yes
runCorrelationId	GUID	The request identifier that is included in each request. This can be used to correlate to the request, if needed.	Yes



Property	Type	Expected value	Required?
status	string	<code>InProgress</code> is returned when the job is still being processed. <code>Failed</code> is returned when there was a catastrophic issue processing the job on the server. There should be more details in the error property. <code>Finished</code> is returned when the job has completed successfully without issues.   <code>FinishedWithErrors</code> is returned when the job has completed successfully, however, one or more rules failed to complete without error. This is purely a signal for you to know that the report may not be complete. Microsoft is aware of these issues in the backend and will work to get things diagnosed and addressed.	Yes
resultFileUri	array of strings	A list of URIs that allow for direct download of the output. There should be one per file that was included in the original analyze API call.	No. This is only included when processing has completed.
issueSummary	IssueSummary	Properties listed below	No. This is only included when processing has completed.

Property	Type	Expected value	Required?
issueSummary.criticalIssueCount	int	Count of issues identified having a critical severity in the result	Yes
issueSummary.highIssueCount	int	Count of issues identified having a high severity in the result	Yes
issueSummary.mediumIssueCount	int	Count of issues identified having a medium severity in the result	Yes
issueSummary.lowIssueCount	int	Count of issues identified having a low severity in the result	Yes
issueSummary.informationalIssueCount	int	Count of issues identified having an informational severity in the result	Yes

## Example: status check when done

This example issues a status check call with the result being a completion.

### Request

HTTP

```
GET [Geographical URI]/api/status/9E378E56-6F35-41E9-BF8B-C0CC88E2B832&api-version=1.0
Accept: application/json
Content-Type: application/json; charset=utf-8
x-ms-correlation-id: 9E378E56-6F35-41E9-BF8B-C0CC88E2B832
x-ms-tenant-id: F2E60E49-CB87-4C24-8D4F-908813B22506
```

### Response

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
```

```
"privacyPolicy":"https://go.microsoft.com/fwlink/?LinkID=310140",
"progress":100,
"resultFileUri":["https://fakeblob.blob.core.windows.net/report-
files/mySolution.zip?sv=2017-11-09&sr=b&sig=xyz&se=2019-06-
11T20%3A27%3A59Z&sp=rd"],"runCorrelationId":"9E378E56-6F35-41E9-BF8B-
C0CC88E2B832","status":"Finished","issueSummary":
{
 "informationalIssueCount":0,
 "lowIssueCount":0,
 "mediumIssueCount":302,
 "highIssueCount":30,
 "criticalIssueCount":0
}
}
```

## See also

[Use the Power Apps checker web API](#)

[Retrieve the list of rulesets](#)

[Retrieve the list of rules](#)

[Upload a file](#)

[Invoke analysis](#)

# Retrieve the list of rules

Article • 05/02/2024

Rules are grouped together using a ruleset. A rule can be in no ruleset, or multiple rulesets. Use a `GET` request to obtain a list of all rules available, rules in a ruleset, or rulesets by calling the API `[Geographical URI]/api/rule`. There are a few variations to calling this API, however, the most common usage is to retrieve the list of rules for a specific ruleset.

### ⓘ Note

This API does require an OAuth token.

## Headers

 Expand table

Name	Type	Expected value	Required?
Accept-Language	string	The language code (for example, en-US). The default is en-US.	no

## Parameters

 Expand table

Name	Type	Expected value	Required?
ruleset	string	The name or ID of the ruleset or a list of ruleset IDs, or names separated by a comma or semicolon (for example, "Solution Checker").	no
includeMessageFormats	bool	When set to <code>true</code> , the list of possible message variations are included in the results of the language(s) requests, if available. This list is useful for translations into multiple languages. If not needed, then don't provide this parameter or provide <code>false</code> as the value as this parameter increases the size of the response and can increase processing time.	no

# Expected responses

 Expand table

HTTP status code	Scenario	Result
200	One or more results were found	See the example later in this article. One or more results may be returned.
204	No results were found	No results in the response body.
401	Authentication Failed	No results in the response body.

## Expected response body

The following table outlines the structure of the response for each request (HTTP 200 response only).

 Expand table

Property	Type	Expected value	Required?
code	string	The identifier of the rule, sometimes referred to as the Rule ID.	Yes
summary	string	A summary of the rule.	Yes
description	string	More detailed description of the rule.	Yes
guidanceUrl	URI	The URL in which to find published guidance. There may be some cases where there isn't a dedicated supporting guidance article.	Yes
include	boolean	Signals to the service that the rule is to be included in the analysis. This value is <code>true</code> for this API.	No
messageTemplates	array	This property value is included only when <code>includeMessageFormats</code> is <code>true</code> .	No

Property	Type	Expected value	Required?
messageTemplates.ruleId	string	Returns the same ID value as the <code>code</code> property.	Yes
messageTemplates.messageTemplateId	string	An identifier used in the Static Analysis Results Interchange Format (SARIF) report to signal an issue message variation for the rule.	Yes
messageTemplates.messageTemplate	string	The text of the message variation for the issue scenario that the rule reports. This text is a format string that may contain tokens in which arguments provided in the SARIF report can be used to construct a detailed message.	Yes

## Example: retrieve rules for a ruleset in another language

This example returns data for all of the rules in the *Solution Checker* ruleset in the French language. If the desired language is English, then just remove the Accept-Language header.

### Request

HTTP

```
GET [Geographical URI]/api/rule?ruleset=083A2EF5-7E0E-4754-9D88-9455142DC08B&api-version=2.0
x-ms-correlation-id: 9E378E56-6F35-41E9-BF8B-C0CC88E2B832
Accept: application/json
Content-Type: application/json; charset=utf-8
Accept-Language: fr
```

### Response

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[
 {
```

```

 "description": "Ne pas implémenter d'activités de workflow Microsoft Dynamics CRM 4.0",
 "guidanceUrl": "https://go.microsoft.com/fwlink/?LinkID=398563&error=il-avoid-crm4-wf&client=PAChecker",
 "include": true,
 "code": "il-avoid-crm4-wf",
 "summary": "Ne pas implémenter d'activités de workflow Microsoft Dynamics CRM 4.0",
 "howToFix": {
 "summary": ""
 }
 },
 {
 "description": "Utiliser InvalidPluginExecutionException dans des plug-ins et activités de workflow",
 "guidanceUrl": "https://go.microsoft.com/fwlink/?LinkID=398563&error=il-use-standard-exception&client=PAChecker",
 "include": true,
 "code": "il-use-standard-exception",
 "summary": "Utiliser InvalidPluginExecutionException dans des plug-ins et activités de workflow",
 "howToFix": {
 "summary": ""
 }
 },
 ...
]

```

## Example: retrieve all

This example returns data for all of the rules available.

### Request

HTTP

```

GET [Geographical URI]/api/rule?api-version=2.0
Accept: application/json
Content-Type: application/json; charset=utf-8

```

### Response

HTTP

```

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

```

```

[
 {
 "description": "Retrieve specific columns for an entity via query

```

```

APIs",
 "guidanceUrl": "https://go.microsoft.com/fwlink/?
LinkID=398563&error=il-specify-column&client=PAChecker",
 "include": true,
 "code": "il-specify-column",
 "summary": "Retrieve specific columns for an entity via query APIs",
 "howToFix": {
 "summary": ""
 }
},
{
 "description": "Do not duplicate plug-in step registration",
 "guidanceUrl": "https://go.microsoft.com/fwlink/?
LinkID=398563&error=meta-remove-dup-reg&client=PAChecker",
 "include": true,
 "code": "meta-remove-dup-reg",
 "summary": "Do not duplicate plug-in step registration",
 "howToFix": {
 "summary": ""
 }
},
...
]

```

## Example: retrieve for a ruleset with message formats

This example returns data for all of the rules in the *Solution Checker* ruleset in the French language. If the desired language is English, then just remove the Accept-Language header.

### Request

HTTP

```

GET [Geographical URI]/api/rule?ruleset=083A2EF5-7E0E-4754-9D88-
9455142DC08B&includeMessageFormats=true&api-version=2.0
Accept: application/json
Content-Type: application/json; charset=utf-8

```

### Response

HTTP

```

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

```

[



```

{
 "description": "Do not implement Microsoft Dynamics CRM 4.0 workflow
activities",
 "guidanceUrl": "https://go.microsoft.com/fwlink/?
LinkID=398563&error=il-avoid-crm4-wf&client=PAChecker",
 "include": true,
 "code": "il-avoid-crm4-wf",
 "summary": "Do not implement Microsoft Dynamics CRM 4.0 workflow
activities",
 "howToFix": {
 "summary": ""
 },
 "messageTemplates": [
 {
 "ruleId": "il-avoid-crm4-wf",
 "messageTemplateId": "message1",
 "messageTemplate": "Update the {0} class to derive from
System.Workflow.Activities.CodeActivity, refactor Execute method
implementation, and remove
Microsoft.Crm.Workflow.CrmWorkflowActivityAttribute from type"
 },
 {
 "ruleId": "il-avoid-crm4-wf",
 "messageTemplateId": "message2",
 "messageTemplate": "Change the {0} property's type from {1}
to {2} Argument <T> type"
 },
 {
 "ruleId": "il-avoid-crm4-wf",
 "messageTemplateId": "message3",
 "messageTemplate": "Replace the
Microsoft.Crm.Workflow.Crm{0}Attribute with Microsoft.Xrm.Sdk.Workflow.
{0}Attribute"
 },
 {
 "ruleId": "il-avoid-crm4-wf",
 "messageTemplateId": "message4",
 "messageTemplate": "Remove the {0}
System.Workflow.ComponentModel.DependencyProperty type field"
 }
]
},
{
 "description": "Use InvalidPluginExecutionException in plug-ins and
workflow activities",
 "guidanceUrl": "https://go.microsoft.com/fwlink/?
LinkID=398563&error=il-use-standard-exception&client=PAChecker",
 "include": true,
 "code": "il-use-standard-exception",
 "summary": "Use InvalidPluginExecutionException in plug-ins and
workflow activities",
 "howToFix": {
 "summary": ""
 },
 "messageTemplates": [

```

```
{
 "ruleId": "il-use-standard-exception",
 "messageTemplateId": "message1",
 "messageTemplate": "An unguarded throw of type {0} was
detected. Refactor this code to either throw an exception of type
InvalidPluginExecutionException or guard against thrown exceptions of other
types."
},
{
 "ruleId": "il-use-standard-exception",
 "messageTemplateId": "message2",
 "messageTemplate": "An unguarded rethrow of type {0} was
detected. Refactor this code to either throw an exception of type
InvalidPluginExecutionException or guard against thrown exceptions of other
types."
}
],
...
]
```

## See also

[Use the Power Apps checker web API](#)

[Retrieve the list of rulesets](#)

[Upload a file](#)

[Invoke analysis](#)

[Check for analysis status](#)

---

## Feedback

Was this page helpful?

[Provide product feedback](#) [↗](#)

# Retrieve the list of rulesets

Article • 05/02/2024

Rules are grouped together using a ruleset. Rulesets can have one or more rules with no limit. A rule can be in no ruleset, or multiple rulesets. Use a `GET` request to obtain a list of all rulesets available by calling the API, [Geographical URI]/api/ruleset.

 **Note**

This API does require an OAuth token.

## Expected responses

 Expand table

HTTP status code	Scenario	Result
200	One or more results were found	See the example later in this article. One or more results may be returned.
204	No results were found	No results response body is returned.
401	Authentication Failed	No results in the response body.

## Expected response body

The following table outlines the structure of the response for each request (HTTP 200 response only).

 Expand table

Property	Type	Expected value	Required?
id	Guid	Identifier of the ruleset	Yes
name	string	Friendly name of the ruleset	Yes

## Example: retrieve all rulesets

This example returns data for all of the rulesets available.

## Request

HTTP

```
GET [Geographical URI]/api/ruleset?api-version=2.0
Accept: application/json
x-ms-correlation-id: 9E378E56-6F35-41E9-BF8B-C0CC88E2B832
Content-Type: application/json; charset=utf-8
```

## Response

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[
 {
 "id": "083a2ef5-7e0e-4754-9d88-9455142dc08b",
 "name": "AppSource Certification"
 },
 {
 "id": "0ad12346-e108-40b8-a956-9a8f95ea18c9",
 "name": "Solution Checker"
 }
]
```

## See also

[Use the Power Apps checker web API](#)

[Retrieve the list of rules](#)

[Upload a file](#)

[Invoke analysis](#)

[Check for analysis status](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# Upload a file for analysis

Article • 11/17/2023

The initiation of an analysis job requires a path to an Azure blob that is accessible by URL. The ability to upload a file to Azure blob storage in the specified geography using the upload service is provided. It is not required that the upload API be used in order to run analysis. You can upload using a `POST` request to the following: `[Geographical URI]/api/upload?api-version=1.0`. Uploading a file up to 30 MB in size is supported. For anything larger you will need to provide your own externally accessible Azure storage and SAS URI.

## ⓘ Note

This API does require an OAuth token.

## Headers

Name	Type	Expected value	Required?
Authorization	string	The OAuth 1 bearer token with Microsoft Entra ID Application ID claim.	yes
x-ms-tenant-id	GUID	The ID of the tenant for the application.	yes
x-ms-correlation-id	GUID	The Identifier for the analysis run. You should provide the same ID for the entire execution (upload, analyze, status).	yes
Content-Type	object	multipart/form-data	yes
Content-Disposition	object	Include the name and filename parameters, for example: <code>form-data; name="solution1.zip"; filename="solution1.zip"</code>	yes

## Expected responses

HTTP status code	Scenario	Result
200	Upload was a success	No result body

HTTP status code	Scenario	Result
400	A non zip file was sent, incorrect parameters, or a file was included with a virus	No result body
413	File is too large	No result body

## Example: upload a file

This example demonstrates how a file can be uploaded that is to be analyzed.

### Request

HTTP

```
POST [Geographical URI]/api/upload
Accept: application/json
x-ms-correlation-id: 9E378E56-6F35-41E9-BF8B-C0CC88E2B832
x-ms-tenant-id: F2E60E49-CB87-4C24-8D4F-908813B22506
Content-Type: multipart/form-data
Content-Disposition: form-data; name=mySolution.zip; filename=mySolution.zip
```

### Response

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

["https://mystorage.blob.core.windows.net/solution-files/0a4cd700-d1d0-4ef8-8318-e4844cc1636c/mySolution.zip?sv=2017-11-09&sr=b&sig=xyz&se=2019-06-11T19%3A05%3A20Z&sp=rd"]
```

## See also

[Use the Power Apps checker web API](#)

[Retrieve the list of rulesets](#)

[Retrieve the list of rules](#)

[Invoke analysis](#)

[Check for analysis status](#)

# Work with solutions using the Dataverse SDK

Article • 02/07/2025

As part of your development to production lifecycle you might want to create custom automation to handle certain tasks. For example, in your DevOps project pipeline you might want to execute some custom code or script that creates a sandbox environment, imports an unmanaged solution, exports that unmanaged solution as a managed solution, and, finally, deletes the environment. You can do these operations and more by using the APIs that are available to you. Here are some examples of what you can accomplish using the [Dataverse SDK for .NET](#) and custom code.

## ⓘ Note

You can also perform these same operations using the Web API. The related actions are: [ImportSolution](#), [ExportSolution](#), [CloneAsPatch](#), and [CloneAsSolution](#).

## Create, export, or import an unmanaged solution

Learn how to perform some common solution operations by using C# code. To view the complete working C# code sample that demonstrates these types of solution operations, and more, go to [Sample: Work with solutions](#).

## Create a publisher

Every solution requires a publisher, represented by the [Publisher table](#). A publisher requires the these properties:

- A customization prefix
- A unique name
- A friendly name

## ⓘ Note

For a healthy application lifecycle management (ALM) approach, always use a custom publisher and solution, not the default solution and publisher, for deploying your customizations.

The following code sample first defines a publisher and then checks to determine whether the publisher already exists based on the unique name. If it already exists, the customization prefix might have been changed. This sample seeks to capture the current customization prefix. The `PublisherId` is also captured so that the publisher record can be deleted. If the publisher isn't found, a new publisher is created using the `IOrganizationService.Create` method.

C#

```
// Define a new publisher
Publisher _myPublisher = new Publisher
{
 UniqueName = "contoso-publisher",
 FriendlyName = "Contoso publisher",
 SupportingWebsiteUrl =
 "https://learn.microsoft.com/powerapps/developer/data-
platform/overview",
 CustomizationPrefix = "contoso",
 EmailAddress = "someone@contoso.com",
 Description = "This publisher was created from sample code"
};

// Does the publisher already exist?
QueryExpression querySamplePublisher = new QueryExpression
{
 EntityName = Publisher.EntityLogicalName,
 ColumnSet = new ColumnSet("publisherid", "customizationprefix"),
 Criteria = new FilterExpression()
};

querySamplePublisher.Criteria.AddCondition("uniquename",
 ConditionOperator.Equal,
 _myPublisher.UniqueName);

EntityCollection querySamplePublisherResults =
 _serviceProxy.RetrieveMultiple(querySamplePublisher);

Publisher SamplePublisherResults = null;

// If the publisher already exists, use it
if (querySamplePublisherResults.Entities.Count > 0)
{
 SamplePublisherResults =
 (Publisher)querySamplePublisherResults.Entities[0];
 _publisherId = (Guid)SamplePublisherResults.PublisherId;
 _customizationPrefix = SamplePublisherResults.CustomizationPrefix;
}
```



```

}

// If the publisher doesn't exist, create it
if (SamplePublisherResults == null)
{
 _publisherId = _serviceProxy.Create(_myPublisher);

 Console.WriteLine(String.Format("Created publisher: {0}.",
 _myPublisher.FriendlyName));

 _customizationPrefix = _myPublisher.CustomizationPrefix;
}

```

## Create an unmanaged solution

After you have a custom publisher available, you can then create an unmanaged solution. The following table lists the columns with descriptions that a solution contains.

[Expand table](#)

Column Label	Description
Display Name	The name for the solution.
Name	Microsoft Dataverse generates a unique name based on the <b>Display Name</b> . You can edit the unique name. The unique name must only contain alphanumeric characters or the underscore character.
Publisher	Use the <b>Publisher</b> lookup to associate the solution with a publisher.
Version	Specify a version by using the format: <i>major.minor.build.revision</i> , for example, 1.0.0.0.
Configuration Page	If you include an HTML Web resource in your solution, you can use this lookup to add it as your designated solution configuration page.
Description	Use this column to include any relevant details about your solution.

Here's sample code to create an unmanaged solution that uses the publisher we created in the previous section.

```

C#

// Create a solution
Solution solution = new Solution
{
 UniqueName = "sample-solution",
 FriendlyName = "Sample solution",
 PublisherId = new EntityReference(Publisher.EntityLogicalName,

```

```

_publisherId),
 Description = "This solution was created by sample code.",
 Version = "1.0"
};

// Check whether the solution already exists
QueryExpression queryCheckForSampleSolution = new QueryExpression
{
 EntityName = Solution.EntityLogicalName,
 ColumnSet = new ColumnSet(),
 Criteria = new FilterExpression()
};

queryCheckForSampleSolution.Criteria.AddCondition("uniquename",
 ConditionOperator.Equal, solution.UniqueName);

// Attempt to retrieve the solution
EntityCollection querySampleSolutionResults =
 _serviceProxy.RetrieveMultiple(queryCheckForSampleSolution);

// Create the solution if it doesn't already exist
Solution SampleSolutionResults = null;

if (querySampleSolutionResults.Entities.Count > 0)
{
 SampleSolutionResults = (Solution)querySampleSolutionResults.Entities[0];
 _solutionsSampleSolutionId = (Guid)SampleSolutionResults.SolutionId;
}

if (SampleSolutionResults == null)
{
 _solutionsSampleSolutionId = _serviceProxy.Create(solution);
}

```

After you create an unmanaged solution, add solution components by creating them in the context of this solution or by adding existing components from other solutions.

More information: [Add a new solution component](#) and [Add an existing solution component](#)

## Export an unmanaged solution

This code sample shows how to export an unmanaged solution or package a managed solution. The code uses the [ExportSolutionRequest](#) class to export a compressed file representing an unmanaged solution. The option to create a managed solution is set by using the [Managed](#) property. This sample saves a file named *samplesolution.zip* to the output folder.

```
// Export a solution
ExportSolutionRequest exportSolutionRequest = new ExportSolutionRequest();
exportSolutionRequest.Managed = false;
exportSolutionRequest.SolutionName = solution.UniqueName;

ExportSolutionResponse exportSolutionResponse =
 (ExportSolutionResponse)_serviceProxy.Execute(exportSolutionRequest);

byte[] exportXml = exportSolutionResponse.ExportSolutionFile;
string filename = solution.UniqueName + ".zip";

File.WriteAllBytes(outputDir + filename, exportXml);

Console.WriteLine("Solution exported to {0}.", outputDir + filename);
```

## Import an unmanaged solution

Importing (or upgrading) a solution by using code is accomplished with [ImportSolutionRequest](#).

C#

```
// Install or upgrade a solution
byte[] fileBytes = File.ReadAllBytes(ManagedSolutionLocation);

ImportSolutionRequest impSolReq = new ImportSolutionRequest()
{
 CustomizationFile = fileBytes
};

_serviceProxy.Execute(impSolReq);
```

## Tracking import success

You can use the `ImportJob` table to capture data about the success of the solution import. When you specify an `ImportJobId` for the [ImportSolutionRequest](#), you can use that value to query the `ImportJob` table about the status of the import. The `ImportJobId` can also be used to download an import log file using the [RetrieveFormattedImportJobResultsRequest](#) message.

C#

```
// Monitor solution import success
byte[] fileBytesWithMonitoring = File.ReadAllBytes(ManagedSolutionLocation);
```

```

ImportSolutionRequest impSolReqWithMonitoring = new ImportSolutionRequest()
{
 CustomizationFile = fileBytes,
 ImportJobId = Guid.NewGuid()
};

_serviceProxy.Execute(impSolReqWithMonitoring);

ImportJob job =
(ImportJob)_serviceProxy.Retrieve(ImportJob.EntityLogicalName,
 impSolReqWithMonitoring.ImportJobId, new ColumnSet(new System.String[] {
"data",
"solutionname" }));

System.Xml.XmlDocument doc = new System.Xml.XmlDocument();
doc.LoadXml(job.Data);

String ImportedSolutionName =
 doc.SelectSingleNode("//solutionManifest/UniqueName").InnerText;

String SolutionImportResult =
 doc.SelectSingleNode("//solutionManifest/result/\\@result").Value;

Console.WriteLine("Report from the ImportJob data");

Console.WriteLine("Solution Unique name: {0}", ImportedSolutionName);

Console.WriteLine("Solution Import Result: {0}", SolutionImportResult);

Console.WriteLine("");

// This code displays the results for Global Option sets installed as part
// of a
// solution.

System.Xml.XmlNodeList optionSets =
doc.SelectNodes("//optionSets/optionSet");

foreach (System.Xml.XmlNode node in optionSets)
{
 string OptionSetName = node.Attributes["LocalizedName"].Value;
 string result = node.FirstChild.Attributes["result"].Value;

 if (result == "success")
 {
 Console.WriteLine("{0} result: {1}", OptionSetName, result);
 }
 else
 {
 string errorCode = node.FirstChild.Attributes["errorcode"].Value;
 string errorText = node.FirstChild.Attributes["errortext"].Value;

 Console.WriteLine("{0} result: {1} Code: {2} Description:
{3}", OptionSetName,
 result, errorCode, errorText);
 }
}

```

```
}
}
```

The content of the `Data` property is a string representing the solution XML file.

## Add and remove solution components

Learn how to add and remove solution components using code.

### Add a new solution component

This sample shows how to create a solution component that is associated with a specific solution. If you don't associate the solution component to a specific solution when it's created it will only be added to the default solution and you must add it to a solution manually or by using the code included in the [Add an existing solution component](#).

This code creates a new global option set and adds it to the solution with a unique name equal to `_primarySolutionName`.

C#

```
OptionSetMetadata optionSetMetadata = new OptionSetMetadata()
{
 Name = _globalOptionSetName,
 DisplayName = new Label("Example Option Set", _languageCode),
 IsGlobal = true,
 OptionSetType = OptionSetType.Picklist,
 Options =
 {
 new OptionMetadata(new Label("Option 1", _languageCode), 1),
 new OptionMetadata(new Label("Option 2", _languageCode), 2)
 }
};
CreateOptionSetRequest createOptionSetRequest = new CreateOptionSetRequest
{
 OptionSet = optionSetMetadata
};

createOptionSetRequest.SolutionUniqueName = _primarySolutionName;
_serviceProxy.Execute(createOptionSetRequest);
```

### Add an existing solution component

This sample shows how to add an existing solution component to a solution.

The following code uses the [AddSolutionComponentRequest](#) to add the `Account` table as a solution component to an unmanaged solution.

C#

```
// Add an existing Solution Component
// Add the Account entity to the solution
RetrieveEntityRequest retrieveForAddAccountRequest = new
RetrieveEntityRequest()
{
 LogicalName = Account.EntityLogicalName
};
RetrieveEntityResponse retrieveForAddAccountResponse =
(RetrieveEntityResponse)_serviceProxy.Execute(retrieveForAddAccountRequest);
AddSolutionComponentRequest addReq = new AddSolutionComponentRequest()
{
 ComponentType = (int)componenttype.Entity,
 ComponentId =
(Guid)retrieveForAddAccountResponse.EntityMetadata.MetadataId,
 SolutionUniqueName = solution.UniqueName
};
_serviceProxy.Execute(addReq);
```

## Remove a solution component

This sample shows how to remove a solution component from an unmanaged solution. The following code uses the [RemoveSolutionComponentRequest](#) to remove a table solution component from an unmanaged solution. The `solution.UniqueName` references the solution created in [Create an unmanaged solution](#).

C#

```
// Remove a Solution Component
// Remove the Account entity from the solution
RetrieveEntityRequest retrieveForRemoveAccountRequest = new
RetrieveEntityRequest()
{
 LogicalName = Account.EntityLogicalName
};
RetrieveEntityResponse retrieveForRemoveAccountResponse =
(RetrieveEntityResponse)_serviceProxy.Execute(retrieveForRemoveAccountRequest);

RemoveSolutionComponentRequest removeReq = new
RemoveSolutionComponentRequest()
{
 ComponentId =
(Guid)retrieveForRemoveAccountResponse.EntityMetadata.MetadataId,
 ComponentType = (int)componenttype.Entity,
 SolutionUniqueName = solution.UniqueName
};
```

```
};
_serviceProxy.Execute(removeReq);
```

## Delete a solution

The following sample shows how to retrieve a solution using the solution `uniquename` and then extract the `solutionid` from the results. The sample then uses the `solutionid` with the [IOrganizationService.Delete](#) method to delete the solution.

```
C#

// Delete a solution

QueryExpression queryImportedSolution = new QueryExpression
{
 EntityName = Solution.EntityLogicalName,
 ColumnSet = new ColumnSet(new string[] { "solutionid", "friendlyname"
}),
 Criteria = new FilterExpression()
};

queryImportedSolution.Criteria.AddCondition("uniquename",
ConditionOperator.Equal, ImportedSolutionName);

Solution ImportedSolution =
(Solution)_serviceProxy.RetrieveMultiple(queryImportedSolution).Entities[0];

_serviceProxy.Delete(Solution.EntityLogicalName,
(Guid)ImportedSolution.SolutionId);

Console.WriteLine("Deleted the {0} solution.",
ImportedSolution.FriendlyName);
```

## Cloning, patching, and upgrading

You can perform additional solution operations by using the available APIs. For cloning and patching solutions, use the [CloneAsPatchRequest](#) and [CloneAsSolutionRequest](#). For information about cloning and patching, see [Create solution patches](#).

When performing solution upgrades use the [StageAndUpgradeRequest](#) and [DeleteAndPromoteRequest](#). For more information about the process of staging and upgrades, go to [Upgrade or update a solution](#).

## Detect solution dependencies

This sample shows how to create a report showing the dependencies between solution components.

This code performs these operations:

- Retrieve all the components for a solution.
- Retrieve all the dependencies for each component.
- For each dependency found display a report describing the dependency.

C#

```
// Grab all Solution Components for a solution.
QueryByAttribute componentQuery = new QueryByAttribute
{
 EntityName = SolutionComponent.EntityLogicalName,
 ColumnSet = new ColumnSet("componenttype", "objectid",
 "solutioncomponentid", "solutionid"),
 Attributes = { "solutionid" },

 // In your code, this value would probably come from another query.
 Values = { _primarySolutionId }
};

IEnumerable<SolutionComponent> allComponents =

_serviceProxy.RetrieveMultiple(componentQuery).Entities.Cast<SolutionComponent>();

foreach (SolutionComponent component in allComponents)
{
 // For each solution component, retrieve all dependencies for the
 component.
 RetrieveDependentComponentsRequest dependentComponentsRequest =
 new RetrieveDependentComponentsRequest
 {
 ComponentType = component.ComponentType.Value,
 ObjectId = component.ObjectId.Value
 };
 RetrieveDependentComponentsResponse dependentComponentsResponse =
 (RetrieveDependentComponentsResponse)_serviceProxy.Execute(dependentComponentsRequest);

 // If there are no dependent components, we can ignore this component.
 if (dependentComponentsResponse.EntityCollection.Entities.Any() ==
 false)
 continue;

 // If there are dependencies upon this solution component, and the
 solution
 // itself is managed, then you will be unable to delete the solution.
```



```

 Console.WriteLine("Found {0} dependencies for Component {1} of type
{2}",
 dependentComponentsResponse.EntityCollection.Entities.Count,
 component.ObjectId.Value,
 component.ComponentType.Value
);
 //A more complete report requires more code
 foreach (Dependency d in
dependentComponentsResponse.EntityCollection.Entities)
 {
 DependencyReport(d);
 }
 }
}

```

The `DependencyReport` method is in the following code sample.

## Dependency report

The `DependencyReport` method provides a friendlier message based on information found within the dependency.

### ⓘ Note

In this sample the method is only partially implemented. It can display messages only for attribute and option set solution components.

C#

```

/// <summary>
/// Shows how to get a more friendly message based on information within the
dependency
/// <param name="dependency">A Dependency returned from the
RetrieveDependentComponents message</param>
/// </summary>
public void DependencyReport(Dependency dependency)
{
 // These strings represent parameters for the message.
 String dependentComponentName = "";
 String dependentComponentTypeName = "";
 String dependentComponentSolutionName = "";
 String requiredComponentName = "";
 String requiredComponentTypeName = "";
 String requiredComponentSolutionName = "";

 // The ComponentType global Option Set contains options for each possible
component.
 RetrieveOptionSetRequest componentTypeRequest = new
RetrieveOptionSetRequest
 {

```

```

 Name = "componenttype"
 };

 RetrieveOptionSetResponse componentTypeResponse =
 (RetrieveOptionSetResponse)_serviceProxy.Execute(componentTypeRequest);
 OptionSetMetadata componentTypeOptionSet =
 (OptionSetMetadata)componentTypeResponse.OptionSetMetadata;
 // Match the Component type with the option value and get the label value
 of the option.
 foreach (OptionMetadata opt in componentTypeOptionSet.Options)
 {
 if (dependency.DependentComponentType.Value == opt.Value)
 {
 dependentComponentTypeName = opt.Label.UserLocalizedLabel.Label;
 }
 if (dependency.RequiredComponentType.Value == opt.Value)
 {
 requiredComponentTypeName = opt.Label.UserLocalizedLabel.Label;
 }
 }
 // The name or display name of the component is retrieved in different ways
 depending on the component type
 dependentComponentName =
 GetComponentName(dependency.DependentComponentType.Value,
 (Guid)dependency.DependentComponentObjectId);
 requiredComponentName =
 GetComponentName(dependency.RequiredComponentType.Value,
 (Guid)dependency.RequiredComponentObjectId);

 // Retrieve the friendly name for the dependent solution.
 Solution dependentSolution = (Solution)_serviceProxy.Retrieve
 (
 Solution.EntityLogicalName,
 (Guid)dependency.DependentComponentBaseSolutionId,
 new ColumnSet("friendlyname")
);
 dependentComponentSolutionName = dependentSolution.FriendlyName;

 // Retrieve the friendly name for the required solution.
 Solution requiredSolution = (Solution)_serviceProxy.Retrieve
 (
 Solution.EntityLogicalName,
 (Guid)dependency.RequiredComponentBaseSolutionId,
 new ColumnSet("friendlyname")
);
 requiredComponentSolutionName = requiredSolution.FriendlyName;

 // Display the message
 Console.WriteLine("The {0} {1} in the {2} depends on the {3} {4} in the
 {5} solution.",
 dependentComponentName,
 dependentComponentTypeName,
 dependentComponentSolutionName,
 requiredComponentName,
 requiredComponentTypeName,

```

```
 requiredComponentSolutionName);
 }
```

## Detect whether a solution component can be deleted

Use the [RetrieveDependenciesForDeleteRequest](#) message to identify any other solution components which would prevent a given solution component from being deleted. The following code sample looks for any attributes using a known global choice column. Any attribute using the global choice would prevent the global choice from being deleted.

```
C#

// Use the RetrieveOptionSetRequest message to retrieve
// a global option set by it's name.
RetrieveOptionSetRequest retrieveOptionSetRequest =
 new RetrieveOptionSetRequest
 {
 Name = _globalOptionSetName
 };

// Execute the request.
RetrieveOptionSetResponse retrieveOptionSetResponse =
 (RetrieveOptionSetResponse)_serviceProxy.Execute(
 retrieveOptionSetRequest);
_globalOptionSetId = retrieveOptionSetResponse.OptionSetMetadata.MetadataId;
if (_globalOptionSetId != null)
{
 // Use the global OptionSet MetadataId with the appropriate componenttype
 // to call RetrieveDependenciesForDeleteRequest
 RetrieveDependenciesForDeleteRequest retrieveDependenciesForDeleteRequest =
 new RetrieveDependenciesForDeleteRequest
 {
 ComponentType = (int)componenttype.OptionSet,
 ObjectId = (Guid)_globalOptionSetId
 };

 RetrieveDependenciesForDeleteResponse retrieveDependenciesForDeleteResponse
 =

 (RetrieveDependenciesForDeleteResponse)_serviceProxy.Execute(retrieveDepende
nciesForDeleteRequest);
 Console.WriteLine("");
 foreach (Dependency d in
retrieveDependenciesForDeleteResponse.EntityCollection.Entities)
 {

 if (d.DependentComponentType.Value == 2)//Just testing for Attributes
 {
 String attributeLabel = "";
 RetrieveAttributeRequest retrieveAttributeRequest = new
RetrieveAttributeRequest
```

```
{
 MetadataId = (Guid)d.DependentComponentObjectId
};
RetrieveAttributeResponse retrieveAttributeResponse =
(RetrieveAttributeResponse)_serviceProxy.Execute(retrieveAttributeRequest);

AttributeMetadata attmet = retrieveAttributeResponse.AttributeMetadata;

attributeLabel = attmet.DisplayName.UserLocalizedLabel.Label;

 Console.WriteLine("An {0} named {1} will prevent deleting the {2} global
option set.",
 (componenttype)d.DependentComponentType.Value,
 attributeLabel,
 _globalOptionSetName);
}
}
}
```

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# Solution staging, with asynchronous import and export

Article • 06/30/2022

Have you ever run into the situation during the import or export of a large solution where the operation times out? If so, you may be a candidate for performing the solution import/export asynchronously. This topic describes how to initiate the asynchronous import or export using the [Dataverse SDK for .NET](#) and Web APIs.

## Staging a solution

In comparison to importing a solution where the solution is imported and available in the environment right away, staging breaks the import process into more controllable phases. The staging process imports the solution as a "holding" solution where the administrator can decide when to make the staged solution available to users, or to perform an upgrade (in the case of a solution upgrade) in the target environment. Part of the staging process is validation of the staged solution. In this way you can stage the solution, know that the solution is valid, and schedule when to apply that solution or upgrade to the target environment.

Operation	Web API	Dataverse SDK for .NET
Stage a solution	<a href="#">StageSolution</a>	<a href="#">StageSolutionRequest</a>

The result of staging the solution will be a collection of validation results indicating success or failure and (if successful) a `StageSolutionUploadId` to be used in the `ImportSolutionAsync` call. See the import solution Web API sample code above for an example of how this is done.

C#

```
public static StageSolutionResults StageSolution(
 IOrganizationService service,
 string solutionFilePath)
{
 // Stage the solution
 var req = new StageSolutionRequest();

 byte[] fileBytes = File.ReadAllBytes(solutionFilePath);
 req["CustomizationFile"] = fileBytes;
 var res = service.Execute(req);
}
```

```

return (res["StageSolutionResults"] as StageSolutionResults);
}

```

## Solution import

`ImportSolution` is the action (or message) that performs the synchronous import operation. To execute the import operation asynchronously use `ImportSolutionAsync`.

Operation	Web API	Dataverse SDK for .NET
Import a solution	<a href="#">ImportSolutionAsync</a>	<a href="#">ImportSolutionAsyncRequest</a>

Now let's take a look at some example code that demonstrates `ImportSolutionAsync`.

C#

```

public static ImportSolutionAsyncResponse ImportSolution(
 IOrganizationService service,
 StageSolutionResults stagingResults,
 Dictionary<string, Guid> connectionIds,
 Dictionary<string, string> envVarValues)
{
 // Import the staged solution
 var componentDetails = stagingResults.SolutionComponentsDetails;

 // TODO These are not referenced in the code but are usefull to explore
 var missingDependencies = stagingResults.MissingDependencies; //
 Contains missing dependencies
 var solutionDetails = stagingResults.SolutionDetails; //
 Contains solution details

 var connectionReferences = componentDetails.Where(x =>
string.Equals(x.ComponentTypeName, "connectionreference"));
 var envVarDef = componentDetails.Where(x =>
string.Equals(x.ComponentTypeName, "environmentvariabledefinition"));
 var envVarValue = componentDetails.Where(x =>
string.Equals(x.ComponentTypeName, "environmentvariablevalue"));

 var componentParams = new EntityCollection();

 // Add each connection reference to the component parmameters entity
 collection.
 foreach (var conn in connectionReferences)
 {
 var e = new Entity("connectionreference")
 {
 ["connectionreferencelogicalname"] =
conn.Attributes["connectionreferencelogicalname"].ToString(),
 ["connectionreferencedisplayname"] =
conn.Attributes["connectionreferencedisplayname"].ToString(),

```

```

 ["connectorid"] = conn.Attributes["connectorid"].ToString(),
 ["connectionid"] = connectionIds[conn.ComponentName]
 };
 componentParams.Entities.Add(e);
}

// Add each environment variable to the component parmameters entity
collection.
foreach (var value in envVarValue)
{
 var e = new Entity("environmentvariablevalue")
 {
 ["schemaname"] = value.Attributes["schemaname"].ToString(),
 ["value"] = envVarValues[value.ComponentName]
 };

 if (value.Attributes.ContainsKey("environmentvariablevalueid"))
 {
 e["environmentvariablevalueid"] =
value.Attributes["environmentvariablevalueid"].ToString();
 }
 componentParams.Entities.Add(e);
}

// Import the solution
var importSolutionReq = new ImportSolutionAsyncRequest();
importSolutionReq.ComponentParameters = componentParams;
importSolutionReq.SolutionParameters = new SolutionParameters {
StageSolutionUploadId = stagingResults.StageSolutionUploadId };
var response = service.Execute(importSolutionReq) as
ImportSolutionAsyncResponse;

return (response);
}

```

`ImportSolutionAsync` shares many input parameters with `ImportSolution` but adds `ComponentParameters` and `SolutionParameters`. `ComponentParameters` can be used to overwrite the component data in the solution's customization XML file. `SolutionParameters` can be used to pass the `StageSolutionUploadId` of a staged solution as was shown in the example Web API code. More information: [Staging a solution](#)

The response returned from `ImportSolutionAsync` contains `ImportJobKey` and `AsyncOperationId`. The `ImportJobKey` value can be used to obtain the import result and the `AsyncOperationId` value can be used to track the import job status.

C#

```

public static void CheckImportStatus(
 IOrganizationService service,
 Guid asyncOperationId,

```

```

 Guid importJobKey)
{
 // Get solution import status
 var finished = false;
 Entity asyncOperation = null;
 // Wait until the async job is finished
 while (!finished)
 {
 asyncOperation = service.Retrieve("asyncoperation",
asyncOperationId, new ColumnSet("statecode", "statuscode"));
 OptionSetValue statecode =
 (OptionSetValue)asyncOperation["statecode"];
 if (statecode.Value == 3)
 {
 finished = true;
 }
 else
 {
 Thread.Sleep(10000);
 }
 }
 // Solution import completed successfully
 OptionSetValue statuscode =
 (OptionSetValue)asyncOperation["statuscode"];
 if (statuscode.Value == 30)
 {
 Console.WriteLine("The solution import completed successfully.");
 }
 else if (asyncOperation["statuscode"].ToString() == "31") // Solution
import failed
 {
 Console.WriteLine("The solution import failed.");
 var getLogReq = new RetrieveFormattedImportJobResultsRequest {
ImportJobId = importJobKey };
 var importJob = service.Execute(getLogReq) as
RetrieveFormattedImportJobResultsResponse;
 // TODO Do something with the import job results
 }
}

```

## Solution export

`ExportSolution` is the action (or message) that performs the synchronous export operation. To execute the export operation asynchronously use `ExportSolutionAsync`.

Operation	Web API	Dataverse SDK for .NET
Export a solution	<a href="#">ExportSolutionAsync</a>	<a href="#">ExportSolutionAsyncRequest</a>



Operation	Web API	Dataverse SDK for .NET
Download an exported solution file	<a href="#">DownloadSolutionExportData</a>	<a href="#">DownloadSolutionExportDataRequest</a>

Now let's take a look at some example code that demonstrates `ExportSolutionAsync`.

C#

```
// Where 'service' is a pre-configured Organization service instance.
var service = (OrganizationServiceProxy)xsc.CreateOrganizationService();

var req = new OrganizationRequest("ExportSolutionAsync");
req.Parameters.Add("SolutionName", "ExportSolutionAsyncTest");
req.Parameters.Add("Managed", false);
var response = service.Execute(req);
```

In the response are the `AsyncOperationId` and `ExportJobId` parameter values. Use the `AsyncOperationId` in the response to verify the success (`statecode == 3; statuscode == 30`) of the asynchronous job. Next, use the `DownloadSolutionExportData` action (or message) with the `ExportJobId` value from the export response to download the exported solution file, which is returned in the `ExportSolutionFile` parameter.

C#

```
// Where 'service' is a pre-configured Organization service instance.
var service = (OrganizationServiceProxy)xsc.CreateOrganizationService();

var req = new OrganizationRequest("DownloadSolutionExportData");
req.Parameters.Add("ExportJobId", Guid.Parse("a9089b53-a1c7-ea11-a813-000d3a14420d"));
var response = service.Execute(req);
```

## See Also

[Sample: Solution staging with asynchronous import](#)

# Create patches to simplify solution updates

Article • 02/07/2025

If you add a table to a solution and export the solution, the table and all of its related assets are exported in that solution. These assets include columns (attributes), forms, views, relationships, and visualizations, and any other assets that are packaged with the table. Exporting all objects means that you can unintentionally modify objects on the target deployment, or carry over unintended dependencies.

To address dependencies, you can create and publish solution patches that contain subcomponents of tables rather than publishing the entire table and all of its assets. The original solution and one or more related patches can be rolled up (merged) at a later time into an updated version of the solution, which then can replace the original solution in the target Microsoft Dataverse organization.

## Patches

You can apply patches to either managed or unmanaged solutions and include only changes to tables and related table assets. Patches don't contain any noncustomized system components or relationships that it depends upon because these components already exist in the deployed-to organization. At some point in your development cycle, you can roll up all the patches into a new solution version to replace the original solution that the patches were created from.

Patches are stored in the Dataverse database as `Solution` table records. A non-null `ParentSolutionId` attribute indicates that the solution is a patch. Patches can be created and managed through the SDK for .NET or Web APIs, which are useful for developing automation such as a product install script. However, the Dataverse web application provides various web forms that enable you to interactively create and manage patches.

- Patches can only be created from a parent solution using [CloneAsPatchRequest](#) or [CloneAsPatch Action](#).
- The patch parent can't be a patch.
- Patches can only have one parent solution.
- A patch creates a dependency, at the solution level, on its parent solution.
- You can only install a patch if the parent solution is present.
- You can't install a patch unless the unique name and major or minor version number of the parent solution, as identified by `ParentSolutionId`, don't match

those of the parent solution installed in the target organization.

- A patch version must have the same major and minor number, but a higher build and release number, than the parent solution version number. The display name can be different.
- If a solution has patches, subsequent patches must have a numerically higher version number than any existing patch for that solution.
- Patches support the same operations as solutions, such as additive update, but not removal. You can't remove components from a solution using a patch. To remove components from a solution perform an upgrade.
- Patches exported as managed must be imported on top of a managed parent solution. The rule is that patch protection (managed or unmanaged) must match its parent.
- Don't use unmanaged patches for production purposes.

The SolutionPackager and PackageDeployer tools support solution patches. Refer to the tool's online help for any command-line options that are related to patches.

## Create a patch

Create a patch from an unmanaged solution in an environment by using the [CloneAsPatchRequest](#) message or the [CloneAsPatch Action](#), or by using the web application. Once you create the patch, the original solution becomes locked and you can't change or export it as long as there are dependent patches that exist in the environment that identify the solution as the parent solution. Patch versioning is similar to solution versioning and specified in the following format: *major.minor.build.release*. You can't make changes to the existing major or minor solution versions when you create a patch.

## Export and import a patch

Use the SDK for .NET or Web APIs, the web application, or the Package Deployer tool to export and import a patch. The relevant SDK for .NET request classes are [ImportSolutionRequest](#) and [ExportSolutionRequest](#). The relevant actions For the Web API are [ImportSolution Action](#) and [ExportSolution Action](#).

## Patching examples

The following table lists the details of a patching example. In this example, the solution and patches are imported in numerical order and are additive, which is consistent with solution import in general.

[Expand table](#)

Patch name	Description
<code>SolutionA</code> , version 1.0 (unmanaged)	Contains <code>entityA</code> with six fields.
<code>SolutionA</code> , version 1.0.1.0 (unmanaged)	Contains <code>entityA</code> with six fields (three updated), and adds <code>entityB</code> with 10 fields.
<code>SolutionA</code> , version 1.0.2.0 (unmanaged)	Contains <code>entityC</code> with 10 fields.

The import process is as follows.

1. The developer or customizer first imports the base solution ( `SolutionA` 1.0) into the environment. The result is `entityA` with six fields in the environment.
2. Next, the `SolutionA` patch 1.0.1.0 is imported. The environment now contains `entityA` with six fields (three have been updated), plus `entityB` with 10 fields.
3. Finally, `SolutionA` patch 1.0.2.0 is imported. The environment now contains `entityA` with six fields (three updated), `entityB` with 10 fields, plus `entityC` with 10 fields.

## Another patching example

Let's take a look at another patching example, with the details listed in the following table.

[Expand table](#)

Patch name	Description
<code>SolutionA</code> , version 1.0 (unmanaged, base solution)	Contains the <code>Account</code> table where the length of the account number column is adjusted from 20 to 30 characters.
<code>SolutionB</code> , version 2.0 (unmanaged, different vendor)	Contains the <code>Account</code> table where the length of the account number column is adjusted to 50 characters.
<code>SolutionA</code> , version 1.0.1.0 (unmanaged, patch)	Contains an update to the <code>Account</code> table where the length of the account number column is adjusted to 35 characters.

The import process is as follows:

1. The developer or customizer first imports the base solution ( `SolutionA` 1.0) into the environment. The result is an `Account` table with an account number column of

30 characters.

2. `SolutionB` is imported. The environment now contains an `Account` table with an account number column of 50 characters.
3. `SolutionA` patch 1.0.1.0 is imported. The environment still contains an `Account` table with an account number column of 50 characters, as applied by `SolutionB`.
4. `SolutionB` is uninstalled. The environment now contains an `Account` table with an account number column of 35 characters, as applied by the `SolutionA` 1.0.1.0 patch.

## Delete a patch

You can delete a patch or base (parent) solution by using [DeleteRequest](#) or, for the Web API, use the `HTTP DELETE` method. The delete process is different for a managed or unmanaged solution that has one or more patches existing in the environment.

For an unmanaged solution, you must uninstall all patches to the base solution first, in reverse version order that they were created, before uninstalling the base solution.

For a managed solution, you simply uninstall the base solution. The Dataverse system automatically uninstalls the patches in reverse version order before uninstalling the base solution. You can also just uninstall a single patch.

## Update a solution

Updating a solution involves rolling up (merging) all patches to that solution into a new version of the solution. Afterwards, that solution becomes unlocked and can once again be modified (unmanaged solution only) or exported. For a managed solution, no further modifications of the solution are allowed except for creating patches from the newly updated solution. To rollup patches into an unmanaged solution, use [CloneAsSolutionRequest](#) or the [CloneAsSolution Action](#). Cloning a solution creates a new version of the unmanaged solution, incorporating all its patches, with a higher *major.minor* version number, the same unique name, and a display name.

For a managed solution things are handled slightly different. You first clone the unmanaged solution (A), incorporating all of its patches and then exporting it as a managed solution (B). In the target organization that contains the managed version of the (A) solution and its patches, you import managed solution (B) and then execute [DeleteAndPromoteRequest](#) or the [DeleteAndPromote Action](#) to replace managed solution (A) and its patches with the upgraded managed solution (B) that has a higher version number.

## See also

[Use segmented solutions](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# When to edit the customizations file

Article • 05/23/2023

The customizations.xml file that is exported as part of an unmanaged solution can be edited to perform specific customization tasks. After editing the file, you can compress the modified file together with the other files exported in the unmanaged solution. You apply the changes by importing that modified unmanaged solution.

Editing a complex XML file like the customizations.xml file is easier and less prone to errors if you use a program designed to support schema validation. While it's possible to edit this file using a simple text editor like Notepad, editing this way isn't recommended unless you're familiar with editing this file. For more information, see [Edit the Customizations file with Schema Validation](#).

## Important

Invalid XML or incorrect definition of solution components can cause errors that will prevent importing a manually edited unmanaged solution.

## Supported tasks

You can edit the customization.xml file to perform the following tasks.

### Editing the ribbon

This documentation describes the process of editing the ribbon by editing the customization.xml file directly. Several people have created ribbon editors that provide a user interface to make editing the ribbon easier. The most popular one so far is the [Ribbon Workbench](#) . For support using this program, contact the program publisher.

For more information about editing the ribbon by editing the customization.xml manually, see [Customize commands and the ribbon](#).

### Editing the SiteMap

The SDK describes the process of editing the SiteMap by editing the customization.xml file directly. However, it's recommended that you use the site map designer in Microsoft Dataverse to create or update site maps. More information: [Create a site map for an app using the site map designer](#)

You can also use one of the community-developed site map editors, such as the [XrmToolBox Site Map Editor](#) .

For more information, see [Change Application Navigation using the SiteMap](#)

### Editing FormXml

FormXml is used to define entity forms and dashboards. The form editor and dashboard designer in the application are the most commonly used tools for this purpose. Editing the customizations.xml file is an alternative method. For more information, see [Customize entity forms](#) and [Create a Dashboard](#).

### Editing saved queries

Definitions of views for entities are included in the customizations.xml file and may be manually edited. The view editor in the application is the most commonly used tool for this purpose. Editing customizations.xml is an alternative method. For more information, see [Customize entity views](#).

### Editing the ISV.config

In earlier versions of Dynamics 365 Dataverse, ISV.Config was the way to add client application extensions and some other configuration options. For Microsoft Dynamics CRM 2011 and Microsoft Dynamics 365 Online, the Ribbon provides the way to extend the application. The only remaining capability left in ISV.Config is to customize the appearance of the Service Calendar. For more information, see [Service Calendar Appearance Configuration](#)

## Unsupported tasks

Defining any other solution components by editing the exported customizations.xml file isn't supported. Other solution components include, *but aren't limited to*, the following:

- Entities
- Attributes
- Entity Relationships
- Entity Messages
- Option Sets
- Web Resources
- Processes (Workflows)
- Plugin Assemblies
- SDK Message Processing steps
- Service Endpoints
- Reports
- Connection Roles
- Article Templates
- Contract Templates
- E-mail Templates



- [Mail Merge Templates](#)
- [Security Roles](#)
- [Field Security Profiles](#)

## See also

[Customization XML Reference](#)

[Customization Solutions File Schema](#)

[Ribbon core schema](#)

[Ribbon types schema](#)

[Ribbon WSS schema](#)

[SiteMap schema](#)

[Form XML schema](#)

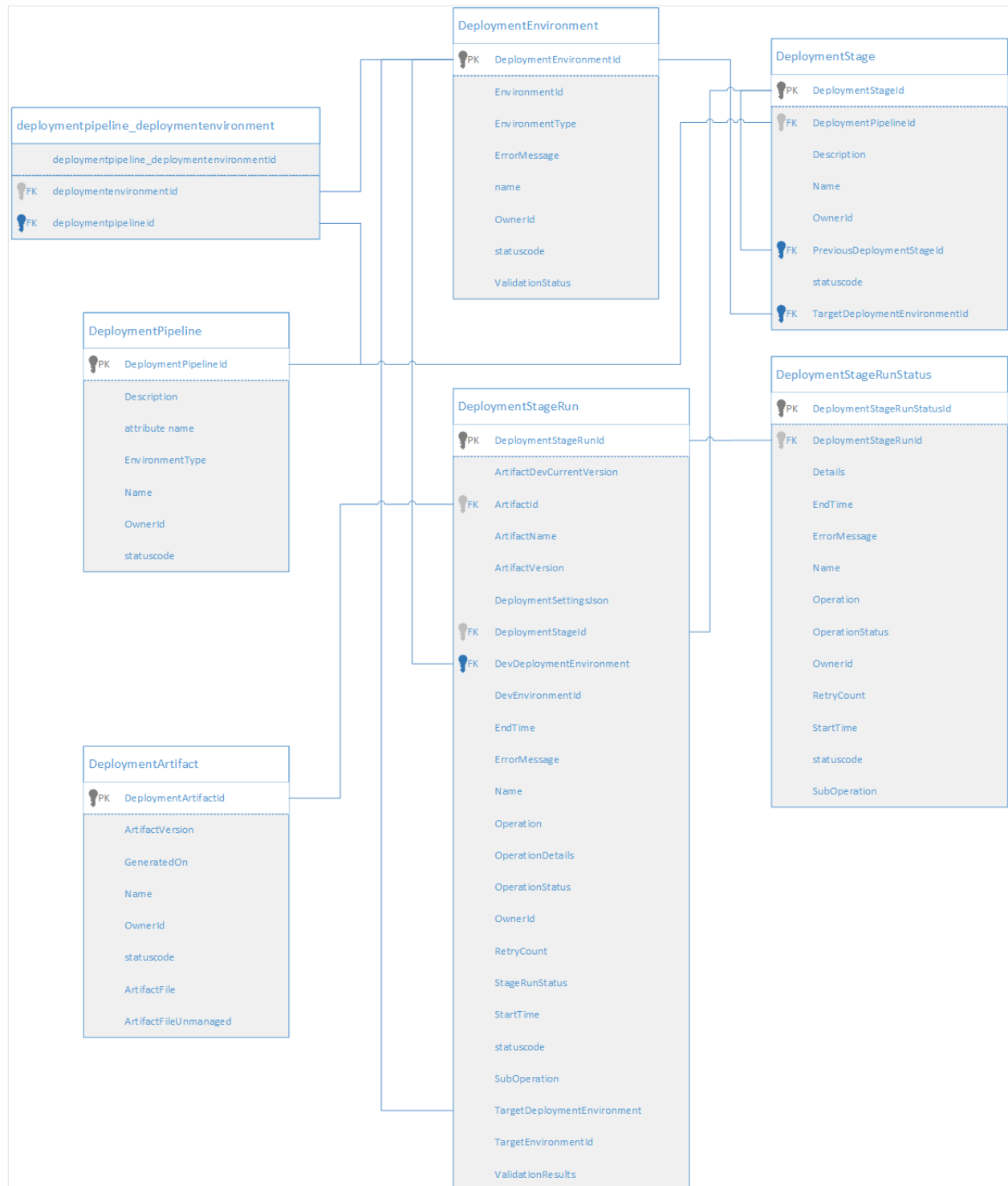
[Schema Support for the Customization File](#)

# Pipeline table reference

Article • 03/16/2023

This article contains the table definitions used in the Pipeline solution. More information:

[Overview of pipelines in Power Platform](#)



## ⓘ Note

For brevity, common columns found on all tables are not included here. The schema name for these columns are: `CreatedBy`, `CreatedOn`, `CreatedOnBehalfBy`,

ImportSequenceNumber, ModifiedBy, ModifiedOn, ModifiedOnBehalfBy, OverriddenCreatedOn, OwningBusinessUnit, OwningTeam, OwningUser, TimeZoneRuleVersionNumber, UTCConversionTimeZoneCode, VersionNumber, and statecode.

## DeploymentArtifact (Deployment Artifact)

Stores solution artifacts that are exported when running pipelines.

[Expand table](#)

SchemaName Display Name	Type	Description
ArtifactVersion Artifact Version	String	Stores the artifact version number.
DeploymentArtifactId DeploymentArtifact	Uniqueidentifier	Unique identifier for entity instances.
GeneratedOn Generated On	DateTime	Date and time when the artifact record was generated.
Name Name	String	The name of the artifact record.
OwnerId Owner	Owner	Owner ID
statuscode Status Reason	Status	Reason for the status of the Deployment Artifact. Value: 1 Label: Active Value: 2 Label: Inactive
ArtifactFile Managed Artifact File	File	Stores the managed version of the artifact. Not valid for Create
ArtifactFileUnmanaged Unmanaged Artifact File	File	Unmanaged Artifact File. Not valid for Create

## DeploymentEnvironment (Deployment Environment)

Stores the environments that are participating in pipelines and configured in the deployment stages.

 Expand table

SchemaName Display Name	Type	Description
<code>DeploymentEnvironmentId</code> Deployment Environment	Uniqueidentifier	Unique identifier for deployment environment instances
<code>EnvironmentId</code> Environment ID	String	The Environment ID of the Power Platform environment.
<code>EnvironmentType</code> Environment Type	Picklist	Indicates if the environment is used for development or as a target environment that a pipeline will deploy to. Value: <code>200000000</code> Label: Development Environment Value: <code>200000001</code> Label: Target Environment
<code>ErrorMessage</code> Error Message	String	Stores the Power Platform environment validation failure error messages.
<code>name</code> Name	String	The name of the Deployment Environment.
<code>OwnerId</code> Owner	Owner	Owner ID
<code>statusCode</code> Status Reason	Status	Reason for the status of the Deployment Environment. Value: <code>1</code> Label: Active Value: <code>2</code> Label: Inactive
<code>ValidationStatus</code> Validation Status	Picklist	Indicates if the Environment ID of the Power Platform environment has been validated. <b>Not valid for Create</b> Value: <code>200000000</code> Label: Pending Value: <code>200000001</code> Label: Success Value: <code>200000002</code> Label: Failed

## DeploymentPipeline (Deployment Pipeline)

Stores the pipeline configurations.

 Expand table

SchemaName Display Name	Type	Description
DeploymentPipelineId Deployment Pipeline	Uniqueidentifier	Unique identifier for the pipeline instances
Description Description	String	Custom description of the pipeline.
Name Name	String	The name of the pipeline record.
OwnerId Owner	Owner	Owner ID
statusCode Status Reason	Status	Reason for the status of the deployment pipeline. Value: 1 Label: Active Value: 2 Label: Inactive

## DeploymentStage (Deployment Stage)

Stores the deployment stage configuration such as target environment and prerequisite deployment stages.

 Expand table

SchemaName Display Name	Type	Description
DeploymentPipelineId Deployment Pipeline	Lookup	Lookup to the pipeline that this stage belongs to.
DeploymentStageId Deployment Stage ID	Uniqueidentifier	Unique identifier for the Deployment Stage instance.
Description Description	String	Custom description of the deployment stage instance.
Name Name	String	The name of the deployment stage instance.
OwnerId Owner	Owner	Owner ID
PreviousDeploymentStageId Previous Deployment Stage	Lookup	Lookup to the previous deployment stage configured in the pipeline. Previous deployment stages must be run successfully

SchemaName Display Name	Type	Description
		before deployments can be run for the current stage.
statusCode <b>Status Reason</b>	Status	Reason for the status of the Deployment Stage. Value: 1 Label: Active Value: 2 Label: Inactive
TargetDeploymentEnvironmentId <b>Target Deployment Environment ID</b>	Lookup	Lookup to the target deployment environment associated with this deployment stage.

## DeploymentStageRun (Deployment Stage Run)

Stores information about a deployment stage execution.

 Expand table

SchemaName Display Name	Type	Description
ArtifactDevCurrentVersion <b>Solution Artifact Current Version</b>	String	Stores the current version of the solution artifact within a development environment.
ArtifactId <b>Artifact ID</b>	Lookup	Internal use only.
ArtifactName <b>Artifact Name</b>	String	The name of the Deployment Artifact being deployed by a Stage Run instance.
ArtifactVersion <b>Solution Artifact Version</b>	String	The version number of the Deployment Artifact being deployed by a Stage Run instance.
DeploymentSettingsJson <b>Deployment Settings JSON</b>	String	Validated JSON payload storing the Connection Reference and Environment Variable information collected when a Deployment Stage is run.
DeploymentStageId <b>Deployment Stage</b>	Lookup	Lookup to the Deployment Stage associated with a Deployment Stage Run instance.
DeploymentStageRunId <b>Deployment Stage Run</b>	Uniqueidentifier	Unique identifier for Deployment Stage Run instances.

SchemaName Display Name	Type	Description
<code>DevDeploymentEnvironment</code> Development Deployment Environment ID	Lookup	Lookup to the development Deployment Environment associated with a Deployment Stage Run.
<code>DevEnvironmentId</code> Dev Environment Id	String	For internal use only.
<code>EndTime</code> End Time	DateTime	Date and time when the deployment succeeded or failed.
<code>ErrorMessage</code> Error Message	String	Stores the deployment failure messages for a stage run instance.
<code>Name</code> Name	String	The name of a Deployment Stage Run. This is auto-generated based on the stage name, the artifact name, and the artifact version number.
<code>Operation</code> Operation	Picklist	Indicates the current operation the deployment. Value: <code>200000200</code> Label: None Value: <code>200000201</code> Label: Validate Value: <code>200000202</code> Label: Deploy
<code>OperationDetails</code> Operation Details	String	Stores the details of the current deployment operation.
<code>OperationStatus</code> Operation Status	Picklist	Stores the status of the current deployment operation. Value: <code>200000000</code> Label: NotStarted Value: <code>200000001</code> Label: Started Value: <code>200000002</code> Label: Succeeded Value: <code>200000003</code> Label: Failed Value: <code>200000004</code> Label: Pending
<code>OwnerId</code> Owner	Owner	Owner ID
<code>RetryCount</code> Retry Count	Integer	For internal use only.
<code>StageRunStatus</code> Stage Run Status	Picklist	Indicates the overall status of a Deployment Stage Run. Value: <code>200000000</code> Label: NotStarted Value: <code>200000001</code> Label: Started Value: <code>200000002</code> Label: Succeeded Value: <code>200000003</code> Label: Failed

SchemaName Display Name	Type	Description
<code>StartTime</code> Start Time	DateTime	Date and time when the deployment started.
<code>statuscode</code> Status Reason	Status	Reason for the status of the Deployment Stage Run. Value: <code>1</code> Label: Active Value: <code>2</code> Label: Inactive
<code>SubOperation</code> SubOperation	Picklist	Represents the sub operation of the current deployment operation. This is managed internally. Value: <code>200000100</code> Label: None Value: <code>200000101</code> Label: Executing Pre-step Value: <code>200000102</code> Label: Retrieving Artifact Configuration Value: <code>200000103</code> Label: Validating Artifact Configuration Value: <code>200000104</code> Label: Retrieving Artifact Value: <code>200000105</code> Label: Deploying Artifact Value: <code>200000106</code> Label: Executing Post-step
<code>TargetDeploymentEnvironment</code> Target Deployment Environment ID	Lookup	Lookup to the target Deployment Environment associated with a Deployment Stage Run.
<code>TargetEnvironmentId</code> Target Environment Id	String	For internal use only.
<code>ValidationResults</code> Validation Results	String	Stores the artifact validation response.

## DeploymentStageRunStatus (Deployment Stage Run Suboperations)

Stores background operation information for a Deployment Stage Run.

[Expand table](#)

SchemaName Display Name	Type	Description
<code>DeploymentStageRunId</code> Deployment Stage Run	Lookup	Lookup to the owning Deployment Stage Run instance.



SchemaName Display Name	Type	Description
DeploymentStageRunStatusId Deployment Stage Run Suboperation ID	Uniqueidentifier	Unique identifier for Deployment Stage Run Suboperation instances.
Details Details	String	Stores information about the Deployment Stage Run Suboperation.
EndTime End Time	DateTime	Date and Time when the deployment operation succeeded or failed.
ErrorMessage Error Message	String	Stores any error message associated with the Deployment Stage Run Suboperation.
Name Name	String	The name of the Deployment Stage Run Suboperation. This is managed internally.
Operation Operation	Picklist	Stores information about the deployment operation being run. Value: 200000200 Label: None Value: 200000201 Label: Validate Value: 200000202 Label: Deploy
OperationStatus Operation Status	Picklist	Stores the status of the deployment sub operation. Value: 200000000 Label: NotStarted Value: 200000001 Label: Started Value: 200000002 Label: Succeeded Value: 200000003 Label: Failed Value: 200000004 Label: Pending
OwnerId Owner	Owner	Owner ID
RetryCount Retry Count	Integer	For internal use only.
StartTime Start Time	DateTime	Date and time when the deployment operation started.
statusCode Status Reason	Status	Reason for the status of the Deployment Stage Run Status. Value: 1 Label: Active Value: 2 Label: Inactive
SubOperation Suboperation	Picklist	Stores additional information about the background operation. Value: 200000100 Label: None Value: 200000101 Label: Executing Pre-step

SchemaName Display Name	Type	Description
		Value: 200000102 Label: Retrieving Artifact Configuration
		Value: 200000103 Label: Validating Artifact Configuration
		Value: 200000104 Label: Retrieving Artifact
		Value: 200000105 Label: Deploying Artifact
		Value: 200000106 Label: Executing Post-step

## See Also

[Overview of pipelines in Power Platform](#)

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

# Responsible AI FAQs for Microsoft Power Platform

05/29/2025

An AI system includes not only the technology, but also the people who use it, the people affected by it, and the environment in which it's deployed. Microsoft's Responsible AI FAQs are intended to help you understand how AI technology works, the choices system owners and users can make that influence system performance and behavior, and the importance of thinking about the whole system, including the technology, the people, and the environment. You can use Responsible AI FAQs to better understand specific AI systems and features that Microsoft develops.

Responsible AI FAQs are part of a broader effort to put Microsoft's AI principles into practice. To find out more, see [Microsoft AI principles](#) .

## AI-driven features in this app

This app contains a growing list of AI-driven features. To learn about the capabilities and impact of specific features, select a feature name from the list.

- [Copilot data security and privacy](#)
- [Copilot data sharing](#)
- [Use Copilot to generate deployment notes in pipelines](#)

## Related content

- [Responsible AI FAQs for Power Apps](#)
- [Responsible AI FAQs for Power Automate](#)
- [Responsible AI FAQs for Power Pages](#)

# FAQ for use Copilot to generate deployment notes in pipelines

05/31/2025

These frequently asked questions (FAQ) describe the AI impact of the use Copilot to generate deployment notes with pipelines feature in Power Platform.

## What's use Copilot to generate deployment notes in pipelines?

AI-generated deployment notes is a feature in pipelines for Power Platform that allows you to seamlessly generate deployment notes to provide more context to your deployment record. The only steps are to review what AI has generated for you (prefilled in the **Deployment notes** field), and continue to deploy.

## What are the system's capabilities?

This capability is powered by [Azure OpenAI Service](#), which uses powerful language models. These language models generate new text in English that looks and sounds like text written by humans.

Using Azure OpenAI service as a foundation, deployment notes are generated in the form of a solution overview, providing a concise solution description and an overview of its components. These notes are based on component metadata such as name, description, and type.

## What is the system's intended use?

AI-generated deployment notes help admins approve and manage deployments in their organization, providing insight into what a solution contains and its purpose. AI-generated content might have mistakes. Make sure it's accurate and appropriate before using it.

## How was use Copilot to generate deployment notes in pipelines evaluated? What metrics are used to measure performance?

Use Copilot to generate deployment notes in pipelines underwent substantial testing before the feature was released in preview. If you encounter issues with the content being generated,

please submit feedback. Your feedback is used to improve Microsoft products and services. IT admins for your organization can view and manage your feedback data.

For more information, go to: [Data, privacy, and security for Azure OpenAI Service](#)

## What are the limitations of use Copilot to generate deployment notes in pipelines? How can users minimize the impact of the AI-generated deployment notes in pipelines limitations when using the system?

This capability is in the process of rolling out, and might not be available in your region yet.

This capability might be subject to usage limits or capacity throttling.

The following are requirements to access the feature:

- This feature is supported in the following languages: English, Chinese (Simplified), Czech, Danish, Dutch, Finnish, French, German, Greek, Italian, Japanese, Korean, Norwegian (Bokmål), Polish, Portuguese (Brazil), Russian, Spanish, Swedish, Thai, and Turkish. Copilot uses the language selected in **Settings > Power Apps settings > Language** in [make.powerapps.com](https://make.powerapps.com).
- AI deployment notes must be enabled for your pipeline to use the Copilot leveraging AI models. To enable the feature:
  1. Go to your pipelines host environment > **Deployment Pipeline Configuration > Pipelines** > and select the desired pipeline.
  2. On the **General** settings page, under **Pipeline Details**, enable or disable **AI deployment notes**.

More information: [Use Copilot to generate deployment notes in pipelines](#).

## What operational factors and settings allow for effective and responsible use of the system?

Complete these tasks to make the most of this feature:

- Name your solution components accurately.
- Add descriptions to your solution components to give them more context.

## See also

- [Use Copilot to generate deployment notes in pipelines](#)